



Ingeniería de Software - 7mo Ciclo
Desarrollo de Soluciones IOT
1ASI0572
Profesor: Marco Antonio León Baca

Report

Tavolo

Team Members:

Member	Code
Baldeon Fabian, Aldo Alberto	u202122633
Cama Salvatierra, Jimena Tamara	u202210778
Castillo Castillo, Jair Alexander	u202211390
La Torre Valle, Franz Jair	u202012378
Quezada Portalatino, Barbara Susana	u202211800

Mayo, 2025

Registro de Versiones del Informe

El objetivo de esta sección es resumir las modificaciones relevantes que se realizan al informe durante el ciclo de vida del proyecto. Esta sección inicia en una página nueva e incluye un cuadro con la siguiente estructura:

Versión	Fecha	Autor	Descripción de modificación
0.1	18/04/2025	Equipo completo	Creación inicial del informe con la estructura base y distribución de responsabilidades

Versión	Fecha	Autor	Descripción de modificación
0.2	19/04/2025	Aldo Baldeon, Jair Castillo	Desarrollo del Capítulo I: Introducción, Startup Profile y Solution Profile
0.3	20/04/2025	Franz La Torre	Desarrollo del Capítulo II: Análisis competitivo y diseño de entrevistas
0.4	20/04/2025	Barbara Quezada, Jimena Cama	Desarrollo inicial del Capítulo IV: Bounded Contexts y Domain-Driven Design
0.5	21/04/2025	Jair Castillo, Aldo Baldeon	Desarrollo del Capítulo III: To-Be Scenario Mapping e Impact Mapping
0.6	22/04/2025	Franz La Torre	Realización y documentación de entrevistas, desarrollo del Needfinding
0.7	23/04/2025	Equipo completo	Revisión general del documento e integración de todas las secciones
0.8	24/04/2025	Barbara Quezada, Jimena Cama	Finalización de diagramas C4 y mejora de los diagramas de clase del dominio
0.9	24/04/2025	Aldo Baldeon	Desarrollo del Product Backlog y mejora de las User Stories
1.0	25/04/2025	Equipo completo	Revisión final, corrección de errores y entrega del documento para TB1
1.1	28/04/2025	Jimena Cama	Redacción de lineamientos de estilo (5.1), diseño de la Landing Page (wireframe y mock-up), prototipado de aplicaciones, y planificación del Sprint 1
1.2	30/04/2025	Jair Castillo	Gestión de la pantalla de Mesas y formulario (agregar/eliminar), desarrollo backend, documentación de testing, servicios y colaboración de equipo en el Sprint
1.3	02/05/2025	Barbara Quezada	Desarrollo de arquitectura de información, implementación de Landing Page, Servicios y Aplicaciones, despliegue del software y pantalla de ver sedes con formulario de reserva
1.4	03/05/2025	Franz La Torre	Diseño UX/UI de aplicaciones (wireframes, wireflows, mock-ups, user flows), configuración del entorno de desarrollo, gestión del código fuente y despliegue; implementación de pantalla menú

Project Report Collaboration Insights

Para desarrollar el proyecto, se empleó GitHub como herramienta para gestionar versiones y facilitar la colaboración en tiempo real. A continuación, se proporciona el enlace al repositorio del proyecto:

Descripción	Enlace
Repositorio del Proyecto	https://github.com/orgs/upc-pre-202510-1asi0572-2952-tavolo/repositories
Link de la Organización	https://github.com/upc-pre-202510-1asi0572-2952-tavolo

TB1:

Para el TB1, se asignaron responsabilidades específicas a cada integrante, las cuales se detallan a continuación:

Miembro del Equipo	Tarea Asignada
Aldo Baldeon	Desarrollo del Capítulo I (Startup Profile, Solution Profile, análisis de antecedentes y problemática) y Capítulo III (To-Be Scenario Mapping, Product Backlog)
Barbara Quezada	Desarrollo del Capítulo IV (Arquitectura del software, diseño estratégico y táctico de Domain-Driven Design, C4 Model)
Franz La Torre	Desarrollo del Capítulo II (Análisis de competidores, Needfinding, diseño y análisis de entrevistas)
Jimena Cama	Desarrollo del Capítulo IV (Diagramas de clase, diagramas de componentes, diseño de BD)
Jair Castillo	Desarrollo del Capítulo I (Lean UX Process, Lean UX Canvas) y Capítulo III (User Stories, Impact Mapping)

TP:

Para el TP, se asignaron responsabilidades específicas a cada integrante, las cuales se detallan a continuación:

Miembro del Equipo	Tareas Asignadas
Aldo Baldeon	Pantalla de Inicio de Sesión, Pantalla de Registro, Pantalla Ver Sedes (Lista y Mapa), Navigation Bar, Desarrollo Backend
Jimena Cama	Documentación del capítulo V (Style Guidelines, General Style Guidelines, Web, Mobile and IoT Style Guidelines, Landing Page UI Design, Landing Page Wireframe, Landing Page Mock-up, Applications Prototyping) y del capítulo VI (Sprint Planning, Sprint Backlog)
Jair Castillo	Documentación de capítulo VI (Testing Suite Evidence for Sprint Review, Services Documentation Evidence, Team Collaboration Insights, Pantalla Gestión de Mesas y Form (Agregar/Eliminar)) Desarrollo Backend
Barbara Quezada	Documentación del capítulo V (Organization Systems, Labeling Systems, SEO Tags, Searching Systems, Navigation Systems), Documentación del capítulo VI (Implementación de Landing Page, Servicios y Aplicaciones, Evidencias de Sprint, Development, Execution, Deployment), Pantalla Ver Sedes - Mesas y Formulario de Reserva

Miembro del Equipo	Tareas Asignadas
Franz La Torre	Documentación del capítulo V (Diseño UX/UI de Aplicaciones, Wireframes, Wireflow Diagrams, Mock-ups, User Flow Diagrams), Documentación del capítulo VI (Gestión de Configuración de Software, entorno de desarrollo, Control de versiones, Guías de estilo, Configuración de despliegue), Pantalla Menú

Cabe destacar que, durante la creación del informe, se realizaron commits específicos para cada tarea asignada, lo que permitió garantizar un control de versiones y una colaboración efectiva en tiempo real. El equipo mantuvo reuniones semanales de sincronización para asegurar la coherencia entre los diferentes capítulos y compartir avances.

Miembros

- Aldo Baldeon (CodAress) - Responsable del análisis de la solución y estructuración del producto
- Barbara Quezada (BarbaraQP15) - Responsable del diseño arquitectónico y modelado estratégico
- Franz La Torre (FranzJairLTV) - Responsable de la investigación de usuarios y análisis competitivo
- Jimena Cama (aksonie) - Responsable del modelado técnico y representación de componentes
- Jair Castillo (U202211390) - Responsable del marco metodológico y especificación de historias

El uso de GitHub ha permitido gestionar versiones y fomentar la colaboración en tiempo real, facilitando que los integrantes del equipo trabajen tanto de forma asincrónica como sincrónica en el proyecto. Asimismo, la metodología Gitflow ha posibilitado la creación de diversas ramas para el desarrollo de las tareas asignadas, promoviendo un trabajo autónomo y colaborativo entre los miembros.

La rama "develop" se utilizó como un entorno para integrar las tareas de cada miembro de manera estructurada y organizada, mientras que la rama "main" se destinó a la entrega de la versión final del proyecto. Esta estrategia permitió una revisión conjunta del contenido antes de su integración final, reduciendo errores y asegurando la calidad del informe presentado.

Contenido

Tabla de Contenidos

[Registro de Versiones del Informe](#)

[Project Report Collaboration Insights](#)

[Contenido](#)

[Student Outcomes](#)

[Capítulo I: Introducción](#)

- [1.1. Startup Profile](#)
 - [1.1.1. Descripción de la Startup](#)
 - [1.1.2. Perfiles de integrantes del equipo](#)
- [1.2. Solution Profile](#)
 - [1.2.1. Antecedentes y problemática](#)
 - [1.2.2. Lean UX Process](#)
 - [1.2.2.1. Lean UX Problem Statements](#)
 - [1.2.2.2. Lean UX Assumptions](#)

- **Business Outcomes:**
- **Users:**
- **User Outcomes & Benefits:**
- **Feature Assumptions:**
- **Business Assumptions:**
- **User Assumptions:**
- 1.2.2.3. Lean UX Hypothesis Statements
- 1.2.2.4. Lean UX Canvas
- 1.3. Segmentos objetivo

Capítulo II: Requirements Elicitation & Analysis

- 2.1. Competidores
 - 2.1.1. Análisis competitivo
 - 2.1.2. Estrategias y tácticas frente a competidores
- 2.2. Entrevistas
 - 2.2.1. Diseño de entrevistas
 - 2.2.2. Registro de entrevistas
 - 2.2.3. Análisis de entrevistas
- 2.3. Needfinding
 - 2.3.1. User Personas
 - 2.3.2. User Task Matrix
 - 2.3.3. User Journey Mapping
 - 2.3.4. Empathy Mapping
 - 2.3.5. As-is Scenario Mapping
- 2.4. Ubiquitous Language

Capítulo III: Requirements Specification

- 3.1. To-Be Scenario Mapping
- 3.2. User Stories
- 3.3. Impact Mapping
- 3.4. Product Backlog

Capítulo IV: Solution Software Design

- 4.1. Strategic-Level Domain-Driven Design
 - 4.1.1. Event Storming
 - 4.1.1.1 Candidate Context Discovery
 - 4.1.1.2 Domain Message Flows Modeling
 - 4.1.1.3 Bounded Context Canvases
 - 4.1.2. Context Mapping
 - 4.1.3. Software Architecture
 - 4.1.3.1. Software Architecture System Landscape Diagram
 - 4.1.3.2. Software Architecture Context Level Diagrams
 - 4.1.3.3. Software Architecture Deployment Diagrams
- 4.2. Tactical-Level Domain-Driven Design
 - 4.2.X. Bounded Context: Bounded Context Name
 - 4.2.X.1. Domain Layer
 - 4.2.X.2. Interface Layer
 - 4.2.X.3. Application Layer
 - 4.2.X.4. Infrastructure Layer

- 4.2.X.6. Bounded Context Software Architecture Component Level Diagrams
- 4.2.X.7. Bounded Context Software Architecture Code Level Diagrams
 - 4.2.X.7.1. Bounded Context Domain Layer Class Diagrams
 - 4.2.X.7.2. Bounded Context Database Design Diagram

Capítulo V: Solution UI/UX Design

- 5.1. Style Guidelines
 - 5.1.1. General Style Guidelines
 - 5.1.2. Web, Mobile and IoT Style Guidelines
- 5.2. Information Architecture
 - 5.2.1. Organization Systems
 - 5.2.2. Labeling Systems
 - 5.2.3. SEO Tags and Meta Tags
 - 5.2.4. Searching Systems
 - 5.2.5. Navigation Systems
- 5.3. Landing Page UI Design
 - 5.3.1. Landing Page Wireframe
 - 5.3.2. Landing Page Mock-up
- 5.4. Applications UX/UI Design
 - 5.4.1. Applications Wireframes
 - 5.4.2. Applications Wireflow Diagrams
 - 5.4.3. Applications Mock-ups
 - 5.4.4. Applications User Flow Diagrams
- 5.5. Applications Prototyping

Capítulo VI: Product Implementation, Validation & Deployment

- 6.1. Software Configuration Management
 - 6.1.1. Software Development Environment Configuration
 - 6.1.2. Source Code Management
 - 6.1.3. Source Code Style Guide & Conventions
 - 6.1.4. Software Deployment Configuration
- 6.2. Landing Page, Services & Applications Implementation
 - 6.2.1. Sprint 1
 - 6.2.1.1. Sprint Planning 1
 - 6.2.1.2. Aspect Leaders and Collaborators
 - 6.2.1.3. Sprint Backlog 1
 - 6.2.1.4. Development Evidence for Sprint Review
 - 6.2.1.5. Testing Suite Evidence for Sprint Review
 - 6.2.1.6. Execution Evidence for Sprint Review
 - 6.2.1.7. Services Documentation Evidence for Sprint Review
 - 6.2.1.8. Software Deployment Evidence for Sprint Review
 - 6.2.1.9. Team Collaboration Insights during Sprint
 - 6.2.2. Sprint 2
 - 6.2.2.1. Sprint Planning 2.
 - 6.2.2.2. Aspect Leaders and Collaborators.
 - 6.2.2.3. Sprint Backlog 2.
 - 6.2.2.4. Development Evidence for Sprint Review.
 - 6.2.2.5. Testing Suite Evidence for Sprint Review.
 - 6.2.2.6. Execution Evidence for Sprint Review.
 - 6.2.2.7. Services Documentation Evidence for Sprint Review.

- 6.2.3. Sprint 3
 - 6.2.3.1. Sprint Planning 3
 - 6.2.3.2. Aspect Leaders and Collaborators
 - 6.2.3.3. Sprint Backlog 3
 - 6.2.3.4. Development Evidence for Sprint Review
 - 6.2.3.5. Testing Suite Evidence for Sprint Review
 - 6.2.3.6. Execution Evidence for Sprint Review
 - 6.2.3.7. Services Documentation Evidence for Sprint Review
 - 6.2.3.8. Software Deployment Evidence for Sprint Review
 - 6.2.3.9. Team Collaboration Insights during Sprint
- 6.3. Validation Interviews
 - 6.3.1. Diseño de Entrevistas
 - 6.3.2. Registro de Entrevistas
 - 6.3.3. Evaluaciones según heurísticas
- 6.4. Video About-the-Product

Conclusiones

Bibliografía

Anexos

Student Outcome

El curso contribuye al cumplimiento del Student Outcome ABET:

ABET – EAC - Student Outcome 5

Criterio: La capacidad de funcionar efectivamente en un equipo cuyos miembros juntos proporcionan liderazgo, crean un entorno de colaboración e inclusivo, establecen objetivos, planifican tareas y cumplen objetivos.

En el siguiente cuadro se describe las acciones realizadas y enunciados de conclusiones por parte del grupo, que permiten sustentar el haber alcanzado el logro del ABET – EAC - Student Outcome 5.

Criterio específico	Acciones realizadas	Conclusiones
Trabaja en equipo para proporcionar liderazgo en forma conjunta	TB1: Baldeon Fabian, Aldo Alberto - He demostrado liderazgo colaborando activamente con Jair en el desarrollo del Capítulo I y en el diseño táctico de la solución. Asumí el liderazgo en la elaboración del perfil de la startup, la definición de los antecedentes, el desarrollo del Lean UX Process, y el diseño a nivel táctico del software con el enfoque de Domain-Driven Design. Esto ha permitido consolidar una visión común del proyecto.	TB1: Hemos demostrado un liderazgo compartido al asumir responsabilidades clave en cada capítulo del proyecto, asegurando que cada integrante tome la iniciativa en distintas áreas. La colaboración entre Aldo y Jair en la introducción y conceptualización del problema, el liderazgo de Franz en la investigación de requerimientos, y la toma de decisiones técnicas y estratégicas por parte de Jimena y Bárbara reflejan un liderazgo distribuido y coherente. Esto ha permitido que el equipo avance con una visión unificada del proyecto y tome decisiones informadas en conjunto.
		TP1: Como equipo, ejercimos un liderazgo conjunto al asumir

Cama Salvatierra, Jimena Tamara

- He demostrado la capacidad de trabajar eficazmente en equipos multidisciplinarios, asumiendo roles de liderazgo compartido para la toma de decisiones y el logro de objetivos comunes. En el desarrollo de proyectos, fomenté un entorno inclusivo donde cada integrante pudo aportar sus fortalezas, promoviendo la comunicación abierta, la planificación conjunta de tareas y la adaptación ante cambios.

Castillo Castillo, Jair Alexander

- He liderado en conjunto con Aldo el desarrollo del Capítulo I, enfocándome especialmente en la elaboración del Lean UX Canvas y la definición de hipótesis. Asimismo, tomé la iniciativa en la elaboración del To-Be Scenario Mapping y el Impact Mapping, consolidando así la visión futura del sistema.

La Torre Valle, Franz Jair

- Asumí un rol de liderazgo en el Capítulo II, enfocándome en el análisis de requerimientos. Me encargué del análisis competitivo, diseño y evaluación de entrevistas, así como del Needfinding completo. Lideré la exploración profunda de las necesidades de los usuarios para alinear los requerimientos con la visión del producto.

Quezada Portalatino, Barbara Susana

- He compartido el liderazgo del Capítulo IV con Jimena, asegurando que el diseño de alto nivel del sistema refleje la arquitectura definida y cumpla con las expectativas funcionales. También aporté activamente en la redacción del backlog y en las historias de usuario del sistema.

TP1:**Baldeon Fabian, Aldo Alberto**

responsabilidades según nuestras fortalezas individuales, distribuyendo tareas de forma equitativa y coordinada. A lo largo del proyecto, tomamos decisiones colaborativas que permitieron mantener una visión unificada del sistema, y nos apoyamos mutuamente para superar obstáculos técnicos y organizativos. Esta dinámica nos permitió liderar desde distintos frentes del desarrollo —diseño, backend, frontend y gestión—, asegurando que todos los módulos avanzaran de forma integrada y con sentido de propósito común.

- He trabajado en equipo asumiendo responsabilidades clave del frontend y backend, lo cual me permitió brindar liderazgo conjunto en el desarrollo de funcionalidades como el inicio de sesión, registro y navegación, asegurando cohesión entre el equipo técnico.

Cama Salvatierra, Jimena Tamara

- Asumí un rol de liderazgo dentro del equipo al desarrollar las directrices de estilo y prototipado de interfaces, guiando visualmente a los demás miembros para mantener la coherencia del diseño en todas las plataformas.

Castillo Castillo, Jair Alexander

- Participé de manera activa en el equipo asumiendo el liderazgo en aspectos técnicos como pruebas, documentación de servicios y la gestión del módulo de mesas, lo que fortaleció la estructura del desarrollo.

La Torre Valle, Franz Jair

- Aporté liderazgo en la definición de la experiencia de usuario, wireframes, flujos y configuración del entorno de desarrollo, asegurando que las decisiones técnicas se alinearan con los objetivos del equipo.

Quezada Portalatino, Barbara Susana

- Desde el área de arquitectura de la información y la implementación de la landing page y servicios, proporcioné liderazgo conjunto asegurando que los contenidos estuvieran correctamente estructurados y fueran accesibles.

Crea un entorno colaborativo e inclusivo, establece metas,

TB1:
Baldeon Fabian, Aldo Alberto
- He contribuido a crear un entorno de trabajo participativo al compartir avances de forma constante, establecer cronogramas claros y

TB1:
A lo largo del desarrollo del proyecto, hemos construido un entorno de trabajo colaborativo y respetuoso, donde cada voz ha sido escuchada y valorada. La planificación de tareas se realizó en base a las fortalezas individuales, estableciendo metas claras para cada entrega. La

**planifica
tareas y
cumple
objetivos.**

ayudar a coordinar las entregas de los distintos capítulos. Me aseguré de cumplir los objetivos asignados dentro de los plazos, especialmente en los apartados de análisis y diseño de la solución.

Cama Salvatierra, Jimena Tamara

- He demostrado la capacidad de crear un entorno de trabajo colaborativo e inclusivo, en el que se valora la participación activa de todos los integrantes del equipo. A lo largo del desarrollo del proyecto, trabajé en la definición de metas comunes, la planificación de tareas con enfoque estratégico y el cumplimiento de los objetivos establecidos, asegurando una gestión eficiente del tiempo y los recursos.

Castillo Castillo, Jair Alexander

- He aportado al trabajo grupal coordinando con los demás miembros la integración de los entregables del Capítulo III, asegurando que los insumos de análisis sean reflejados en las historias de usuario y el backlog. Mi planificación detallada ha permitido cumplir los entregables sin contratiempos.

La Torre Valle, Franz Jair

- Organicé sesiones para compartir hallazgos con el equipo, facilitando un espacio abierto para retroalimentación. Además, establecí objetivos claros para cada sección del capítulo y cumplí con cada uno dentro del tiempo estimado.

Quezada Portalatino, Barbara Susana

- Promoví la colaboración mediante la constante retroalimentación con el equipo y la organización de sesiones de revisión conjunta. Gracias a una planificación ordenada y objetiva, logré cumplir con todos los entregables asignados, manteniendo una sinergia adecuada con Jimena y el

comunicación constante entre los miembros del grupo permitió resolver dudas, alinear criterios y asegurar el cumplimiento de los objetivos establecidos. Esta dinámica no solo optimizó la ejecución del proyecto, sino que fortaleció la cohesión del equipo.

TP1:

Durante todo el proyecto, generamos un ambiente de trabajo colaborativo e inclusivo, en el que todas las ideas fueron escuchadas y consideradas. Establecimos metas claras en cada sprint, planificamos tareas basadas en prioridades reales del sistema y mantuvimos una comunicación constante entre los miembros. Esta organización nos permitió cumplir con los objetivos trazados, mejorar la calidad del producto final y fortalecer nuestras habilidades para trabajar de forma efectiva como equipo multidisciplinario.

resto del equipo.

TP1:

Baldeon Fabian, Aldo Alberto

- Contribuí activamente a un entorno colaborativo e inclusivo, proponiendo metas claras para el avance del desarrollo de pantallas y el backend, planificando tareas específicas y cumpliendo los entregables según lo definido en los sprints.

Cama Salvatierra, Jimena Tamara

- Fomenté un entorno colaborativo e inclusivo durante el diseño UX/UI, facilitando la planificación de tareas de diseño, estableciendo metas claras por sprint y asegurando que los entregables cumplan los estándares de calidad definidos.

Castillo Castillo, Jair Alexander

- Establecí y cumplí metas relacionadas al testing y documentación, planificando mis tareas de manera coordinada con el equipo, fomentando una colaboración efectiva que permitió lograr los objetivos del sprint.

La Torre Valle, Franz Jair

- Planifiqué y ejecuté tareas relacionadas a la configuración del software y estándares de código, promoviendo un entorno colaborativo, con comunicación clara y objetivos bien definidos, logrando avances significativos en los entregables del proyecto.

Quezada Portalatino, Barbara

Susana

- Fui parte activa en la planificación de tareas y definición de objetivos del equipo, garantizando que tanto los sistemas de navegación como el contenido cumplieran con los estándares de calidad y usabilidad esperados.

Capítulo I: Introducción

1.1. Startup Profile

1.1.1. Descripción de la Startup

TAVOLO es una solución tecnológica desarrollada para **optimizar la experiencia en cafeterías peruanas** mediante el uso de **dispositivos IoT** integrados a un sistema web y móvil. Su propuesta se enfoca exclusivamente en resolver los problemas relacionados con la ocupación de mesas, la gestión de reservas y la visualización de información relevante para los comensales, administradores y usuarios superadministradores.

Para los comensales, TAVOLO permite visualizar el menú digital de una sede, conocer la disponibilidad de mesas en tiempo real, realizar reservas y consultar detalles de las mismas. También se ofrece la posibilidad de visualizar la ubicación de todas las sedes en un mapa interactivo, registrarse como nuevo usuario o iniciar sesión para acceder a funcionalidades personalizadas. Estas características permiten al usuario planificar su visita con antelación y evitar tiempos de espera innecesarios. Un estudio reciente de Apoyo Consultoría (2023) reveló que el 62 % de los limeños evita acudir a locales con alta concurrencia si no puede consultar previamente la disponibilidad, lo cual respalda la utilidad de este enfoque preventivo basado en datos en tiempo real.

Desde el panel de administración, los administradores pueden gestionar la sede que se les ha asignado. Esto incluye agregar nuevas mesas, eliminar aquellas que ya no estén en uso, visualizar todas las reservas registradas en su local y confirmar que están gestionando la sede correcta. Esta funcionalidad permite a los encargados mantener un control actualizado del aforo, planificar la atención de manera eficiente y optimizar la logística operativa. La digitalización de estas tareas manuales no solo mejora la precisión en la gestión, sino que también incrementa la productividad del personal administrativo.

TAVOLO se presenta así como una respuesta concreta y realista a los desafíos cotidianos que enfrentan las cafeterías en Perú. No pretende cubrir todo el ciclo de negocio de un restaurante, sino **centrarse exclusivamente en la mejora del proceso de aforo, reservas y atención al cliente**. Este enfoque especializado y realista busca generar impacto directo en la experiencia del usuario y en la eficiencia del negocio, con una implementación que puede ser gradual, asequible y con bajo nivel de complejidad técnica.

Misión:

Brindar una solución tecnológica eficiente y accesible para la gestión de aforo en cafeterías peruanas, utilizando dispositivos IoT y plataformas digitales que permitan a los comensales acceder a información en tiempo real, y a los administradores optimizar la ocupación de mesas y las reservas, **mejorando así la experiencia del cliente y la operación del negocio**.

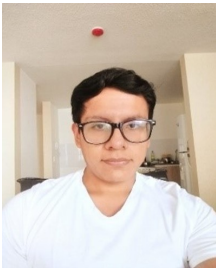
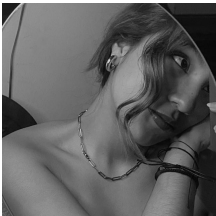
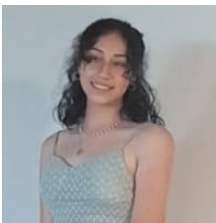
Visión:

Consolidarse como la **solución de referencia en el Perú** para la digitalización de procesos de atención en cafeterías, mediante el uso de tecnología embebida, edge computing y sistemas en la nube, promoviendo una experiencia de consumo más ordenada, rápida y satisfactoria para los comensales, y una gestión más eficiente para los negocios del sector gastronómico.

1.1.2. Perfiles de integrantes del equipo

1.1.2. Perfiles de integrantes del equipo

Foto	Información
------	-------------

Foto	Información
	Nombre y Apellido: Aldo Alberto Baldeon Fabian Código: U202122633 Carrera: Ingeniería de Software Acerca de: Soy Aldo Alberto Baldeon Fabian, estudio la carrera de Ingeniería de Software en la UPC. Escogí esta carrera porque me interesó el desarrollo de aplicaciones. Soy responsable y me gusta trabajar en equipo. Poseo conocimientos básicos en C#, java, JavaScript, HTML y CSS. También, poseo conocimientos intermedios en C + +, SQL y MongoDB, además de contar con experiencia en Git y GitHub.
	Nombre y Apellido: Barbara Susana Quezada Portalatino Código: U202211800 Carrera: Ingeniería de Software Acerca de: Barbara Susana Quezada Portalatino, cursando el séptimo ciclo de la carrera de software, trabajo mayormente bajo un rol de líder que me ayuda a poder organizar no solo la ideas de mi grupo sino que las ideas aterricen en la ejecución. Soy una persona muy disciplinada y detallista.
	Nombre y Apellido: Jimena Tamara Cama Salvatierra Código: U2022210778 Carrera: Ingeniería de Software Acerca de: Soy estudiante de la carrera de Ingeniería de Software en la UPC y actualmente estoy cursando el 7mo ciclo. Me considero una persona curiosa, determinada y organizada. Con la experiencia en proyectos de startup y trabajos en equipo, trabajaré junto a mis compañeros para lograr un óptimo resultado del proyecto.
	Nombre y Apellido: Jair Alexander Castillo Castillo Código: U202211390 Carrera: Ingeniería de Software Acerca de: Soy estudiante de la carrera de Ingeniería de Software en la UPC y me encuentro en el 7to ciclo. Me considero una persona dedicada, comprensiva y metódica. Con mis habilidades de liderazgo y mi capacidad para trabajar en equipo en un ambiente de respeto, estoy segura de que podré dirigir la implementación de la startup de nuestro proyecto de manera exitosa.
	Nombre y Apellido: Franz Jair La Torre Valle Código: U202012378 Carrera: Ingeniería de Software Acerca de: Soy estudiante de la carrera de Ingeniería de Software en la Universidad Peruana de Ciencias Aplicadas. Mi compromiso con el equipo se basa en mantener una participación activa en todas las actividades, estar siempre disponible para resolver dudas y contribuir de manera significativa al desarrollo del proyecto. Busco aportar con soluciones innovadoras, apoyo constante y un aprendizaje continuo para alcanzar los objetivos propuestos.

1.2. Solution Profile

TAVOLO es una solución tecnológica integral basada en IoT que transforma la forma en que los comensales interactúan con las cafeterías. El sistema utiliza sensores de peso (con un margen de error inferior al 5 %) para detectar en tiempo real la ocupación de las mesas. Esta información se transmite automáticamente a una aplicación web y móvil dirigida a los comensales, permitiéndoles consultar la disponibilidad de mesas desde sus dispositivos personales antes de llegar al establecimiento.

El objetivo principal es minimizar los tiempos de espera, asegurando que el **usuario final no espere más de 2 minutos para acceder a una mesa libre**. Para ello, el sistema realiza **actualizaciones cada 10 segundos** y cuenta con una interfaz optimizada con tiempos de carga inferiores a 5 segundos. De este modo, se mejora significativamente la experiencia del cliente, brindándole información clara, precisa y en tiempo real que le facilite tomar decisiones sobre su visita.

Actualmente, la implementación del sistema, tanto a nivel físico como tecnológico, está a cargo del equipo de desarrollo, quienes conocen a fondo su funcionamiento. Sin embargo, contemplamos una fase de escalamiento en la que el personal de cada cafetería podrá instalar el sistema en menos de una hora, sin necesidad de asistencia técnica especializada.

Además, TAVOLO incluye una interfaz administrativa exclusiva para cada sede, donde los administradores pueden agregar mesas, consultar la sede asignada, visualizar reservas y, próximamente estadísticas. Esta funcionalidad permitirá optimizar la rotación de mesas y mejorar la gestión del flujo de comensales.

1.2.1. Antecedentes y problemática

En los últimos años, el sector gastronómico en el Perú ha mostrado un crecimiento sostenido, especialmente en ciudades como Lima, Arequipa y Cusco, donde las cafeterías han ganado popularidad como espacios de encuentro social y laboral. Según el Ministerio de la Producción (2022), **los negocios relacionados con alimentos y bebidas representaron más del 35 % del total de microempresas en el sector servicios**. Sin embargo, la mayoría de estos establecimientos aún carece de tecnologías que les permitan gestionar de forma eficiente su aforo y rotación de clientes, especialmente en horas pico. Pese al avance de la digitalización en otros rubros, la implementación de tecnologías inteligentes, como sensores IoT o aplicaciones móviles personalizadas, aún es limitada en el sector gastronómico peruano (INEI, 2021).

Actualmente, uno de los principales desafíos para los clientes de cafeterías en el Perú es la incertidumbre sobre la disponibilidad de mesas, lo que genera pérdidas de tiempo, experiencias insatisfactorias y, en algunos casos, la decisión de no ingresar al local. Esta problemática se intensifica en zonas urbanas con alta demanda, donde la falta de sistemas de gestión en tiempo real impide una distribución óptima del aforo. Según un estudio de Apoyo Consultoría (2023), **más del 60 % de los consumidores en Lima indicaron que evitarían acudir a locales concurridos si tuvieran una alternativa digital que les informe sobre la disponibilidad antes de salir de casa**. Esta situación representa no solo una oportunidad de mejora para la experiencia del cliente, sino también una necesidad de transformación digital en la industria.

Aplicando la técnica de las 5 W y 2 H:

What? - ¿Qué?

TAVOLO es una solución tecnológica integrada con IoT que permite **monitorear en tiempo real la ocupación de mesas en cafeterías**, proporcionando a los clientes información precisa sobre la disponibilidad antes de llegar al establecimiento. Además, ofrece a los administradores la posibilidad de **gestionar las mesas y reservas de comensales** mediante una interfaz intuitiva.

Why? - ¿Por qué?

El problema se vuelve crítico en momentos de alta afluencia, como durante almuerzos, cenas, fines de semana o eventos especiales, en los cuales la ocupación irregular de las mesas puede generar aglomeraciones y una mala experiencia para los clientes.

Where? - ¿Dónde?

El sistema está destinado a **cafeterías ubicadas en áreas urbanas** de alta demanda, comenzando con Lima, y se proyecta expandir a otras ciudades del Perú y eventualmente a Latinoamérica. La interfaz web y móvil estará disponible

para clientes en cualquier lugar, mientras que la instalación física de los sensores de ocupación será realizada dentro de las cafeterías.

When? - ¿Cuándo?

La implementación inicial del sistema se desarrollará en un corto plazo, con un escalamiento proyectado a futuro para permitir la instalación por parte del personal de las cafeterías sin soporte técnico especializado.

Who? - ¿Quién?

El sistema está dirigido a comensales y administradores de cafeterías. Los comensales utilizan la aplicación para verificar la disponibilidad de mesas y realizar reservas. Los administradores gestionan las mesas y reservas.

How? - ¿Cómo?

La solución se implementará instalando sensores de peso en cada mesa, conectados a una red IoT que transmite datos en tiempo real a una plataforma centralizada. A través de una aplicación móvil y web, los usuarios podrán visualizar la disponibilidad de mesas y, a su vez, el personal del establecimiento podrá gestionar la ocupación de manera óptima y recibir alertas en caso de anomalías.

How much? - ¿Cuánto?

¿Cuánto afecta este problema?

La falta de control de aforo puede traducirse en pérdidas económicas para los establecimientos debido a la ineficiencia en la asignación de mesas, así como en sanciones por incumplimiento de normativas de seguridad.

¿Cuánto costará resolver este problema?

El costo de implementación dependerá del número de mesas y la infraestructura necesaria para la conexión y mantenimiento de los sensores, así como del desarrollo de la plataforma digital. Se estima que la inversión inicial se recuperará rápidamente mediante la optimización del uso del espacio y el incremento en la satisfacción del cliente.

¿Cuántas personas se beneficiarán?

Esta solución beneficiará directamente a los clientes y al personal de los establecimientos, con un impacto indirecto en el sector gastronómico en general. Se estima que, en establecimientos de tamaño mediano, la adopción de la tecnología podría mejorar la eficiencia operativa en un 30-40%, beneficiando a cientos de usuarios diarios.

La problemática identificada no requiere una transformación completa del modelo de negocio de las cafeterías, sino una **intervención técnica puntual** que resuelva los cuellos de botella más frecuentes: **el acceso a información de disponibilidad, la planificación anticipada y la gestión eficiente del espacio físico**. En ese sentido, **TAVOLO no busca reemplazar los procesos tradicionales**, sino complementarlos con herramientas tecnológicas que permitan a las cafeterías peruanas operar con mayor previsibilidad, rapidez y satisfacción del cliente.

1.2.2. Lean UX Process

1.2.2.1. Lean UX Problem Statements

Visitante

Para los visitantes que desean informarse sobre soluciones tecnológicas aplicadas a cafeterías, **que necesitan** comprender qué es TAVOLO, cómo funciona y qué valor ofrece, **pero** actualmente no encuentran en el ecosistema digital peruano una presentación clara ni accesible que comunique

efectivamente los beneficios de una solución basada en IoT para el sector gastronómico.

Esto les impide considerar su adopción, recomendarla o confiar en su utilidad real.

El 35% de las microempresas peruanas del sector servicios corresponde a negocios de alimentos y bebidas, pero más del 70% de estas aún no cuenta con estrategias digitales de comunicación técnica o funcional.

TAVOLO busca resolver este vacío mediante una landing page visual, estructurada y orientada a comunicar desde el **primer contacto su propuesta de valor**, casos de uso y beneficios diferenciadores.

Comensal

Para comensales que frecuentan cafeterías en zonas urbanas del Perú,

que necesitan conocer en tiempo real la disponibilidad de mesas, visualizar el menú digital y reservar con antelación, **pero** actualmente enfrentan esperas innecesarias, incertidumbre al llegar al local y una experiencia de consumo poco predecible,

lo que afecta su comodidad y decisión de volver a consumir en el mismo establecimiento.

Según un estudio de Apoyo Consultoría (2023), **el 62 % de los consumidores limeños evitaría ingresar a un local si no puede verificar previamente su nivel de aforo.**

TAVOLO responde a esta necesidad con una plataforma sincronizada con sensores embebidos, que ofrece al comensal una app móvil para autogestión de su visita y planificación de su consumo con mínima fricción.

Administrador

Para los administradores de cafeterías encargados de operar las sedes en tiempo real,

que necesitan gestionar la ocupación de mesas, controlar reservas y visualizar su sede asignada mediante una interfaz digital centralizada,

pero actualmente dependen de métodos manuales y no cuentan con herramientas que integren sensores físicos con flujos operativos,

lo que genera errores en la asignación, sobreaforo no controlado y decisiones reactivas.

En cafeterías de tamaño mediano en Lima, la falta de automatización genera pérdidas operativas y afecta la experiencia del cliente. Según datos internos y análisis del sector, **la eficiencia puede incrementarse en un 30–40 % con tecnologías de digitalización inteligente.**

TAVOLO busca transformar este escenario mediante un panel de administración en la nube, que unifica la gestión de sedes, mesas y reservas en una interfaz intuitiva con datos en tiempo real.

1.2.2.2. Lean UX Assumptions

A partir del análisis del contexto y las necesidades identificadas en los distintos perfiles de usuario, se establecen los siguientes supuestos iniciales que guían el diseño y validación del producto TAVOLO. Estos supuestos serán **contrastados durante el desarrollo iterativo** del proyecto para asegurar que el producto responda realmente a los objetivos planteados.

Business Outcomes:

- Aumento del 25% en la eficiencia operativa de los establecimientos que adopten la solución.
- Reducción del 30% en incidencias por sobreaforo y problemas de asignación de mesas.
- Incremento del 20% en la satisfacción de los clientes, medido a través de encuestas de experiencia.
- Incremento del 15% en el flujo de clientes al evitar tiempos de espera prolongados.
- Mejora en el cumplimiento de protocolos de seguridad y normativas vigentes.

Users:

Los usuarios primarios son:

- Propietarios y administradores cafeterías: Interesados en optimizar el uso del espacio y garantizar la seguridad en el establecimiento.
- Clientes: Usuarios que buscan información en tiempo real sobre la disponibilidad de mesas para planificar su visita de manera eficiente.

User Outcomes & Benefits:

- Propietarios/Administradores:
 - Control en tiempo real del aforo y asignación de mesas.
 - Reducción de errores manuales en la gestión de espacios.
 - Acceso a estadísticas e informes que faciliten la toma de decisiones.
- Clientes:
 - Información precisa sobre la disponibilidad de mesas.
 - Reducción de tiempos de espera al llegar al local.
 - Mejora en la experiencia y comodidad al planificar su visita.

Feature Assumptions:

- Visualización en tiempo real del estado de ocupación de las mesas.
- Funcionalidad de reservas digitales.
- Menú accesible por sede desde la aplicación.
- Gestión de mesas y reservas para administradores.
- Acceso completo a la plataforma para superadministradores.
- Seguridad basada en autenticación por roles.

Business Assumptions:

1. **Creemos que nuestros usuarios necesitan** un sistema automatizado y en tiempo real que facilite la gestión del aforo en establecimientos de hostelería.
2. **Estas necesidades se pueden satisfacer** mediante la implementación de sensores IoT y el desarrollo de una plataforma digital accesible desde dispositivos móviles y computadoras.
3. **Nuestros clientes serán** cafeterías que buscan mejorar la eficiencia en la gestión de sus espacios y la experiencia de sus clientes.
4. **El valor más importante que un cliente quiere de nuestros servicios es** la capacidad de optimizar la ocupación de mesas en tiempo real, garantizando tanto la seguridad como la eficiencia operativa.
5. **El cliente también va obtener** beneficios adicionales como la recopilación de datos para análisis futuros.
6. **Vamos a obtener la mayoría de nuestros clientes mediante** estrategias de marketing digital, participación en ferias de tecnología y colaboraciones con asociaciones del sector de gastronomía.
7. **Vamos a obtener ingresos mediante** la venta de la solución IoT como hardware y software.
8. **Nuestra competencia principal** son los sistemas tradicionales de control de aforo y soluciones de monitoreo manual.

9. **Vamos a tener ventaja frente a nuestra competencia debido a** la automatización en tiempo real, la facilidad de uso y la capacidad de integración con otras plataformas, lo que mejora significativamente la eficiencia operativa.
10. **El mayor riesgo del servicio es** la posible resistencia de los establecimientos a la adopción de nuevas tecnologías y la integración con sistemas ya existentes.
11. **Lo resolveremos mediante** demostraciones prácticas, pruebas piloto y estrategias de capacitación para asegurar una transición suave y una rápida adopción de la tecnología.

User Assumptions:

¿Quién es el usuario?

Los usuarios son propietarios, administradores y clientes de restaurantes, cafeterías y otros locales de hostelería, que buscan una solución tecnológica para gestionar de manera eficiente la ocupación de mesas.

¿Qué problemas tiene nuestro producto? ¿Resolver?

El principal problema es la falta de información en tiempo real sobre la disponibilidad de mesas, lo que genera tiempos de espera, errores en la asignación de asientos y riesgos en el cumplimiento de normativas de seguridad. Nuestra solución automatiza este proceso, reduciendo la incertidumbre y mejorando la experiencia del usuario.

¿Qué características son importantes?

- Monitoreo en tiempo real de la ocupación mediante sensores de peso.
- Interfaz intuitiva y amigable en la aplicación móvil y web.
- Alertas y notificaciones automáticas para el personal del local.
- Reportes históricos y en tiempo real para el análisis del uso del espacio.
- Integración con otros sistemas de gestión interna del establecimiento.

¿Dónde encaja nuestro producto en su trabajo o vida?

La solución se integra en la operación diaria de las cafeterías, facilitando tanto la gestión interna (para administradores) como la experiencia del cliente al planificar su visita.

¿Cuándo y cómo es nuestro producto? ¿Usado?

El producto es utilizado en tiempo real, especialmente durante horas pico y en momentos de alta afluencia, permitiendo a los establecimientos gestionar el aforo de forma continua y los clientes verificar la disponibilidad antes de llegar.

¿Cómo debe verse nuestro producto y cómo debe comportarse?

La interfaz visual debe ser moderna, limpia e intuitiva, con gráficos y datos en tiempo real que faciliten la rápida comprensión del estado de ocupación. Además, el sistema debe ser ágil, confiable y accesible desde múltiples dispositivos, garantizando una experiencia fluida y sin complicaciones.

1.2.2.3. Lean UX Hypothesis Statements

A continuación, se detallan las hipótesis formuladas para abordar la problemática mediante las funcionalidades incluidas en nuestra propuesta de solución IoT. Cada hipótesis está diseñada para ser clara, cuantificable y permitir una evaluación objetiva de su éxito.

- **Hypothesis Statement 1:**

Creemos que los propietarios y administradores de cafeterías necesitan una herramienta en tiempo real que les permita monitorear la ocupación de mesas de forma automatizada.

Sabremos que hemos tenido éxito.

Cuando al menos el 75% de estos usuarios informen una mejora en la eficiencia operativa y una reducción del 30% en incidencias por sobreaforo.

- **Hypothesis Statement 2:**

Creemos que los clientes valorarán disponer de información actualizada sobre la disponibilidad de mesas a través de una aplicación móvil y web.

Sabremos que hemos tenido éxito.

Cuando el 80% de los clientes reporten una experiencia mejorada y una disminución notable en los tiempos de espera al llegar al establecimiento.

- **Hypothesis Statement 3:**

Creemos que la implementación de sensores IoT integrados en las mesas, que transmitan datos en tiempo real a una plataforma centralizada, facilitará la gestión del aforo y optimizará el flujo de clientes.

Sabremos que hemos tenido éxito.

Cuando los establecimientos vean un incremento del 25% en la eficiencia en la asignación de mesas y una recuperación de la inversión en menos de 12 meses.

- **Hypothesis Statement 4:**

Creemos que una interfaz intuitiva y amigable en la aplicación digital fomentará la adopción tanto por parte de administradores como de clientes, simplificando el proceso de toma de decisiones y la planificación de visitas.

Sabremos que hemos tenido éxito.

Cuando el 80% de los usuarios completen tareas básicas en la aplicación sin necesidad de asistencia y se mantenga un alto nivel de satisfacción en encuestas posteriores a su experiencia.

1.2.2.4. Lean UX Canvas



1.3. Segmentos objetivo

TAVOLO está diseñado para atender las necesidades reales de usuarios que interactúan directamente con el sistema dentro del entorno de atención y operación en cafeterías. A partir del análisis funcional y del levantamiento de requisitos, se identifican dos segmentos objetivos clave, sobre los cuales se construye y valida el producto.

1. Comensales recurrentes en zonas urbanas de alta afluencia

Este segmento está compuesto por usuarios finales que visitan frecuentemente cafeterías ubicadas en distritos con alto tránsito peatonal, como San Isidro, Miraflores, Surco y San Miguel. Se trata principalmente de estudiantes universitarios, trabajadores de oficina y profesionales jóvenes que priorizan la optimización de su tiempo durante el desayuno, el almuerzo o reuniones breves. Son usuarios digitales familiarizados con el uso de aplicaciones móviles y esperan poder visualizar la disponibilidad de mesas, hacer reservas y consultar el menú digital desde sus dispositivos personales. Este segmento es fundamental para validar la propuesta de valor del producto y su utilidad en entornos reales de consumo.

2. Administradores operativos de cafeterías

Este grupo corresponde a los encargados de gestionar el funcionamiento de una sede física. Tienen bajo su responsabilidad el control del aforo, la gestión de mesas y la administración de reservas. También requieren saber qué sede tienen asignada y mantener el sistema actualizado según los cambios del local. Este segmento valida la funcionalidad del panel de gestión administrativa, que incluye herramientas visuales, control de datos en tiempo real y facilidad de uso en entornos dinámicos como una cafetería con alta rotación.

Cabe precisar que, si bien el sistema contempla un **rol de superadministrador** con acceso total a las funcionalidades, este **no representa un segmento objetivo del producto**, sino que corresponde exclusivamente al equipo de desarrollo encargado de configurar el sistema, realizar pruebas y brindar soporte técnico al propietario del negocio.

Capítulo II: Requirements Elicitation & Analysis

2.1. Competidores

Algunos de los competidores a los que Tavolo podría enfrentarse son:

- **Eat App:** Plataforma de gestión de reservas y mesas que permite a los restaurantes automatizar la asignación de mesas, mejorar la experiencia del cliente y analizar datos de comportamiento. Eat App se destaca por su CRM integrado y herramientas de marketing, aunque su sistema depende de reservas manuales y no detecta la ocupación en tiempo real.
- **OpenTable:** Uno de los líderes globales en reservas en línea para restaurantes. Ofrece una experiencia fluida para los usuarios al permitir reservas anticipadas y gestionar la disponibilidad de mesas. Sin embargo, su enfoque está más orientado a restaurantes formales y no cuenta con integración de sensores ni monitoreo automático de mesas.
- **Waitlist Me:** Aplicación que digitaliza la lista de espera tradicional, permitiendo a los clientes apuntarse desde sus teléfonos y recibir notificaciones cuando su mesa esté lista. Aunque mejora la organización y reduce aglomeraciones, depende de la actualización manual por parte del personal y no ofrece información en tiempo real sobre la ocupación real del local.

2.1.1. Análisis competitivo

Competitive Analysis Landscape

¿Por qué llevar a cabo este análisis?		Este análisis se lleva a cabo para poder investigar, analizar y comparar el comportamiento de los competidores directos o indirectos en el mercado.			
Nombre		Tavolo	Eat App	OpenTable	Waitlist Me
Logo					
Perfil	Overview	Es una solución tecnológica integral basada en IoT, diseñada específicamente para cafeterías. Utiliza sensores de peso para detectar automáticamente la ocupación de mesas y actualiza esta información cada 10 segundos en una app web y móvil. El sistema también ofrece una interfaz administrativa para la sede,	Es una plataforma de gestión de restaurantes basada en la nube que permite manejar reservas, asignación de mesas, datos de clientes y campañas de marketing automatizadas. Su enfoque está en ofrecer una experiencia completa para restaurantes que desean profesionalizar la	Es una de las plataformas más reconocidas a nivel mundial para reservas en restaurantes. Los usuarios pueden buscar establecimientos, ver disponibilidad en tiempo real, leer reseñas y reservar directamente desde la app o el sitio web. También ofrece herramientas para los restaurantes,	Es una solución que digitaliza el proceso de espera en locales físicos, como restaurantes, cafeterías y tiendas. Los clientes pueden apuntarse a una lista desde sus dispositivos y reciben notificaciones cuando su turno se acerca. También permite al staff gestionar

		facilitando la gestión del flujo de comensales y mejorando la rotación de mesas.	relación con sus clientes, utilizando herramientas de CRM, reportes y análisis de comportamiento.	aunque su enfoque principal está en mejorar la experiencia del comensal antes de llegar al lugar.	el flujo de clientes de forma más organizada, aunque depende de la intervención manual.
	Ventaja competitiva ¿Qué valor ofrece a los clientes?	Brinda información precisa y en tiempo real sobre la disponibilidad de mesas, ayudando a evitar esperas y optimizando su visita desde antes de llegar al local.	Permite reservar mesas de forma sencilla y recibir un servicio más personalizado, gracias al uso de datos históricos y preferencias del cliente.	Facilita la reserva anticipada en restaurantes de forma rápida y confiable, brindando acceso a una amplia variedad de opciones en una sola plataforma.	Reduce la incertidumbre y el tiempo de espera físico, al permitirles ocupar su tiempo libremente mientras esperan su turno.
Perfil de Marketing	Mercado objetivo	Cafeterías que buscan innovar en la gestión de mesas y mejorar la experiencia del cliente mediante tecnología avanzada.	Restaurantes que buscan mejorar su gestión de reservas y la experiencia del cliente. Su mercado abarca desde pequeños establecimientos hasta grandes cadenas hoteleras y restaurantes de alta gama.	Gama de restaurantes, desde pequeños locales hasta grandes cadenas internacionales, que buscan gestionar reservas en línea y mejorar la experiencia del cliente.	Restaurantes, cafeterías y otros establecimientos de servicio rápido que buscan gestionar eficientemente las listas de espera y mejorar la experiencia del cliente.
	Estrategias de Marketing	Demostraciones en vivo. Alianzas estratégicas. Publicidad en redes sociales. Marketing de contenidos.	Optimización de motores de búsqueda. Publicidad de pago por clic. Alianzas estratégicas.	Publicidad y promoción. Análisis de datos. Segmentación de clientes.	Pruebas gratuitas y demostraciones. Testimonios y reseñas.
Perfil de producto	Productos y Servicios	Solución basada en IoT que utiliza sensores de peso para detectar la ocupación de las mesas en tiempo	Proporciona un sistema de reservas en línea, un CRM para gestionar relaciones con los	Ofrece a los restaurantes herramientas para gestionar la disponibilidad de mesas, realizar	Ofrece una solución digital para gestionar las listas de espera en tiempo real. Los

real. Esta información se refleja automáticamente en una app web y móvil, permitiendo a los comensales ver la disponibilidad de mesas antes de llegar. Además, incluye una interfaz administrativa para la gestión del flujo de comensales y optimización de la rotación de mesas.	clientes, herramientas de marketing automatizado, análisis de datos y reportes detallados, así como la integración con otros sistemas de restaurante como POS (punto de venta) y plataformas de pago.	reservas, y personalizar la experiencia del cliente. También proporciona servicios de marketing y publicidad, herramientas de CRM y análisis detallados del rendimiento del restaurante, como la ocupación y los tiempos de espera.	restaurantes pueden asignar mesas y notificar a los clientes mediante SMS o la app cuando su mesa esté lista. Además, la plataforma ofrece análisis sobre los tiempos de espera y la rotación de mesas para mejorar la eficiencia del establecimiento.
--	---	---	--

Precios y Costos	Ofrece un modelo de suscripción mensual, con precios que varían según el número de mesas y funcionalidades requeridas. A diferencia de sus competidores, no cobra comisiones por cada reserva, lo que lo convierte en una opción más rentable a largo plazo. Puede haber un costo inicial por la instalación de los sensores.	Sigue un modelo de suscripción mensual, con precios que empiezan desde \$99 por mes dependiendo del tamaño del restaurante y las funcionalidades necesarias. Además, en algunos casos, cobran comisiones por cada reserva realizada a través de la plataforma.	Opera con un modelo basado en suscripción mensual, cuyo precio inicial ronda los \$39 por mes. Además, cobra una comisión por cada reserva realizada a través de la plataforma, que varía entre el 1% y el 5%. También puede haber costos adicionales por servicios de promoción o características premium.	Opera con un modelo de suscripción mensual, comenzando desde \$29 al mes para restaurantes pequeños. No cobra comisiones por reserva, lo que lo hace una opción más accesible en términos de costos continuos. Los precios varían según el tamaño del restaurante y las funcionalidades adicionales requeridas.
-------------------------	---	--	---	---

Canales de distribución	- Página web - Aplicaciones móviles en	- Página web - Aplicaciones móviles en	- Página web - Aplicaciones móviles en	- Página web - Aplicaciones móviles en
--------------------------------	---	---	---	---

	dispositivos iOS y Android.	dispositivos iOS y Android.	dispositivos iOS y Android.	dispositivos iOS y Android.	
Análisis FODA	Fortalezas	- Detección automática de ocupación en tiempo real.	- Plataforma integral con CRM y analítica.	- Marca reconocida globalmente.	- Fácil de usar y configurar.
		- App ligera, actualizaciones rápidas (cada 10 segundos).	- Buena integración con otros sistemas.	- Gran base de usuarios y restaurantes.	- Económico y accesible.
		- Instalación simple sin técnicos.	- Funciona en móviles y tablets.	- Interfaz amigable para el cliente.	- Sin comisiones por cliente.
	Debilidades	- Requiere hardware (sensores).	- No detecta ocupación en tiempo real.	- Comisiones por reserva.	- Dependencia del ingreso manual del personal.
		- Etapa temprana de adopción y posicionamiento.	- Costos altos para pequeños restaurantes.	- Menor personalización para negocios pequeños.	- Sin monitoreo físico de mesas.
	Oportunidades	- Escalamiento a grandes cadenas de cafeterías.	- Expansión en mercados emergentes.	- Ampliar servicios para cafeterías o locales pequeños.	- Penetrar en mercados de comida rápida y cafeterías.
		- Expansión hacia restaurantes u hoteles.	- Alianzas con grandes cadenas.	- Integración con experiencias gastronómicas.	- Mejora con inteligencia artificial.
	Amenazas	- Copia del modelo por grandes competidores.	- Competencia con soluciones más automatizadas.	- Nuevas plataformas sin comisiones.	- Soluciones con detección automática.
- Desconfianza inicial por ser una tecnología emergente.		- Cambios en las tecnologías de reservas.	- Desintermediación (restaurantes que prefieren sistemas propios).	- Apps integrales que lo superen en funcionalidades.	

2.1.2. Estrategias y tácticas frente a competidores

Hemos empleado un análisis FODA para identificar las posibilidades y desafíos en el mercado, así como para evaluar nuestras fortalezas y debilidades internas. Esta metodología nos ha permitido concebir estrategias y tácticas que se ajusten de manera coherente a nuestro entorno y a los recursos disponibles.

Estrategia de diferenciación tecnológica:

La principal estrategia de Tavolo es posicionarse como una solución única en el mercado al incorporar tecnología IoT para detectar en tiempo real la ocupación de mesas, eliminando por completo la necesidad de intervención humana o registro manual. Esto permite ofrecer al cliente final información confiable y constante, algo que sus competidores no alcanzan con precisión.

Estrategia de enfoque de nicho en cafeterías:

Otra estrategia clave es enfocarse en un nicho específico y muy definido: las cafeterías. A diferencia de otras plataformas que buscan abarcar todo tipo de restaurantes, Tavolo se especializa en un entorno donde la rotación rápida, la autogestión y la simplicidad son fundamentales. En línea con esto, la táctica es desarrollar un lenguaje de marca, interfaz y funcionalidades adaptadas a este tipo de negocio, además de lanzar campañas dirigidas a franquicias de cafeterías o locales independientes que no requieren una infraestructura compleja, pero sí soluciones eficientes.

Estrategia de escalabilidad y fácil instalación:

Pensando en el crecimiento, Tavolo adopta una estrategia de escalabilidad técnica, diseñando un sistema que puede instalarse sin necesidad de personal técnico, en menos de una hora, lo que facilita su expansión sin altos costos logísticos. Como táctica, se crean tutoriales paso a paso, videos cortos y documentación visual clara que permiten a cualquier miembro del personal ejecutar la instalación.

Estrategia de posicionamiento de marca y educación del mercado:

Tavolo impulsa una estrategia de posicionamiento de marca basada en innovación y liderazgo de pensamiento en el sector horeca. La táctica principal para esto es la generación de contenido educativo (blogs, infografías, webinars) que informan sobre la importancia del monitoreo en tiempo real, las pérdidas por esperas largas y cómo la tecnología puede resolverlo, posicionando a Tavolo como referente en soluciones inteligentes para la industria.

2.2. Entrevistas

2.2.1. Diseño de entrevistas

Propietarios de cafeterías

Preguntas de presentación:

1. ¿Podrías indicarnos tu nombre y el de tu cafetería?
2. ¿Qué puesto ocupas dentro de la empresa?
3. ¿Desde cuándo está funcionando tu cafetería?

Propósito: Queremos entender tus necesidades y retos para mejorar la gestión del aforo en tu cafetería.

Entrevista:

1. ¿Cómo gestionas actualmente las reservas y la disponibilidad de mesas?
2. ¿Qué problemas enfrentas con el tiempo de espera de los clientes?
3. ¿Estarías interesado en una solución tecnológica para gestionar las mesas en tiempo real?
4. ¿Qué características consideras más importantes en una plataforma de gestión de aforo?
5. ¿Qué tan importante es para ti la facilidad de instalación y el soporte técnico?
6. ¿Cómo crees que la tecnología puede mejorar la experiencia del cliente y optimizar la operación de tu cafetería?
7. ¿Algo más que te gustaría agregar sobre tus necesidades o expectativas?

Clientes de establecimiento

Preguntas de presentación:

1. ¿Podrías indicarnos tu nombre?
2. ¿Con qué frecuencia sueles visitar cafeterías?
3. ¿Qué tipo de cafeterías prefieres (por ejemplo, locales, cadenas, cafeterías gourmet)?

Propósito: Queremos conocer tu experiencia como cliente en cafeterías y cómo la tecnología podría mejorarla.

Entrevista:

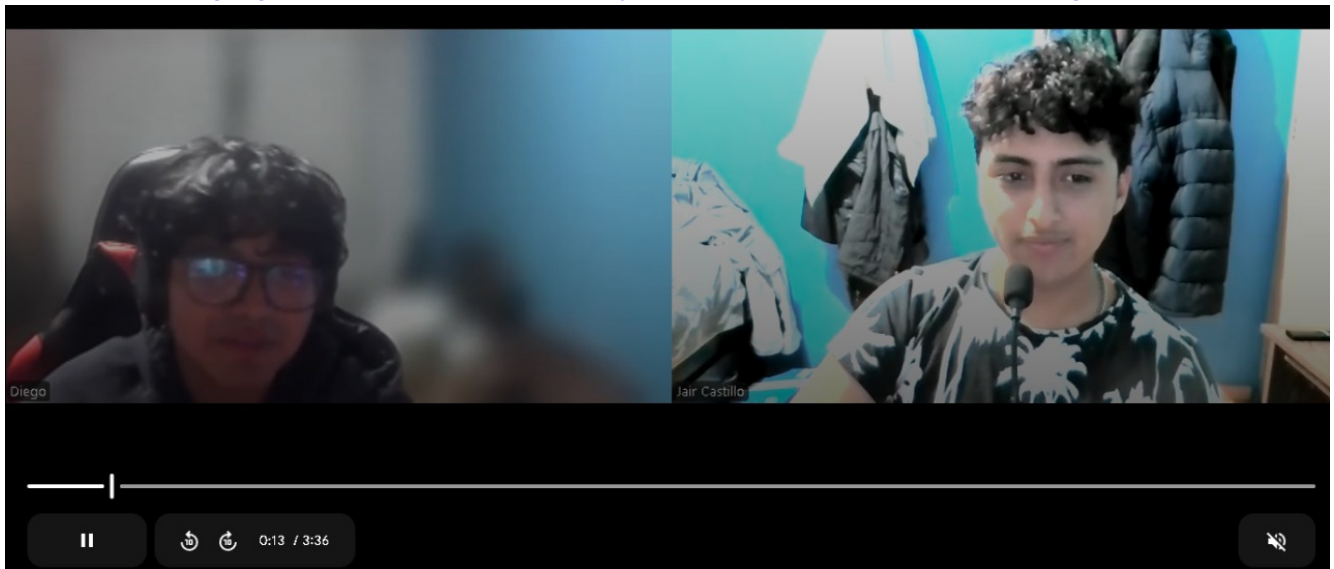
1. ¿Cómo sueles consultar la disponibilidad de mesas antes de ir a una cafetería?
2. ¿Qué tan importante es para ti poder hacer reservas online para evitar esperas?
3. ¿Te gustaría ver la disponibilidad de mesas en tiempo real a través de una app?
4. ¿Qué características debería tener una app de cafetería para que la uses con frecuencia?
5. ¿Cómo valoras la rapidez y facilidad de uso de una aplicación para gestionar tu experiencia en la cafetería?
6. ¿Algo más que consideras importante para mejorar tu experiencia como cliente?

2.2.2. Registro de entrevistas

Propietarios de cafeterías

Entrevista #1

Link: <https://drive.google.com/file/d/1CVDQx8Ue15xk7yt8oukkN1S87bAc7TeB/view?usp=sharing>



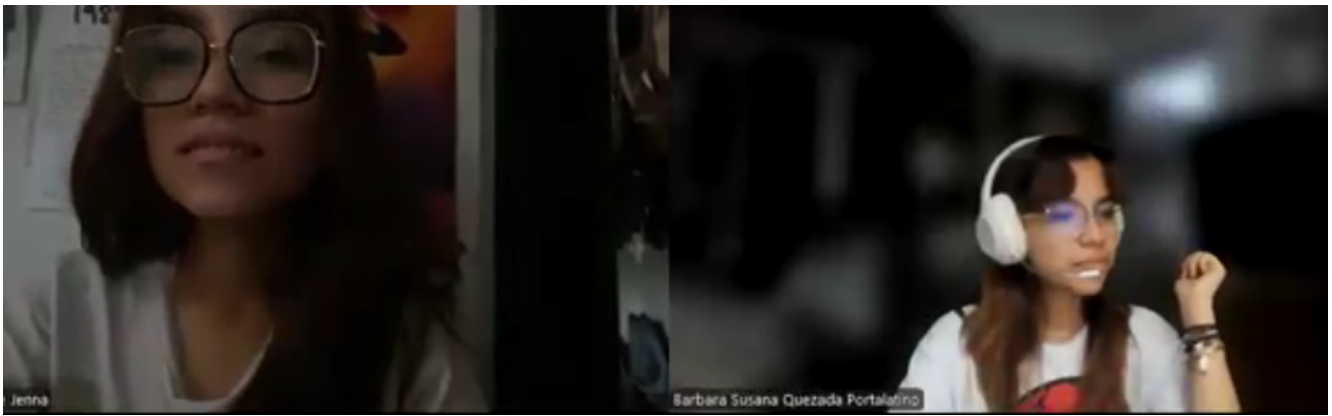
Entrevistado: Diego Morales

Duración: 5:20

Resumen: La persona entrevistada señala que es un cliente frecuente de diversas cafeterías, desde locales con propuestas culinarias elaboradas hasta las más conocidas franquicias. Si las circunstancias lo permiten, suele consultar si hay mesas libres antes de dirigirse al lugar, aunque reconoce que no siempre sigue esta práctica. Manifiesta un deseo por contar con herramientas que le permitan conocer la ocupación y otros datos de interés de antemano.

Entrevista #2

Link: https://drive.google.com/file/d/1WTxiDxw4vaabZhFvibL8mHLINcrJR0_9/view?usp=sharing



Entrevistado: Jenna Vasquez

Duración: 4:21

Resumen: En la entrevista, Jenna, dueña de una cafetería, destacó la necesidad crucial de implementar tecnología de fácil acceso y uso, específicamente sensores que no requieran una capacitación extensa del personal, lo que representaría un ahorro significativo de tiempo y recursos. Adicionalmente, expresó su frustración con el sistema actual de gestión manual de reservas, el cual es propenso a errores frecuentes que generan inconvenientes tanto para los clientes como para la operación del negocio, describiéndolo como un verdadero "dolor de cabeza". Jenna ve en una aplicación como TAVOLO una solución potencial que podría automatizar el proceso de reservas, reducir los errores humanos y aliviar la carga operativa de su personal. Reintentar Claude puede cometer errores. Verifique las respuestas.

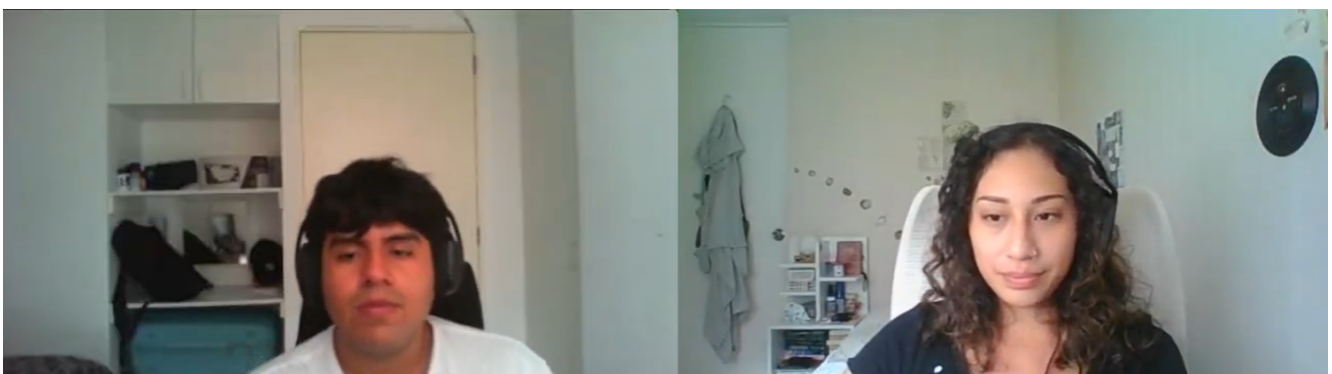
Clientes de establecimiento

Entrevista #1

Link: <https://drive.google.com/file/d/1CVDQx8Ue15xk7yt8oukkN1S87bAc7TeB/view?usp=sharing>

Entrevistado: Elias Yauri Paucar

Duración: 3:18



Resumen: El entrevistado indica que acostumbra visitar diferentes tipos de cafeterías, incluyendo tanto establecimientos gourmet como cadenas comerciales. Cuando tiene la opción, verifica la disponibilidad de espacio antes de ir, aunque no siempre lo hace. Expresa interés en poder acceder a información sobre disponibilidad y otros detalles similares.

Respecto a aplicaciones, valora principalmente tres aspectos: una interfaz intuitiva, facilidad de uso y funcionalidades relevantes. Finalmente, para el entrevistado es fundamental que la navegación dentro de la aplicación sea sencilla de comprender y que ofrezca una experiencia fluida.

Entrevista #2

Link: <https://youtu.be/QGSisj7iF-k>

Entrevistada: Aleshka Fernández

Duración: 3:54



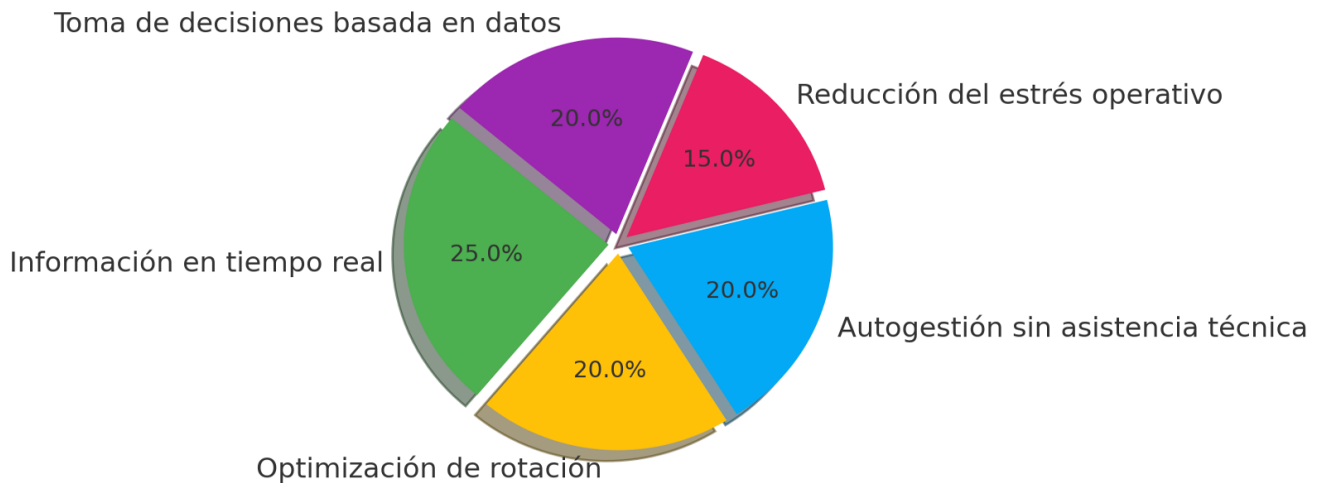
Resumen: La entrevistada menciona que suele visitar cafeterías locales que ofrezcan cafés artesanales o exclusivos del establecimiento. Destaca que, cuando tiene poco tiempo, valora poder reservar sus lugares favoritos. También muestra interés en conocer el aforo en tiempo real. Le gustaría que la aplicación fuera intuitiva, atractiva y visualmente llamativa. Además, considera ideal que la app le sugiera las mejores cafeterías cercanas, según su ubicación y el tipo de cafetería que prefiere. También desea encontrar fácilmente un buscador dentro de la aplicación, dejar calificaciones sobre los lugares visitados y recibir recomendaciones de cafeterías similares en caso de que el tiempo de espera en su lugar habitual sea muy largo.

2.2.3. Análisis de entrevistas

Propietarios de cafeterías

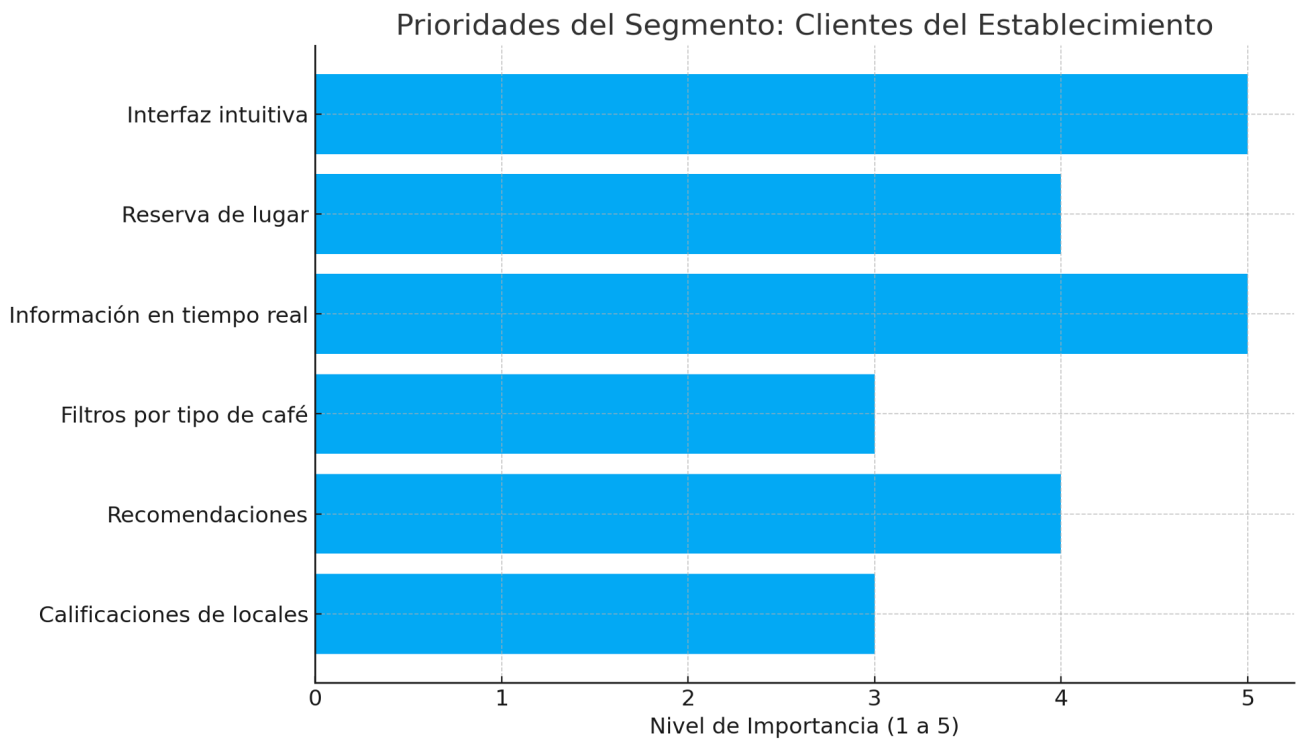
Los propietarios entrevistados son también usuarios frecuentes de cafeterías, lo que les permite empatizar con las necesidades del cliente. Aunque no siempre consultan la disponibilidad de mesas antes de asistir a un local, valoran contar con herramientas tecnológicas que les brinden información en tiempo real sobre la ocupación y otros datos operativos. Identifican como problemáticas la falta de visibilidad para anticiparse a la demanda y la toma de decisiones basada en intuiciones. Por ello, ven como una gran oportunidad integrar soluciones que mejoren la rotación de mesas, optimicen la operación y reduzcan el estrés, siempre y cuando sean fáciles de usar y autogestionables.

Prioridades del Segmento: Propietarios de Cafeterías



Clientes de establecimiento

Los clientes entrevistados visitan diversos tipos de cafeterías y, cuando tienen la oportunidad, valoran consultar la disponibilidad de mesas antes de llegar. Expresan interés en aplicaciones que ofrezcan una experiencia fluida, intuitiva y visualmente atractiva, que les permita ahorrar tiempo, reservar su lugar favorito o encontrar recomendaciones personalizadas según sus preferencias y ubicación. Las principales frustraciones se centran en no saber si encontrarán espacio y en apps poco claras o con funciones irrelevantes. Ven como ganancia una app que no solo informe disponibilidad en tiempo real, sino que también les permita planificar mejor y descubrir nuevas cafeterías fácilmente.



2.3. Needfinding

2.3.1. User Personas

Propietarios de cafeterías

NAME

Daniel Villanueva

TYPE

Artisan



Goals

Reducir los tiempos de espera y evitar que los clientes se vayan si no encuentran mesa
Optimizar la rotación y uso de mesas sin comprometer la experiencia
Aumentar la fidelización de los clientes a través de una experiencia más fluida
Contar con información para tomar decisiones operativas

Quote

“
Quiero que cada persona que entre por la puerta tenga una buena experiencia desde el primer minuto, incluso antes de llegar
”

Demographic

♂ Male 42 years

📍 Mexico

Married

Gerente de cafetería

Background

Daniel fundó su cafetería hace 5 años con un enfoque en café de origen y atención personalizada. Supervisa las operaciones diarias y también gestiona redes sociales y proveedores. No es experto en tecnología, pero le gusta implementarla si mejora la experiencia del cliente o la eficiencia del negocio

Motivations

Ser reconocido por ofrecer un servicio eficiente y de calidad
Modernizar su negocio sin complicaciones
Mantener un equilibrio entre atención personalizada y operaciones optimizadas
Diferenciarse de la competencia con innovación útil

Frustrations

Clientes molestos por no encontrar mesa o tener que esperar demasiado
Perder ventas por falta de información en tiempo real
Soluciones tecnológicas complicadas de instalar o difíciles de usar

Skills

Gestión operativa



Marketing digital



Trabajo en equipo



Technology



UXPRESSIA

This persona was built in uxpressia.com

Clientes de establecimiento

PERSONA: Catalina Vásquez

NAME

Catalina Vásquez

TYPE

Idealist



Goals

Encontrar una mesa disponible sin tener que esperar o "hacer scouting" en varios locales
Optimizar su tiempo libre o de trabajo con ambientes cómodos y sin contratiempos
Vivir una experiencia agradable desde que decide salir hasta que está en la mesa

Quote

“
Si tengo que esperar más de 5 minutos, busco otro lugar, así de simple
”

Background

Suele visitar cafeterías 3-4 veces por semana, tanto para trabajar como para reunirse con amigos o simplemente relajarse. Valora un buen café, el ambiente y la comodidad del lugar. Además, se informa antes de ir a algún lugar (reseñas, disponibilidad, ambiente) y toma decisiones rápidas desde el celular.

Demographic

Female 27 years

Lima

Single

Diseñadora gráfica

Motivations

Aprovechar bien su tiempo libre o sus pausas laborales
Ir a lugares donde se sienta bienvenido y pueda instalarse sin estrés
Compartir buenas experiencias en redes sociales o con amigos

Frustrations

Llegar a un local y encontrarlo lleno
Perder tiempo caminando entre cafeterías buscando lugar
Interfaz lenta o confusa en las apps o webs de los locales

Skills

Gestión del tiempo



Uso de apps



Creatividad



Technology



UXPRESSIA

This persona was built in uxpressia.com

2.3.2. User Task Matrix

Propietarios de cafeterías

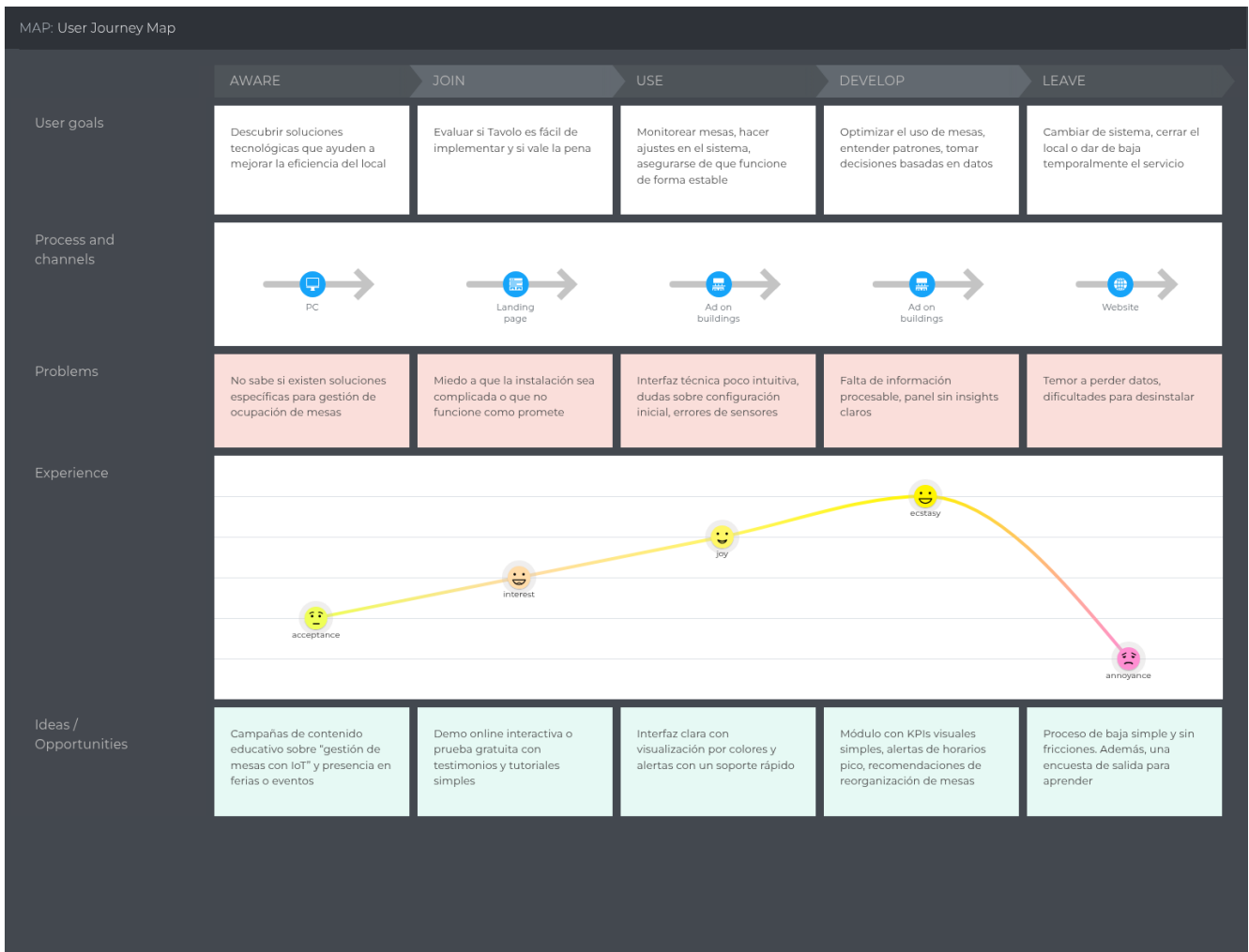
Tarea	Importancia	Frecuencia
Consultar el estado de ocupación de mesas	Alta	Alta
Agregar/modificar mesas desde el panel administrativo	Media	Media
Gestionar reservas desde el panel	Media	Media
Configurar parámetros iniciales (sede, nombre, etc.)	Media	Baja
Supervisar que el sistema esté funcionando correctamente	Alta	Alta
Ver reportes o estadísticas de uso (futura función)	Alta	Media

Clientes de establecimiento

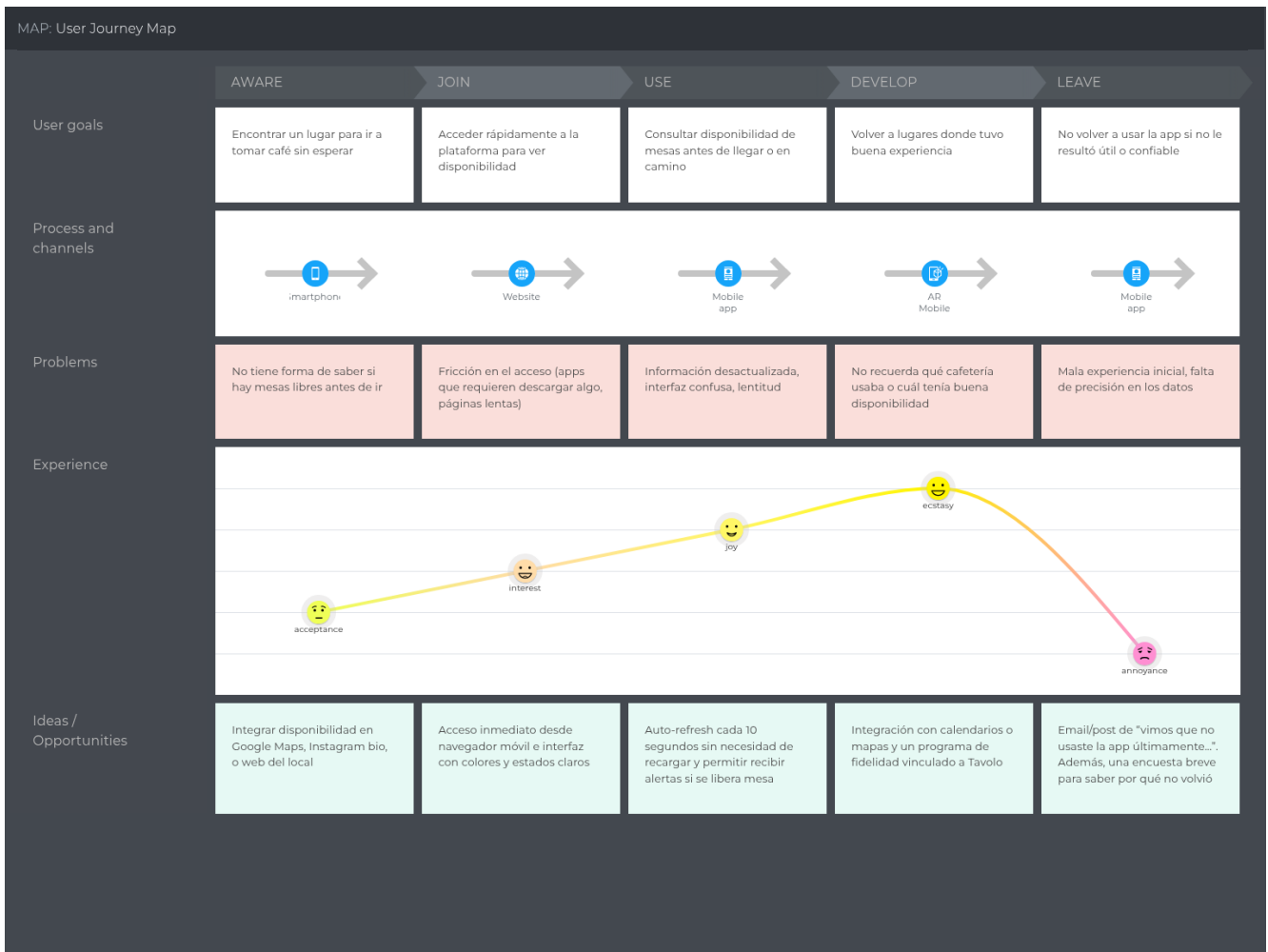
Tarea	Importancia	Frecuencia
Consultar disponibilidad de mesas en tiempo real	Alta	Alta
Acceder rápidamente desde el celular	Alta	Alta
Buscar cafeterías cercanas con disponibilidad	Alta	Media
Guardar su cafetería favorita (opcional futuro)	Media	Baja
Compartir o recomendar la cafetería con amigos	Media	Media

2.3.3. User Journey Mapping

Propietarios de cafeterías



Cientes de establecimiento



2.3.4. Empathy Mapping

Propietarios de cafeterías

¿Con quién estamos empatizando?

Con emprendedores/as o administradores/as de cafeterías, generalmente independientes o con una pequeña cadena, que buscan optimizar la operación de su local sin depender de soluciones costosas o difíciles de implementar

¿Cómo se siente y qué piensa?

“Quiero que mi cafetería sea eficiente, rentable y con buena reputación. Me preocupo por ofrecer una buena experiencia al cliente sin saturar a mi equipo. Además, tengo miedo de invertir en tecnología que no entienda o que me complique más que ayudar

¿Qué necesita hacer?

Monitorear ocupación de mesas sin estar físicamente presente
Implementar soluciones que no le saquen tiempo ni lo obliguen a depender de soporte técnico
Tomar decisiones rápidas basadas en datos claros

¿Qué está escuchando?

Opiniones de otros dueños de cafeterías sobre herramientas útiles
Feedback de sus empleados sobre los problemas en la atención en horas pico
Sugerencias de clientes sobre largas esperas o mala organización



¿Qué está viendo?

Competencia que se moderniza
Tendencias digitales en la industria gastronómica
Apps que ayudan a organizar pedidos o reservas

¿Qué está haciendo?

Supervisando al personal, ajustando la operación manualmente
Haciendo malabares entre proveedores, turnos y clientes
Buscando herramientas útiles pero que no requieran gran capacitación

¿Qué está diciendo?

“No puedo permitirme perder clientes por esperas innecesarias, necesito soluciones que me den control sin complicarme la vida

PAINS

No tener datos para mejorar
Falta de tiempo para probar nuevas herramientas
Miedo a depender de soporte técnico o instalaciones complicadas

GAINS

Clientes más felices y menos presión para el equipo
Control y visibilidad en tiempo real
Instalación rápida y gestión simple

UXPRESSIA

This persona was built in uxpessia.com

Clientes de establecimiento

¿Con quién estamos empatizando?

Con jóvenes profesionales, estudiantes o freelancers que visitan cafeterías para trabajar, reunirse o relajarse. Buscan comodidad, disponibilidad y eficiencia sin perder tiempo

¿Qué está escuchando?

Recomendaciones de amigos o influencers sobre cafeterías
Reviews online que mencionan esperas largas o buen ambiente

¿Cómo se siente y qué piensa?

“
No quiero esperar ni perder tiempo buscando lugar; de hecho, quiero sentirme cómodo, relajado, con espacio para estar tranquilo/a y aprecio la transparencia: Quiero saber cómo está el panorama antes de ir
”



¿Qué necesita hacer?

Ver desde su celular si hay mesas disponibles
Decidir rápido a qué cafetería ir
Evitar filas, tiempos de espera o moverse innecesariamente entre locales

¿Qué está viendo?

Apps que permiten reservar en restaurantes, pero no aplican a cafeterías
Cafeterías modernas que integran tecnología en su experiencia
Lugares visualmente atractivos, con buena iluminación y diseño

¿Qué está diciendo?

“
No me quiero mover hasta estar segura de que hay mesa, ojalá pudiera ver en el celular si está lleno
”

¿Qué está haciendo?

Buscando cafeterías en Google Maps o Instagram
Revisando fotos, horarios y reseñas
Evitando moverse si no tiene garantías de encontrar lugar

PAINS

Frustración por llegar y no encontrar mesa
Pérdida de tiempo caminando entre locales
Información poco clara sobre disponibilidad

GAINS

Mejor uso del tiempo libre o de trabajo
Experiencia moderna, rápida y sin fricción
Aumento en la fidelización; vuelve a donde sabe que la valoran

UXPRESSIA

This persona was built in uxpressia.com

2.3.5. As-is Scenario Mapping

Propietarios de cafeterías

FASES	Apertura del local	Horas pico	Tiempos muertos o baja rotación	Cierre y evaluación
DOING	Llego temprano a preparar el salón y reviso la limpieza	Superviso el movimiento del salón y doy indicaciones rápidas	Trato de motivar al personal a optimizar ocupación	Reviso la caja, analizo el desempeño del día
	Organizo al personal según la estimación de flujo del día	Coordino con el personal para liberar mesas	Me enfoco en redes, promociones o planificación de eventos	Comento con el equipo los problemas que se presentaron
	Activo los pedidos de proveedores y abro caja	Intento gestionar manualmente el flujo de comensales		

THINKING	"¿Cómo puedo organizar mejor al equipo si se llena?"	"¿Hay alguien esperando afuera? ¿Ya se fue esa mesa del fondo?"	"¿Se estará yendo gente por no ver mesas disponibles?"	"Hoy podríamos haber ganado más si organizábamos mejor"
		"Estamos perdiendo ventas por no rotar bien las mesas"	"¿Debemos reducir el horario de atención?"	"¿Y si hubiera una forma de saber cuántas mesas se usan realmente?"

FEELING	Expectativa positiva	Estrés por la carga simultánea de tareas	Impotencia por no tener visibilidad clara del flujo	Frustración al no tener datos para tomar decisiones
	Ligera ansiedad por la incertidumbre del día	Frustración al ver ineficiencia en la atención	Curiosidad por probar nuevas soluciones, pero con dudas	Motivación parcial por mejorar, pero sin claridad sobre el cómo

Clientes de establecimiento

FASES	Decisión de salir	Camino al local	Llegada y evaluación del lugar	Cambio de plan o espera
DOING	Consulto Google Maps o redes sociales para elegir una cafetería	Voy caminando o en transporte mientras reviso reseñas	Me asomo o entro para ver si hay mesas	Decido esperar de pie, buscar otro local o cancelar mi plan
	Pregunto a amigos si conocen un lugar con espacio	Consulto stories o publicaciones recientes del local	Hablo con un mozo o espero que me asignen espacio	Dejo una reseña o me quejo en redes si tuve mala experiencia
	Busco una opción con buen ambiente y sin demasiada gente		Evalúo el ambiente	

FASES	Decisión de salir	Camino al local	Llegada y evaluación del lugar	Cambio de plan o espera
THINKING	"¿Estará lleno ese lugar que me gusta?"	"No tengo ganas de andar buscando otro lugar"	"No hay lugar, otra vez"	"No vuelvo más a este lugar sin saber si hay lugar antes"
	"Sería genial saber desde casa si hay mesas"	"Si llego y no hay lugar, capaz me vuelvo a casa"	"¿Por qué no hay una app para esto?"	
FEELING	Entusiasmo por salir a tomar algo o trabajar fuera	Expectativa mezclada con incertidumbre	Decepción si no encuentra lugar	Frustración, molestia, desilusión
	Inseguridad por no saber si encontrará lugar	Poca paciencia, sobre todo si va con tiempo limitado	Irritación si ve mesas vacías sin uso	Sensación de que no valoraron su tiempo

2.4. Ubiquitous Language

Estos son los términos y conceptos comunes utilizados en nuestro proyecto:

- **Sede:** Unidad donde opera una cafetería. Cada sede puede tener múltiples mesas y su propio panel administrativo.
- **Mesa:** Elemento disponible para ser ocupado por los clientes. Puede ser activada o desactivada desde el panel.
- **Silla con sensor:** Elemento equipado con un sensor de peso que detecta si está siendo utilizada. Su estado ayuda a determinar la ocupación de la mesa.
- **Sensor:** Dispositivo IoT instalado en cada silla que detecta cambios de peso para determinar si hay una persona sentada.
- **Ocupación:** Estado de una mesa, determinado por la detección de presencia en una o más sillas asociadas.
- **Panel administrativo:** Interfaz web usada por el propietario o administrador para gestionar mesas, consultar disponibilidad y visualizar estadísticas.
- **Administrador:** Usuario con control sobre una sede, encargado de gestionar mesas, configuraciones y visualización de datos.
- **Cliente:** Persona que visita la cafetería. Puede usar la app para ver disponibilidad de mesas en tiempo real.
- **Actualización de estado:** Proceso automático mediante el cual el sistema recibe datos de los sensores en las sillas y actualiza la información visible para clientes y administradores.
- **Rotación de mesas:** Número de veces que una mesa es ocupada en un período determinado. Métrica usada para medir eficiencia.

Capítulo III: Requirements Specification

3.1. To-Be Scenario Mapping

Este mapeo "To-Be" presenta el escenario ideal posterior a la implementación de Tavolo, describiendo cómo cambiarán las experiencias del propietario de la cafetería y del comensal gracias a la solución IoT propuesta. Se detallan acciones, pensamientos y emociones alineadas a los beneficios que aporta el sistema.

¿Qué hace el Dueño del Café?

Phases	Doing	Thinking	Feeling
--------	-------	----------	---------

Phases	Doing	Thinking	Feeling
Registro del local	• Instala la app Tavolo. • Crea su cuenta como dueño de local. • Registra su cafetería y sedes.	• "Qué bueno que puedo registrar todas mis sedes fácil." • "Ahora tengo visibilidad online."	• Aliviado • Organizado
Gestión del aforo	• Consulta la ocupación en tiempo real de su sede. • Revisa cada sede desde su celular o computadora.	• "Qué fácil es ver cuántas mesas están ocupadas." • "Ya no dependo del personal para esto." • "Qué accesible que pueda ver desde cualquier dispositivo la app de Tavolo."	• En control • Eficiente
Gestión del tiempo de espera	• Ve cómo los clientes reservan online. • Evita colas en la entrada.	• "Qué bueno que ya no hay filas innecesarias." • "Mis clientes llegan más contentos."	• Relajado • Optimista
Monitoreo de mesas	• Agrega o elimina mesas. • Coordina instalación de sensores IoT.	• "Wow, es sencillo gestionar las mesas con ayuda." • "No imaginé tanta flexibilidad."	• Empoderado • Innovador
Visibilidad en Google Maps	• Permite que los clientes lo busquen por nombre. • Revisa su sede en la app de Tavolo.	• "Qué útil que aparezcamos en Google con mis datos." • "Nos encuentran más rápido."	• Conectado • Satisfecho

¿Qué hace el Usuario?

Phases	Descubrimiento de la aplicación	Registro de datos personales	Búsqueda de cafeterías	Reserva de asientos
Doing	• Descarga Tavolo desde la tienda de aplicaciones. • Explora las funcionalidades principales.	• Registra sus datos en Tavolo. • Configura sus cafeterías favoritas.	• Utiliza la función de búsqueda por ubicación. • Revisa la disponibilidad en tiempo real.	• Selecciona una mesa disponible. • Confirma la reserva para una hora específica.
Thinking	"Esta app me ayudará a no perder tiempo buscando lugares disponibles."	"El proceso de registro es rápido."	"Me encanta poder ver exactamente dónde hay espacio antes de ir."	"La reserva fue sencilla y ahora tengo mi lugar asegurado."
Feeling	Curiosidad	Comodidad	Alivio	Satisfacción

3.2. User Stories

Epics

Epic ID	Name
EPIC-LANDING-001	Funcionalidades landing page
EPIC-CLIENT-002	Funcionalidades user comensal

Epic ID	Name
EPIC-ADMIN-003	Funcionalidades user administrador
EPIC-SUPERADMIN-004	Funcionalidades user superadmin
EPIC-BOOKING-TECH	Funcionalidades de desarrolladores

Story ID	Título	Descripción	Epic ID	Usuario	Criterios de aceptación
US001	Ver sección "¿Qué es Tavolo?"	Como visitante de la página, quiero entender qué es Tavolo desde la landing page para conocer su propósito y cómo puede ayudarme como comensal o administrador de cafetería.	EPIC-LANDING-001	Visitante landing page	Escenario 1: Dado que el visitante accede a la landing page, Cuando hace scroll en "¿Qué es Tavolo?", Entonces el sistema le muestra la sección donde se describe claramente el objetivo de la solución. Escenario 2: Dado que el visitante accede a la landing page, Cuando hace click en "¿Qué es Tavolo?", Entonces el sistema le muestra la sección donde se describe claramente el objetivo de la solución.

Story ID	Título	Descripción	Epic ID	Usuario	Criterios de aceptación
US002	Ver sección "Características clave"	Como visitante, quiero revisar una lista de características principales de Tavolo para evaluar si cubre mis necesidades como usuario final o administrador.	EPIC-LANDING-001	Visitante landing page	Escenario 1: Dado que el usuario navega la landing, Cuando cuando hace scroll a la sección "Características clave", Entonces el sistema le muestra un resumen de las funcionalidades más importantes del sistema. Escenario 2: Dado que el visitante está en la landing, Cuando da click sobre el botón "Características clave", Entonces el sistema le muestra un resumen de las funcionalidades más importantes del sistema.

Story ID	Título	Descripción	Epic ID	Usuario	Criterios de aceptación
US003	Ver la galería virtual	Como visitante, quiero ver una galería virtual en la landing para visualizar cómo luce Tavolo, su interfaz y los sensores en acción.	EPIC-LANDING-001	Visitante landing page	Escenario 1: Dado que el usuario navega la landing, Cuando cuando hace scroll a la sección "Galería Virtual" Entonces visualiza el contenido visual ilustrativo del sistema en uso Escenario 2: Dado que el usuario navega la landing, Cuando cuando hace click a la sección "Galería Virtual" Entonces visualiza el contenido visual ilustrativo del sistema en uso

Story ID	Título	Descripción	Epic ID	Usuario	Criterios de aceptación
US004	Usar el formulario de contacto	Como visitante interesado, quiero tener acceso a un formulario o medio de contacto en la landing para comunicarme con el equipo de Tavolo.	EPIC-LANDING-001	Visitante landing page	Escenario 1: Dado que el visitante accede a "Contáctanos" Cuando completa el formulario con sus datos Entonces el sistema debe notificar al equipo de Tavolo Escenario 2: Dado que el visitante intenta enviar el formulario, Cuando deja campos obligatorios vacíos, Entonces el sistema debe mostrar mensajes de validación para completar la información.

Story ID	Título	Descripción	Epic ID	Usuario	Criterios de aceptación
US005	Ver menú digital de una sede	Como comensal, quiero visualizar el menú de la cafetería para conocer los productos disponibles antes de asistir.	EPIC-CLIENT-002	Comensal	Escenario 1: Dado que el comensal accede a la plataforma, Cuando le da click a la sección "Menú", Entonces el sistema debe mostrar el menú completo de la cafetería. Escenario 2: Dado que el comensal visualiza el menú, Cuando selecciona una categoría como "Bebidas" o "Postres", Entonces el sistema filtra los productos mostrados según esa categoría.

Story ID	Título	Descripción	Epic ID	Usuario	Criterios de aceptación
US006	Visualizar reserva activa	Como comensal, quiero poder visualizar mi reserva activa desde la aplicación, para confirmar los detalles de mi mesa y el horario reservado antes de llegar a la cafetería.	EPIC-CLIENT-002	Comensal	Escenario 1: Dado que el comensal ha iniciado sesión en la plataforma Y tiene al menos una reserva activa, Cuando accede a la sección principal, Entonces el sistema le muestra la información de la reserva actual incluyendo horario, sede y número de mesa. Escenario 2: Dado que el comensal visualiza su reserva, Cuando esta ya ha expirado, Entonces el sistema la remueve de la sección activa.

Story ID	Título	Descripción	Epic ID	Usuario	Criterios de aceptación
US007	Ver disponibilidad de mesas en tiempo real	Como comensal, quiero conocer la disponibilidad de mesas para decidir que sede visitar de la cafetería.	EPIC-CLIENT-002	Comensal	Escenario 1: Dado que el comensal accede a la plataforma Cuando le da click a la sección "Explorar Sedes" Entonces el sistema debe mostrarle la información de la sedes con sus respectivos aforos disponibles
					Escenario 2: Dado que el comensal visualiza el estado de aforo de las sedes Cuando hay una actualización desde los sensores Entonces el sistema debe reflejar el nuevo estado en menos de 5 segundos
					Escenario 3: Dado que todas las mesas están ocupadas en una sede Cuando el comensal visualiza la disponibilidad Entonces el sistema debe informar que no hay mesas disponibles actualmente

Story ID	Título	Descripción	Epic ID	Usuario	Criterios de aceptación
US008	Reservar una mesa	Como comensal, quiero reservar una mesa para asegurarme un lugar al llegar a la cafetería.	EPIC-CLIENT-002	Comensal	Escenario 1: Dado que el comensal selecciona una sede en la sección "Explorar Sedes" Cuando el comensal selecciona la hora aproximada de estadía Y selecciona una mesa disponible Entonces el sistema debe permitir realizar la reserva
					Escenario 2: Dado que el comensal ya tiene una reserva activa Cuando intenta crear una nueva Entonces el sistema debe evitarlo e informar que ya tiene una reserva activa

Story ID	Título	Descripción	Epic ID	Usuario	Criterios de aceptación
US009	Visualizar más sedes en un mapa	Como comensal, quiero poder ver más sedes en el mapa para poder identificar el más cercano.	EPIC-CLIENT-002	Comensal	Escenario 1: Dado que el comensal quiere ver más sedes Cuando le da click a la sección "Sedes" Entonces el sistema debe mostrar todas las sedes de la cafetería
					Escenario 2: Dado que el comensal se encuentra en la sección "Sedes" Y puede ver todas las sedes de la cafetería Cuando le da click al botón de "Ver en mapa" Entonces el sistema muestra un pop up con la dirección en un mapa interactivo

Story ID	Título	Descripción	Epic ID	Usuario	Criterios de aceptación
US010	Registro de cuenta de comensal	Como visitante, quiero poder registrarme como comensal en el sistema Tavolo para poder acceder a las funcionalidades de reserva y visualización de mesas.	EPIC-CLIENT-002	Visitante	Escenario 1: Dado que un visitante accede a la plataforma Cuando completa el formulario de registro con nombre y contraseña Y acepta los términos y condiciones Entonces el sistema debe crear una cuenta nueva Y permitir el acceso a las funcionalidades de comensal Escenario 2: Dado que un visitante intenta registrarse con un nombre de usuario ya usado, Cuando envía el formulario, Entonces el sistema debe impedir el registro y mostrar un mensaje de error indicando que el correo ya está registrado.

Story ID	Título	Descripción	Epic ID	Usuario	Criterios de aceptación
US011	Inicio de sesión de cuenta de comensal	Como comensal registrado, quiero poder iniciar sesión en el sistema para acceder a mis reservas y funcionalidades personalizadas.	EPIC-CLIENT-002	Comensal	Escenario 1: Dado que un comensal registrado accede a la pantalla de login Cuando ingresa sus credenciales correctas (email y contraseña) Entonces el sistema debe autenticarlo Y redirigirlo al dashboard principal
					Escenario 2: Dado que un usuario ingresa credenciales incorrectas Cuando intenta iniciar sesión Entonces el sistema debe mostrar un mensaje de error Y permitir reintentar

Story ID	Título	Descripción	Epic ID	Usuario	Criterios de aceptación
US012	Agregar nuevas mesas	Como administrador de cafetería, quiero agregar mesas al sistema para gestionar mejor la ocupación.	EPIC-ADMIN-003	Administrador	Escenario 1: Dado que el admin accede al panel de su sede Cuando le da click a la sección "Gestión de Mesas" Y posteriormente a la opción "Agregar una nueva mesa" Entonces el sistema le brinda un formulario simple de registro de mesa. Escenario 2: Dado que el admin ingresa los datos correctos de la mesa como su identificador y capacidad de sillas Y le da click al botón "Guardar" Entonces el sistema guarda con su identificador y capacidad de sillas la mesa registrada

Story ID	Título	Descripción	Epic ID	Usuario	Criterios de aceptación
US013	Visualizar reservas realizadas	Como administrador, quiero ver las reservas registradas en mi sede para planificar el servicio.	EPIC-ADMIN-003	Administrador	Escenario 1: Dado que el administrador accede a su sede Cuando le da clic a la sección "Visualizar Reservas" Entonces el sistema debe mostrar las reservas con usuario, hora y mesa Escenario 2: Dado que el administrador ha filtrado por persona o mesa Cuando aplica el filtro Entonces el sistema debe mostrar solo las reservas que coinciden con los criterios seleccionados

Story ID	Título	Descripción	Epic ID	Usuario	Criterios de aceptación
US014	Ver la sede asignada	Como administrador, quiero confirmar qué sede tengo asignada para asegurarme de gestionar la correcta.	EPIC-ADMIN-003	Administrador	Escenario 1: Dado que el administrador inicia sesión Cuando accede a su información Entonces el sistema debe mostrar la sede asociada a su cuenta Escenario 2: Dado que el administrador tiene asignada una sede Cuando intenta acceder a otra sede que no le corresponde Entonces el sistema le impide el acceso

Story ID	Título	Descripción	Epic ID	Usuario	Criterios de aceptación
US015	Eliminar mesas	Como administrador, quiero poder eliminar mesas que ya no están en uso en mi sede, para mantener el sistema actualizado y evitar asignaciones incorrectas.	EPIC-ADMIN-003	Administrador	Escenario 1: Dado que el administrador ha iniciado sesión Y se encuentra en la sección "Ver Mesas" Cuando selecciona una mesa existente Y confirma su eliminación Entonces la mesa es eliminada del sistema Y deja de aparecer en la lista Escenario 2: Dado que la mesa tiene una reserva activa Cuando el administrador intenta eliminarla Entonces el sistema debe impedir la eliminación Y mostrar un mensaje indicando que no puede eliminar mesas con reservas activas

Story ID	Título	Descripción	Epic ID	Usuario	Criterios de aceptación
US016	Inicio de sesión de administrador	Como administrador, quiero poder iniciar sesión en el sistema con mis credenciales específicas para acceder al panel de administración de mi sede.	EPIC-ADMIN-003	Administrador	Escenario 1: Dado que un administrador accede a la pantalla de login Cuando ingresa sus credenciales válidas Entonces el sistema debe autenticarlo Y mostrar el panel de administración con las funcionalidades asociadas a su sede Escenario 2: Dado que un administrador ingresa credenciales incorrectas Cuando intenta acceder Entonces el sistema debe mostrar un mensaje de error Y registrar el intento fallido por seguridad

Story ID	Título	Descripción	Epic ID	Usuario	Criterios de aceptación
US017	Resgistro de cuenta de administrador	Como superadministrador, quiero poder registrar administradores como administradores de una sede en el sistema Tavolo para poder acceder a las funcionalidades de ver reserva y la gestión de mesas.	EPIC-SUPERADMIN-004	Superadmin	Escenario 1: Dado que un superadmin se encuentra en su cuenta Cuando se dirija a la seccion de "Administradores" Y dan click a agregar administrador Entonces pueden registrar un administrador y asignarle una sede Escenario 2: Dado que un superadmin ha registrado a un administrador Cuando se dirige a la sección de "Administradores" Entonces puede visualizar los administradores que han registrado

Story ID	Título	Descripción	Epic ID	Usuario	Criterios de aceptación
US018	Acceder a todos los módulos del sistema	Como superadmin, quiero tener acceso a todas las funcionalidades del sistema para realizar pruebas, crear sedes y admins.	EPIC-SUPERADMIN-004	Superadmin	Escenario 1: Dado que el superadmin accede al sistema Cuando desea visualizar sedes o usuarios Entonces el sistema debe permitirle ver toda la información
					Escenario 2: Dado que el superadmin crea un nuevo admin o sede Cuando registra los datos Entonces el sistema debe almacenarlos y mostrar confirmación

Technical Stories

Story ID	Título	Descripción	Epic ID	Usuario	Criterios de aceptación
----------	--------	-------------	---------	---------	-------------------------

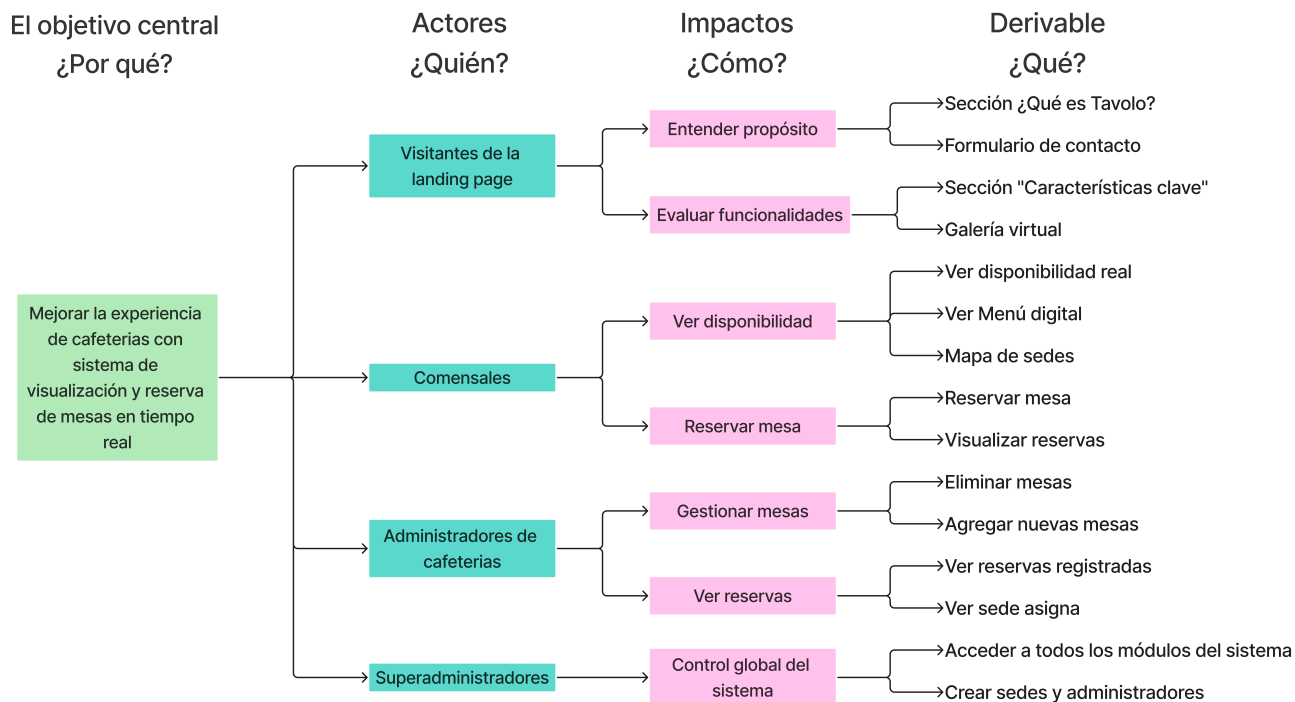
Story ID	Título	Descripción	Epic ID	Usuario	Criterios de aceptación
TAVOLO-TS001	Implementar Endpoint para Obtener Mesas	Como Developer, quiero implementar un endpoint de API que permita a las aplicaciones frontend obtener la lista de mesas disponibles en una sede específica, para que los usuarios puedan ver el estado de ocupación.	EPIC-BOOKING-TECH	Developer	Escenario 1: Obtener mesas disponibles exitosamente Dado que existe una sede con mesas registradas Cuando un cliente solicita la lista de mesas para esa sede Entonces el sistema responde con un código 200 OK y una lista de mesas con su estado de ocupación. Escenario 2: Sede no encontrada Dado que se solicita una sede que no existe Cuando el sistema intenta obtener las mesas Entonces el sistema responde con un código 404 Not Found.

Story ID	Título	Descripción	Epic ID	Usuario	Criterios de aceptación
TAVOLO-TS002	Procesar Datos de Sensor de Ocupación	Como Developer, quiero implementar la lógica de backend para recibir y procesar los datos de ocupación enviados por los sensores IoT, para que el sistema actualice el estado de las mesas en tiempo real.	EPIC-IOT-PROCESSING	Developer	Escenario 1: Datos de ocupación recibidos y procesados Dado que el Edge API recibe datos de ocupación de un sensor Cuando el backend procesa estos datos Entonces el estado de la mesa correspondiente se actualiza en la base de datos. Escenario 2: Datos inválidos recibidos Dado que el Edge API recibe datos con formato inválido Cuando el backend intenta procesarlos Entonces el sistema registra un error y descarta los datos inválidos.

Story ID	Título	Descripción	Epic ID	Usuario	Criterios de aceptación
TAVOLO-TS003	Integrar con Google Maps API (Backend)	Como Developer, quiero implementar la integración con la API de Google Maps en el backend para obtener información geográfica de las sedes, para que las aplicaciones frontend puedan mostrar las ubicaciones en un mapa.	EPIC-BRANCHING-TECH	Developer	Escenario 1: Obtener coordenadas de sede exitosamente Dado que se registra una nueva sede con una dirección válida Cuando el backend llama a la API de Google Maps con la dirección Entonces la API responde con las coordenadas geográficas correctas. Escenario 2: Dirección inválida Dado que se registra una sede con una dirección inválida Cuando el backend llama a la API de Google Maps Entonces el sistema responde con un error o datos de ubicación nulos.

Story ID	Título	Descripción	Epic ID	Usuario	Criterios de aceptación
TAVOLO-TS004	Implementar Lógica de Reserva de Mesa API	Como Developer, quiero implementar la lógica de backend para gestionar el proceso de reserva de mesas a través de la API, asegurando la disponibilidad y registrando la reserva correctamente.	EPIC-BOOKING-TECH	Developer	Escenario 1: Reserva de mesa exitosa Dado que un usuario solicita reservar una mesa disponible Cuando la lógica de reserva valida la disponibilidad y registra la reserva Entonces la reserva se crea en la base de datos y el estado de la mesa se actualiza a "Reservado". Escenario 2: Mesa no disponible Dado que un usuario intenta reservar una mesa ya ocupada o reservada Cuando la lógica de reserva valida la disponibilidad Entonces el sistema rechaza la reserva y responde con un mensaje de error de disponibilidad.

3.3. Impact mapping.



3.4. Product Backlog.

#	Story ID	Título	Descripción	Story Points
1	US008	Reservar una mesa	Como comensal, quiero reservar una mesa para asegurarme un lugar al llegar a la cafetería.	13
2	US018	Acceder a todos los módulos del sistema	Como superadmin, quiero tener acceso a todas las funcionalidades del sistema para realizar pruebas, crear sedes y admins.	13
3	US007	Ver disponibilidad de mesas en tiempo real	Como comensal, quiero conocer la disponibilidad de mesas para decidir que sede visitar de la cafetería.	8
4	US012	Agregar nuevas mesas	Como administrador de cafetería, quiero agregar mesas al sistema para gestionar mejor la ocupación.	5
5	US009	Visualizar más sedes en un mapa	Como comensal, quiero poder ver más sedes en el mapa para poder identificar el más cercano.	5
6	US006	Visualizar reserva activa	Como comensal, quiero poder visualizar mi reserva activa desde la aplicación, para confirmar los detalles de mi mesa y el horario reservado antes de llegar a la cafetería.	5
7	US013	Visualizar reservas realizadas	Como administrador, quiero ver las reservas registradas en mi sede para planificar el servicio.	5

#	Story ID	Título	Descripción	Story Points
8	US017	Resgistro de cuenta de administrador	Como superadministrador, quiero poder registrar administradores como administradores de una sede en el sistema Tavolo para poder acceder a las funcionalidades de ver reserva y la gestión de mesas.	5
9	US005	Ver menú digital de una sede	Como comensal, quiero visualizar el menú de la cafetería para conocer los productos disponibles antes de asistir.	3
10	US004	Usar el formulario de contacto	Como visitante interesado, quiero tener acceso a un formulario o medio de contacto en la landing para comunicarme con el equipo de Tavolo.	3
11	US014	Ver la sede asignada	Como administrador, quiero confirmar qué sede tengo asignada para asegurarme de gestionar la correcta.	3
12	US015	Eliminar mesas	Como administrador, quiero poder eliminar mesas que ya no están en uso en mi sede, para mantener el sistema actualizado y evitar asignaciones incorrectas.	3
13	US010	Registro de cuenta de comensal	Como visitante, quiero poder registrarme como comensal en el sistema Tavolo para poder acceder a las funcionalidades de reserva y visualización de mesas.	3
14	US011	Inicio de sesión de cuenta de comensal	Como comensal registrado, quiero poder iniciar sesión en el sistema para acceder a mis reservas y funcionalidades personalizadas.	3
15	US016	Inicio de sesión de administrador	Como administrador, quiero poder iniciar sesión en el sistema con mis credenciales específicas para acceder al panel de administración de mi sede.	3
16	US001	Ver sección "¿Qué es Tavolo?"	Como visitante de la página, quiero entender qué es Tavolo desde la landing page para conocer su propósito y cómo puede ayudarme como comensal o administrador de cafetería.	2
17	US002	Ver sección "Características clave"	Como visitante, quiero revisar una lista de características principales de Tavolo para evaluar si cubre mis necesidades como usuario final o administrador.	2
18	US003	Ver la galería virtual	Como visitante, quiero ver una galería virtual en la landing para visualizar cómo luce Tavolo, su interfaz y los sensores en acción.	2

Capítulo IV: Solution Software Design

4.1. Strategic-Level Domain-Driven Design.

4.1.1. EventStorming.

Con el objetivo de comprender en profundidad el dominio de Tavolo, se llevó a cabo una sesión de Event Storming de aproximadamente 1 hora. Esto permitió que nosotros como equipo organizáramos nuestras ideas y pensamientos sobre sistema desde múltiples perspectivas: negocio, usuario final, administración y experiencia. A través de esta

dinámica, identificamos eventos clave, comandos, usuarios y agregados que nos ayudaron a esbozar una primera visión integral del sistema.

Se vieron los siguientes puntos en la reunión:

- Exploración del dominio general

Se partió desde la experiencia del visitante en la landing page, avanzando por el flujo de registro e inicio de sesión como comensal o administrador, hasta la reserva de mesas, visualización en tiempo real y gestión desde el panel administrativo.

- Identificación de eventos y comandos clave

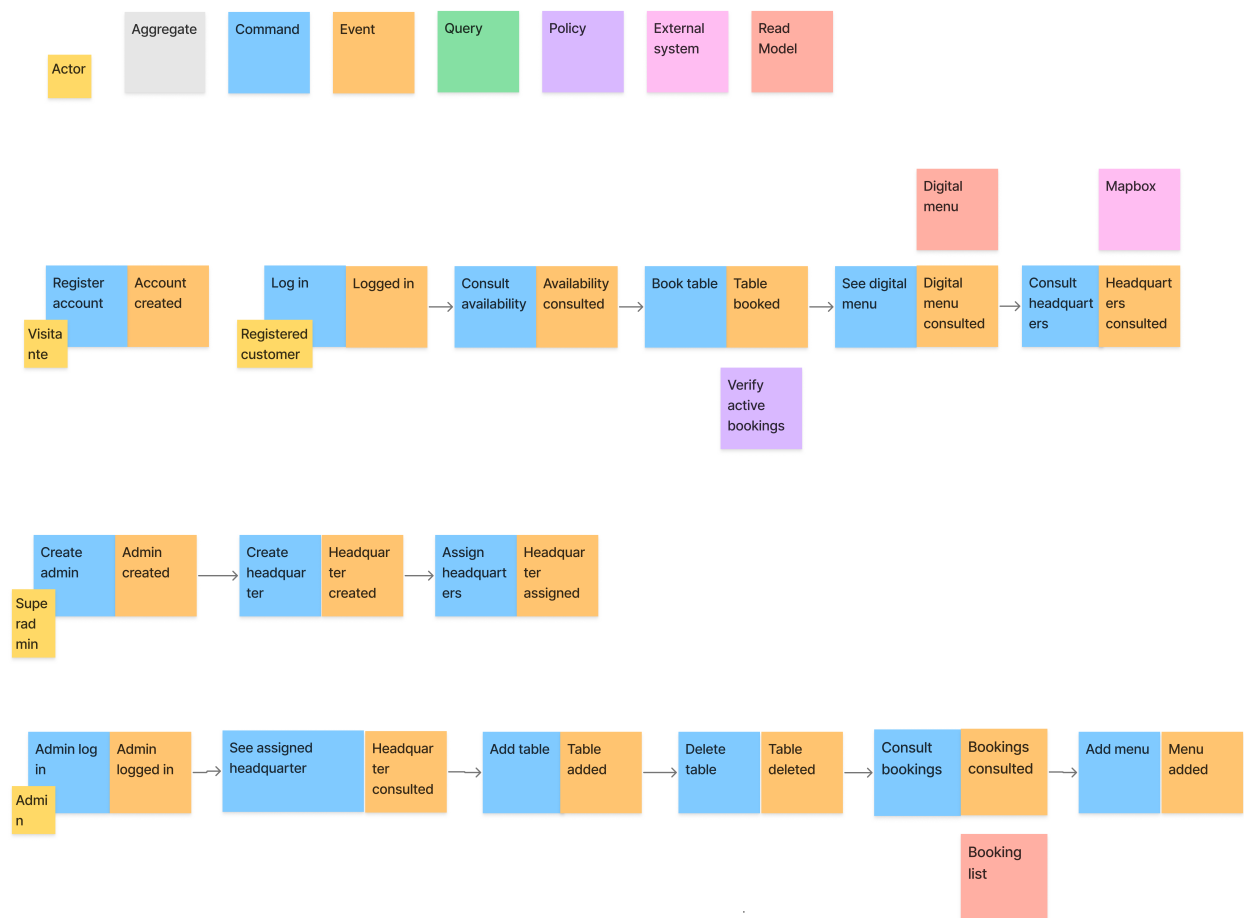
Se colocaron notas naranjas para eventos, y se complementaron con comandos en azul. EL equipo siguió de las User Stories previamente realizadas, lo que aseguró la coherencia y el flujo de ña solución.

- Asignación de roles y responsables

Se diferenciaron los actores como: comensal, administrador, superadmin, visitante, para asociar claramente qué parte del sistema controlan o en qué puntos interactúan. Al dividirlo de esta manera es más fácil detectar posibles conflictos o áreas de mejora en la experiencia.

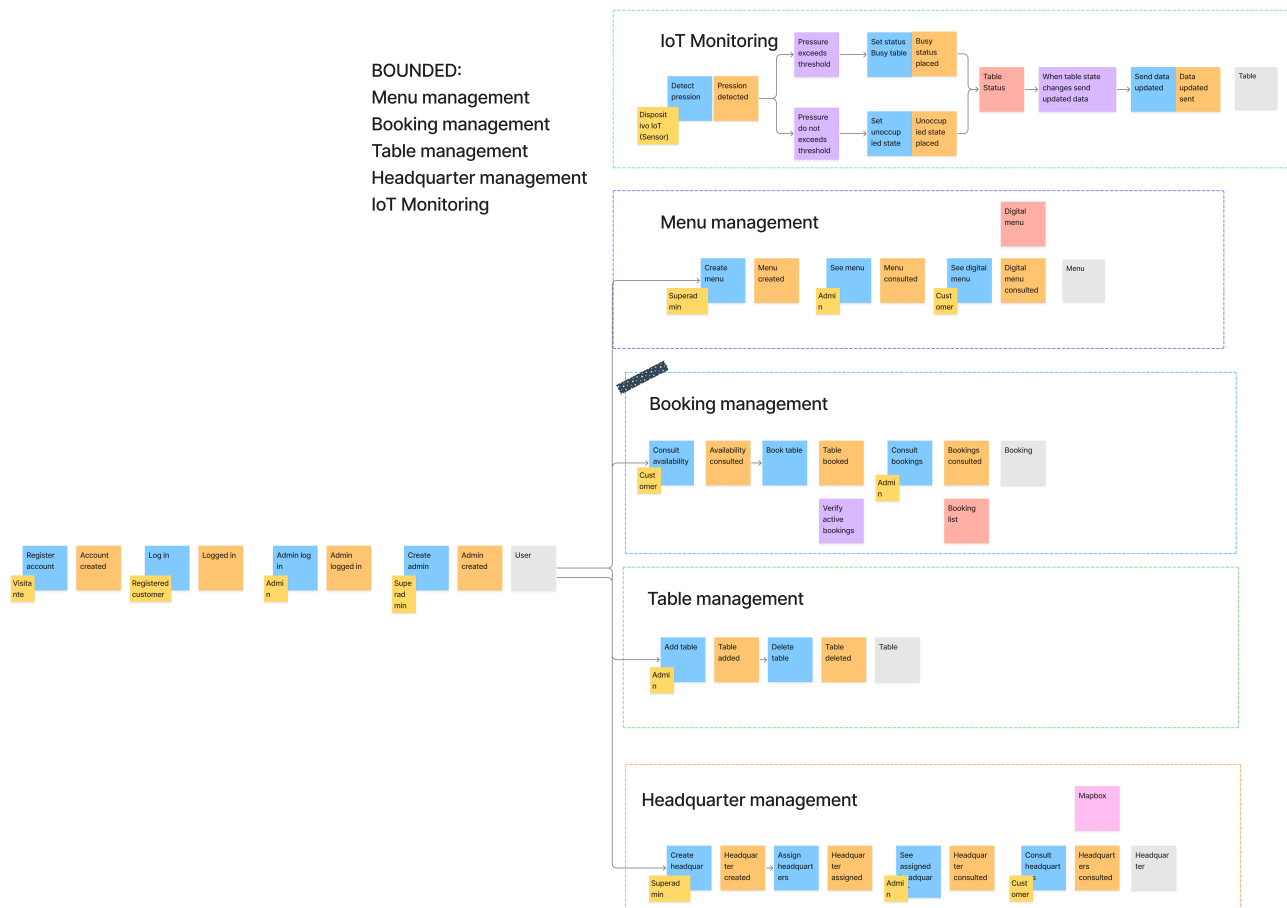
- Evidencia de lo relizado en la reunion

Legend



4.1.1.1 Candidate Context Discovery.

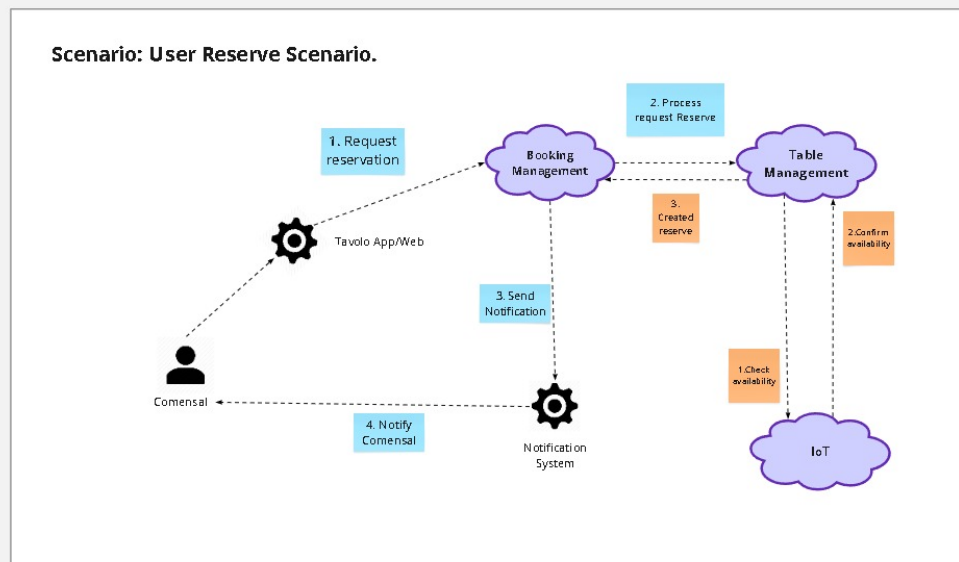
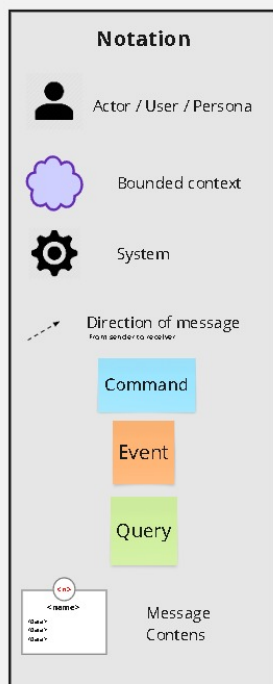
Identificar contextos candidatos es un paso clave para gestionar la complejidad en el desarrollo de sistemas. Se trata de un análisis minucioso que busca entender los elementos centrales del sistema y sus interconexiones. A partir de ahí, se procede a agrupar estos elementos en 'contextos delimitados' lógicos y coherentes. Esta separación no solo facilita el diseño y la implementación, sino que también tiene como meta principal potenciar la escalabilidad, el desempeño y la mantenibilidad del sistema resultante.



4.1.1.2 Domain Message Flows Modeling.

Para analizar y diseñar sistemas de software, se usa el Modelado de Flujos de Mensajes de Dominio, un método que ilustra la transferencia de información entre componentes mediante mensajes. Este proceso se centra en especificar los mensajes enviados y recibidos por los diferentes actores del sistema y en descifrar sus relaciones. El uso de esta metodología aporta claridad para entender y representar las vías de información del sistema, permitiendo detectar problemas potenciales más fácilmente y optimizar la estructura del diseño. A modo de ejemplo, mostraremos a continuación algunos diagramas aplicados a nuestro sistema.

Domain Message Flows Modeling



DESCRIPCIÓN DEL SCENARIO:

El **Comensal** usa la app/web de Tavolo para **solicitar una reserva** especificando sede y hora. El sistema **Booking Management** recibe la solicitud, **verifica la disponibilidad** de mesas (consultando a **Table Management / IoT**). Si hay mesas libres, **crea la reserva** (evento "Reserva Creada"), **notifica la confirmación** al Comensal a través del **Notification System**, y actualiza el estado de la mesa en **Table Management** para marcarla como ocupada en ese horario.

4.1.1.3 Bounded Context Canvases.

Booking Management

Bounded Context Canvases

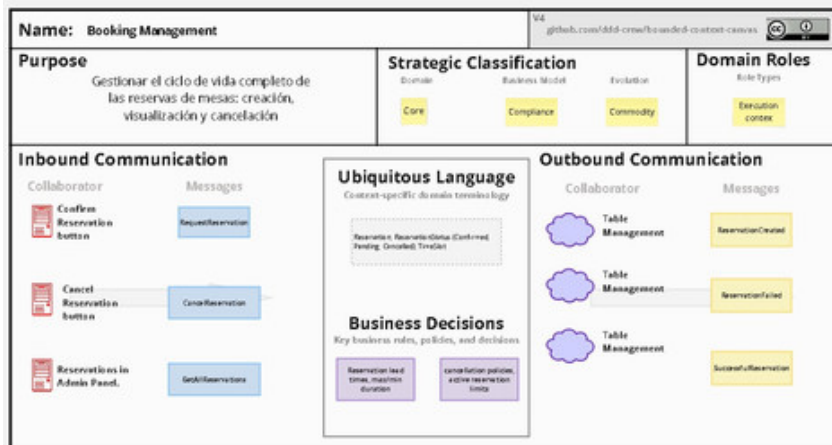
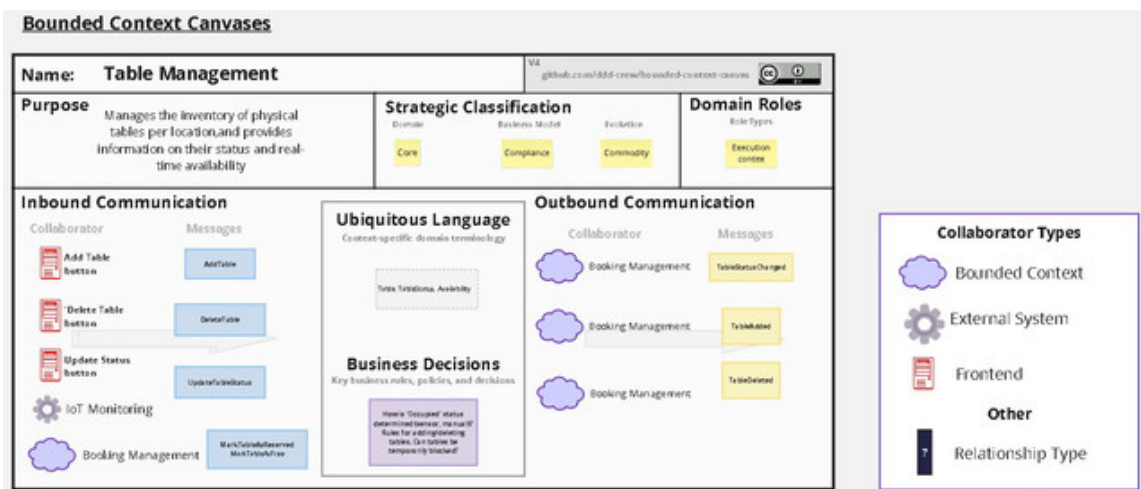
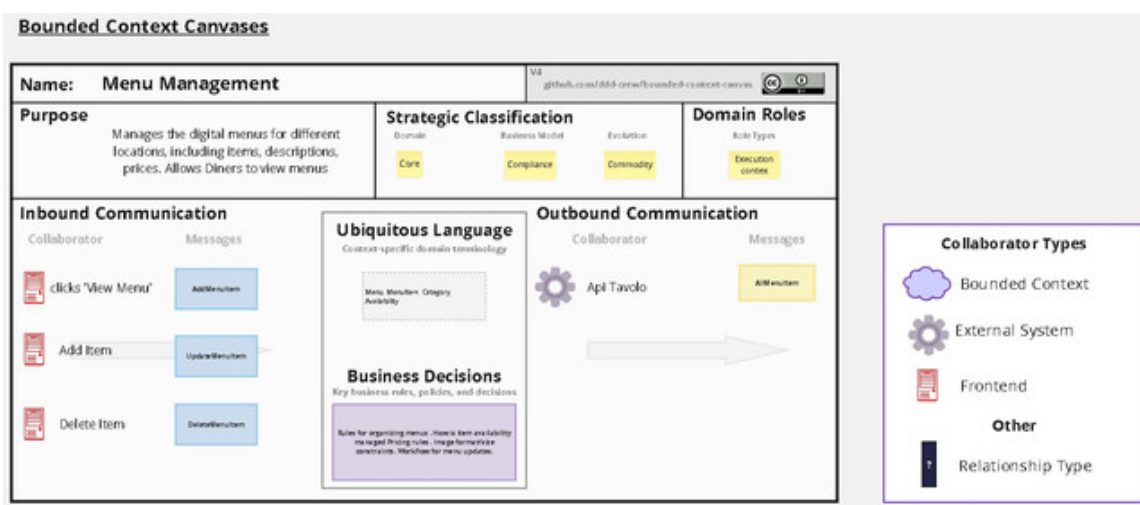


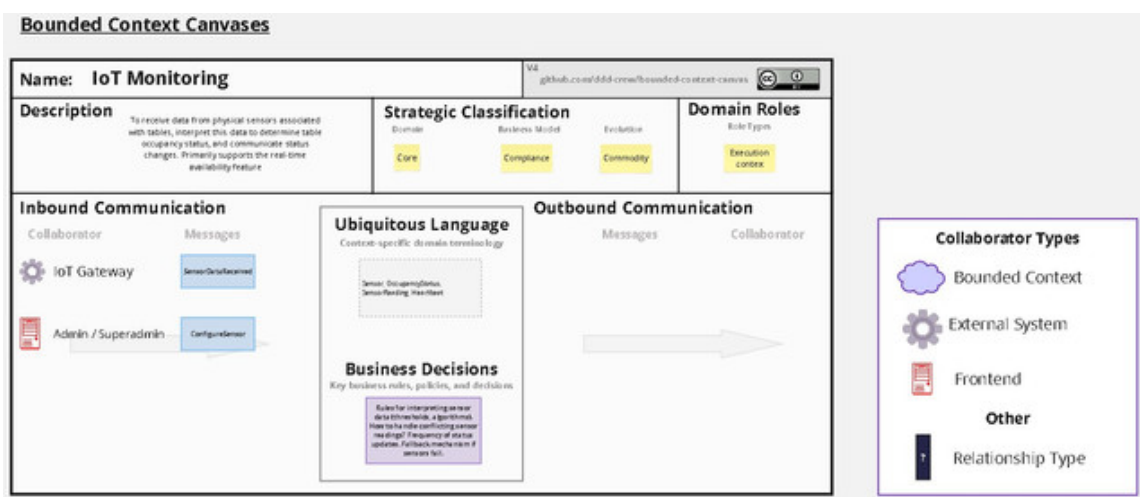
Table Management



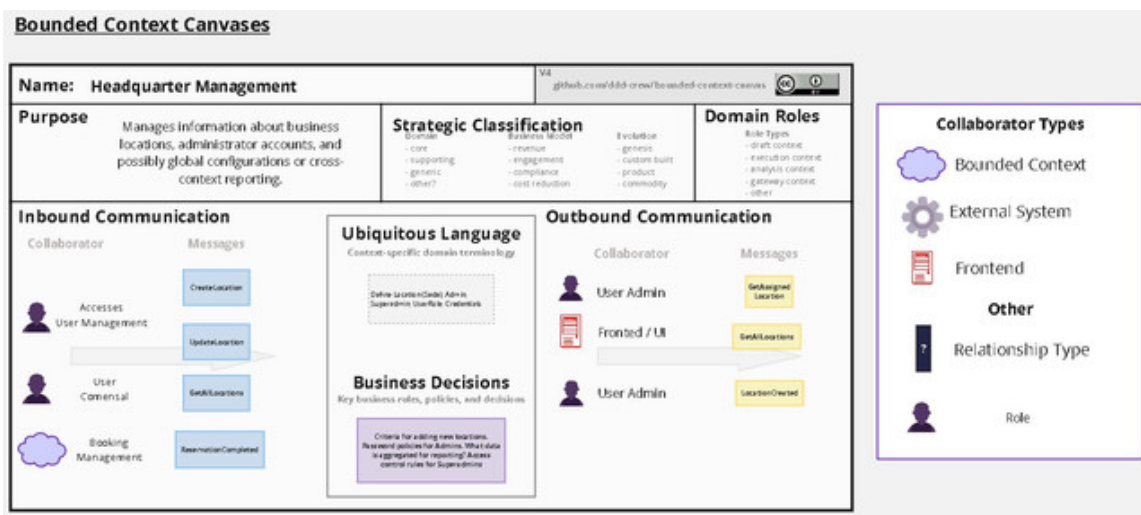
Menu Management



IoT Monitoring



Headquarter Management

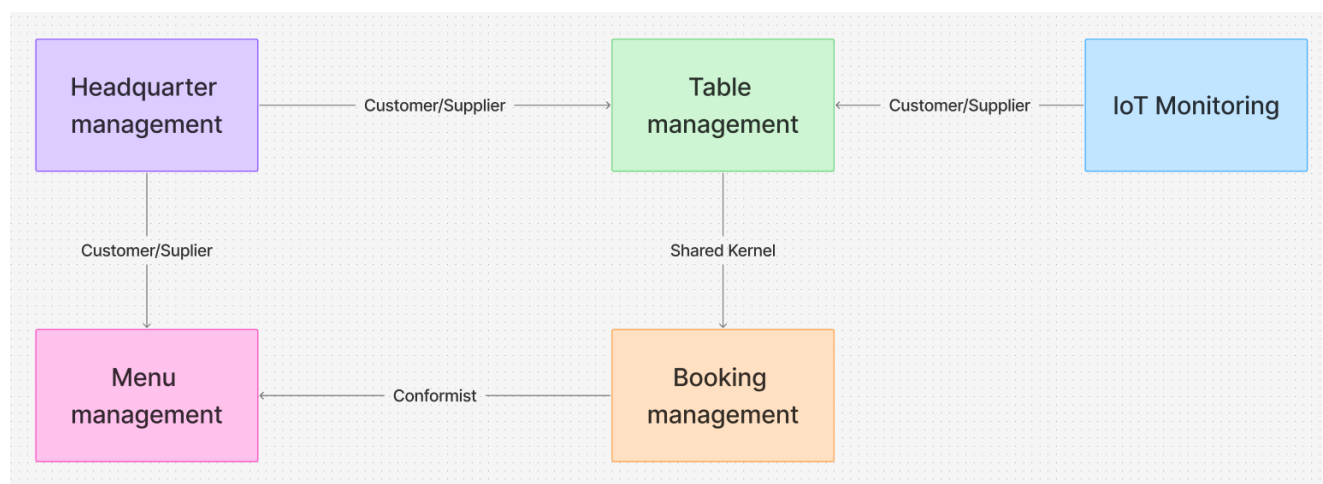


4.1.2. Context Mapping.

En esta sección desarrollamos un conjunto de *context maps* para visualizar las relaciones entre los *bounded contexts* del sistema. A partir de la información recolectada, exploramos distintas alternativas de diseño, cuestionando cómo cambiaría la estructura si reubicamos, dividimos o agrupamos *capabilities*. Finalmente, evaluamos cada propuesta considerando patrones como *Anti-corruption Layer*, *Conformist*, *Customer/Supplier* y *Shared Kernel*, con el fin de definir la mejor aproximación para la arquitectura del dominio. A continuación presentaremos las opciones que contemplamos para Tavolo y la estructura final.

Opción 1

En esta estructura mantenemos los cinco bounded contexts separados con relaciones claramente definidas. Las ventajas de este tipo de contexto son por un lado la clara separación de responsabilidades y por otro lado, se especifica que cada contexto se enfoca en una funcionalidad específica. Una de las principales desventajas es que hay una mayor complejidad en la sincronización entre contextos.



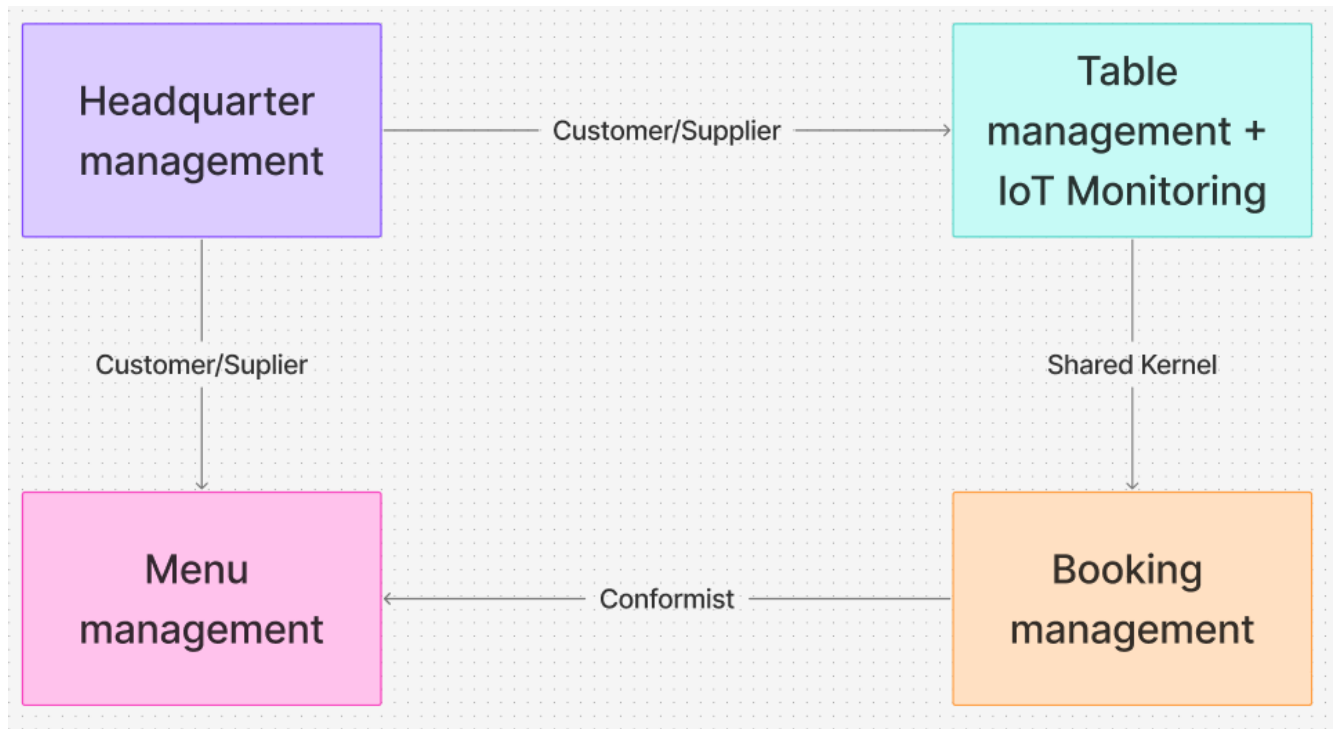
Opción 2

Esta alternativa propone unir los contextos de IoT Monitoring y Table Management en un solo bounded context. Al hacerlo, se elimina la necesidad de sincronización externa entre ambos, manteniendo relaciones similares con los demás contextos del sistema.

Esta combinación presenta ventajas como la simplificación de la arquitectura al disminuir la cantidad de bounded contexts, una comunicación más directa entre la detección de ocupación y la gestión de mesas.

No obstante, una desventajas es la combinación de responsabilidades distintas, ya que una parte se enfoca en la infraestructura de sensores y la otra en procesos administrativos. Esto podría dificultar que el personal de cafetería

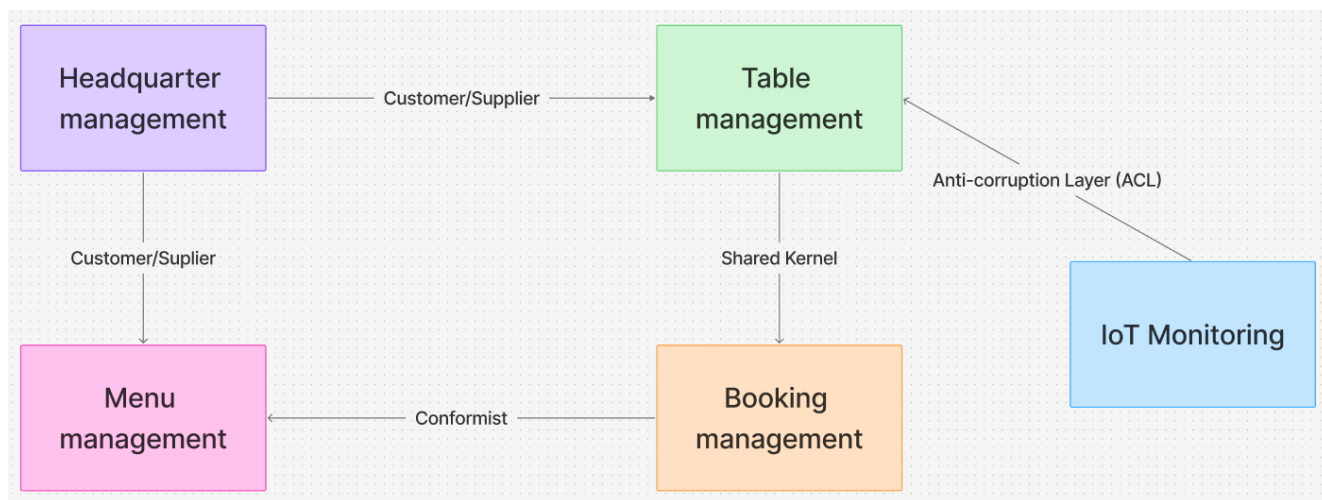
realice la instalación del sistema en menos de una hora sin ayuda técnica, y además genera el riesgo de que un solo contexto asuma demasiadas funciones.



Opcion 3

Esta alternativa propone una arquitectura compuesta por cinco bounded contexts bien definidos, con relaciones claras entre ellos. La estructura busca equilibrar la separación de responsabilidades, para permitir que el sistema escale y se mantenga con facilidad. Además, facilita la instalación sin asistencia técnica y asegura tiempos de respuesta adecuados, lo que contribuye directamente a mejorar la experiencia del cliente en el uso del sistema Tavolo.

- Headquarter Management se comunica con Menu Management y Table Management, proporcionando la información de las sedes. Menu Management adapta su contenido según cada sede, y Table Management utiliza la información organizativa que viene de Headquarter. En ambos casos, la relación es del tipo Customer/Supplier, donde Headquarter es el proveedor.
- Table Management y Booking Management comparten el modelo de "mesa" y su disponibilidad. Por eso, tienen una relación de tipo Shared Kernel, lo que asegura que ambos usen los mismos conceptos para evitar errores o confusión.
- Booking Management también se relaciona con Menu Management, pero en este caso la relación es Conformist. Booking utiliza información del menú, pero se adapta a su estructura sin modificarla.
- IoT Monitoring se conecta con Table Management mediante una Anti-corruption Layer. Esta capa traduce los datos que vienen de los sensores a un formato que Table Management pueda entender. Así, se protege el sistema de los detalles técnicos del IoT y se facilita la instalación del sistema sin ayuda especializada.



Elección

Elegimos la **opción 3**, ya que proporciona el mejor equilibrio entre la separación de responsabilidades, la fácil de implementación y el cumplimiento de los requisitos del sistema.

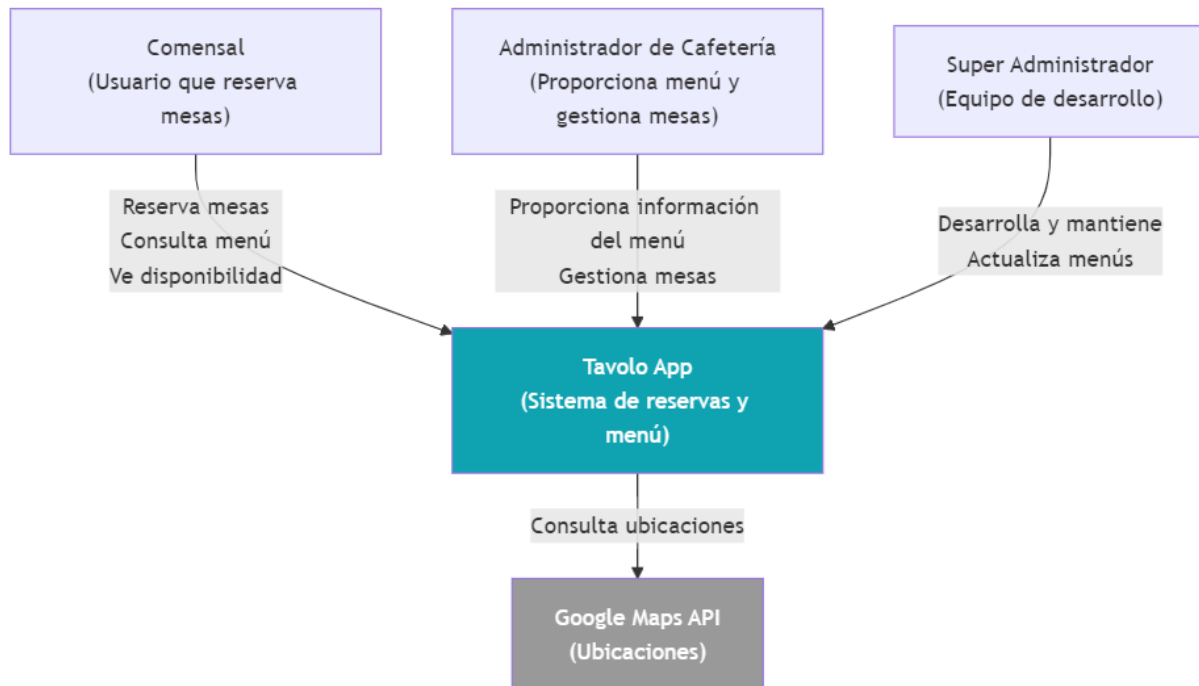
Al definir cinco bounded contexts con relaciones claras, se facilita la evolución independiente de cada parte del sistema, lo que mejora su escalabilidad y mantenibilidad. Además, al separar gestiones, como la de gestión de IoT mediante una Anti-corruption Layer, se simplifica la instalación. Asimismo, esta estructura garantiza tiempos de respuesta rápidos y optimiza la gestión de mesas, reservas y menús, brindando una experiencia más fluida y eficiente para los usuarios y el personal.

4.1.3. Software Architecture.

En esta sección, se presenta y explica la representación de la arquitectura de software para Tavolo utilizando el C4 Model. A través de estos diagramas, se busca proporcionar una comprensión clara y accesible de la arquitectura de Tavolo, permitiendo a los miembros del equipo, stakeholders y futuros desarrolladores entiendan cómo se organiza y comunica el sistema a diferentes niveles.

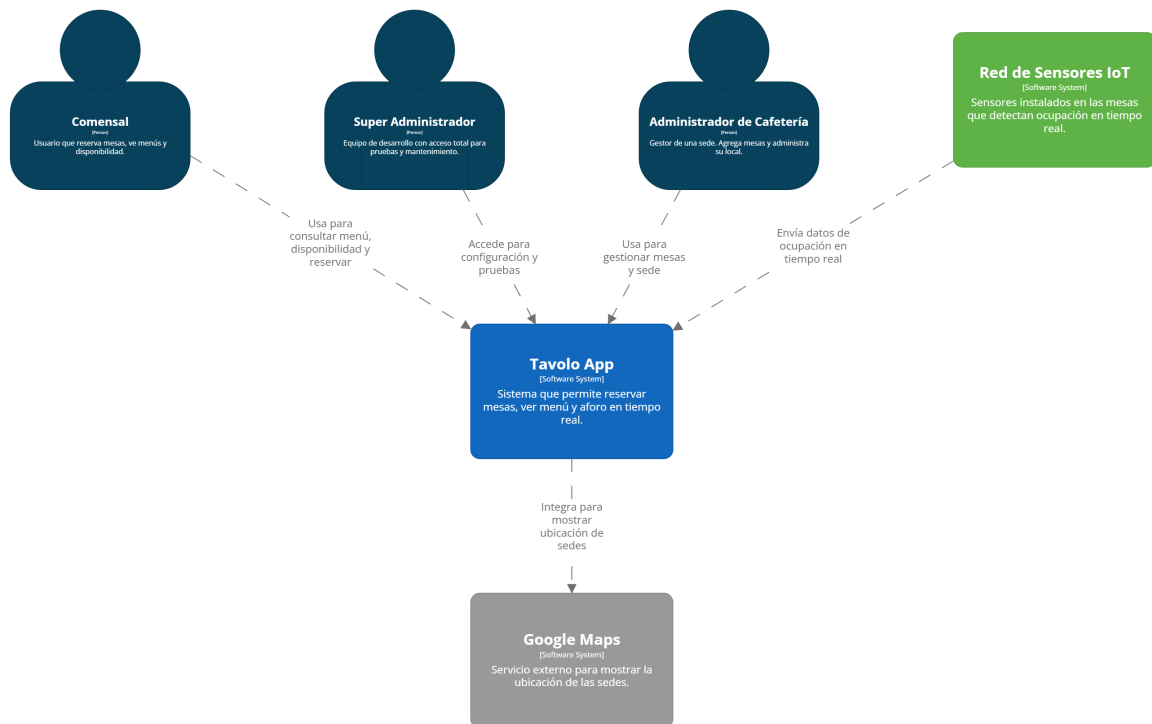
4.1.3.1. Software Architecture System Landscape Diagram.

Este diagrama muestra que Tavolo opera en un ecosistema compuesto por tres tipos de usuarios (Comensales, Administradores de Cafeterías y Super Administradores) que interactúan directamente con el sistema principal Tavolo App. El sistema se conecta con un servicio externo: Google Maps API para mostrar ubicaciones de cafeterías. Los Comensales usan Tavolo para reservar mesas y consultar menús, los Administradores de Cafeterías proporcionan información sobre menús y gestionan sus mesas, mientras que los Super Administradores (equipo de desarrollo) mantienen y actualizan el sistema.



4.1.3.2. Software Architecture Context Level Diagrams.

Este diagrama muestra que el sistema Tavolo (representado como una única entidad) interactúa con tres tipos de usuarios: Comensales que utilizan el sistema para encontrar cafeterías, ver menús y realizar reservas; Administradores de Cafeterías que actualizan la información de menús y mesas; y Super Administradores que gestionan el mantenimiento general del sistema. Tavolo también se comunica con dos sistemas externos: Google Maps para mostrar ubicaciones geográficas de cafeterías y un sistema de Sensores IoT que proporciona datos en tiempo real sobre la ocupación de mesas en los establecimientos.

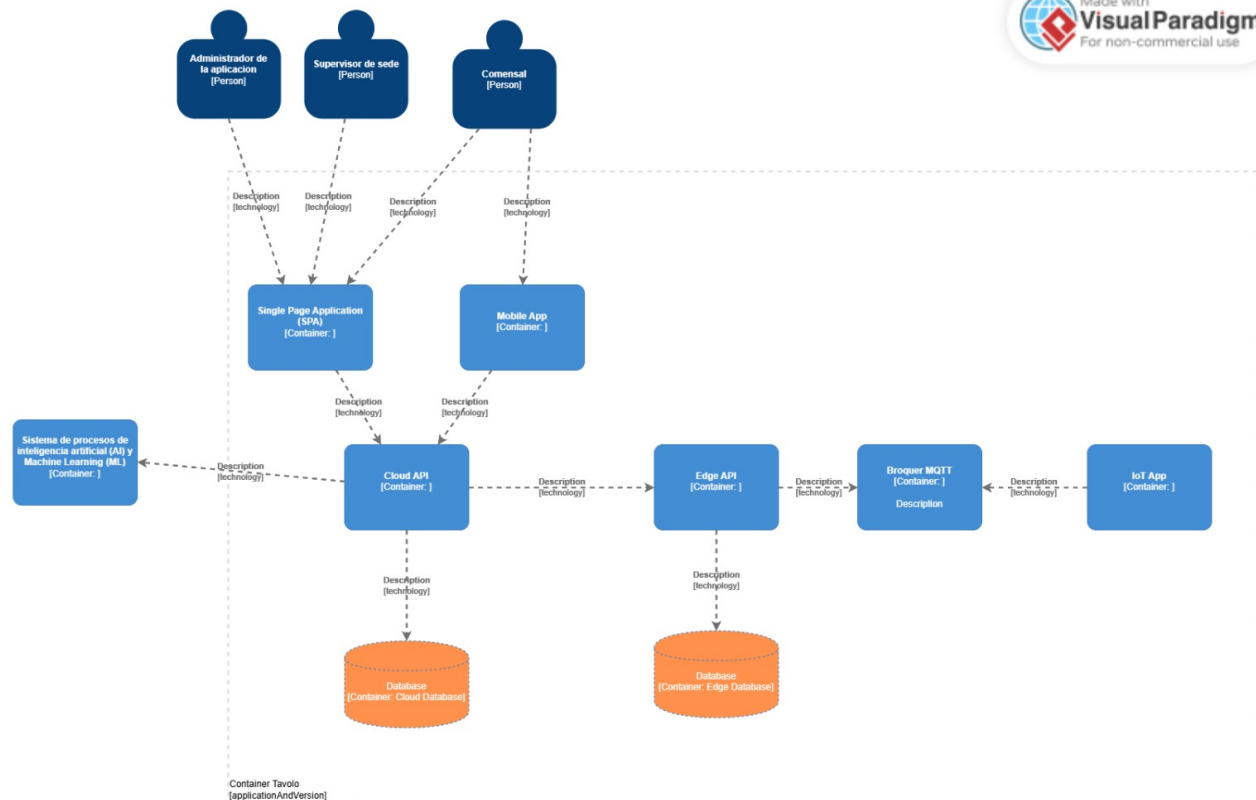


[System Context] Tavolo App

Diagrama de contexto del sistema Tavolo y su relación con usuarios, sensores IoT y servicios externos.
viernes, 11 de abril de 2025, 18:22 hora estándar de Perú

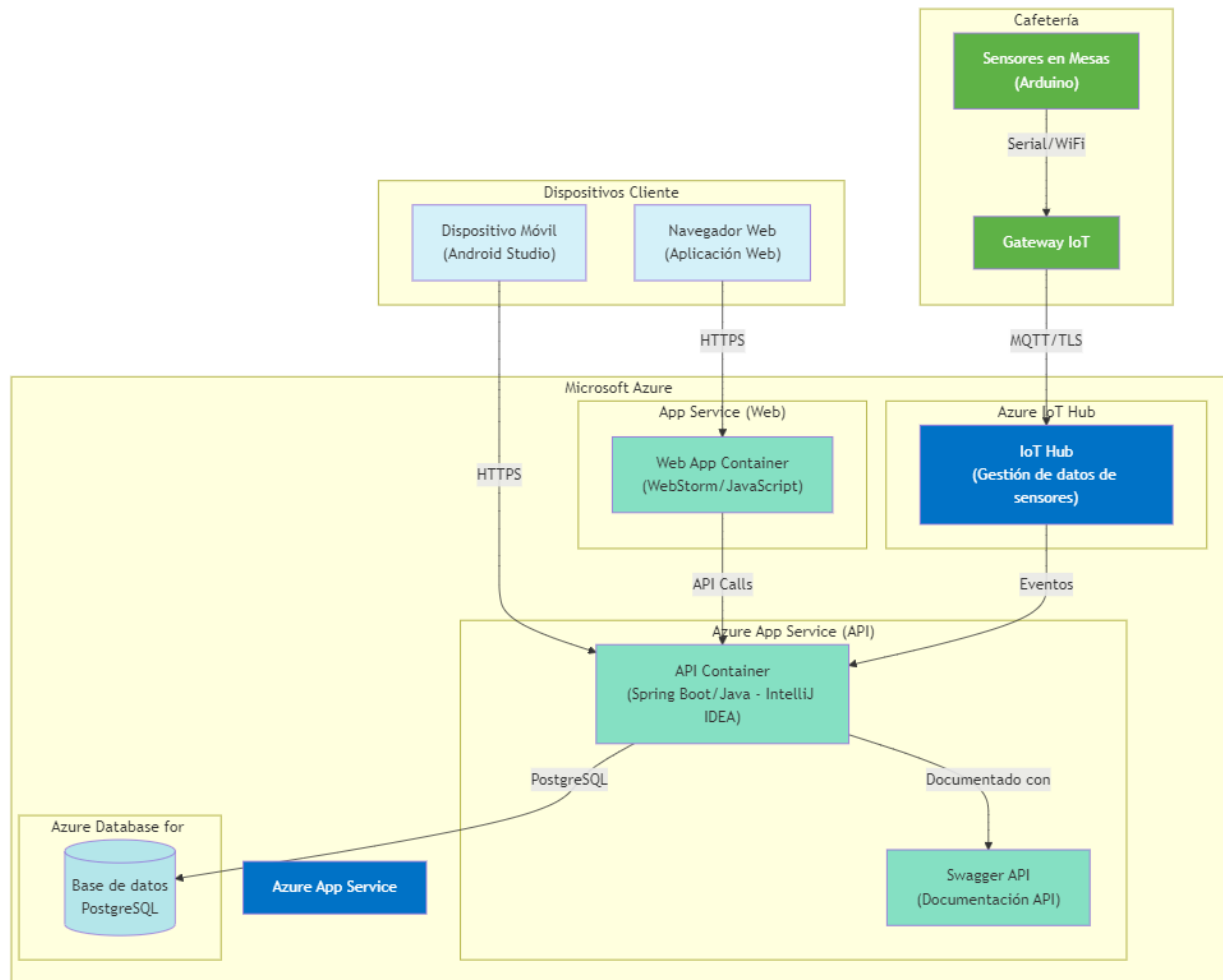
4.1.3.2. Software Architecture Container Level Diagrams.

Este diagrama muestra que el sistema Tavolo está compuesto por cuatro contenedores principales: una aplicación web accesible desde navegadores, una aplicación móvil para dispositivos portátiles, una API que gestiona toda la lógica de negocio y una base de datos que almacena la información del sistema. Además, incluye un módulo de sensores encargado de procesar los datos provenientes de dispositivos Arduino ubicados en las mesas. Por último, la API también se conecta con Google Maps, una aplicación externa, permitiendo que el comensal acceda a un mapa interactivo con las ubicaciones de las distintas sedes de la cafetería.



4.1.3.3. Software Architecture Deployment Diagrams.

Este diagrama muestra que el sistema Tavolo se despliega en tres entornos principales: Microsoft Azure Cloud, Dispositivos Cliente y Cafeterías físicas. En Azure, el sistema utiliza App Service para alojar la aplicación web frontend (WebStorm), otro App Service para la API (Spring Boot/Java) con documentación Swagger, Azure Database for PostgreSQL para los datos, y Azure IoT Hub para gestionar comunicaciones con dispositivos IoT. Los usuarios acceden al sistema a través de navegadores web o dispositivos Android. En cada cafetería, se instalan sensores Arduino en las mesas que envían datos de ocupación a un Gateway Arduino local, el cual transmite esta información al IoT Hub en Azure. Todas las comunicaciones entre componentes utilizan protocolos seguros como HTTPS, MQTT/TLS o conexiones PostgreSQL.



4.2. Tactical-Level Domain-Driven-Design

4.2.1 Bounded Context: IAM Bounded Context

El **IAM (Identity and Access Management) Bounded Context** es responsable de gestionar la autenticación y la creación de usuarios en el sistema. Este contexto asegura que los usuarios puedan registrarse, iniciar sesión y gestionar sus credenciales de manera segura, cumpliendo con los estándares de seguridad y privacidad.

4.2.1.1. Domain Layer

La **Domain Layer** del IAM Bounded Context encapsula la lógica de negocio relacionada con la autenticación y la gestión de usuarios. En esta capa, se definen los elementos principales del dominio, como agregados, entidades y objetos de valor, que representan los conceptos clave del sistema.

Aggregates

1. User

- **Propósito:** El agregado principal es el usuario (**User**), que encapsula la lógica de negocio relacionada con la autenticación y la gestión de roles.
- **Atributos:**
 - **username:** Identificador único del usuario, validado para cumplir con restricciones de longitud y unicidad.
 - **password:** Contraseña del usuario, almacenada de forma segura.

- **roles**: Conjunto de roles asociados al usuario, representados como una relación **ManyToMany** con la entidad **Role**.
- **Métodos**:
 - **addRole(Role role)**: Agrega un rol al usuario.
 - **addRoles(List<Role> roles)**: Agrega múltiples roles al usuario, validando que sean roles válidos.
- **Características**:
 - Extiende **AuditableAbstractAggregateRoot**, lo que permite auditar cambios en los usuarios.
 - Gestiona la relación entre usuarios y roles, asegurando consistencia y validación.

Entities

1. Role

- **Propósito**: La entidad **Role** representa los roles que pueden ser asignados a los usuarios.
- **Atributos**:
 - **id**: Identificador único del rol.
 - **name**: Nombre del rol, representado como un valor enumerado (**Roles**).
- **Métodos**:
 - **getDefaultRole()**: Devuelve el rol predeterminado (**ROLE_USER**).
 - **toRoleFromName(String name)**: Convierte un nombre de rol en una instancia de **Role**.
 - **validateRoleSet(List<Role> roles)**: Valida un conjunto de roles, asignando un rol predeterminado si el conjunto está vacío o es nulo.
- **Características**:
 - Define una relación única entre los nombres de roles y sus representaciones en la base de datos.
 - Facilita la validación y asignación de roles a los usuarios.

Value Objects:

1. Roles

- **Propósito**: El objeto de valor **Roles** es una enumeración que define los roles disponibles en el sistema.
- **Valores**:
 - **ROLE_ADMIN**: Rol de administrador.
 - **ROLE_SUPERVISOR**: Rol de supervisor.
 - **ROLE_USER**: Rol de usuario estándar.
- **Características**:
 - Representa roles como valores inmutables, asegurando consistencia en su uso dentro del dominio.

En conjunto, estos elementos permiten modelar de manera robusta y segura la lógica de negocio relacionada con la autenticación y la gestión de usuarios, asegurando que las reglas del dominio se cumplan de manera consistente.

4.2.1.2. Interface Layer

La **Interface Layer** del IAM Bounded Context expone los puntos de entrada al sistema a través de controladores REST. Estos controladores permiten la interacción con las entidades del dominio mediante solicitudes HTTP, facilitando la comunicación entre los clientes y el sistema. Además, esta capa incluye recursos y transformadores que aseguran una representación adecuada de los datos y su conversión entre las capas de la aplicación.

Controllers

Los controladores son responsables de manejar las solicitudes HTTP y delegar la lógica de negocio a los servicios correspondientes. A continuación, se describen los principales controladores:

1. UsersController

- **Propósito:** Gestiona las operaciones relacionadas con los usuarios.
- **Endpoints:**
 - `GET /api/v1/users`: Obtiene la lista de todos los usuarios.
 - `GET /api/v1/users/{userId}`: Obtiene los detalles de un usuario específico por su ID.
- **Dependencias:**
 - `UserQueryService`: Servicio encargado de manejar las consultas relacionadas con los usuarios.

2. RolesController

- **Propósito:** Gestiona las operaciones relacionadas con los roles.
- **Endpoints:**
 - `GET /api/v1/roles`: Obtiene la lista de todos los roles disponibles.
- **Dependencias:**
 - `RoleQueryService`: Servicio encargado de manejar las consultas relacionadas con los roles.

3. AuthenticationController

- **Propósito:** Gestiona las operaciones de autenticación y registro de usuarios.
- **Endpoints:**
 - `POST /api/v1/authentication/sign-up`: Registra un nuevo usuario en el sistema.
 - `POST /api/v1/authentication/sign-in`: Autentica a un usuario y genera un token de acceso.
- **Dependencias:**
 - `UserCommandService`: Servicio encargado de manejar los comandos relacionados con los usuarios.

Resources

Los recursos representan los datos que se exponen a través de la API REST. Estos recursos son utilizados para estructurar las respuestas de los controladores y asegurar una representación clara y consistente de los datos. A continuación, se describen los principales recursos:

1. UserResource

Representa un usuario en el sistema.

- **Atributos:**
 - `id`: Identificador único del usuario.
 - `username`: Nombre de usuario.
 - `roles`: Lista de roles asociados al usuario.

2. RoleResource

Representa un rol en el sistema.

- **Atributos:**
 - `id`: Identificador único del rol.
 - `name`: Nombre del rol.

3. AuthenticatedUserResource

Representa un usuario autenticado junto con su token de acceso.

- **Atributos:**
 - `id`: Identificador único del usuario.
 - `username`: Nombre de usuario.
 - `token`: Token de acceso generado.

4. SignUpResource

Representa los datos necesarios para registrar un nuevo usuario.

- **Atributos:**
 - `username`: Nombre de usuario.
 - `password`: Contraseña.
 - `roles`: Lista de roles asignados al usuario.

5. `SignInResource`

Representa los datos necesarios para autenticar a un usuario.

- **Atributos:**
 - `username`: Nombre de usuario.
 - `password`: Contraseña.

Transformers

Los transformadores son responsables de convertir las entidades del dominio en recursos y viceversa. Esto asegura que los datos expuestos a través de la API REST sean consistentes y estén en el formato esperado. A continuación, se describen los principales transformadores:

1. `UserResourceFromEntityAssembler`

Convierte una entidad `User` en un recurso `UserResource`.

- **Método principal:**
 - `toResourceFromEntity(User entity)`: Transforma un usuario del dominio en un recurso.

2. `RoleResourceFromEntityAssembler`

Convierte una entidad `Role` en un recurso `RoleResource`.

- **Método principal:**
 - `toResourceFromEntity(Role entity)`: Transforma un rol del dominio en un recurso.

3. `AuthenticatedUserResourceFromEntityAssembler`

Convierte una entidad `User` y un token en un recurso `AuthenticatedUserResource`.

- **Método principal:**
 - `toResourceFromEntity(User entity, String token)`: Transforma un usuario autenticado en un recurso.

4. `SignUpCommandFromResourceAssembler`

Convierte un recurso `SignUpResource` en un comando `SignUpCommand`.

- **Método principal:**
 - `toCommandFromResource(SignUpResource resource)`: Transforma los datos de registro en un comando.

5. `SignInCommandFromResourceAssembler`

Convierte un recurso `SignInResource` en un comando `SignInCommand`.

- **Método principal:**
 - `toCommandFromResource(SignInResource resource)`: Transforma los datos de inicio de sesión en un comando.

Relaciones entre componentes

- Los controladores utilizan los servicios de consulta (`UserQueryService`, `RoleQueryService`) y de comandos (`UserCommandService`) para delegar la lógica de negocio.

- Los transformadores convierten las entidades del dominio en recursos para las respuestas HTTP y viceversa para las solicitudes entrantes.
- Los recursos estructuran los datos expuestos a los clientes, asegurando una representación clara y consistente.

Esta estructura asegura que la **Interface Layer** sea modular, reutilizable y fácil de mantener, facilitando la interacción entre los clientes y el sistema.

4.2.1.3 Application Layer

La **Application Layer** del IAM Bounded Context actúa como un intermediario entre la **Domain Layer** y las capas externas, como la **Interface Layer** y la **Infrastructure Layer**. Su propósito principal es coordinar las operaciones de negocio, manejar comandos y consultas, y orquestar la lógica de aplicación sin exponer directamente los detalles del dominio.

Command Services

Los servicios de comandos son responsables de ejecutar operaciones que modifican el estado del sistema. A continuación, se describen los principales servicios de comandos:

1. RoleCommandServiceImpl

- **Propósito:** Gestiona las operaciones relacionadas con los roles, como la creación inicial de roles en el sistema.
- **Métodos principales:**
 - `handle(SeedRolesCommand command)`: Verifica si los roles predefinidos existen en el sistema y los crea si no están presentes.
- **Dependencias:**
 - `RoleRepository`: Interactúa con la base de datos para verificar y guardar roles.

2. UserCommandServiceImpl

- **Propósito:** Gestiona las operaciones relacionadas con los usuarios, como el registro y la autenticación.
- **Métodos principales:**
 - `handle(SignUpCommand command)`: Registra un nuevo usuario en el sistema, asignándole roles y almacenando su contraseña de forma segura.
 - `handle(SignInCommand command)`: Autentica a un usuario verificando sus credenciales y generando un token de acceso.
- **Dependencias:**
 - `UserRepository`: Interactúa con la base de datos para guardar y recuperar usuarios.
 - `HashingService`: Codifica y verifica contraseñas.
 - `TokenService`: Genera y valida tokens de acceso.
 - `RoleRepository`: Recupera roles asignados a los usuarios.

Query Services

Los servicios de consultas son responsables de recuperar información del sistema sin modificar su estado. A continuación, se describen los principales servicios de consultas:

1. RoleQueryServiceImpl

- **Propósito:** Gestiona las consultas relacionadas con los roles.
- **Métodos principales:**
 - `handle(GetAllRolesQuery query)`: Recupera todos los roles disponibles en el sistema.
 - `handle(GetRoleByNameQuery query)`: Recupera un rol específico por su nombre.

- **Dependencias:**
 - **RoleRepository**: Interactúa con la base de datos para recuperar roles.

2. UserQueryServiceImpl

- **Propósito:** Gestiona las consultas relacionadas con los usuarios.
- **Métodos principales:**
 - **handle(GetAllUsersQuery query)**: Recupera todos los usuarios registrados en el sistema.
 - **handle(GetUserByIdQuery query)**: Recupera un usuario específico por su ID.
 - **handle(GetUserByUsernameQuery query)**: Recupera un usuario específico por su nombre de usuario.
- **Dependencias:**
 - **UserRepository**: Interactúa con la base de datos para recuperar usuarios.

Relaciones entre componentes

- Los **Command Services** interactúan con los repositorios para modificar el estado del sistema y con los servicios externos para operaciones auxiliares, como el hashing de contraseñas y la generación de tokens.
- Los **Query Services** interactúan únicamente con los repositorios para recuperar información del sistema.

Esta estructura asegura que la **Application Layer** sea modular, reutilizable y fácil de mantener, permitiendo una separación clara de responsabilidades y facilitando la evolución del sistema.

4.2.1.4. Infrastructure Layer

La **Infrastructure Layer** del IAM Bounded Context proporciona las implementaciones técnicas necesarias para soportar las operaciones del sistema. Esta capa incluye configuraciones de seguridad, servicios de hashing, manejo de tokens, repositorios para la persistencia de datos y componentes relacionados con la autorización y autenticación. Su objetivo principal es conectar la lógica de negocio con los recursos externos, como bases de datos, servicios de seguridad y APIs externas.

Persistencia (JPA Repositories)

1. RoleRepository

- **Propósito:** Proporciona métodos para interactuar con la base de datos de roles.
- **Métodos principales:**
 - **findByName**: Busca un rol por su nombre.
 - **existsByName**: Verifica si un rol existe en la base de datos.

2. UserRepository

- **Propósito:** Proporciona métodos para interactuar con la base de datos de usuarios.
- **Métodos principales:**
 - **findByUsername**: Busca un usuario por su nombre de usuario.
 - **existsByUsername**: Verifica si un usuario existe en la base de datos.

Relaciones entre componentes

- **Persistencia:** Los repositorios (**RoleRepository**, **UserRepository**) proporcionan acceso a los datos almacenados en la base de datos, permitiendo a las capas superiores interactuar con las entidades del dominio.

Esta estructura asegura que la **Infrastructure Layer** sea modular, reutilizable y fácil de mantener, facilitando la integración con otros sistemas y servicios externos.

4.2.1.5. Bounded Context Software Architecture Component Level Diagrams

En esta sección se presenta el diagrama de componentes del **IAM Bounded Context**, el cual detalla los principales módulos y sus interacciones dentro del contexto delimitado. Este diagrama sigue el enfoque del C4 Model para representar los componentes clave, como servicios de aplicación, controladores, repositorios y servicios externos, junto con sus relaciones.

El propósito de este diagrama es proporcionar una visión clara y estructurada de cómo se organizan los componentes dentro del contexto, facilitando la comprensión de su arquitectura y permitiendo identificar puntos de integración y responsabilidades.

El **IAM Bounded Context** está compuesto por los siguientes módulos principales:

1. Application Layer:

- Coordina las operaciones de negocio relacionadas con la autenticación y la gestión de usuarios.
- Incluye servicios de comandos y consultas que interactúan con la **Domain Layer** y la **Infrastructure Layer**.
- Maneja eventos relacionados con el registro de usuarios, la asignación de roles y la autenticación.

2. Interface Layer:

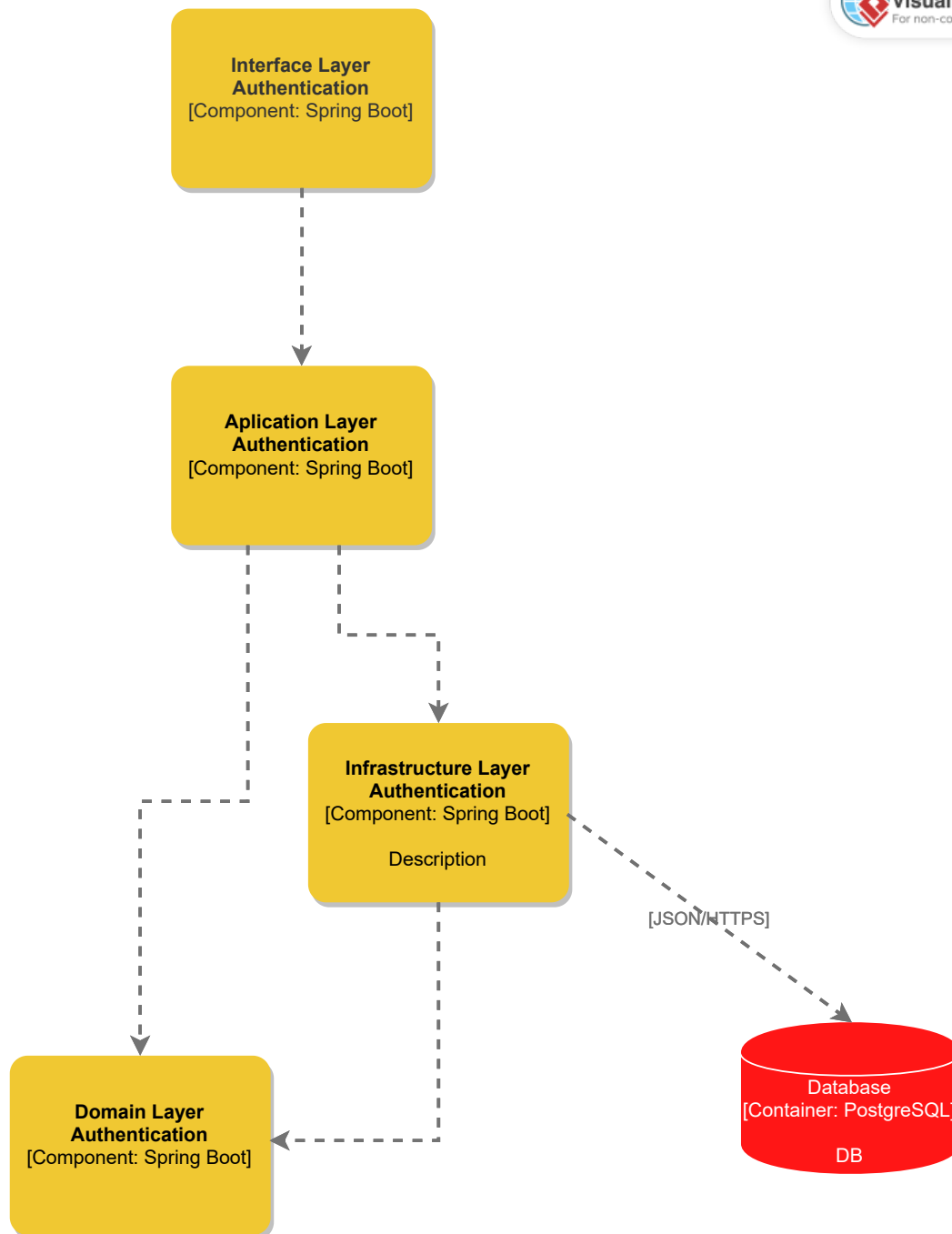
- Expone los puntos de entrada al sistema a través de controladores REST.
- Incluye recursos y transformadores que aseguran una representación adecuada de los datos y su conversión entre las capas de la aplicación.
- Proporciona endpoints para operaciones como el registro de usuarios, la autenticación y la consulta de roles.

3. Domain Layer:

- Encapsula la lógica de negocio relacionada con la autenticación y la gestión de usuarios.
- Define los agregados, entidades y objetos de valor que representan los conceptos clave del dominio, como usuarios, roles y permisos.
- Asegura que las reglas de negocio, como la validación de contraseñas y la asignación de roles, se cumplan de manera consistente.

4. Infrastructure Layer:

- Proporciona las implementaciones técnicas necesarias para soportar las operaciones del sistema.
- Incluye repositorios para la persistencia de datos y componentes que conectan la lógica de negocio con los recursos externos, como bases de datos y servicios de seguridad.
- Implementa servicios auxiliares como el hashing de contraseñas y la generación de tokens de autenticación.



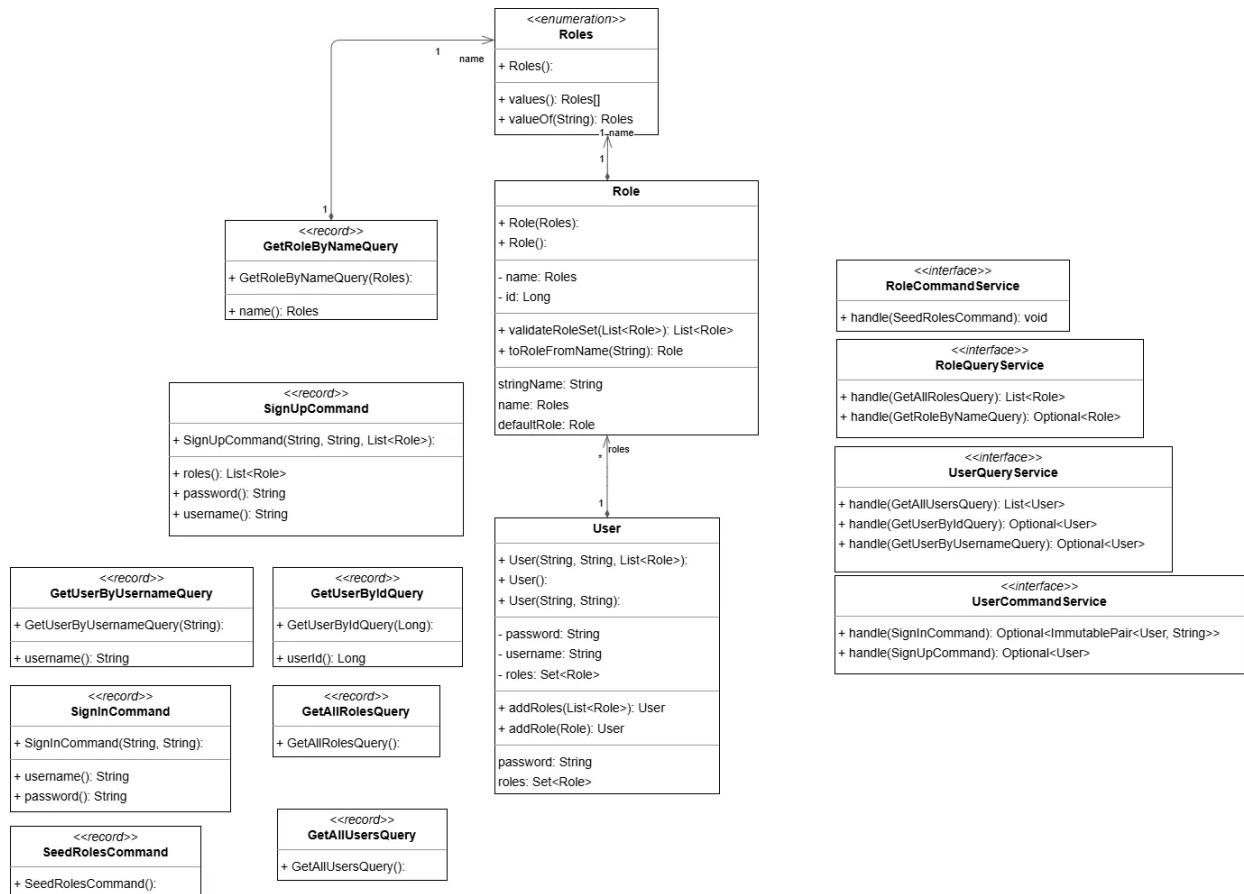
4.2.1.6. Bounded Context Software Architecture Code Level Diagrams

En este apartado se presentan los diagramas que ofrecen un mayor nivel de detalle sobre la implementación de los componentes del **IAM Bounded Context**. Estos diagramas están diseñados para ilustrar cómo se estructuran las clases, interfaces y relaciones dentro de las capas del contexto, proporcionando una visión técnica que facilita el desarrollo, mantenimiento y evolución del sistema.

4.2.1.6.1. Bounded Context Domain Layer Class Diagrams

El diagrama de clases correspondiente a la **Domain Layer** del **IAM Bounded Context** incluye las clases principales, como agregados, entidades y objetos de valor, así como las interfaces y enumeraciones que definen el

comportamiento del dominio. También se destacan las relaciones entre estos elementos, como asociaciones, composiciones y dependencias.



Elementos principales del diagrama:

1. Aggregates:

- **User**: Agregado principal que encapsula la lógica de negocio relacionada con la autenticación y la gestión de usuarios. Incluye atributos como **username**, **password** y **roles**.
 - **Atributos**:
 - **username**: Identificador único del usuario.
 - **password**: Contraseña almacenada de forma segura.
 - **roles**: Lista de roles asignados al usuario.
 - **Métodos**:
 - **addRole(Role role)**: Agrega un rol al usuario.
 - **addRoles(List<Role> roles)**: Agrega múltiples roles al usuario, validando que sean válidos.

2. Entities:

- **Role**: Representa los roles que pueden ser asignados a los usuarios.
 - **Atributos**:
 - **id**: Identificador único del rol.
 - **name**: Nombre del rol.
 - **Métodos**:
 - **getDefaultRole()**: Devuelve el rol predeterminado (**ROLE_USER**).
 - **validateRoleSet(List<Role> roles)**: Valida un conjunto de roles asignados.

3. Value Objects:

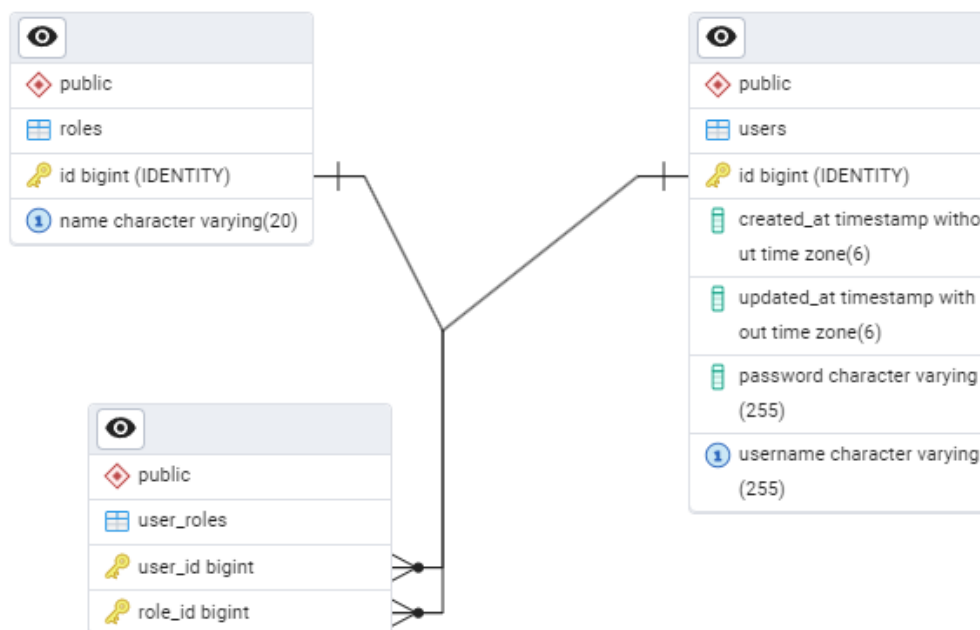
- **Roles**: Enumeración que define los roles disponibles en el sistema (**ROLE_ADMIN**, **ROLE_SUPERVISOR**, **ROLE_USER**).
- **Password**: Representa la contraseña del usuario, asegurando que cumpla con las políticas de seguridad.
- **Token**: Representa un token de autenticación generado para un usuario.

Relaciones destacadas:

- El agregado **User** gestiona las relaciones con la entidad **Role**, asegurando que los usuarios tengan roles válidos asignados.
- Los objetos de valor encapsulan datos inmutables y validaciones específicas, como la seguridad de contraseñas y la unicidad de roles.
- La entidad **Role** define los permisos y responsabilidades que pueden ser asignados a los usuarios.

4.2.1.6.2. Bounded Context Database Design Diagram.

El diseño de la base de datos para el **IAM Bounded Context** refleja la estructura del dominio, asegurando que las entidades y relaciones definidas en la **Domain Layer** se representen de manera eficiente en el modelo relacional.



Este diseño incluye las siguientes tablas principales:

1. Users:

- Representa los usuarios registrados en el sistema.
- **Atributos principales:**
 - **id**: Identificador único del usuario.
 - **username**: Nombre de usuario único.
 - **password**: Contraseña almacenada de forma segura.
 - **created_at**: Fecha de creación del usuario.

2. Roles:

- Representa los roles disponibles en el sistema.
- **Atributos principales:**
 - **id**: Identificador único del rol.
 - **name**: Nombre del rol.

3. UserRoles:

- Representa la relación entre usuarios y roles.
- **Atributos principales:**
 - **user_id**: Identificador del usuario.
 - **role_id**: Identificador del rol.

Este diseño asegura que las operaciones relacionadas con la autenticación y la gestión de usuarios sean eficientes y consistentes, facilitando la integración con la lógica de negocio definida en la **Domain Layer**.

Voy a completar el análisis táctico de Domain-Driven Design para los bounded contexts faltantes: Table Management, Menu Management y IoT Monitoring. Desarrollaré cada sección siguiendo la estructura que ya tiene tu informe.

4.2.2 Bounded Context: Table Management Bounded Context

El **Table Management Bounded Context** es responsable de gestionar la información y el estado de las mesas en las cafeterías. Este contexto se encarga de registrar, actualizar y consultar el estado de las mesas, así como su capacidad, ubicación dentro de la sede y otros atributos relevantes. Es el núcleo central que permite la visualización en tiempo real del estado de ocupación de las mesas para los usuarios.

4.2.2.1. Domain Layer

La **Domain Layer** del Table Management Bounded Context encapsula la lógica de negocio relacionada con la gestión de mesas. En esta capa, se definen los elementos principales del dominio, como agregados, entidades y objetos de valor, que representan los conceptos clave del sistema.

Aggregates

1. Table

- **Propósito:** El agregado principal es la mesa (**Table**), que encapsula la lógica de negocio relacionada con la gestión de la mesa y su estado de ocupación.
- **Atributos:**
 - **tableNumber**: Número identificativo de la mesa, representado como un objeto de valor **TableNumber**.
 - **headquarterId**: Identificador de la sede a la que pertenece la mesa, representado como un objeto de valor **HeadquarterId**.
 - **capacity**: Capacidad de la mesa (número de sillas), representada como un objeto de valor **Capacity**.
 - **status**: Estado actual de la mesa, representado como un objeto de valor **TableStatus**.
 - **location**: Ubicación física de la mesa dentro de la sede, representada como un objeto de valor **Location**.

- **Métodos:**
 - `updateStatus(TableStatus status)`: Actualiza el estado de la mesa.
 - `assignToHeadquarter(HeadquarterId headquarterId)`: Asigna la mesa a una sede específica.
 - `changeCapacity(Capacity capacity)`: Modifica la capacidad de la mesa.
 - `relocate(Location location)`: Actualiza la ubicación de la mesa dentro de la sede.
- **Características:**
 - Extiende `AuditableAbstractAggregateRoot`, lo que permite auditar cambios en las mesas.
 - Gestiona la validación de estados y cambios según reglas de negocio.

Entities

1. Chair

- **Propósito:** La entidad `Chair` representa cada silla asociada a una mesa, con su estado de ocupación.
- **Atributos:**
 - `id`: Identificador único de la silla.
 - `chairNumber`: Número identificativo de la silla dentro de la mesa.
 - `status`: Estado actual de la silla (ocupada o libre), representado como un objeto de valor `ChairStatus`.
- **Métodos:**
 - `updateStatus(ChairStatus status)`: Actualiza el estado de la silla.
 - `isOccupied()`: Verifica si la silla está ocupada.

Value Objects

1. TableNumber

- **Propósito:** Representa el número identificativo de una mesa.
- **Validaciones:**
 - El número debe ser positivo.
 - El número debe ser único dentro de una sede.

2. HeadquarterId

- **Propósito:** Representa el identificador único de una sede.
- **Validaciones:**
 - El identificador no puede ser nulo ni negativo.

3. Capacity

- **Propósito:** Representa la capacidad de una mesa en términos de número de personas que pueden sentarse.
- **Validaciones:**
 - La capacidad debe ser un número positivo.
 - La capacidad máxima permitida es de 12 personas.

4. TableStatus

- **Propósito:** Enumera los estados posibles de una mesa.
- **Valores:**
 - `AVAILABLE`: La mesa está disponible para ser ocupada.
 - `OCCUPIED`: La mesa está ocupada por comensales.
 - `RESERVED`: La mesa está reservada para un futuro cercano.
 - `MAINTENANCE`: La mesa está fuera de servicio temporalmente.

5. ChairStatus

- **Propósito:** Enumera los estados posibles de una silla.
- **Valores:**
 - **OCCUPIED:** La silla está ocupada por un comensal.
 - **FREE:** La silla está libre.

6. Location

- **Propósito:** Representa la ubicación física de una mesa dentro de la sede.
- **Atributos:**
 - **zone:** Zona de la cafetería (interior, terraza, etc.).
 - **coordinates:** Coordenadas relativas dentro del plano de la sede.
- **Validaciones:**
 - Las coordenadas deben estar dentro de los límites del plano de la sede.

Commands

1. CreateTableCommand

- **Propósito:** Comando para crear una nueva mesa en una sede específica.
- **Atributos:**
 - **tableNumber:** Número de la mesa.
 - **headquarterId:** ID de la sede donde se ubicará la mesa.
 - **capacity:** Capacidad de la mesa.
 - **location:** Ubicación de la mesa dentro de la sede.

2. UpdateTableStatusCommand

- **Propósito:** Comando para actualizar el estado de una mesa existente.
- **Atributos:**
 - **tableId:** ID de la mesa a actualizar.
 - **status:** Nuevo estado de la mesa.

3. DeleteTableCommand

- **Propósito:** Comando para eliminar una mesa existente.
- **Atributos:**
 - **tableId:** ID de la mesa a eliminar.
 - **headquarterId:** ID de la sede a la que pertenece la mesa.

Queries

1. GetTableByIdQuery

- **Propósito:** Recupera una mesa específica por su ID.
- **Atributos:**
 - **tableId:** ID de la mesa a consultar.

2. GetTablesByHeadquarterQuery

- **Propósito:** Recupera todas las mesas asociadas a una sede específica.
- **Atributos:**
 - **headquarterId:** ID de la sede a consultar.

3. GetTablesByStatusQuery

- **Propósito:** Recupera todas las mesas que tienen un estado específico.
- **Atributos:**
 - **status:** Estado de las mesas a consultar.
 - **headquarterId:** ID de la sede a consultar.

Events

1. TableCreatedEvent

- **Propósito:** Evento que se dispara cuando se crea una nueva mesa.
- **Atributos:**
 - **tableId:** ID de la mesa creada.
 - **headquarterId:** ID de la sede donde se ubicó la mesa.

2. TableStatusChangedEvent

- **Propósito:** Evento que se dispara cuando cambia el estado de una mesa.
- **Atributos:**
 - **tableId:** ID de la mesa.
 - **previousStatus:** Estado anterior de la mesa.
 - **newStatus:** Nuevo estado de la mesa.
 - **timestamp:** Momento en que ocurrió el cambio.

4.2.2.2. Interface Layer

La **Interface Layer** del Table Management Bounded Context expone los puntos de entrada al sistema a través de controladores REST. Esta capa permite la interacción con las entidades del dominio mediante solicitudes HTTP, facilitando la comunicación entre los clientes y el sistema. Además, incluye recursos y transformadores que aseguran una representación adecuada de los datos y su conversión entre las capas de la aplicación.

Access Control Layer (ACL)

1. TableManagementContextFacade

- **Propósito:** Proporciona una interfaz simplificada para interactuar con el dominio del Table Management Bounded Context desde otros contextos. Permite consultar información sobre las mesas, como su disponibilidad, capacidad y ubicación.
- **Métodos principales:**
 - **getTableStatus(Long tableId):** Devuelve el estado actual de una mesa.
 - **getTablesByStatus(Long headquarterId, String status):** Devuelve las mesas de una sede con un estado específico.
 - **existsTable(Long tableId):** Verifica si una mesa existe en el sistema.
- **Dependencias:**
 - **TableQueryService:** Servicio encargado de manejar las consultas relacionadas con las mesas.

Controllers

Los controladores son responsables de manejar las solicitudes HTTP y delegar la lógica de negocio a los servicios correspondientes. A continuación, se describen los principales controladores:

1. TableController

- **Propósito:** Gestiona las operaciones relacionadas con las mesas.
- **Endpoints:**
 - **POST /api/v1/tables:** Crea una nueva mesa.

- GET /api/v1/tables/{tableId}: Obtiene los detalles de una mesa específica por su ID.
- GET /api/v1/tables: Obtiene la lista de todas las mesas, con opción a filtrar por sede.
- GET /api/v1/headquarters/{headquarterId}/tables: Obtiene las mesas de una sede específica.
- PUT /api/v1/tables/{tableId}/status: Actualiza el estado de una mesa.
- DELETE /api/v1/tables/{tableId}: Elimina una mesa existente.
- **Dependencias:**
 - **TableCommandService:** Servicio encargado de manejar los comandos relacionados con las mesas.
 - **TableQueryService:** Servicio encargado de manejar las consultas relacionadas con las mesas.

Resources

Los recursos representan los datos que se exponen a través de la API REST. Estos recursos son utilizados para estructurar las respuestas de los controladores y asegurar una representación clara y consistente de los datos. A continuación, se describen los principales recursos:

1. CreateTableResource

- **Propósito:** Representa los datos necesarios para crear una nueva mesa.
- **Atributos:**
 - **tableNumber:** Número de la mesa.
 - **headquarterId:** ID de la sede donde se ubicará la mesa.
 - **capacity:** Capacidad de la mesa.
 - **zone:** Zona de la cafetería donde se ubicará la mesa.
 - **xPosition:** Coordenada X de la mesa en el plano.
 - **yPosition:** Coordenada Y de la mesa en el plano.

2. TableResource

- **Propósito:** Representa una mesa en el sistema.
- **Atributos:**
 - **id:** ID único de la mesa.
 - **tableNumber:** Número de la mesa.
 - **headquarterId:** ID de la sede donde está ubicada la mesa.
 - **capacity:** Capacidad de la mesa.
 - **status:** Estado actual de la mesa.
 - **zone:** Zona de la cafetería donde está ubicada la mesa.
 - **position:** Coordenadas de la mesa en el plano.

3. UpdateTableStatusResource

- **Propósito:** Representa los datos necesarios para actualizar el estado de una mesa.
- **Atributos:**
 - **status:** Nuevo estado de la mesa.

Transformers

Los transformadores son responsables de convertir las entidades del dominio en recursos y viceversa. Esto asegura que los datos expuestos a través de la API REST sean consistentes y estén en el formato esperado. A continuación, se describen los principales transformadores:

1. CreateTableCommandFromResourceAssembler

- **Propósito:** Convierte un recurso **CreateTableResource** en un comando **CreateTableCommand**.

- **Método principal:**
 - `toCommandFromResource(CreateTableResource resource)`: Transforma los datos de creación de una mesa en un comando.

2. TableResourceFromEntityAssembler

- **Propósito:** Convierte una entidad `Table` en un recurso `TableResource`.
- **Método principal:**
 - `toResourceFromEntity(Table entity)`: Transforma una mesa del dominio en un recurso.

3. UpdateTableStatusCommandFromResourceAssembler

- **Propósito:** Convierte un recurso `UpdateTableStatusResource` en un comando `UpdateTableStatusCommand`.
- **Método principal:**
 - `toCommandFromResource(UpdateTableStatusResource resource, Long tableId)`: Transforma los datos de actualización de estado en un comando.

4.2.2.3. Application Layer

La **Application Layer** del Table Management Bounded Context actúa como un intermediario entre la **Domain Layer** y las capas externas, como la **Interface Layer** y la **Infrastructure Layer**. Su propósito principal es coordinar las operaciones de negocio, manejar comandos y consultas, y orquestar la lógica de aplicación sin exponer directamente los detalles del dominio.

Command Services

Los servicios de comandos son responsables de ejecutar operaciones que modifican el estado del sistema. A continuación, se describen los principales servicios de comandos:

1. TableCommandServiceImpl

- **Propósito:** Gestiona las operaciones relacionadas con la creación, actualización y eliminación de mesas.
- **Métodos principales:**
 - `handle(CreateTableCommand command)`: Crea una nueva mesa validando que la sede exista y que no haya duplicados.
 - `handle(UpdateTableStatusCommand command)`: Actualiza el estado de una mesa existente.
 - `handle>DeleteTableCommand command)`: Elimina una mesa existente.
- **Validaciones:**
 - Verifica que la sede exista utilizando el servicio externo `ExternalHeadquarterService`.
 - Asegura que no existan mesas con el mismo número en la misma sede.
 - Valida que la mesa exista antes de actualizar su estado o eliminarla.
- **Dependencias:**
 - `TableRepository`: Persistencia de mesas.
 - `ExternalHeadquarterService`: Verifica la existencia de la sede.

Query Services

Los servicios de consultas son responsables de recuperar información del sistema sin modificar su estado. A continuación, se describen los principales servicios de consultas:

1. TableQueryServiceImpl

- **Propósito:** Gestiona las consultas relacionadas con las mesas.
- **Métodos principales:**
 - `handle(GetTableByIdQuery query)`: Recupera una mesa específica por su ID.

- `handle(GetTablesByHeadquarterQuery query)`: Recupera todas las mesas de una sede específica.
- `handle(GetTablesByStatusQuery query)`: Recupera todas las mesas que tienen un estado específico en una sede determinada.
- **Dependencias:**
 - `TableRepository`: Persistencia de mesas.

Event Handlers

Los manejadores de eventos son responsables de reaccionar a eventos específicos del sistema. A continuación, se describen los principales manejadores de eventos:

1. TableStatusChangedEventHandler

- **Propósito:** Maneja el evento `TableStatusChangedEvent`, que se dispara cuando cambia el estado de una mesa.
- **Método principal:**
 - `on(TableStatusChangedEvent event)`: Realiza acciones adicionales cuando cambia el estado de una mesa, como notificar a otros sistemas o actualizar estadísticas.
- **Dependencias:**
 - `NotificationService`: Servicio para enviar notificaciones.
 - `StatisticsService`: Servicio para actualizar estadísticas de uso de mesas.

Outbound Services (ACL)

Los servicios externos proporcionan funcionalidades auxiliares que no forman parte del dominio principal. A continuación, se describen los principales servicios externos:

1. ExternalHeadquarterService

- **Propósito:** Interactúa con el Branching Bounded Context para verificar la existencia de sedes.
- **Método principal:**
 - `existsHeadquarter(Long headquarterId)`: Verifica si una sede existe en el sistema.

4.2.2.4. Infrastructure Layer

La **Infrastructure Layer** del Table Management Bounded Context proporciona las implementaciones técnicas necesarias para soportar las operaciones del sistema relacionadas con la gestión de mesas. Esta capa incluye repositorios para la persistencia de datos y componentes que conectan la lógica de negocio con los recursos externos, como bases de datos. Su objetivo principal es garantizar que las operaciones de almacenamiento y recuperación de información sean eficientes, consistentes y seguras.

Persistencia (JPA Repositories)

1. TableRepository

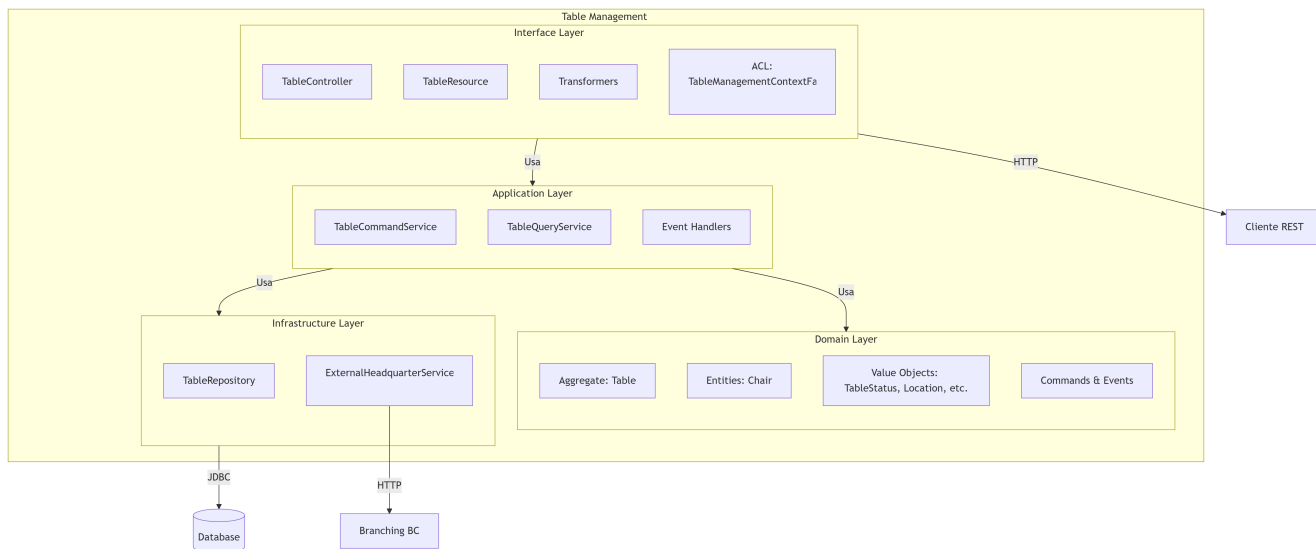
- **Propósito:** Proporciona métodos para interactuar con la base de datos de mesas.
- **Métodos principales:**
 - `findByHeadquarterId(Long headquarterId)`: Recupera todas las mesas de una sede específica.
 - `findByHeadquarterIdAndStatus(Long headquarterId, TableStatus status)`: Recupera las mesas de una sede con un estado específico.
 - `existsByHeadquarterIdAndTableNumber(Long headquarterId, Integer tableNumber)`: Verifica si existe una mesa con un número específico en una sede.
- **Características:**
 - Extiende `JpaRepository`, lo que permite realizar operaciones CRUD sobre las entidades `Table`.
 - Facilita la recuperación de mesas según diferentes criterios de filtro.

Relaciones entre componentes

- **Persistencia:** El repositorio **TableRepository** proporciona acceso a los datos almacenados en la base de datos, permitiendo a las capas superiores (como la **Application Layer**) interactuar con las entidades del dominio.
- **Validación:** Los métodos personalizados en **TableRepository** son utilizados para validar la existencia de mesas y recuperar información específica, asegurando la consistencia de los datos durante las operaciones de negocio.

4.2.2.5. Bounded Context Software Architecture Component Level Diagrams

En esta sección se presenta el diagrama de componentes del **Table Management Bounded Context**, el cual detalla los principales módulos y sus interacciones dentro del contexto delimitado. Este diagrama sigue el enfoque del C4 Model para representar los componentes clave, como servicios de aplicación, controladores, repositorios y servicios externos, junto con sus relaciones.



El **Table Management Bounded Context** está compuesto por los siguientes módulos principales:

1. Application Layer:

- Coordina las operaciones de negocio relacionadas con la gestión de mesas.
- Incluye servicios de comandos y consultas que interactúan con la **Domain Layer** y la **Infrastructure Layer**.
- Maneja eventos relacionados con cambios en el estado de las mesas.

2. Interface Layer:

- Expone los puntos de entrada al sistema a través de controladores REST.
- Incluye recursos y transformadores que aseguran una representación adecuada de los datos y su conversión entre las capas de la aplicación.
- Proporciona una ACL (Access Control Layer) para facilitar la integración con otros contextos.

3. Domain Layer:

- Encapsula la lógica de negocio relacionada con la gestión de mesas.
- Define los agregados, entidades y objetos de valor que representan los conceptos clave del dominio.
- Incluye eventos de dominio que representan cambios significativos en el estado del sistema.

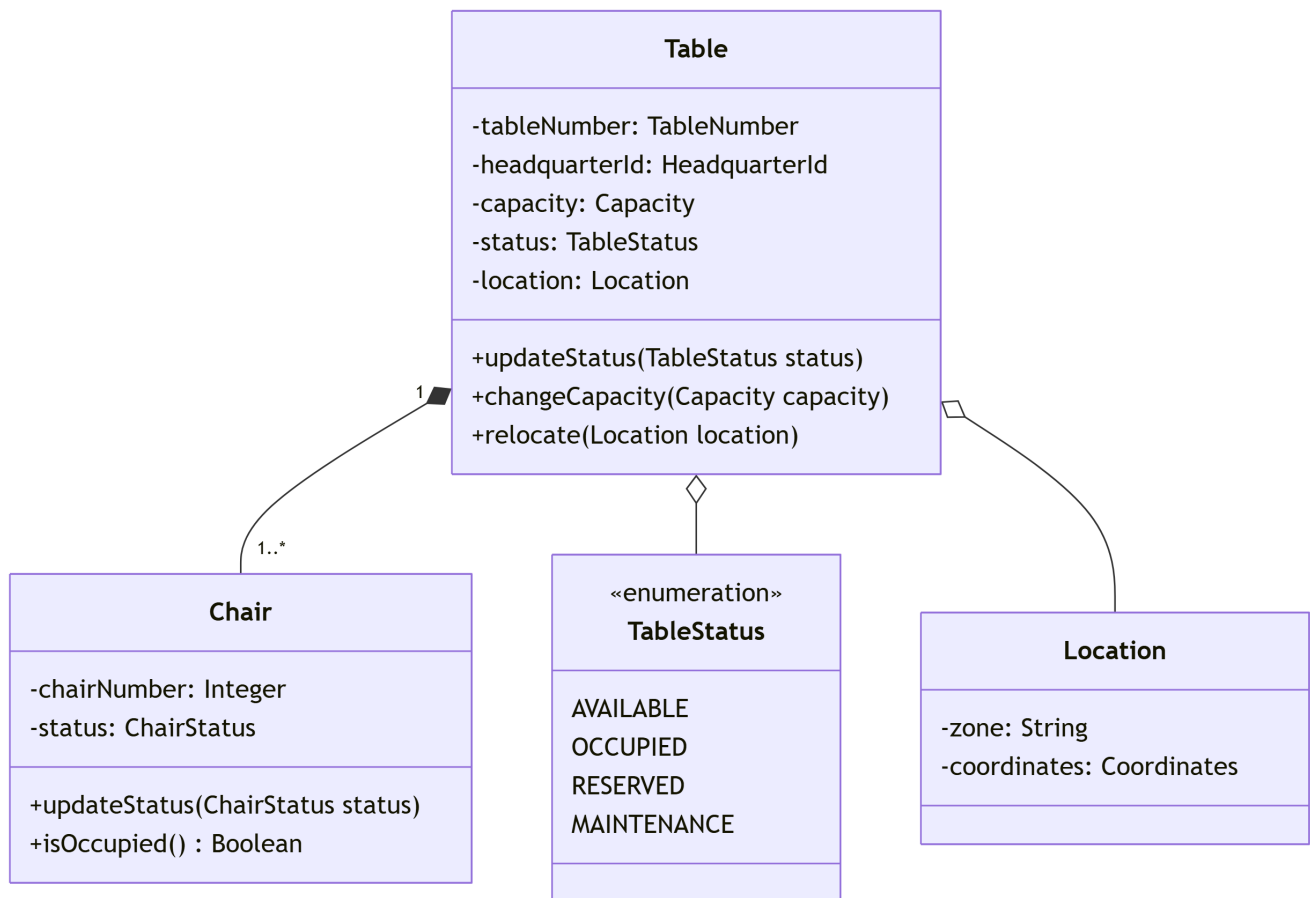
4. Infrastructure Layer:

- Proporciona las implementaciones técnicas necesarias para soportar las operaciones del sistema.
- Incluye repositorios para la persistencia de datos y componentes que conectan la lógica de negocio con los recursos externos, como bases de datos.

4.2.2.6. Bounded Context Software Architecture Code Level Diagrams

4.2.2.6.1. Bounded Context Domain Layer Class Diagrams

El diagrama de clases correspondiente a la **Domain Layer** del **Table Management Bounded Context** incluye las clases principales, como agregados, entidades y objetos de valor, así como las interfaces y enumeraciones que definen el comportamiento del dominio.



Elementos principales del diagrama:

1. Aggregates:

- **Table**: Agregado principal que encapsula la lógica de negocio relacionada con la gestión de mesas.
 - **Atributos**:
 - **tableNumber**: Número identificativo de la mesa.
 - **headquarterId**: ID de la sede a la que pertenece la mesa.
 - **capacity**: Capacidad de la mesa.
 - **status**: Estado actual de la mesa.
 - **location**: Ubicación física de la mesa dentro de la sede.
 - **Métodos**:
 - **updateStatus(TableStatus status)**: Actualiza el estado de la mesa.
 - **assignToHeadquarter(HeadquarterId headquarterId)**: Asigna la mesa a una sede específica.
 - **changeCapacity(Capacity capacity)**: Modifica la capacidad de la mesa.

2. Entities:

- **Chair**: Entidad que representa cada silla asociada a una mesa.
 - **Atributos**:

- **chairNumber**: Número identificativo de la silla dentro de la mesa.
- **status**: Estado actual de la silla.
- **Métodos**:
 - **updateStatus(ChairStatus status)**: Actualiza el estado de la silla.
 - **isOccupied()**: Verifica si la silla está ocupada.

3. Value Objects:

- **TableNumber**: Representa el número identificativo de una mesa.
- **Capacity**: Representa la capacidad de una mesa.
- **TableStatus**: Enumera los estados posibles de una mesa.
- **ChairStatus**: Enumera los estados posibles de una silla.
- **Location**: Representa la ubicación física de una mesa dentro de la sede.
- **HeadquarterId**: Representa el identificador único de una sede.

4.2.2.6.2. Bounded Context Database Design Diagram.

El diseño de la base de datos para el **Table Management Bounded Context** refleja la estructura del dominio, asegurando que las entidades y relaciones definidas en la **Domain Layer** se representen de manera eficiente en el modelo relacional.

TABLES		
bigint	id	PK
integer	table_number	
bigint	headquarter_id	FK
integer	capacity	
varchar(20)	status	
varchar(50)	name	

varchar(50)	zone	
integer	x_position	
integer	y_position	
datetime	created_at	
datetime	updated_at	



1 a muchos



CHAIRS		
bigint	id	PK
bigint	table_id	FK
integer	chair_number	

varchar(10)	status	
-------------	--------	--

Este diseño incluye las siguientes tablas principales:

1. Tables:

- Representa las mesas en el sistema.
- **Atributos principales:**
 - **id**: Identificador único de la mesa.
 - **table_number**: Número de la mesa dentro de la sede.
 - **headquarter_id**: Identificador de la sede a la que pertenece la mesa.
 - **capacity**: Capacidad de la mesa en términos de personas.
 - **status**: Estado actual de la mesa (AVAILABLE, OCCUPIED, RESERVED, MAINTENANCE).
 - **zone**: Zona de la cafetería donde está ubicada la mesa.
 - **x_position**: Coordenada X de la mesa en el plano.
 - **y_position**: Coordenada Y de la mesa en el plano.

2. Chairs:

- Representa las sillas asociadas a las mesas.
- **Atributos principales:**
 - **id**: Identificador único de la silla.
 - **table_id**: Identificador de la mesa a la que pertenece la silla.
 - **chair_number**: Número de la silla dentro de la mesa.
 - **status**: Estado actual de la silla (OCCUPIED, FREE).

4.2.3 Bounded Context: Branching Bounded Context

El **Branching Bounded Context** es responsable de gestionar la información de las sedes (headquarters) del sistema Tavolo. Este contexto asegura que las sedes puedan ser registradas, actualizadas y gestionadas de manera eficiente, incluyendo detalles como horarios de atención, ubicación geográfica, información de contacto y dirección. Este contexto es clave para garantizar que las operaciones relacionadas con las sedes sean consistentes y cumplan con los estándares de calidad y precisión requeridos.

4.2.3.1. Domain Layer

La **Domain Layer** del Branching Bounded Context encapsula la lógica de negocio relacionada con la gestión de sedes. En esta capa, se definen los elementos principales del dominio, como agregados, entidades y objetos de valor, que representan los conceptos clave del sistema.

La **Domain Layer** del Branching Bounded Context encapsula la lógica de negocio relacionada con la gestión de sedes. En esta capa, se definen los elementos principales del dominio, como agregados, entidades y objetos de valor, que representan los conceptos clave del sistema.

Aggregates

1. Headquarter

- **Propósito:** El agregado principal es la sede (**Headquarter**), que encapsula la lógica de negocio relacionada con la gestión de información de las sedes.
- **Atributos:**
 - **name:** Nombre de la sede, representado como un objeto de valor **NameHeadquarter**.
 - **contactNumbers:** Números de contacto de la sede, representados como un objeto de valor **ContactNumbers**.
 - **coordinates:** Coordenadas geográficas de la sede, representadas como un objeto de valor **Coordinates**.
 - **schedule:** Horarios de atención de la sede, representados como una entidad **Schedule**.
 - **address:** Dirección de la sede, representada como un objeto de valor **StreetAddress**.
- **Métodos:**
 - **getOpeningTime():** Devuelve la hora de apertura de la sede.
 - **getClosingTime():** Devuelve la hora de cierre de la sede.
 - **getIntervalMinutes():** Devuelve el intervalo de tiempo en minutos entre reservas.
- **Características:**
 - Extiende **AuditableAbstractAggregateRoot**, lo que permite auditar cambios en las sedes.
 - Gestiona la relación entre los horarios, la dirección y los datos de contacto de la sede, asegurando consistencia y validación.

Entities

1. Schedule

- **Propósito:** Representa los horarios de atención de una sede.
- **Atributos:**
 - **businessHours:** Horarios de apertura y cierre, representados como un objeto de valor **BusinessHours**.
 - **intervalMinutes:** Intervalo de tiempo en minutos entre reservas.
- **Características:**
 - Permite definir y validar los horarios de atención de la sede.
 - Incluye validaciones para garantizar que los horarios sean consistentes (por ejemplo, la hora de apertura no puede ser posterior a la hora de cierre).

Value Objects

1. BusinessHours

- **Propósito:** Representa los horarios de apertura y cierre de una sede.
- **Atributos:**
 - **openingTime:** Hora de apertura.
 - **closingTime:** Hora de cierre.
- **Validaciones:**
 - La hora de apertura no puede ser nula.
 - La hora de cierre no puede ser nula.
 - La hora de apertura no puede ser posterior a la hora de cierre.

2. ContactNumbers

- **Propósito:** Representa los números de contacto de una sede.
- **Atributos:**
 - **landlinePhone:** Número de teléfono fijo.
 - **mobilePhone:** Número de teléfono móvil.
- **Validaciones:**

- Ambos números deben ser válidos y no nulos.

3. Coordinates

- **Propósito:** Representa las coordenadas geográficas de una sede.
- **Atributos:**
 - **latitude:** Latitud de la sede.
 - **longitude:** Longitud de la sede.
- **Validaciones:**
 - La latitud y la longitud no pueden ser nulas.

4. NameHeadquarter

- **Propósito:** Representa el nombre de una sede.
- **Atributos:**
 - **name:** Nombre de la sede.
- **Validaciones:**
 - El nombre no puede ser nulo ni vacío.
 - El nombre debe cumplir con un formato válido (solo letras y espacios).

5. StreetAddress

- **Propósito:** Representa la dirección de una sede.
- **Atributos:**
 - **street:** Calle.
 - **number:** Número de la dirección.
 - **city:** Ciudad.
 - **postalCode:** Código postal.
 - **country:** País.
- **Validaciones:**
 - Los campos **street**, **city**, **postalCode** y **country** no pueden ser nulos ni vacíos.
 - La calle debe cumplir con un formato válido (solo letras, espacios y puntos).

Relaciones entre componentes

- El agregado **Headquarter** actúa como el núcleo del dominio, gestionando las relaciones con los objetos de valor (**NameHeadquarter**, **ContactNumbers**, **Coordinates**, **StreetAddress**) y la entidad **Schedule**.
- Los objetos de valor encapsulan datos inmutables y validaciones específicas, asegurando consistencia en el dominio.
- La entidad **Schedule** permite modelar horarios complejos, incluyendo intervalos de tiempo entre reservas.

Esta estructura asegura que la lógica de negocio relacionada con la gestión de sedes sea robusta, consistente y fácil de mantener, cumpliendo con los requisitos del sistema.

4.2.3.2. Interface Layer

La **Interface Layer** del Branching Bounded Context expone los puntos de entrada al sistema a través de controladores REST y una ACL (Access Control Layer). Esta capa permite la interacción con las entidades del dominio mediante solicitudes HTTP, facilitando la comunicación entre los clientes y el sistema. Además, incluye recursos y transformadores que aseguran una representación adecuada de los datos y su conversión entre las capas de la aplicación.

Access Control Layer (ACL)

1. HeadquarterContextFacade

- **Propósito:** Proporciona una interfaz simplificada para interactuar con el dominio del Branching Bounded Context desde otros contextos. Permite consultar información clave de las sedes, como horarios de apertura y cierre, intervalos de servicio y existencia de una sede.
- **Métodos principales:**
 - `getOpeningTime(Long headquarterId)`: Devuelve la hora de apertura de una sede.
 - `getClosingTime(Long headquarterId)`: Devuelve la hora de cierre de una sede.
 - `getIntervalMinutes(Long headquarterId)`: Devuelve el intervalo de servicio en minutos de una sede.
 - `existsHeadquarter(Long headquarterId)`: Verifica si una sede existe en el sistema.
- **Dependencias:**
 - `HeadquarterQueryService`: Servicio encargado de manejar las consultas relacionadas con las sedes.

Controllers

Los controladores son responsables de manejar las solicitudes HTTP y delegar la lógica de negocio a los servicios correspondientes. A continuación, se describen los principales controladores:

1. HeadquarterController

- **Propósito:** Gestiona las operaciones relacionadas con las sedes.
- **Endpoints:**
 - `POST /api/v1/headquarters`: Crea una nueva sede.
 - `GET /api/v1/headquarters/{headquarterId}`: Obtiene los detalles de una sede específica por su ID.
 - `GET /api/v1/headquarters`: Obtiene la lista de todas las sedes.
- **Dependencias:**
 - `HeadquarterCommandService`: Servicio encargado de manejar los comandos relacionados con las sedes.
 - `HeadquarterQueryService`: Servicio encargado de manejar las consultas relacionadas con las sedes.

Resources

Los recursos representan los datos que se exponen a través de la API REST. Estos recursos son utilizados para estructurar las respuestas de los controladores y asegurar una representación clara y consistente de los datos. A continuación, se describen los principales recursos:

1. CreateHeadquarterResource

- **Propósito:** Representa los datos necesarios para crear una nueva sede.
- **Atributos:**
 - `name`: Nombre de la sede.
 - `landlinePhone`: Número de teléfono fijo.
 - `mobilePhone`: Número de teléfono móvil.
 - `latitude`: Latitud de la sede.
 - `longitude`: Longitud de la sede.
 - `street`: Calle de la dirección.
 - `number`: Número de la dirección.
 - `city`: Ciudad.
 - `postalCode`: Código postal.
 - `country`: País.
 - `openingTime`: Hora de apertura.

- **closingTime**: Hora de cierre.
- **intervalMinutes**: Intervalo de servicio en minutos.

2. HeadquarterResource

- **Propósito**: Representa una sede en el sistema.
- **Atributos**:
 - **id**: Identificador único de la sede.
 - **name**: Nombre de la sede.
 - **landlinePhone**: Número de teléfono fijo.
 - **mobilePhone**: Número de teléfono móvil.
 - **latitude**: Latitud de la sede.
 - **longitude**: Longitud de la sede.
 - **streetAddress**: Dirección completa de la sede.
 - **openingTime**: Hora de apertura.
 - **closingTime**: Hora de cierre.
 - **intervalMinutes**: Intervalo de servicio en minutos.

Transformers

Los transformadores son responsables de convertir las entidades del dominio en recursos y viceversa. Esto asegura que los datos expuestos a través de la API REST sean consistentes y estén en el formato esperado. A continuación, se describen los principales transformadores:

1. CreateHeadquarterCommandFromResourceAssembler

- **Propósito**: Convierte un recurso **CreateHeadquarterResource** en un comando **CreateHeadquarterCommand**.
- **Método principal**:
 - **toCommandFromResource(CreateHeadquarterResource resource)**: Transforma los datos de creación de una sede en un comando.

2. HeadquarterResourceFromEntityAssembler

- **Propósito**: Convierte una entidad **Headquarter** en un recurso **HeadquarterResource**.
- **Método principal**:
 - **toResourceFromEntity(Headquarter entity)**: Transforma una sede del dominio en un recurso.

Relaciones entre componentes

- El **HeadquarterController** utiliza los servicios de comandos (**HeadquarterCommandService**) y consultas (**HeadquarterQueryService**) para delegar la lógica de negocio.
- La **ACL (HeadquarterContextFacade)** proporciona una interfaz simplificada para consultar información clave de las sedes desde otros contextos.
- Los transformadores convierten las entidades del dominio en recursos para las respuestas HTTP y viceversa para las solicitudes entrantes.
- Los recursos estructuran los datos expuestos a los clientes, asegurando una representación clara y consistente.

Esta estructura asegura que la **Interface Layer** sea modular, reutilizable y fácil de mantener, facilitando la interacción entre los clientes y el sistema.

4.2.3.4. Infrastructure Layer

La **Infrastructure Layer** del Branching Bounded Context proporciona las implementaciones técnicas necesarias para soportar las operaciones del sistema relacionadas con la gestión de sedes. Esta capa incluye repositorios para la persistencia de datos y componentes que conectan la lógica de negocio con los recursos externos, como bases de datos. Su objetivo principal es garantizar que las operaciones de almacenamiento y recuperación de información sean eficientes, consistentes y seguras.

Persistencia (JPA Repositories)

1. HeadquarterRepository

- **Propósito:** Proporciona métodos para interactuar con la base de datos de sedes.
- **Métodos principales:**
 - `existsByName(NameHeadquarter name)`: Verifica si existe una sede con un nombre específico.
 - `existsByCoordinates(Coordinates coordinates)`: Verifica si existe una sede en una ubicación geográfica específica.
- **Características:**
 - Extiende `JpaRepository`, lo que permite realizar operaciones CRUD (Crear, Leer, Actualizar, Eliminar) sobre las entidades `Headquarter`.
 - Facilita la validación de unicidad para nombres y coordenadas de sedes, asegurando que no haya duplicados en el sistema.

Relaciones entre componentes

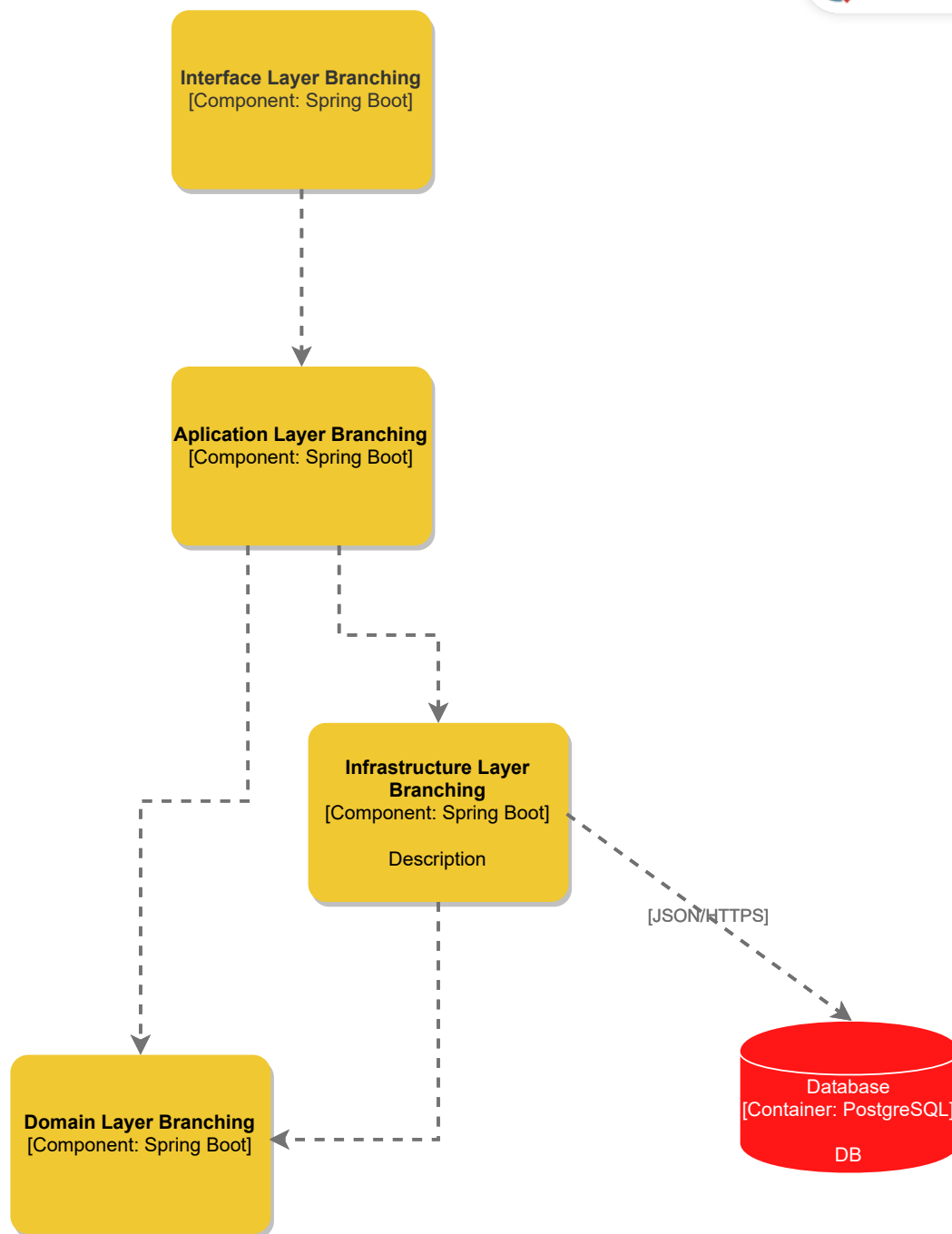
- **Persistencia:** El repositorio `HeadquarterRepository` proporciona acceso a los datos almacenados en la base de datos, permitiendo a las capas superiores (como la **Application Layer**) interactuar con las entidades del dominio.
- **Validación:** Los métodos `existsByName` y `existsByCoordinates` son utilizados para validar la unicidad de las sedes durante las operaciones de creación o actualización, asegurando la consistencia de los datos.

La **Infrastructure Layer** del Branching Bounded Context asegura que las operaciones relacionadas con la persistencia de datos sean robustas y confiables. Al proporcionar un repositorio especializado para las sedes, esta capa facilita la integración con la base de datos y garantiza que las reglas de negocio, como la unicidad de nombres y ubicaciones, se cumplan de manera eficiente. Esta estructura modular y reutilizable permite que el sistema sea escalable y fácil de mantener.

4.2.3.5. Bounded Context Software Architecture Component Level Diagrams

En esta sección se presenta el diagrama de componentes del **Branching Bounded Context**, el cual detalla los principales módulos y sus interacciones dentro del contexto delimitado. Este diagrama sigue el enfoque del C4 Model para representar los componentes clave, como servicios de aplicación, controladores, repositorios y servicios externos, junto con sus relaciones.

El propósito de este diagrama es proporcionar una visión clara y estructurada de cómo se organizan los componentes dentro del contexto, facilitando la comprensión de su arquitectura y permitiendo identificar puntos de integración y responsabilidades.



El **Branching Bounded Context** está compuesto por los siguientes módulos principales:

1. Application Layer:

- Coordina las operaciones de negocio relacionadas con la gestión de sedes.
- Incluye servicios de comandos y consultas que interactúan con la **Domain Layer** y la **Infrastructure Layer**.
- Maneja eventos relacionados con la creación y actualización de sedes.

2. Interface Layer:

- Expone los puntos de entrada al sistema a través de controladores REST.

- Incluye recursos y transformadores que aseguran una representación adecuada de los datos y su conversión entre las capas de la aplicación.

3. **Domain Layer:**

- Encapsula la lógica de negocio relacionada con la gestión de sedes.
- Define los agregados, entidades y objetos de valor que representan los conceptos clave del dominio.

4. **Infrastructure Layer:**

- Proporciona las implementaciones técnicas necesarias para soportar las operaciones del sistema.
- Incluye repositorios para la persistencia de datos y componentes que conectan la lógica de negocio con los recursos externos, como bases de datos.

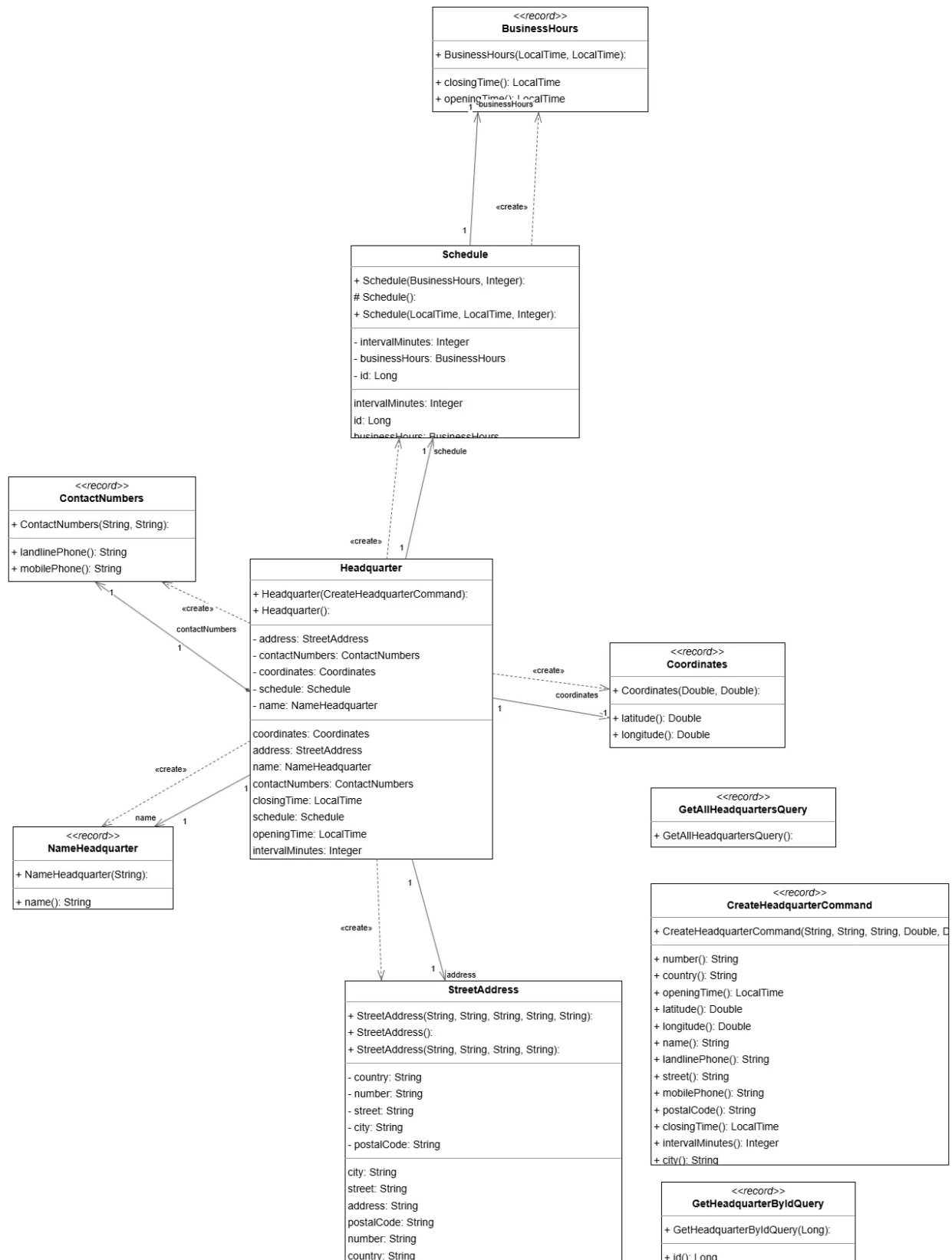
4.2.3.6. **Bounded Context Software Architecture Code Level Diagrams**

En este apartado se presentan los diagramas que ofrecen un mayor nivel de detalle sobre la implementación de los componentes del **Branching Bounded Context**. Estos diagramas están diseñados para ilustrar cómo se estructuran las clases, interfaces y relaciones dentro de las capas del contexto, proporcionando una visión técnica que facilita el desarrollo, mantenimiento y evolución del sistema.

4.2.3.6.1. **Bounded Context Domain Layer Class Diagrams**

El diagrama de clases correspondiente a la **Domain Layer** del **Branching Bounded Context** incluye las clases principales, como agregados, entidades y objetos de valor, así como las interfaces y enumeraciones que definen el comportamiento del dominio. También se destacan las relaciones entre estos elementos, como asociaciones, composiciones y dependencias.

El objetivo de este diagrama es proporcionar una representación detallada de la lógica de negocio encapsulada en la capa del dominio, asegurando que las reglas del negocio estén claramente definidas y alineadas con los requisitos del sistema.



Elementos principales del diagrama:

1. Aggregates:

- **Headquarter**: Agregado principal que encapsula la lógica de negocio relacionada con la gestión de sedes. Incluye atributos como **name**, **contactNumbers**, **coordinates**, **schedule** y **address**.

2. Entities:

- **Schedule**: Representa los horarios de atención de una sede, incluyendo atributos como **businessHours** e **intervalMinutes**.

3. Value Objects:

- **NameHeadquarter**: Representa el nombre de una sede.
- **ContactNumbers**: Representa los números de contacto de una sede.
- **Coordinates**: Representa las coordenadas geográficas de una sede.
- **StreetAddress**: Representa la dirección de una sede.
- **BusinessHours**: Representa los horarios de apertura y cierre de una sede.

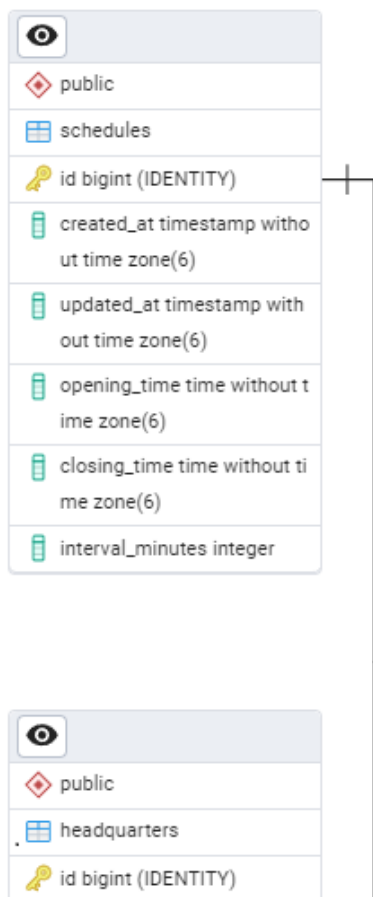
Relaciones destacadas:

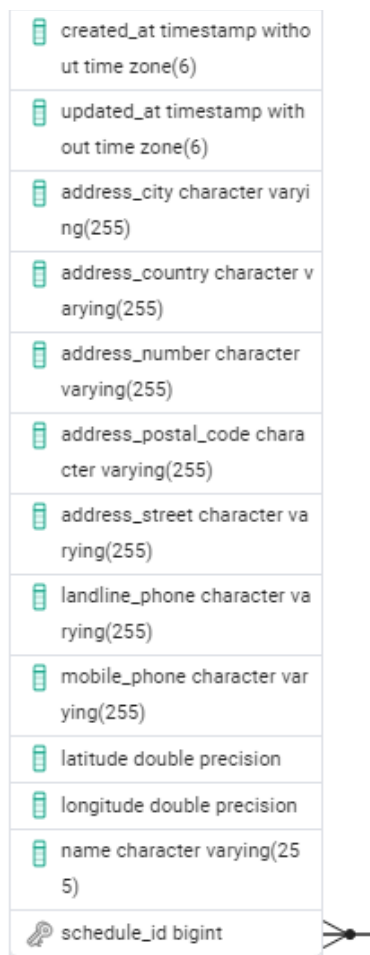
- El agregado **Headquarter** actúa como el núcleo del dominio, gestionando las relaciones con los objetos de valor (**NameHeadquarter**, **ContactNumbers**, **Coordinates**, **StreetAddress**) y la entidad **Schedule**.
- Los objetos de valor encapsulan datos inmutables y validaciones específicas, asegurando consistencia en el dominio.
- La entidad **Schedule** permite modelar horarios complejos, incluyendo intervalos de tiempo entre reservas.

4.2.3.6.2. Bounded Context Database Design Diagram.

4.2.3.6.2. Bounded Context Database Design Diagram

El diseño de la base de datos para el **Branching Bounded Context** refleja la estructura del dominio, asegurando que las entidades y relaciones definidas en la **Domain Layer** se representen de manera eficiente en el modelo relacional.





Este diseño incluye las siguientes tablas principales:

1. Headquarters

- **Propósito:** Representa las sedes registradas en el sistema.
- **Atributos principales:**
 - **id:** Identificador único de la sede.
 - **name:** Nombre de la sede.
 - **landline_phone:** Número de teléfono fijo de la sede.
 - **mobile_phone:** Número de teléfono móvil de la sede.
 - **latitude:** Latitud de la ubicación de la sede.
 - **longitude:** Longitud de la ubicación de la sede.
 - **address_street:** Calle de la dirección de la sede.
 - **address_number:** Número de la dirección.
 - **address_city:** Ciudad donde se encuentra la sede.
 - **address_postal_code:** Código postal de la sede.
 - **address_country:** País donde se encuentra la sede.
 - **schedule_id:** Relación con la tabla **Schedules**.

2. Schedules

- **Propósito:** Representa los horarios de atención de las sedes.

- **Atributos principales:**
 - **id**: Identificador único del horario.
 - **opening_time**: Hora de apertura de la sede.
 - **closing_time**: Hora de cierre de la sede.
 - **interval_minutes**: Intervalo de tiempo en minutos entre reservas.

Este diseño asegura que las operaciones relacionadas con la gestión de sedes sean eficientes y consistentes, facilitando la integración con la lógica de negocio definida en la **Domain Layer**.

4.2.4 Bounded Context: Booking Bounded Context

El **Booking Bounded Context** es responsable de gestionar las reservas de mesas en el sistema Tavolo. Este contexto asegura que los usuarios puedan realizar reservas de manera eficiente, gestionando la disponibilidad de mesas, horarios y slots de tiempo. Además, permite a los administradores supervisar y gestionar las reservas activas, asegurando que las operaciones relacionadas con las reservas sean consistentes y cumplan con los estándares de calidad y precisión requeridos.

4.2.4.1. Domain Layer

La **Domain Layer** del Booking Bounded Context encapsula la lógica de negocio relacionada con la gestión de reservas. En esta capa, se definen los elementos principales del dominio, como agregados, entidades, objetos de valor, comandos, consultas y eventos, que representan los conceptos clave del sistema.

Aggregates

1. Booking

- **Propósito:** Representa una reserva realizada por un cliente para una mesa específica en una fecha y horario determinado.
- **Atributos:**
 - **userId**: Identificador del cliente que realiza la reserva, representado como un objeto de valor **UserId**.
 - **tableId**: Mesa reservada, representada como una referencia a la entidad **Table**.
 - **bookingDate**: Fecha de la reserva.
 - **bookingSlots**: Conjunto de intervalos de tiempo reservados, representados como una colección de entidades **BookingSlot**.
- **Características:**
 - Extiende **AuditTableAbstractAggregateRoot**, lo que permite auditar cambios en las reservas.
 - Gestiona la relación entre el cliente, la mesa y los slots reservados, asegurando consistencia y validación.

2. Table

- **Propósito:** Representa una mesa en una sede específica, incluyendo su disponibilidad y detalles.
- **Atributos:**
 - **tableDetails**: Detalles de la mesa, como número de mesa y cantidad de asientos, representados como un objeto de valor **TableDetails**.
 - **headquarterId**: Identificador de la sede a la que pertenece la mesa, representado como un objeto de valor **HeadquarterId**.
 - **status**: Estado actual de la mesa (**AVAILABLE**, **RESERVED**, **OCCUPIED**), representado como un objeto de valor **TableStatus**.
 - **availabilitySlots**: Conjunto de slots de disponibilidad generados para la mesa, representados como una colección de entidades **AvailabilitySlot**.

- **Características:**
 - Extiende `AuditableAbstractAggregateRoot`, lo que permite auditar cambios en las mesas.
 - Gestiona la generación y actualización de slots de disponibilidad.

Entities

1. AvailabilitySlot

- **Propósito:** Representa un intervalo de tiempo disponible para una mesa en una fecha específica.
- **Atributos:**
 - `dateOfSlot`: Fecha del slot.
 - `timeInterval`: Intervalo de tiempo del slot, representado como un objeto de valor `TimeSlot`.
 - `status`: Estado del slot (`AVAILABLE`, `RESERVED`), representado como un objeto de valor `ScheduleSlotStatus`.
- **Métodos:**
 - `updateStatus(ScheduleSlotStatus status)`: Actualiza el estado del slot.

2. BookingSlot

- **Propósito:** Representa un intervalo de tiempo reservado por un cliente.
- **Atributos:**
 - `timeInterval`: Intervalo de tiempo reservado, representado como un objeto de valor `TimeSlot`.

Value Objects

1. UserId

- **Propósito:** Representa el identificador único de un cliente.
- **Validaciones:**
 - El identificador no puede ser negativo.

2. HeadquarterId

- **Propósito:** Representa el identificador único de una sede.
- **Validaciones:**
 - El identificador no puede ser nulo ni negativo.

3. TableDetails

- **Propósito:** Representa los detalles de una mesa, como su número y cantidad de asientos.
- **Validaciones:**
 - El número de mesa no puede ser negativo.
 - La cantidad de asientos debe ser mayor a cero.

4. TableStatus

- **Propósito:** Enumera los estados posibles de una mesa (`AVAILABLE`, `RESERVED`, `OCCUPIED`).

5. TimeSlot

- **Propósito:** Representa un intervalo de tiempo con una hora de inicio y una hora de fin.
- **Validaciones:**
 - La hora de inicio y la hora de fin no pueden ser nulas.
 - La hora de inicio debe ser anterior a la hora de fin.

6. ScheduleSlotStatus

- **Propósito:** Enumera los estados posibles de un slot de disponibilidad (**AVAILABLE**, **RESERVED**).

7. MaximumDuration

- **Propósito:** Representa la duración máxima permitida para una reserva.
- **Validaciones:**
 - La duración debe ser un número positivo.
 - La duración no puede exceder las 5 horas.

Commands

1. CreateBookingCommand

- **Propósito:** Representa la solicitud para crear una nueva reserva.
- **Atributos:**
 - **clientId:** Identificador del cliente.
 - **tableId:** Identificador de la mesa.
 - **bookingDate:** Fecha de la reserva.
 - **slotIds:** Lista de identificadores de slots reservados.

2. CreateTableCommand

- **Propósito:** Representa la solicitud para crear una nueva mesa.
- **Atributos:**
 - **tableNumber:** Número de la mesa.
 - **seats:** Cantidad de asientos.
 - **headquartersId:** Identificador de la sede.

3. CreateTableScheduleCommand

- **Propósito:** Representa la solicitud para generar los slots de disponibilidad de una mesa.
- **Atributos:**
 - **tableId:** Identificador de la mesa.

Queries

1. GetAllBookingsQuery

- **Propósito:** Recupera todas las reservas registradas en el sistema.

2. GetAllTablesQuery

- **Propósito:** Recupera todas las mesas registradas en el sistema.

3. GetBookingByIdQuery

- **Propósito:** Recupera una reserva específica por su identificador.

4. GetTableByIdQuery

- **Propósito:** Recupera una mesa específica por su identificador.

5. GetTableScheduleByIdAndDateQuery

- **Propósito:** Recupera los slots de disponibilidad de una mesa para una fecha específica.

Events

1. SingleTableAvailabilitySlotsGeneratedEvent

- **Propósito:** Evento que se dispara cuando se generan los slots de disponibilidad para una mesa.
- **Atributos:**
 - `tableId`: Identificador de la mesa.

Relaciones entre componentes

- El agregado `Booking` actúa como el núcleo del dominio, gestionando las relaciones con las entidades `BookingSlot` y `Table`.
- El agregado `Table` gestiona la generación y actualización de los slots de disponibilidad (`AvailabilitySlot`).
- Los objetos de valor encapsulan datos inmutables y validaciones específicas, asegurando consistencia en el dominio.
- Los comandos y consultas permiten interactuar con el sistema de manera estructurada, facilitando la creación de reservas y la recuperación de información.
- Los eventos aseguran que las operaciones críticas, como la generación de slots de disponibilidad, sean comunicadas de manera eficiente a otros componentes del sistema.

Esta estructura asegura que la lógica de negocio relacionada con la gestión de reservas sea robusta, consistente y fácil de mantener, cumpliendo con los requisitos del sistema.

4.2.4.2. Interface Layer

La **Interface Layer** del Booking Bounded Context expone los puntos de entrada al sistema a través de controladores REST. Esta capa permite la interacción con las entidades del dominio mediante solicitudes HTTP, facilitando la comunicación entre los clientes y el sistema. Además, incluye recursos y transformadores que aseguran una representación adecuada de los datos y su conversión entre las capas de la aplicación.

Controllers

Los controladores son responsables de manejar las solicitudes HTTP y delegar la lógica de negocio a los servicios correspondientes. A continuación, se describen los principales controladores:

1. BookingController

- **Propósito:** Gestiona las operaciones relacionadas con las reservas.
- **Endpoints:**
 - `POST /api/v1/bookings`: Crea una nueva reserva.
 - `GET /api/v1/bookings/{id}`: Obtiene los detalles de una reserva específica por su ID.
 - `GET /api/v1/bookings`: Obtiene la lista de todas las reservas.
- **Dependencias:**
 - `BookingCommandService`: Servicio encargado de manejar los comandos relacionados con las reservas.
 - `BookingQueryService`: Servicio encargado de manejar las consultas relacionadas con las reservas.

2. TableController

- **Propósito:** Gestiona las operaciones relacionadas con las mesas.
- **Endpoints:**
 - `POST /api/v1/tables`: Crea una nueva mesa.
 - `GET /api/v1/tables/{tableId}`: Obtiene los detalles de una mesa específica por su ID.
 - `GET /api/v1/tables`: Obtiene la lista de todas las mesas.
 - `GET /api/v1/tables/{tableId}/schedule`: Obtiene los slots de disponibilidad de una mesa para una fecha específica.

- **Dependencias:**
 - **TableCommandService:** Servicio encargado de manejar los comandos relacionados con las mesas.
 - **TableQueryService:** Servicio encargado de manejar las consultas relacionadas con las mesas.

Resources

Los recursos representan los datos que se exponen a través de la API REST. Estos recursos son utilizados para estructurar las respuestas de los controladores y asegurar una representación clara y consistente de los datos. A continuación, se describen los principales recursos:

1. AvailabilitySlotResource

- **Propósito:** Representa un slot de disponibilidad de una mesa.
- **Atributos:**
 - **id:** Identificador único del slot.
 - **date:** Fecha del slot.
 - **startTime:** Hora de inicio del slot.
 - **endTime:** Hora de fin del slot.
 - **status:** Estado del slot (**AVAILABLE**, **RESERVED**).

2. BookingResource

- **Propósito:** Representa una reserva en el sistema.
- **Atributos:**
 - **id:** Identificador único de la reserva.
 - **clientId:** Identificador del cliente que realizó la reserva.
 - **tableId:** Identificador de la mesa reservada.
 - **bookingDate:** Fecha de la reserva.
 - **bookingSlots:** Lista de slots reservados.

3. BookingSlotResource

- **Propósito:** Representa un slot reservado por un cliente.
- **Atributos:**
 - **startTime:** Hora de inicio del slot reservado.
 - **endTime:** Hora de fin del slot reservado.

4. CreateBookingResource

- **Propósito:** Representa los datos necesarios para crear una nueva reserva.
- **Atributos:**
 - **clientId:** Identificador del cliente.
 - **tableId:** Identificador de la mesa.
 - **bookingDate:** Fecha de la reserva.
 - **slotIds:** Lista de identificadores de slots reservados.

5. CreateTableResource

- **Propósito:** Representa los datos necesarios para crear una nueva mesa.
- **Atributos:**
 - **headquarterId:** Identificador de la sede a la que pertenece la mesa.
 - **tableNumber:** Número de la mesa.
 - **seats:** Cantidad de asientos.

6. TableResource

- **Propósito:** Representa una mesa en el sistema.
- **Atributos:**
 - **id:** Identificador único de la mesa.
 - **headquarterId:** Identificador de la sede a la que pertenece la mesa.
 - **tableNumber:** Número de la mesa.
 - **seats:** Cantidad de asientos.
 - **status:** Estado actual de la mesa (**AVAILABLE**, **RESERVED**, **OCCUPIED**).

Transformers

Los transformadores son responsables de convertir las entidades del dominio en recursos y viceversa. Esto asegura que los datos expuestos a través de la API REST sean consistentes y estén en el formato esperado. A continuación, se describen los principales transformadores:

1. AvailabilitySlotResourceFromEntityAssembler

- **Propósito:** Convierte una entidad **AvailabilitySlot** en un recurso **AvailabilitySlotResource**.
- **Métodos principales:**
 - **toResourceFromEntity(AvailabilitySlot entity):** Transforma un slot de disponibilidad en un recurso.
 - **toResourceListFromEntities(List<AvailabilitySlot> entities):** Transforma una lista de slots de disponibilidad en una lista de recursos.

2. BookingResourceFromEntityAssembler

- **Propósito:** Convierte una entidad **Booking** en un recurso **BookingResource**.
- **Método principal:**
 - **toResourceFromEntity(Booking booking):** Transforma una reserva en un recurso.

3. CreateBookingCommandFromResourceAssembler

- **Propósito:** Convierte un recurso **CreateBookingResource** en un comando **CreateBookingCommand**.
- **Método principal:**
 - **toCommandFromResource(CreateBookingResource resource):** Transforma los datos de creación de una reserva en un comando.

4. CreateTableCommandFromResourceAssembler

- **Propósito:** Convierte un recurso **CreateTableResource** en un comando **CreateTableCommand**.
- **Método principal:**
 - **toCommandFromResource(CreateTableResource resource):** Transforma los datos de creación de una mesa en un comando.

5. TableResourceFromEntityAssembler

- **Propósito:** Convierte una entidad **Table** en un recurso **TableResource**.
- **Método principal:**
 - **toResourceFromEntity(Table entity):** Transforma una mesa en un recurso.

Relaciones entre componentes

- Los controladores (**BookingController**, **TableController**) utilizan los servicios de comandos y consultas para delegar la lógica de negocio.
- Los transformadores convierten las entidades del dominio en recursos para las respuestas HTTP y viceversa para las solicitudes entrantes.

- Los recursos estructuran los datos expuestos a los clientes, asegurando una representación clara y consistente.

Esta estructura asegura que la **Interface Layer** sea modular, reutilizable y fácil de mantener, facilitando la interacción entre los clientes y el sistema.

4.2.4.3. Application Layer

La **Application Layer** del Booking Bounded Context actúa como un intermediario entre la **Domain Layer** y las capas externas, como la **Interface Layer** y la **Infrastructure Layer**. Su propósito principal es coordinar las operaciones de negocio, manejar comandos y consultas, orquestar la lógica de aplicación y garantizar que las reglas del dominio se cumplan de manera consistente. Además, esta capa incluye manejadores de eventos y servicios externos (ACL) para interactuar con otros contextos delimitados.

Command Services

Los servicios de comandos son responsables de ejecutar operaciones que modifican el estado del sistema. A continuación, se describen los principales servicios de comandos:

1. BookingCommandServiceImpl

- **Propósito:** Gestiona las operaciones relacionadas con la creación de reservas.
- **Métodos principales:**
 - `handle(CreateBookingCommand command)`: Crea una nueva reserva, validando la disponibilidad de los slots y asegurando que las reglas de negocio, como la duración máxima de la reserva, se cumplan.
- **Validaciones:**
 - Verifica que el cliente exista utilizando el servicio externo `ExternalUserService`.
 - Valida que los slots solicitados estén disponibles y sean consecutivos.
 - Asegura que la duración total de la reserva no exceda las 2 horas.
- **Dependencias:**
 - `BookingRepository`: Persistencia de reservas.
 - `TableRepository`: Gestión de mesas y sus slots de disponibilidad.
 - `ExternalUserService`: Verifica la existencia del cliente.

2. TableCommandServiceImpl

- **Propósito:** Gestiona las operaciones relacionadas con la creación de mesas y la generación de slots de disponibilidad.
- **Métodos principales:**
 - `handle(CreateTableCommand command)`: Crea una nueva mesa en una sede específica, validando que la sede exista y que no haya duplicados.
 - `handle(CreateTableScheduleCommand command)`: Genera los slots de disponibilidad para una mesa, basándose en los horarios y el intervalo de la sede.
- **Validaciones:**
 - Verifica que la sede exista utilizando el servicio externo `ExternalHeadquarterService`.
 - Asegura que no existan mesas duplicadas en la misma sede.
- **Dependencias:**
 - `TableRepository`: Persistencia de mesas y sus slots de disponibilidad.
 - `ExternalHeadquarterService`: Obtiene información de la sede, como horarios y intervalos.

Query Services

Los servicios de consultas son responsables de recuperar información del sistema sin modificar su estado. A continuación, se describen los principales servicios de consultas:

1. BookingQueryServiceImpl

- **Propósito:** Gestiona las consultas relacionadas con las reservas.
- **Métodos principales:**
 - `handle(GetBookingByIdQuery query)`: Recupera una reserva específica por su ID.
 - `handle(GetAllBookingsQuery query)`: Recupera todas las reservas registradas en el sistema.
- **Dependencias:**
 - `BookingRepository`: Persistencia de reservas.

2. TableQueryServiceImpl

- **Propósito:** Gestiona las consultas relacionadas con las mesas y sus slots de disponibilidad.
- **Métodos principales:**
 - `handle(GetTableByIdQuery query)`: Recupera una mesa específica por su ID.
 - `handle(GetAllTablesQuery query)`: Recupera todas las mesas registradas en el sistema.
 - `handle(GetTableScheduleByIdAndDateQuery query)`: Recupera los slots de disponibilidad de una mesa para una fecha específica.
- **Dependencias:**
 - `TableRepository`: Persistencia de mesas y sus slots de disponibilidad.

Event Handlers

Los manejadores de eventos son responsables de reaccionar a eventos específicos del sistema. A continuación, se describe el principal manejador de eventos:

1. SingleTableAvailabilitySlotsGeneratedEventHandler

- **Propósito:** Maneja el evento `SingleTableAvailabilitySlotsGeneratedEvent`, que se dispara cuando se generan los slots de disponibilidad para una mesa.
- **Método principal:**
 - `on(SingleTableAvailabilitySlotsGeneratedEvent event)`: Genera los slots de disponibilidad para una mesa específica.
- **Dependencias:**
 - `TableCommandService`: Servicio encargado de manejar los comandos relacionados con las mesas.

Outbound Services (ACL)

Los servicios externos proporcionan funcionalidades auxiliares que no forman parte del dominio principal. A continuación, se describen los principales servicios externos:

1. ExternalHeadquarterService

- **Propósito:** Interactúa con el Branching Bounded Context para obtener información de las sedes.
- **Métodos principales:**
 - `getHeadquarterOpeningTime(Long headquarterId)`: Obtiene la hora de apertura de una sede.
 - `getHeadquarterClosingTime(Long headquarterId)`: Obtiene la hora de cierre de una sede.
 - `getHeadquarterIntervalMinutes(Long headquarterId)`: Obtiene el intervalo de servicio en minutos de una sede.
 - `existsHeadquarter(Long headquarterId)`: Verifica si una sede existe.

2. ExternalUserService

- **Propósito:** Interactúa con el IAM Bounded Context para verificar la existencia de usuarios.

- **Método principal:**
 - `existUserById(Long userId)`: Verifica si un usuario existe en el sistema.

Relaciones entre componentes

- Los **Command Services** interactúan con los repositorios para modificar el estado del sistema y con los servicios externos (ACL) para validar información de otros contextos.
- Los **Query Services** interactúan únicamente con los repositorios para recuperar información del sistema.
- Los **Event Handlers** reaccionan a eventos del dominio para ejecutar lógica adicional, como la generación de slots de disponibilidad.
- Los **Outbound Services** (ACL) permiten la integración con otros contextos delimitados, como el Branching Bounded Context y el IAM Bounded Context.

La **Application Layer** del Booking Bounded Context asegura que las operaciones relacionadas con la gestión de reservas y mesas sean robustas, consistentes y fáciles de mantener. Al coordinar la lógica de negocio, manejar eventos y facilitar la integración con otros contextos, esta capa garantiza que las reglas del dominio se cumplan de manera eficiente y que el sistema sea escalable y extensible.

4.2.4.4. Infrastructure Layer

La **Infrastructure Layer** del Booking Bounded Context proporciona las implementaciones técnicas necesarias para soportar las operaciones del sistema relacionadas con la gestión de reservas y mesas. Esta capa incluye repositorios para la persistencia de datos y componentes que conectan la lógica de negocio con los recursos externos, como bases de datos. Su objetivo principal es garantizar que las operaciones de almacenamiento y recuperación de información sean eficientes, consistentes y seguras.

Persistencia (JPA Repositories)

1. BookingRepository

- **Propósito:** Proporciona métodos para interactuar con la base de datos de reservas.
- **Características:**
 - Extiende `JpaRepository`, lo que permite realizar operaciones CRUD (Crear, Leer, Actualizar, Eliminar) sobre las entidades `Booking`.
 - Facilita la persistencia y recuperación de reservas, asegurando que las operaciones relacionadas con las reservas sean eficientes y confiables.

2. TableRepository

- **Propósito:** Proporciona métodos para interactuar con la base de datos de mesas y sus slots de disponibilidad.
- **Métodos principales:**
 - `existsByHeadquarterIdAndTableDetails_TableNumber(HeadquarterId headquarterId, Integer tableDetails_tableNumber)`: Verifica si existe una mesa con un número específico en una sede.
 - `findAvailabilitySlotsByTableIdAndDate(Long tableId, LocalDate date)`: Recupera los slots de disponibilidad de una mesa para una fecha específica.
 - `findByIdWithSlotsForUpdate(Long id)`: Recupera una mesa junto con sus slots de disponibilidad utilizando un bloqueo pesimista para evitar modificaciones concurrentes.
- **Características:**
 - Extiende `JpaRepository`, lo que permite realizar operaciones CRUD sobre las entidades `Table`.
 - Incluye consultas personalizadas para manejar la relación entre mesas y sus slots de disponibilidad.

- Utiliza un bloqueo pesimista (**PESSIMISTIC_WRITE**) para garantizar la consistencia de los datos durante las operaciones críticas, como la creación de reservas.

Relaciones entre componentes

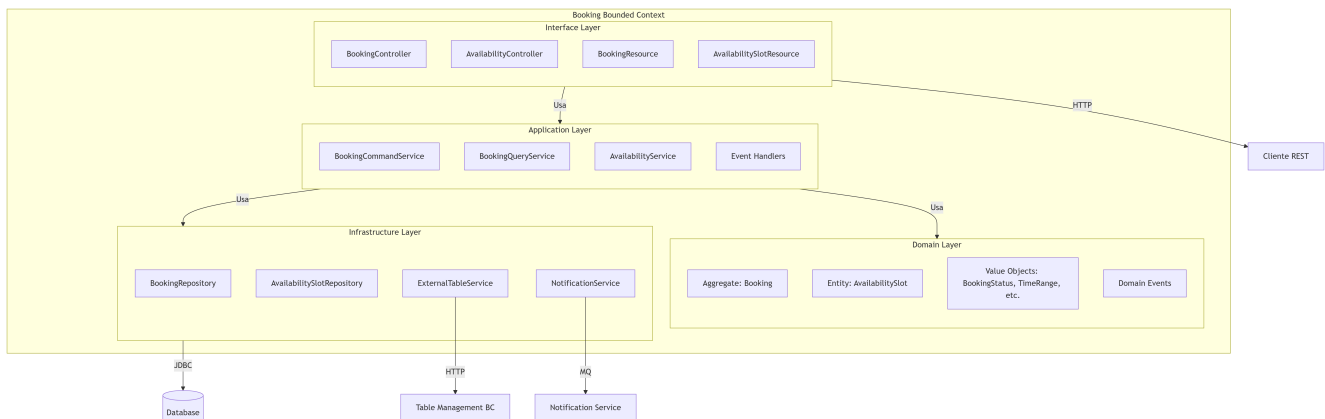
- **Persistencia:** Los repositorios **BookingRepository** y **TableRepository** proporcionan acceso a los datos almacenados en la base de datos, permitiendo a las capas superiores (como la **Application Layer**) interactuar con las entidades del dominio.
- **Validación:** Los métodos personalizados en **TableRepository** son utilizados para validar la existencia de mesas y recuperar información específica, como los slots de disponibilidad, asegurando la consistencia de los datos durante las operaciones de negocio.

La **Infrastructure Layer** del Booking Bounded Context asegura que las operaciones relacionadas con la persistencia de datos sean robustas y confiables. Al proporcionar repositorios especializados para las reservas y mesas, esta capa facilita la integración con la base de datos y garantiza que las reglas de negocio, como la validación de slots de disponibilidad y la unicidad de mesas, se cumplan de manera eficiente. Esta estructura modular y reutilizable permite que el sistema sea escalable y fácil de mantener.

4.2.4.5. Bounded Context Software Architecture Component Level Diagrams

En esta sección se presenta el diagrama de componentes del **Booking Bounded Context**, el cual detalla los principales módulos y sus interacciones dentro del contexto delimitado. Este diagrama sigue el enfoque del C4 Model para representar los componentes clave, como servicios de aplicación, controladores, repositorios y servicios externos, junto con sus relaciones.

El propósito de este diagrama es proporcionar una visión clara y estructurada de cómo se organizan los componentes dentro del contexto, facilitando la comprensión de su arquitectura y permitiendo identificar puntos de integración y responsabilidades.



El **Booking Bounded Context** está compuesto por los siguientes módulos principales:

1. Application Layer:

- Coordina las operaciones de negocio relacionadas con la gestión de reservas y mesas.
- Incluye servicios de comandos y consultas que interactúan con la **Domain Layer** y la **Infrastructure Layer**.
- Maneja eventos relacionados con la creación de reservas y la generación de slots de disponibilidad.

2. Interface Layer:

- Expone los puntos de entrada al sistema a través de controladores REST.
- Incluye recursos y transformadores que aseguran una representación adecuada de los datos y su conversión entre las capas de la aplicación.

3. Domain Layer:

- Encapsula la lógica de negocio relacionada con la gestión de reservas y mesas.
- Define los agregados, entidades y objetos de valor que representan los conceptos clave del dominio.

4. Infrastructure Layer:

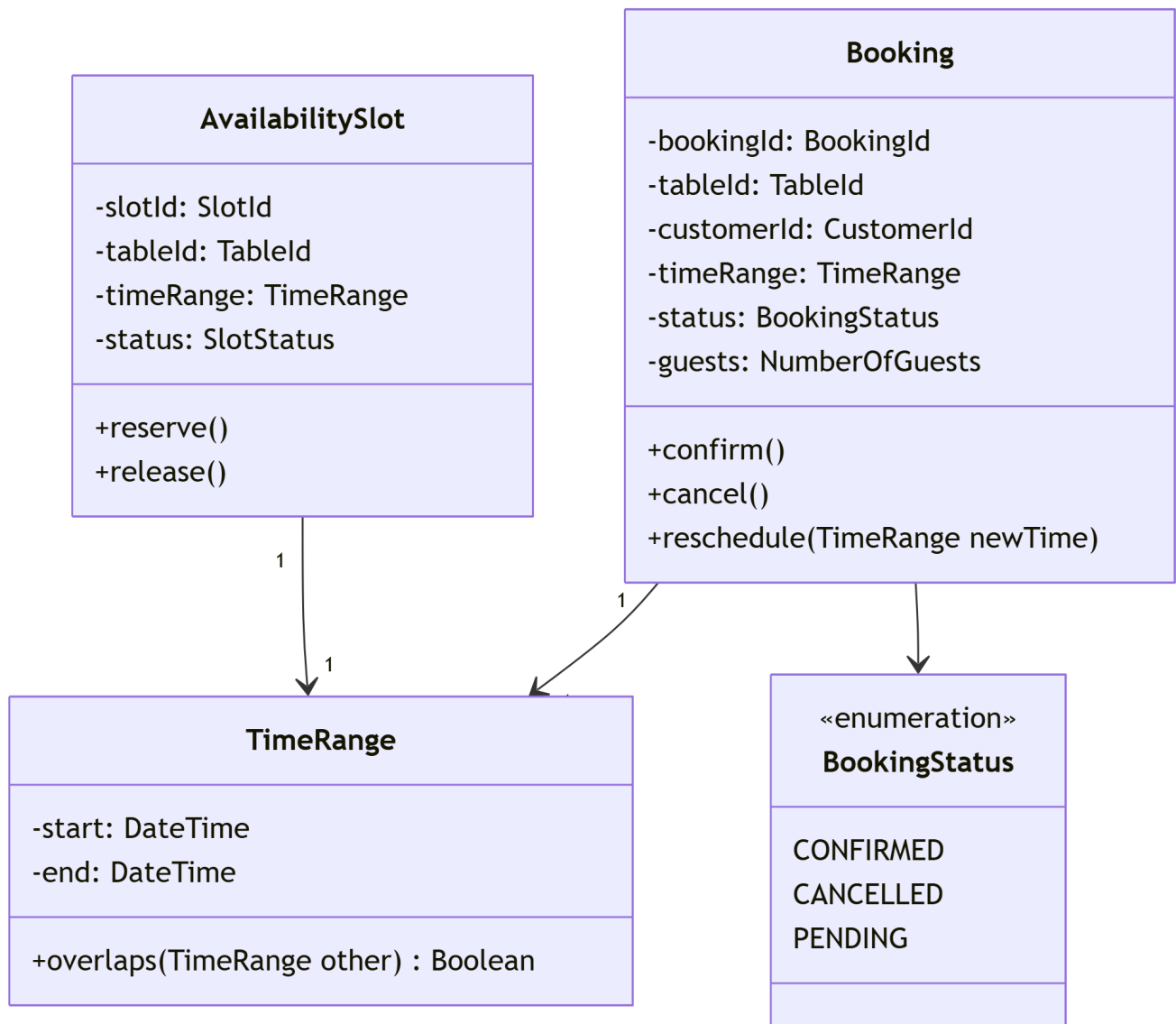
- Proporciona las implementaciones técnicas necesarias para soportar las operaciones del sistema.
- Incluye repositorios para la persistencia de datos y componentes que conectan la lógica de negocio con los recursos externos, como bases de datos.

4.2.4.6. Bounded Context Software Architecture Code Level Diagrams

En este apartado se presentan los diagramas que ofrecen un mayor nivel de detalle sobre la implementación de los componentes del **Booking Bounded Context**. Estos diagramas están diseñados para ilustrar cómo se estructuran las clases, interfaces y relaciones dentro de las capas del contexto, proporcionando una visión técnica que facilita el desarrollo, mantenimiento y evolución del sistema.

4.2.4.6.1. Bounded Context Domain Layer Class Diagrams

El diagrama de clases correspondiente a la **Domain Layer** del **Booking Bounded Context** incluye las clases principales, como agregados, entidades y objetos de valor, así como las interfaces y enumeraciones que definen el comportamiento del dominio. También se destacan las relaciones entre estos elementos, como asociaciones, composiciones y dependencias.



Elementos principales del diagrama:

1. Aggregates:

- **Booking**: Agregado principal que encapsula la lógica de negocio relacionada con las reservas. Incluye atributos como **userId**, **tableId**, **bookingDate** y **bookingSlots**.
- **Table**: Agregado que representa una mesa en una sede específica, incluyendo su disponibilidad y detalles.

2. Entities:

- **AvailabilitySlot**: Representa un intervalo de tiempo disponible para una mesa en una fecha específica.
- **BookingSlot**: Representa un intervalo de tiempo reservado por un cliente.

3. Value Objects:

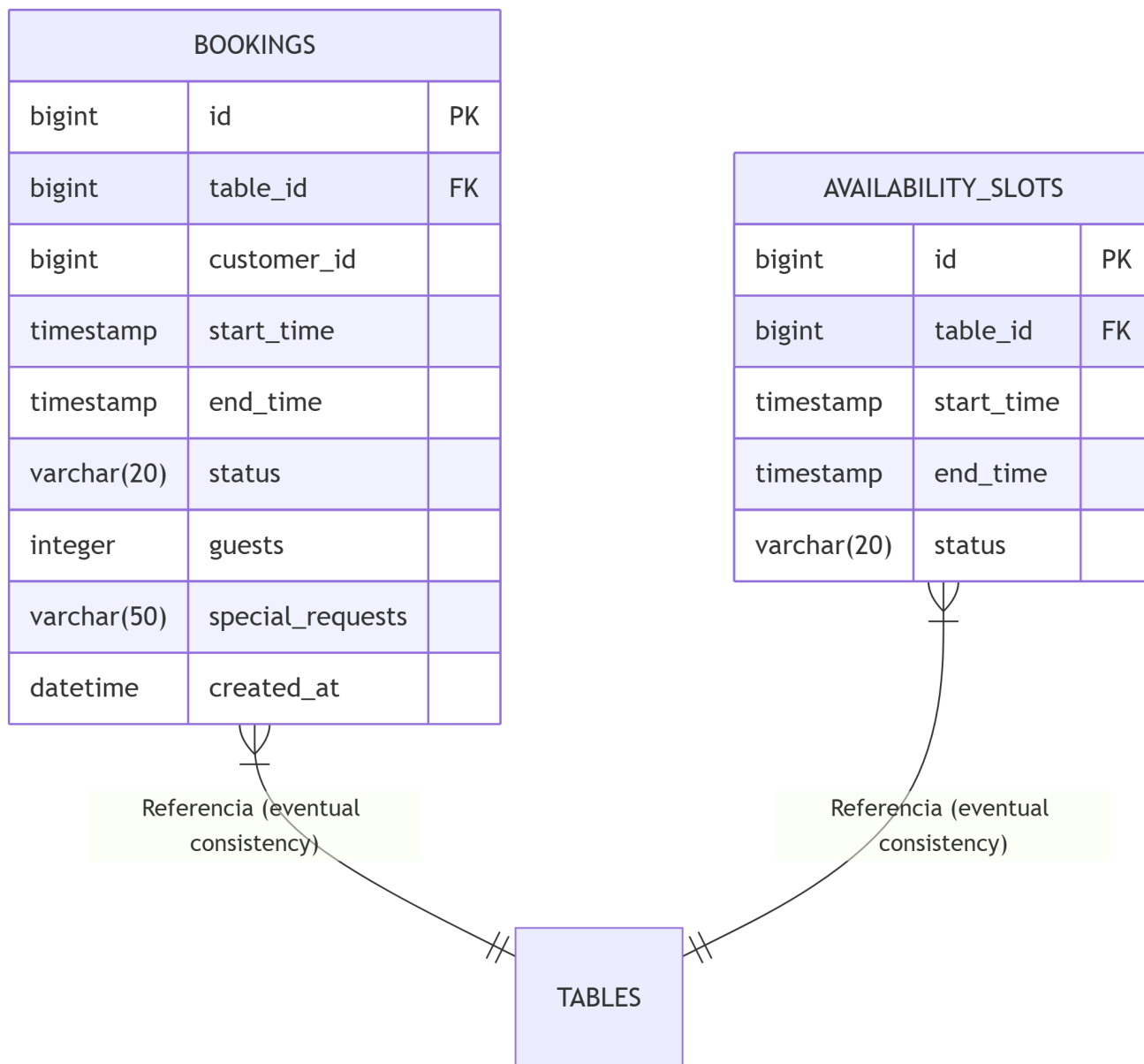
- **UserId**: Representa el identificador único de un cliente.
- **HeadquarterId**: Representa el identificador único de una sede.
- **TableDetails**: Representa los detalles de una mesa, como su número y cantidad de asientos.
- **TimeSlot**: Representa un intervalo de tiempo con una hora de inicio y una hora de fin.
- **TableStatus**: Enumera los estados posibles de una mesa (**AVAILABLE**, **RESERVED**, **OCCUPIED**).
- **ScheduleSlotStatus**: Enumera los estados posibles de un slot de disponibilidad (**AVAILABLE**, **RESERVED**).

Relaciones destacadas:

- El agregado **Booking** gestiona las relaciones con las entidades **BookingSlot** y **Table**.
- El agregado **Table** gestiona la generación y actualización de los slots de disponibilidad (**AvailabilitySlot**).
- Los objetos de valor encapsulan datos inmutables y validaciones específicas, asegurando consistencia en el dominio.

4.2.4.6.2. Bounded Context Database Design Diagram.

El diseño de la base de datos para el **Booking Bounded Context** refleja la estructura del dominio, asegurando que las entidades y relaciones definidas en la **Domain Layer** se representen de manera eficiente en el modelo relacional.



Este diseño incluye las siguientes tablas principales:

1. Bookings:

- Representa las reservas realizadas por los clientes.
- Atributos principales:
 - **id**: Identificador único de la reserva.
 - **user_id**: Identificador del cliente que realizó la reserva.
 - **table_id**: Identificador de la mesa reservada.
 - **booking_date**: Fecha de la reserva.

2. Tables:

- Representa las mesas disponibles en las sedes.
- Atributos principales:
 - **id**: Identificador único de la mesa.
 - **headquarter_id**: Identificador de la sede a la que pertenece la mesa.
 - **table_number**: Número de la mesa.
 - **seats**: Cantidad de asientos.
 - **status**: Estado actual de la mesa.

3. AvailabilitySlots:

- Representa los slots de disponibilidad de las mesas.
- Atributos principales:
 - **id**: Identificador único del slot.
 - **table_id**: Identificador de la mesa asociada.
 - **date_of_slot**: Fecha del slot.
 - **start_time**: Hora de inicio del slot.
 - **end_time**: Hora de fin del slot.
 - **status**: Estado del slot (**AVAILABLE**, **RESERVED**).

4. BookingSlots:

- Representa los slots reservados por los clientes.
- Atributos principales:
 - **id**: Identificador único del slot reservado.
 - **booking_id**: Identificador de la reserva asociada.
 - **start_time**: Hora de inicio del slot reservado.
 - **end_time**: Hora de fin del slot reservado.

Voy a completar el análisis táctico de Domain-Driven Design para los bounded contexts faltantes: Table Management, Menu Management y IoT Monitoring. Desarrollaré cada sección siguiendo la estructura que ya tiene tu informe.

El **Table Management Bounded Context** es responsable de gestionar la información y el estado de las mesas en las cafeterías. Este contexto se encarga de registrar, actualizar y consultar el estado de las mesas, así como su capacidad, ubicación dentro de la sede y otros atributos relevantes. Es el núcleo central que permite la visualización en tiempo real del estado de ocupación de las mesas para los usuarios.

4.2.5 Bounded Context: Menu Management Bounded Context

El **Menu Management Bounded Context** es responsable de gestionar la información de los menús disponibles en las cafeterías. Este contexto asegura que los productos, categorías y precios puedan ser registrados, actualizados y consultados de manera eficiente, permitiendo a los comensales visualizar el menú digital antes de acudir al establecimiento. Este contexto es clave para proporcionar una experiencia completa a los usuarios de la plataforma Tavolo.

4.2.5.1. Domain Layer

La **Domain Layer** del Menu Management Bounded Context encapsula la lógica de negocio relacionada con la gestión de menús. En esta capa, se definen los elementos principales del dominio, como agregados, entidades y objetos de valor, que representan los conceptos clave del sistema.

Aggregates

1. Menu

- **Propósito:** El agregado principal es el menú (**Menu**), que encapsula la lógica de negocio relacionada con la gestión de productos disponibles en una sede específica.
- **Atributos:**
 - **headquarterId**: Identificador de la sede a la que pertenece el menú, representado como un objeto de valor **HeadquarterId**.
 - **name**: Nombre del menú, representado como un objeto de valor **MenuName**.
 - **description**: Descripción del menú, representada como un objeto de valor **MenuDescription**.
 - **categories**: Conjunto de categorías que componen el menú, representadas como una colección de entidades **Category**.
- **Métodos:**
 - **addCategory(Category category)**: Agrega una nueva categoría al menú.
 - **removeCategory(CategoryId categoryId)**: Elimina una categoría del menú.
 - **updateName(MenuName name)**: Actualiza el nombre del menú.
 - **updateDescription(MenuDescription description)**: Actualiza la descripción del menú.
- **Características:**
 - Extiende **AuditableAbstractAggregateRoot**, lo que permite auditar cambios en los menús.
 - Gestiona la relación entre la sede y las categorías de productos, asegurando consistencia y validación.

Entities

1. Category

- **Propósito:** La entidad **Category** representa una categoría de productos dentro de un menú.
- **Atributos:**
 - **id**: Identificador único de la categoría, representado como un objeto de valor **CategoryId**.
 - **name**: Nombre de la categoría, representado como un objeto de valor **CategoryName**.
 - **description**: Descripción de la categoría, representada como un objeto de valor **CategoryDescription**.
 - **products**: Conjunto de productos que pertenecen a la categoría, representados como una colección de entidades **Product**.
- **Métodos:**
 - **addProduct(Product product)**: Agrega un nuevo producto a la categoría.
 - **removeProduct(ProductId productId)**: Elimina un producto de la categoría.
 - **updateName(CategoryName name)**: Actualiza el nombre de la categoría.
 - **updateDescription(CategoryDescription description)**: Actualiza la descripción de la categoría.

2. Product

- **Propósito:** La entidad **Product** representa un producto dentro de una categoría del menú.
- **Atributos:**
 - **id**: Identificador único del producto, representado como un objeto de valor **ProductId**.
 - **name**: Nombre del producto, representado como un objeto de valor **ProductName**.
 - **description**: Descripción del producto, representada como un objeto de valor **ProductDescription**.
 - **price**: Precio del producto, representado como un objeto de valor **Money**.
 - **imageUrl**: URL de la imagen del producto, representada como un objeto de valor **ImageUrl**.
 - **availability**: Disponibilidad del producto, representada como un objeto de valor **Availability**.
- **Métodos:**
 - **updateName(ProductName name)**: Actualiza el nombre del producto.

- `updateDescription(ProductDescription description)`: Actualiza la descripción del producto.
- `updatePrice(Money price)`: Actualiza el precio del producto.
- `updateImageUrl(ImageUrl imageUrl)`: Actualiza la URL de la imagen del producto.
- `updateAvailability(Availability availability)`: Actualiza la disponibilidad del producto.

Value Objects

1. HeadquarterId

- **Propósito:** Representa el identificador único de una sede.
- **Validaciones:**
 - El identificador no puede ser nulo ni negativo.

2. MenuName

- **Propósito:** Representa el nombre de un menú.
- **Validaciones:**
 - El nombre no puede ser nulo ni vacío.
 - El nombre no puede exceder los 100 caracteres.

3. MenuDescription

- **Propósito:** Representa la descripción de un menú.
- **Validaciones:**
 - La descripción no puede exceder los 500 caracteres.

4. CategoryId

- **Propósito:** Representa el identificador único de una categoría.
- **Validaciones:**
 - El identificador no puede ser nulo ni negativo.

5. CategoryName

- **Propósito:** Representa el nombre de una categoría.
- **Validaciones:**
 - El nombre no puede ser nulo ni vacío.
 - El nombre no puede exceder los 50 caracteres.

6. CategoryDescription

- **Propósito:** Representa la descripción de una categoría.
- **Validaciones:**
 - La descripción no puede exceder los 200 caracteres.

7. ProductId

- **Propósito:** Representa el identificador único de un producto.
- **Validaciones:**
 - El identificador no puede ser nulo ni negativo.

8. ProductName

- **Propósito:** Representa el nombre de un producto.
- **Validaciones:**
 - El nombre no puede ser nulo ni vacío.

- El nombre no puede exceder los 100 caracteres.

9. ProductDescription

- **Propósito:** Representa la descripción de un producto.
- **Validaciones:**
 - La descripción no puede exceder los 500 caracteres.

10. Money

- **Propósito:** Representa un importe monetario con una divisa.
- **Atributos:**
 - **amount:** Importe.
 - **currency:** Divisa.
- **Validaciones:**
 - El importe no puede ser negativo.
 - La divisa debe ser válida según las divisas soportadas.

11. ImageUrl

- **Propósito:** Representa la URL de una imagen.
- **Validaciones:**
 - La URL debe ser válida.

12. Availability

- **Propósito:** Enumera los estados posibles de disponibilidad de un producto.
- **Valores:**
 - **AVAILABLE:** El producto está disponible para ser ordenado.
 - **SOLD_OUT:** El producto está agotado temporalmente.
 - **DISCONTINUED:** El producto ha sido discontinuado.

Commands

1. CreateMenuCommand

- **Propósito:** Comando para crear un nuevo menú para una sede específica.
- **Atributos:**
 - **headquarterId:** ID de la sede.
 - **name:** Nombre del menú.
 - **description:** Descripción del menú.

2. AddCategoryCommand

- **Propósito:** Comando para agregar una nueva categoría a un menú existente.
- **Atributos:**
 - **menuId:** ID del menú.
 - **name:** Nombre de la categoría.
 - **description:** Descripción de la categoría.

3. AddProductCommand

- **Propósito:** Comando para agregar un nuevo producto a una categoría existente.
- **Atributos:**
 - **categoryId:** ID de la categoría.
 - **name:** Nombre del producto.

- **description**: Descripción del producto.
- **price**: Precio del producto.
- **imageUrl**: URL de la imagen del producto.
- **availability**: Disponibilidad inicial del producto.

Queries

1. GetMenuByIdQuery

- **Propósito**: Recupera un menú específico por su ID.
- **Atributos**:
 - **menuId**: ID del menú a consultar.

2. GetMenuByHeadquarterQuery

- **Propósito**: Recupera el menú asociado a una sede específica.
- **Atributos**:
 - **headquarterId**: ID de la sede a consultar.

3. GetCategoryByIdQuery

- **Propósito**: Recupera una categoría específica por su ID.
- **Atributos**:
 - **categoryId**: ID de la categoría a consultar.

4. GetProductByIdQuery

- **Propósito**: Recupera un producto específico por su ID.
- **Atributos**:
 - **productId**: ID del producto a consultar.

Events

1. MenuCreatedEvent

- **Propósito**: Evento que se dispara cuando se crea un nuevo menú.
- **Atributos**:
 - **menuId**: ID del menú creado.
 - **headquarterId**: ID de la sede asociada.

2. CategoryAddedEvent

- **Propósito**: Evento que se dispara cuando se agrega una nueva categoría a un menú.
- **Atributos**:
 - **menuId**: ID del menú.
 - **categoryId**: ID de la categoría agregada.

3. ProductAddedEvent

- **Propósito**: Evento que se dispara cuando se agrega un nuevo producto a una categoría.
- **Atributos**:
 - **categoryId**: ID de la categoría.
 - **productId**: ID del producto agregado.

Relaciones entre componentes

- El agregado **Menu** actúa como el núcleo del dominio, gestionando las relaciones con las entidades **Category** y, a través de ellas, con las entidades **Product**.
- Las entidades **Category** y **Product** tienen sus propias identidades y ciclos de vida dentro del contexto.
- Los objetos de valor encapsulan datos inmutables y validaciones específicas, asegurando consistencia en el dominio.
- Los comandos, consultas y eventos permiten interactuar con el sistema de manera estructurada, facilitando la creación y gestión de menús, categorías y productos.

4.2.5.2. Interface Layer

La **Interface Layer** del Menu Management Bounded Context expone los puntos de entrada al sistema a través de controladores REST. Esta capa permite la interacción con las entidades del dominio mediante solicitudes HTTP, facilitando la comunicación entre los clientes y el sistema. Además, incluye recursos y transformadores que aseguran una representación adecuada de los datos y su conversión entre las capas de la aplicación.

Access Control Layer (ACL)

1. MenuManagementContextFacade

- **Propósito:** Proporciona una interfaz simplificada para interactuar con el dominio del Menu Management Bounded Context desde otros contextos. Permite consultar información sobre menús, categorías y productos.
- **Métodos principales:**
 - `getMenuByHeadquarter(Long headquarterId)`: Obtiene el menú asociado a una sede específica.
 - `existsMenu(Long menuId)`: Verifica si un menú existe en el sistema.
- **Dependencias:**
 - `MenuQueryService`: Servicio encargado de manejar las consultas relacionadas con los menús.

Controllers

Los controladores son responsables de manejar las solicitudes HTTP y delegar la lógica de negocio a los servicios correspondientes. A continuación, se describen los principales controladores:

1. MenuController

- **Propósito:** Gestiona las operaciones relacionadas con los menús.
- **Endpoints:**
 - `POST /api/v1/menus`: Crea un nuevo menú.
 - `GET /api/v1/menus/{menuId}`: Obtiene los detalles de un menú específico por su ID.
 - `GET /api/v1/headquarters/{headquarterId}/menu`: Obtiene el menú asociado a una sede específica.
- **Dependencias:**
 - `MenuCommandService`: Servicio encargado de manejar los comandos relacionados con los menús.
 - `MenuQueryService`: Servicio encargado de manejar las consultas relacionadas con los menús.

2. CategoryController

- **Propósito:** Gestiona las operaciones relacionadas con las categorías de los menús.
- **Endpoints:**
 - `POST /api/v1/menus/{menuId}/categories`: Agrega una nueva categoría a un menú existente.
 - `GET /api/v1/categories/{categoryId}`: Obtiene los detalles de una categoría específica por su ID.
- **Dependencias:**

- **CategoryCommandService**: Servicio encargado de manejar los comandos relacionados con las categorías.
- **CategoryQueryService**: Servicio encargado de manejar las consultas relacionadas con las categorías.

3. ProductController

- **Propósito**: Gestiona las operaciones relacionadas con los productos de las categorías.
- **Endpoints**:
 - **POST /api/v1/categories/{categoryId}/products**: Agrega un nuevo producto a una categoría existente.
 - **GET /api/v1/products/{productId}**: Obtiene los detalles de un producto específico por su ID.
 - **PUT /api/v1/products/{productId}/availability**: Actualiza la disponibilidad de un producto.
- **Dependencias**:
 - **ProductCommandService**: Servicio encargado de manejar los comandos relacionados con los productos.
 - **ProductQueryService**: Servicio encargado de manejar las consultas relacionadas con los productos.

Resources

Los recursos representan los datos que se exponen a través de la API REST. Estos recursos son utilizados para estructurar las respuestas de los controladores y asegurar una representación clara y consistente de los datos. A continuación, se describen los principales recursos:

1. CreateMenuResource

- **Propósito**: Representa los datos necesarios para crear un nuevo menú.
- **Atributos**:
 - **headquarterId**: ID de la sede asociada al menú.
 - **name**: Nombre del menú.
 - **description**: Descripción del menú.

2. MenuResource

- **Propósito**: Representa un menú en el sistema, incluyendo sus categorías y productos.
- **Atributos**:
 - **id**: ID único del menú.
 - **headquarterId**: ID de la sede asociada.
 - **name**: Nombre del menú.
 - **description**: Descripción del menú.
 - **categories**: Lista de categorías del menú.

3. CreateCategoryResource

- **Propósito**: Representa los datos necesarios para crear una nueva categoría.
- **Atributos**:
 - **name**: Nombre de la categoría.
 - **description**: Descripción de la categoría.

4. CategoryResource

- **Propósito**: Representa una categoría en el sistema, incluyendo sus productos.
- **Atributos**:
 - **id**: ID único de la categoría.

- **name**: Nombre de la categoría.
- **description**: Descripción de la categoría.
- **products**: Lista de productos de la categoría.

5. CreateProductResource

- **Propósito**: Representa los datos necesarios para crear un nuevo producto.
- **Atributos**:
 - **name**: Nombre del producto.
 - **description**: Descripción del producto.
 - **price**: Precio del producto.
 - **currency**: Moneda del precio.
 - **imageUrl**: URL de la imagen del producto.
 - **availability**: Disponibilidad inicial del producto.

6. ProductResource

- **Propósito**: Representa un producto en el sistema.
- **Atributos**:
 - **id**: ID único del producto.
 - **name**: Nombre del producto.
 - **description**: Descripción del producto.
 - **price**: Precio del producto.
 - **currency**: Moneda del precio.
 - **imageUrl**: URL de la imagen del producto.
 - **availability**: Disponibilidad actual del producto.

Transformers

Los transformadores son responsables de convertir las entidades del dominio en recursos y viceversa. Esto asegura que los datos expuestos a través de la API REST sean consistentes y estén en el formato esperado. A continuación, se describen los principales transformadores:

1. CreateMenuCommandFromResourceAssembler

- **Propósito**: Convierte un recurso **CreateMenuResource** en un comando **CreateMenuCommand**.
- **Método principal**:
 - **toCommandFromResource(CreateMenuResource resource)**: Transforma los datos de creación de un menú en un comando.

2. MenuResourceFromEntityAssembler

- **Propósito**: Convierte una entidad **Menu** en un recurso **MenuResource**.
- **Método principal**:
 - **toResourceFromEntity(Menu entity)**: Transforma un menú del dominio en un recurso.

3. CreateCategoryCommandFromResourceAssembler

- **Propósito**: Convierte un recurso **CreateCategoryResource** en un comando **AddCategoryCommand**.
- **Método principal**:
 - **toCommandFromResource(CreateCategoryResource resource, Long menuId)**: Transforma los datos de creación de una categoría en un comando.

4. CategoryResourceFromEntityAssembler

- **Propósito:** Convierte una entidad `Category` en un recurso `CategoryResource`.
- **Método principal:**
 - `toResourceFromEntity(Category entity)`: Transforma una categoría del dominio en un recurso.

5. `CreateProductCommandFromResourceAssembler`

- **Propósito:** Convierte un recurso `CreateProductResource` en un comando `AddProductCommand`.
- **Método principal:**
 - `toCommandFromResource(CreateProductResource resource, Long categoryId)`: Transforma los datos de creación de un producto en un comando.

6. `ProductResourceFromEntityAssembler`

- **Propósito:** Convierte una entidad `Product` en un recurso `ProductResource`.
- **Método principal:**
 - `toResourceFromEntity(Product entity)`: Transforma un producto del dominio en un recurso.

4.2.5.3. Application Layer

La **Application Layer** del Menu Management Bounded Context actúa como un intermediario entre la **Domain Layer** y las capas externas, como la **Interface Layer** y la **Infrastructure Layer**. Su propósito principal es coordinar las operaciones de negocio, manejar comandos y consultas, y orquestar la lógica de aplicación sin exponer directamente los detalles del dominio.

Command Services

Los servicios de comandos son responsables de ejecutar operaciones que modifican el estado del sistema. A continuación, se describen los principales servicios de comandos:

1. `MenuCommandServiceImpl`

- **Propósito:** Gestiona las operaciones relacionadas con la creación y modificación de menús.
- **Métodos principales:**
 - `handle(CreateMenuCommand command)`: Crea un nuevo menú para una sede específica.
- **Validaciones:**
 - Verifica que la sede exista utilizando el servicio externo `ExternalHeadquarterService`.
 - Asegura que no exista ya un menú para la sede especificada.
- **Dependencias:**
 - `MenuRepository`: Persistencia de menús.
 - `ExternalHeadquarterService`: Verifica la existencia de la sede.

2. `CategoryCommandServiceImpl`

- **Propósito:** Gestiona las operaciones relacionadas con la creación y modificación de categorías.
- **Métodos principales:**
 - `handle(AddCategoryCommand command)`: Agrega una nueva categoría a un menú existente.
- **Validaciones:**
 - Verifica que el menú exista.
 - Asegura que no exista ya una categoría con el mismo nombre en el menú.
- **Dependencias:**
 - `MenuRepository`: Persistencia de menús.
 - `CategoryRepository`: Persistencia de categorías.

3. `ProductCommandServiceImpl`

- **Propósito:** Gestiona las operaciones relacionadas con la creación y modificación de productos.
- **Métodos principales:**
 - `handle(AddProductCommand command)`: Agrega un nuevo producto a una categoría existente.
 - `handle(UpdateProductAvailabilityCommand command)`: Actualiza la disponibilidad de un producto existente.
- **Validaciones:**
 - Verifica que la categoría exista.
 - Asegura que no exista ya un producto con el mismo nombre en la categoría.
- **Dependencias:**
 - `CategoryRepository`: Persistencia de categorías.
 - `ProductRepository`: Persistencia de productos.

Query Services

Los servicios de consultas son responsables de recuperar información del sistema sin modificar su estado. A continuación, se describen los principales servicios de consultas:

1. MenuQueryServiceImpl

- **Propósito:** Gestiona las consultas relacionadas con los menús.
- **Métodos principales:**
 - `handle(GetMenuByIdQuery query)`: Recupera un menú específico por su ID.
 - `handle(GetMenuByHeadquarterQuery query)`: Recupera el menú asociado a una sede específica.
- **Dependencias:**
 - `MenuRepository`: Persistencia de menús.

2. CategoryQueryServiceImpl

- **Propósito:** Gestiona las consultas relacionadas con las categorías de los menús.
- **Métodos principales:**
 - `handle(GetCategoryByIdQuery query)`: Recupera una categoría específica por su ID.
- **Dependencias:**
 - `CategoryRepository`: Persistencia de categorías.

3. ProductQueryServiceImpl

- **Propósito:** Gestiona las consultas relacionadas con los productos de las categorías.
- **Métodos principales:**
 - `handle(GetProductByIdQuery query)`: Recupera un producto específico por su ID.
- **Dependencias:**
 - `ProductRepository`: Persistencia de productos.

Event Handlers

Los manejadores de eventos son responsables de reaccionar a eventos específicos del sistema. A continuación, se describen los principales manejadores de eventos:

1. MenuCreatedEventHandler

- **Propósito:** Maneja el evento `MenuCreatedEvent`, que se dispara cuando se crea un nuevo menú.
- **Método principal:**
 - `on(MenuCreatedEvent event)`: Realiza acciones adicionales cuando se crea un menú, como notificar a otros sistemas.
- **Dependencias:**
 - `NotificationService`: Servicio para enviar notificaciones.

Outbound Services (ACL)

Los servicios externos proporcionan funcionalidades auxiliares que no forman parte del dominio principal. A continuación, se describen los principales servicios externos:

1. ExternalHeadquarterService

- **Propósito:** Interactúa con el Branching Bounded Context para verificar la existencia de sedes.
- **Método principal:**
 - `existsHeadquarter(Long headquarterId)`: Verifica si una sede existe en el sistema.

4.2.5.4. Infrastructure Layer

La **Infrastructure Layer** del Menu Management Bounded Context proporciona las implementaciones técnicas necesarias para soportar las operaciones del sistema relacionadas con la gestión de menús, categorías y productos. Esta capa incluye repositorios para la persistencia de datos y componentes que conectan la lógica de negocio con los recursos externos, como bases de datos. Su objetivo principal es garantizar que las operaciones de almacenamiento y recuperación de información sean eficientes, consistentes y seguras.

Persistencia (JPA Repositories)

1. MenuRepository

- **Propósito:** Proporciona métodos para interactuar con la base de datos de menús.
- **Métodos principales:**
 - `findByHeadquarterId(HeadquarterId headquarterId)`: Recupera el menú asociado a una sede específica.
 - `existsByHeadquarterId(HeadquarterId headquarterId)`: Verifica si existe un menú para una sede específica.
- **Características:**
 - Extiende `JpaRepository`, lo que permite realizar operaciones CRUD sobre las entidades `Menu`.
 - Facilita la recuperación de menús según diferentes criterios de filtro.

2. CategoryRepository

- **Propósito:** Proporciona métodos para interactuar con la base de datos de categorías.
- **Métodos principales:**
 - `findByMenuIdAndName(Long menuId, CategoryName name)`: Recupera una categoría específica por su menú y nombre.
 - `existsByMenuIdAndName(Long menuId, CategoryName name)`: Verifica si existe una categoría con un nombre específico en un menú.
- **Características:**
 - Extiende `JpaRepository`, lo que permite realizar operaciones CRUD sobre las entidades `Category`.
 - Facilita la recuperación de categorías según diferentes criterios de filtro.

3. ProductRepository

- **Propósito:** Proporciona métodos para interactuar con la base de datos de productos.
- **Métodos principales:**
 - `findByCategoryIdAndName(Long categoryId, ProductName name)`: Recupera un producto específico por su categoría y nombre.
 - `existsByCategoryIdAndName(Long categoryId, ProductName name)`: Verifica si existe un producto con un nombre específico en una categoría.
- **Características:**
 - Extiende `JpaRepository`, lo que permite realizar operaciones CRUD sobre las entidades `Product`.

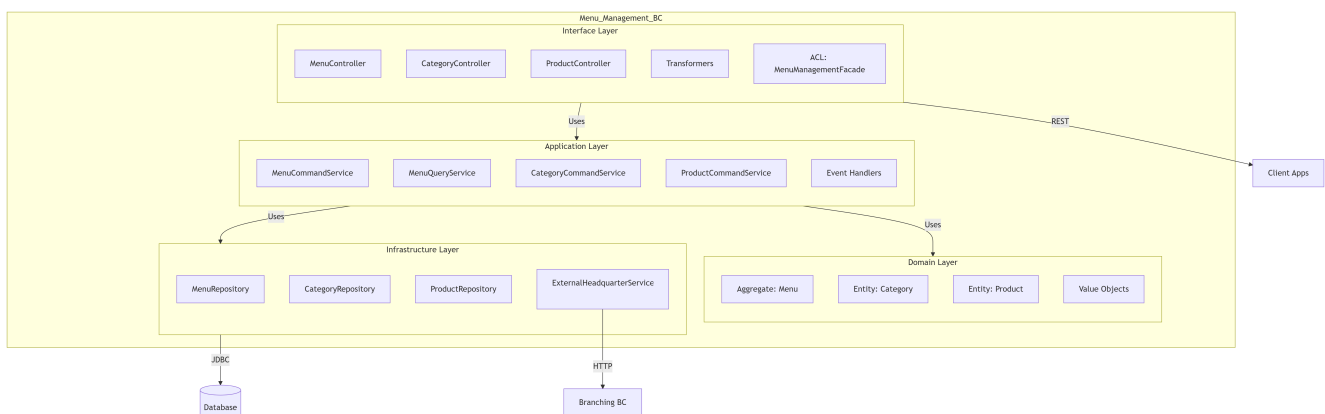
- Facilita la recuperación de productos según diferentes criterios de filtro.

Relaciones entre componentes

- **Persistencia:** Los repositorios **MenuRepository**, **CategoryRepository** y **ProductRepository** proporcionan acceso a los datos almacenados en la base de datos, permitiendo a las capas superiores (como la **Application Layer**) interactuar con las entidades del dominio.
- **Validación:** Los métodos personalizados en los repositorios son utilizados para validar la existencia y unicidad de menús, categorías y productos, asegurando la consistencia de los datos durante las operaciones de negocio.

4.2.5.5. Bounded Context Software Architecture Component Level Diagrams

En esta sección se presenta el diagrama de componentes del **Menu Management Bounded Context**, el cual detalla los principales módulos y sus interacciones dentro del contexto delimitado. Este diagrama sigue el enfoque del C4 Model para representar los componentes clave, como servicios de aplicación, controladores, repositorios y servicios externos, junto con sus relaciones.



El **Menu Management Bounded Context** está compuesto por los siguientes módulos principales:

1. Application Layer:

- Coordina las operaciones de negocio relacionadas con la gestión de menús, categorías y productos.
- Incluye servicios de comandos y consultas que interactúan con la **Domain Layer** y la **Infrastructure Layer**.
- Maneja eventos relacionados con la creación y modificación de menús, categorías y productos.

2. Interface Layer:

- Expone los puntos de entrada al sistema a través de controladores REST.
- Incluye recursos y transformadores que aseguran una representación adecuada de los datos y su conversión entre las capas de la aplicación.
- Proporciona una ACL (Access Control Layer) para facilitar la integración con otros contextos.

3. Domain Layer:

- Encapsula la lógica de negocio relacionada con la gestión de menús, categorías y productos.
- Define los agregados, entidades y objetos de valor que representan los conceptos clave del dominio.
- Incluye eventos de dominio que representan cambios significativos en el estado del sistema.

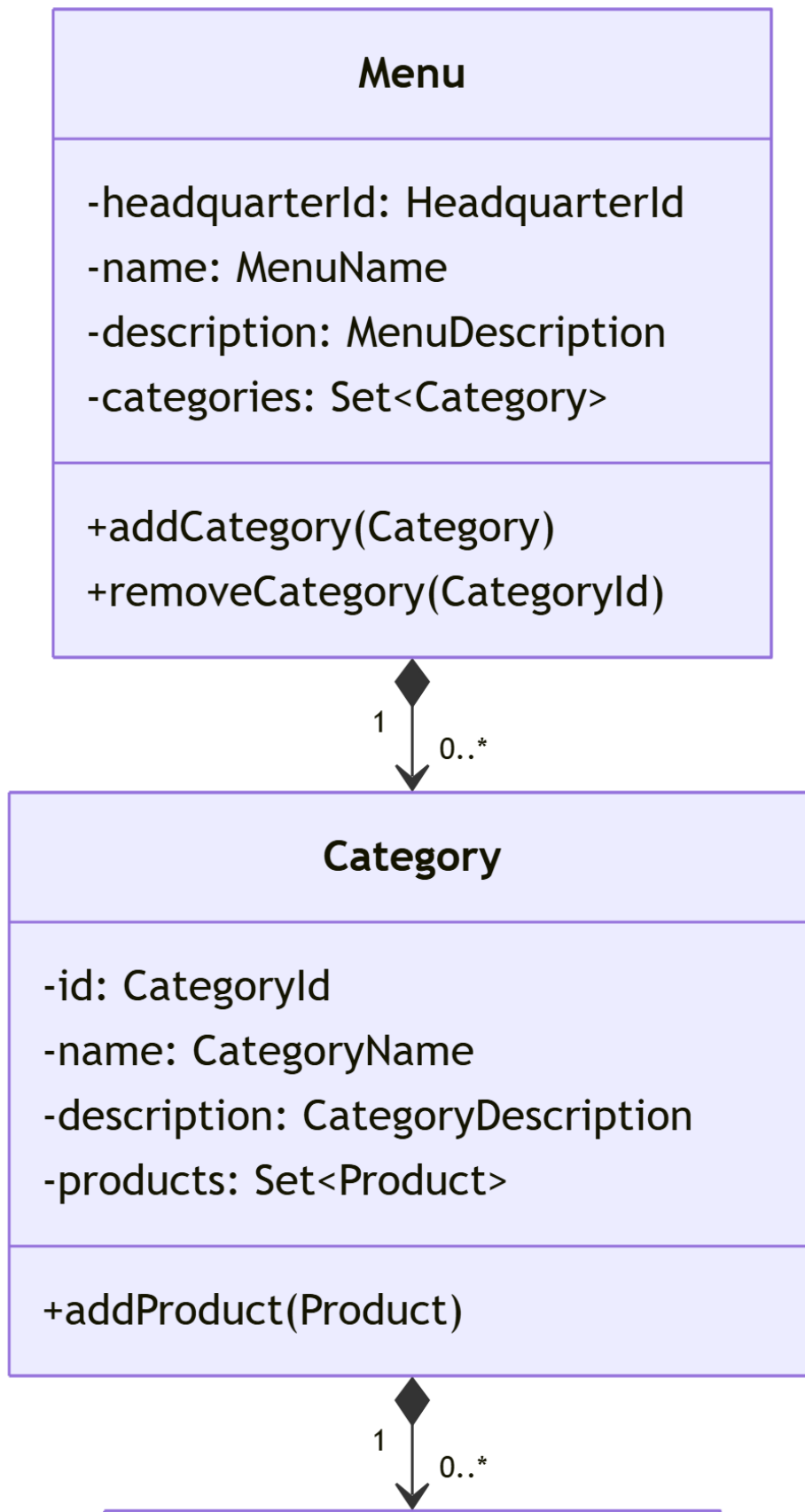
4. Infrastructure Layer:

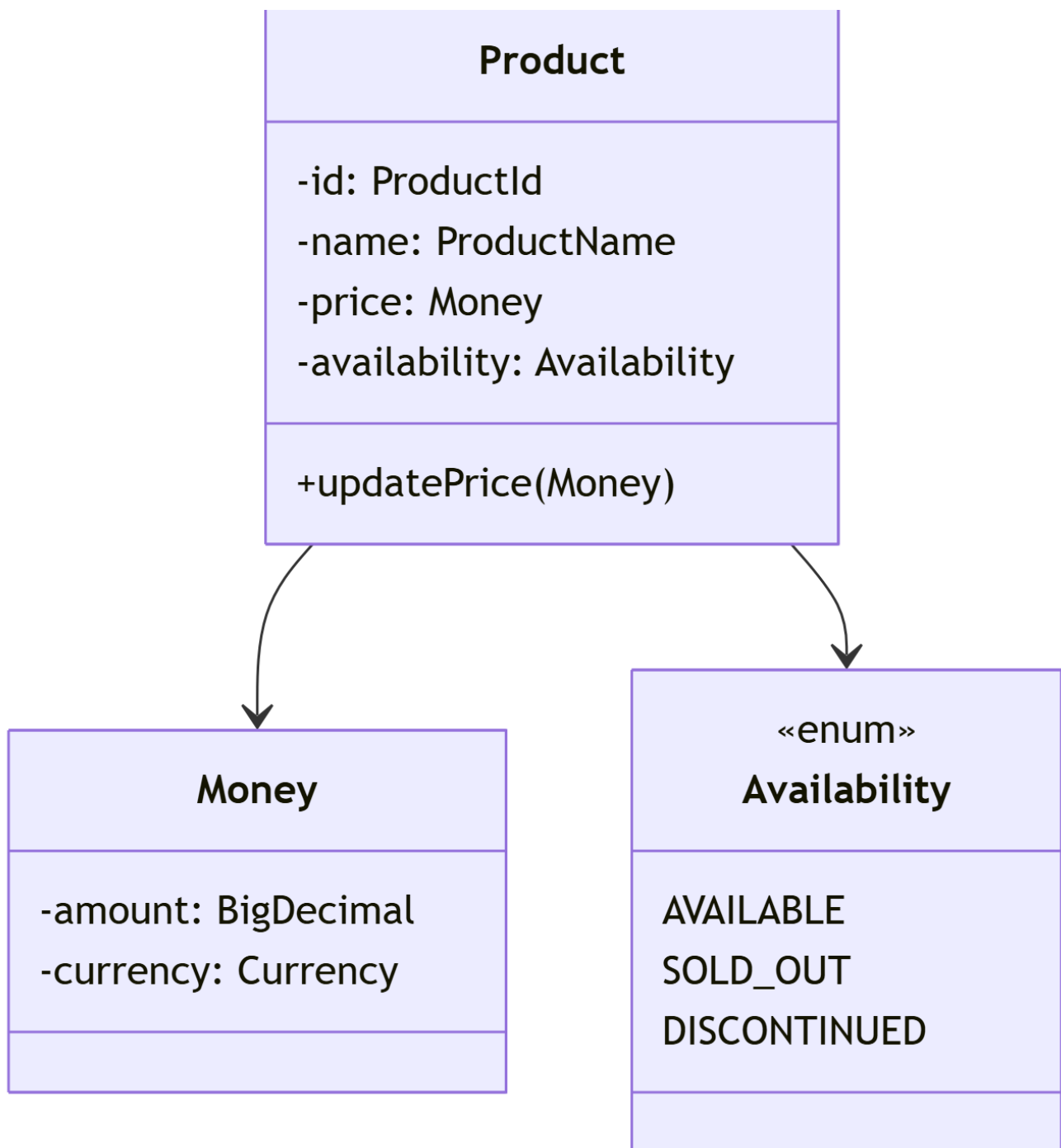
- Proporciona las implementaciones técnicas necesarias para soportar las operaciones del sistema.
- Incluye repositorios para la persistencia de datos y componentes que conectan la lógica de negocio con los recursos externos, como bases de datos.

4.2.5.6. Bounded Context Software Architecture Code Level Diagrams

4.2.5.6.1. Bounded Context Domain Layer Class Diagrams

El diagrama de clases correspondiente a la **Domain Layer** del **Menu Management Bounded Context** incluye las clases principales, como agregados, entidades y objetos de valor, así como las interfaces y enumeraciones que definen el comportamiento del dominio.





Elementos principales del diagrama:

1. Aggregates:

- **Menu**: Agregado principal que encapsula la lógica de negocio relacionada con la gestión de menús.
 - **Atributos**:
 - `headquarterId`: ID de la sede a la que pertenece el menú.
 - `name`: Nombre del menú.
 - `description`: Descripción del menú.
 - `categories`: Conjunto de categorías que componen el menú.
 - **Métodos**:
 - `addCategory(Category category)`: Agrega una nueva categoría al menú.
 - `removeCategory(CategoryId categoryId)`: Elimina una categoría del menú.
 - `updateName(MenuName name)`: Actualiza el nombre del menú.
 - `updateDescription(MenuDescription description)`: Actualiza la descripción del menú.

2. Entities:

- **Category**: Entidad que representa una categoría de productos dentro de un menú.
 - **Atributos**:
 - **id**: ID único de la categoría.
 - **name**: Nombre de la categoría.
 - **description**: Descripción de la categoría.
 - **products**: Conjunto de productos que pertenecen a la categoría.
 - **Métodos**:
 - **addProduct(Product product)**: Agrega un nuevo producto a la categoría.
 - **removeProduct(ProductId productId)**: Elimina un producto de la categoría.
 - **updateName(CategoryName name)**: Actualiza el nombre de la categoría.
 - **updateDescription(CategoryDescription description)**: Actualiza la descripción de la categoría.
- **Product**: Entidad que representa un producto dentro de una categoría del menú.
 - **Atributos**:
 - **id**: ID único del producto.
 - **name**: Nombre del producto.
 - **description**: Descripción del producto.
 - **price**: Precio del producto.
 - **imageUrl**: URL de la imagen del producto.
 - **availability**: Disponibilidad del producto.
 - **Métodos**:
 - **updateName(ProductName name)**: Actualiza el nombre del producto.
 - **updateDescription(ProductDescription description)**: Actualiza la descripción del producto.
 - **updatePrice(Money price)**: Actualiza el precio del producto.
 - **updateImageUrl(ImageUrl imageUrl)**: Actualiza la URL de la imagen del producto.
 - **updateAvailability(Availability availability)**: Actualiza la disponibilidad del producto.

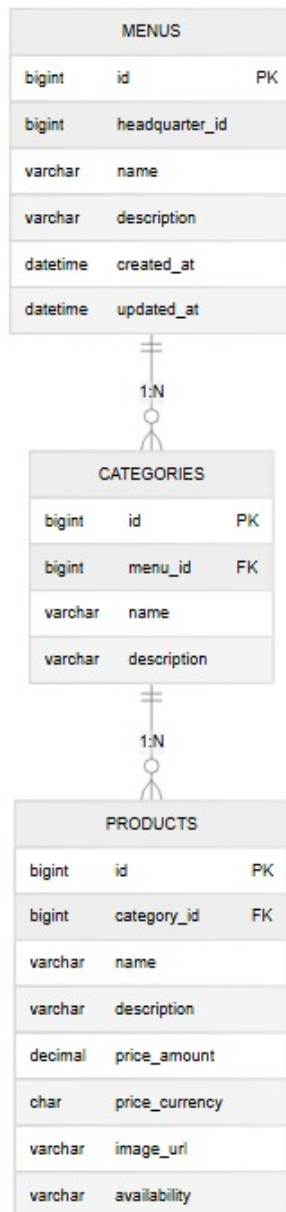
3. Value Objects:

- **HeadquarterId**: Representa el identificador único de una sede.
- **MenuName**: Representa el nombre de un menú.
- **MenuDescription**: Representa la descripción de un menú.
- **CategoryId**: Representa el identificador único de una categoría.
- **CategoryName**: Representa el nombre de una categoría.
- **CategoryDescription**: Representa la descripción de una categoría.
- **ProductId**: Representa el identificador único de un producto.
- **ProductName**: Representa el nombre de un producto.
- **ProductDescription**: Representa la descripción de un producto.
- **Money**: Representa un importe monetario con una divisa.
- **ImageUrl**: Representa la URL de una imagen.
- **Availability**: Enumera los estados posibles de disponibilidad de un producto.

4.2.5.6.2. Bounded Context Database Design Diagram.

El diseño de la base de datos para el **Menu Management Bounded Context** refleja la estructura del dominio, asegurando que las entidades y relaciones definidas en la **Domain Layer** se representen de manera eficiente en el

modelo relacional.



Este diseño incluye las siguientes tablas principales:

1. **Menus:**

- Representa los menús disponibles en las sedes.
- **Atributos principales:**
 - **id**: Identificador único del menú.
 - **headquarter_id**: Identificador de la sede a la que pertenece el menú.
 - **name**: Nombre del menú.
 - **description**: Descripción del menú.

2. **Categories:**

- Representa las categorías de productos dentro de los menús.
- **Atributos principales:**
 - **id**: Identificador único de la categoría.
 - **menu_id**: Identificador del menú al que pertenece la categoría.
 - **name**: Nombre de la categoría.

- **description**: Descripción de la categoría.

3. Products:

- Representa los productos disponibles en las categorías.
- **Atributos principales**:
 - **id**: Identificador único del producto.
 - **category_id**: Identificador de la categoría a la que pertenece el producto.
 - **name**: Nombre del producto.
 - **description**: Descripción del producto.
 - **price_amount**: Importe del precio del producto.
 - **price_currency**: Divisa del precio del producto.
 - **image_url**: URL de la imagen del producto.
 - **availability**: Disponibilidad del producto (AVAILABLE, SOLD_OUT, DISCONTINUED).

4.2.6 Bounded Context: IoT Monitoring Bounded Context

El **IoT Monitoring Bounded Context** es responsable de gestionar la interacción con los dispositivos IoT instalados en las cafeterías, como los sensores de peso en las sillas. Este contexto se encarga de recopilar, procesar e interpretar los datos enviados por estos dispositivos para determinar el estado de ocupación de las mesas en tiempo real. Además, proporciona mecanismos para la configuración y monitoreo de los dispositivos, asegurando su correcto funcionamiento y la precisión de los datos recopilados.

Capítulo V: Solution UI/UX Design

5.1. Style Guidelines

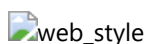
5.1.1. General Style Guidelines

En esta sección definimos los principios visuales y de interacción que rigen toda la experiencia Tavolo, asegurando coherencia entre plataformas. Establecemos una identidad visual clara mediante el uso de paleta de colores, tipografía, iconografía, espaciado y tono comunicacional unificado.



5.1.2. Web, Mobile and IoT Style Guidelines

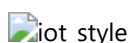
Diseñamos interfaces web centradas en la claridad y eficiencia, priorizando la experiencia del usuario en pantallas grandes. Aplicamos estructuras responsivas, patrones de navegación consistentes y un uso óptimo del grid para garantizar accesibilidad y fluidez en entornos de escritorio.



Adaptamos la experiencia Tavolo a dispositivos móviles optimizando la usabilidad táctil, la jerarquía visual y la agilidad en la navegación. Nos enfocamos en gestos intuitivos, tiempos de carga mínimos y transiciones suaves entre pantallas, respetando las limitaciones y ventajas del entorno móvil.



Extendemos nuestros lineamientos a dispositivos IoT garantizando interfaces funcionales y simples. Diseñamos visuales claras y compactas, con códigos de color efectivos y elementos visuales que facilitan la comprensión.



5.2. Information Architecture

5.2.1. Organization Systems

Para Tavolo, hemos implementado sistemas de organización que facilitan la navegación y acceso a la información tanto para comensales como para administradores. Estos sistemas están diseñados para proporcionar una experiencia de usuario intuitiva y eficiente.

 Diagrama de Organization System de Tavolo

Sistema jerárquico

Organizamos la información en Tavolo siguiendo una estructura jerárquica clara:

Para comensales:

El proceso comienza en la Landing Page, donde los usuarios son redirigidos a la aplicación web o móvil según su preferencia. Una vez allí, los comensales pueden registrarse o iniciar sesión con sus credenciales, accediendo a un panel principal que les muestra información relevante como reservas activas, sedes disponibles y menús de productos, incluyendo opciones de café y sandwiches. Para realizar una reserva, los comensales seleccionan una sede, visualizan las mesas disponibles y completan un formulario con los detalles de su reserva, como fecha, hora y número de personas. Además, pueden ubicar la sede en un mapa integrado para mayor comodidad.

Para administradores:

Por otro lado, los administradores acceden a un panel de gestión después de iniciar sesión. Desde allí, pueden administrar las mesas de las sedes, agregando nuevas o eliminando las existentes mediante un formulario sencillo. También tienen acceso a información detallada de las sedes y pueden visualizar las reservas de diferentes maneras: por mesa, por persona o en una lista completa. Esto les permite mantener un control eficiente sobre las operaciones del establecimiento.

Sistema secuencial

También implementamos una organización secuencial para procesos específicos:

Proceso de reserva:

1. Selección de sede
2. Visualización de mesas disponibles
3. Selección de mesa
4. Selección de fecha y hora
5. Confirmación de reserva

Proceso de gestión de mesas:

1. Visualización del inventario actual de las mesas
2. Selección de acción (agregar/eliminar)
3. Introducción de datos de la mesa
4. Confirmación de la operación

5.2.2. Labeling Systems

Antes de implementar las etiquetas en nuestra plataforma, determinamos qué requisitos necesitamos cumplir. Las etiquetas nos permiten agregar información contextual a diferentes elementos de la plataforma Tavolo. A continuación,

se detalla el sistema de etiquetado implementado:

Etiquetas para mesas

Etiqueta	Descripción
[DISPONIBLE]	Mesa que puede ser reservada en este momento
[OCUPADA]	Mesa actualmente en uso por comensales

Etiquetas para reservas

Etiqueta	Descripción
[ACTIVA]	Reserva en curso (dentro del rango horario seleccionado)

Etiquetas para productos en menú

Etiqueta	Descripción
[CAFÉ]	Bebidas de café
[REPOSTERÍA]	Pasteles, galletas y postres
[SANDWICH]	Opciones de sándwiches y bocadillos

Etiquetas para notificaciones de sistema

Etiqueta	Descripción
[ÉXITO]	Confirmación de operación completada exitosamente
[ERROR]	Problema durante una operación

Este sistema de etiquetas proporciona contexto visual inmediato y mejora la comprensión de la información presentada tanto para comensales como para administradores.

5.2.3. SEO Tags and Meta Tags

La implementación adecuada de etiquetas SEO es crucial para mejorar la visibilidad de Tavolo en los motores de búsqueda. A continuación, detallamos las principales etiquetas implementadas:

Título

Indica el tema de la página, debe ser corto y descriptivo, manteniéndose entre 55-60 caracteres.

```
<title>Tavolo - Sistema de Gestión de Reservas y Ocupación para Cafeterías</title>
```

Descripción

Breve descripción del propósito de la plataforma.

```
<meta name="description" content="Tavolo optimiza la experiencia en cafeterías con tecnología IoT, visualización en tiempo real de mesas disponibles y reservas digitales. ¡Mejora la experiencia de tus clientes!"/>
```

Robots

Indican a los motores de búsqueda qué hacer con la página.

```
<meta name="robots" content="index, follow">
```

Tipo de contenido

Útil para que los motores de búsqueda identifiquen el idioma de la página.

```
<meta http-equiv="Content-Type" content="text/html; charset=UTF-8">
<meta http-equiv="Content-Language" content="es">
```

Viewport Meta Tag

Crucial para asegurar que el contenido se vea bien en dispositivos móviles.

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Open Graph Tags

Para mejorar la apariencia cuando se comparte en redes sociales.

```
<meta property="og:title" content="Tavolo - Gestión Inteligente para Cafeterías">
<meta property="og:description" content="Sistema IoT para optimizar la ocupación de mesas
en cafeterías">
<meta property="og:image" content="https://tavolo.com/images/tavolo-preview.jpg">
<meta property="og:url" content="https://tavolo.com">
<meta property="og:type" content="website">
```

Keywords

Aunque tienen menor importancia actualmente, aún pueden ser útiles.

```
<meta name="keywords" content="reservas cafeterías, mesas disponibles, IoT cafeterías,
gestión restaurantes, sistema reservas tiempo real, ocupación mesas">
```

5.2.4. Searching Systems

El sistema de búsqueda propuesto para Tavolo permitirá a los usuarios encontrar fácilmente información relevante según sus necesidades específicas. Este sistema está diseñado para facilitar tanto a comensales como a administradores la localización de sedes, reservas y disponibilidad de mesas.

Sistema de búsqueda para comensales

Nombre del filtro	Descripción
Ubicación	El usuario podrá filtrar cafeterías por distrito o zona de la ciudad
Disponibilidad inmediata	Muestra sólo las cafeterías con mesas disponibles en el momento actual
Capacidad	Permite filtrar por mesas disponibles con capacidad específica (2, 4, 6 personas)
Horario de atención	Filtra por cafeterías abiertas en un rango horario específico
Distancia	Ordena los resultados desde la ubicación actual del usuario

Sistema de búsqueda para administradores

Nombre del filtro	Descripción
Búsqueda por número de mesa	Permite encontrar rápidamente información de una mesa específica
Búsqueda por nombre de cliente	Localiza reservas realizadas por un cliente específico
Historial de reservas	Busca en el histórico de reservas completadas

Características adicionales del sistema de búsqueda

- **Búsqueda en tiempo real:** Los resultados se actualizan automáticamente cada 10 segundos.
- **Historial de búsquedas:** Mantiene un registro de búsquedas recientes.

Este sistema de búsqueda está diseñado para ser intuitivo y proporcionar resultados precisos, mejorando así la experiencia del usuario y la eficiencia operativa de la plataforma Tavolo.

5.2.5. Navigation Systems

El sistema de navegación de la plataforma Tavolo guiará a los usuarios a través de las distintas secciones y pantallas, proporcionándoles acceso intuitivo a todas las funcionalidades. A continuación, se detallan los elementos que facilitan la navegación de los usuarios:

Navegación global

Nombre	Descripción
Inicio	Página principal que muestra un resumen de la información relevante según el tipo de usuario
Perfil	Permite al usuario gestionar su información personal y preferencias

Navegación para comensales

Nombre	Descripción
Explorar sedes	Muestra todas las cafeterías disponibles con información sobre su ubicación y disponibilidad
Mis reservas	Permite al usuario ver sus reservas activas y su historial de reservas anteriores
Ver menú	Acceso a los menús digitales de las diferentes cafeterías
Mapa	Visualización geográfica de las sedes en un mapa interactivo

Navegación para administradores

Nombre	Descripción
--------	-------------

Nombre	Descripción
Dashboard	Panel principal con indicadores clave sobre la operación de la sede
Gestión de mesas	Interfaz para agregar, eliminar o modificar mesas en la sede asignada
Reservas	Visualización y gestión de las reservas actuales y futuras

Elementos de navegación

- **Menú principal:** Barra de navegación fija en la parte superior que contiene los enlaces principales.
- **Menú lateral desplegable:** Para acceso a funciones secundarias y configuraciones.
- **Botones de acción flotantes:** Para acciones principales como "Realizar reserva" o "Agregar mesa".
- **Gestos táctiles:** En la versión móvil, se implementan gestos como deslizar para ver más opciones o pinchar para ampliar el mapa.

Patrones de navegación

- **Hub and spoke:** La pantalla principal actúa como centro desde donde se accede a las diferentes funcionalidades.
- **Navegación por pestañas:** Para alternar entre diferentes vistas de una misma categoría.
- **Navegación jerárquica:** Para profundizar en detalles (sede → mesas → reservas).
- **Desplazamiento infinito:** En listados de sedes o mesas para cargar más resultados al desplazarse.

Este sistema de navegación está diseñado para ser consistente, intuitivo y adaptable a diferentes dispositivos, asegurando que los usuarios puedan encontrar rápidamente la información o funcionalidad que necesitan sin esfuerzo excesivo.

5.3. Landing Page UI Design

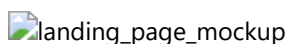
5.3.1. Landing Page Wireframe

En esta sección representamos una estructura funcional preliminar que organiza los principales bloques de contenido sin detalles visuales. Su objetivo es definir la jerarquía informativa y el flujo de navegación del usuario. Este diseño inicial se enfoca en la lógica y disposición del contenido, sin aplicar aún colores, imágenes o estilos gráficos.



5.3.2. Landing Page Mock-up

Por otro lado, el mockup de la landing page de Tavolo muestra una versión visual refinada y casi final del diseño. Siguiendo el General Style Guidelines, respetamos la paleta de colores, la tipografía, etc. El contenido está organizado de manera clara y atractiva, destacando el valor principal de Tavolo: permitir a los usuarios monitorear el aforo en tiempo real y reservar su lugar en cafeterías de forma inteligente. Las secciones explican detalladamente cómo funciona la aplicación, sus beneficios para los usuarios y cafeterías, e incluyen botones de llamado a la acción, un tour virtual y un formulario de contacto. Este mockup transmite una experiencia profesional, acogedora y tecnológica.

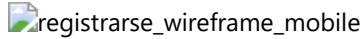
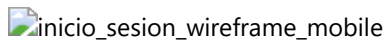
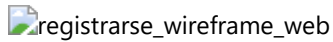
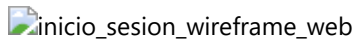


5.4. Applications UX/UI Design

5.4.1. Applications Wireframes

Esta sección presenta los wireframes de la aplicación web y móvil de Tavolo, diseñados para los dos perfiles de usuario: Clientes de establecimiento y propietarios de cafeterías. Cada wireframe representa la estructura y jerarquía de los elementos antes de aplicar estilo visual o contenido final.

Los primeros wireframes son compartidos por ambos usuarios ya que nos referimos al inicio de sesión y registro, estos representan las pantallas de acceso al sistema, con formularios simples y enfocados en la usabilidad. Se prioriza una navegación rápida y accesible para usuarios recurrentes y nuevos:



Clientes

- **Pantalla de inicio:**

Muestra un resumen de reservas activas del cliente y acceso directo al menú del establecimiento. Está diseñada para brindar información clave de forma inmediata.



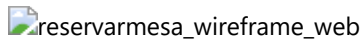
- **Menú digital:**

Organizado por categorías. La estructura está pensada para facilitar la exploración y eventual selección en el pedido.



- **Reservar mesa:**

Incluye selector de sede, visualización de mesas disponibles, horarios, y acceso a un mapa con la ubicación exacta de la cafetería. Se destaca la usabilidad móvil responsive y la claridad en la disponibilidad visual.



Propietarios

- **Pantalla de inicio:**

Resume información clave de cada sede: mesas activas y reservas por horario.



- **Gestionar mesas:**

Visualiza todas las mesas registradas por sede, permite su eliminación o la adición de nuevas con datos como zona, capacidad e identificador único.

gestion_wireframe_web

mesa_wireframe_web

agregarmesa_wireframe_web

gestion_wireframe_mobile

form_wireframe_mobile

mesa_wireframe_mobile

- **Ver reservas:**

Incluye filtros para buscar reservas por mesa o por cliente, optimizando la gestión diaria del flujo de comensales.

filtro_wireframe_web

pormesa_wireframe_web

persona_wireframe_web

pormesa_wireframe_mobile

persona_wireframe_mobile

5.4.2. Applications Wireflow Diagrams

Los diagramas de Wireflow desarrollados para Tavolo permiten visualizar de manera gráfica la transición entre pantallas dentro de la aplicación, integrando el diseño de la interfaz con el flujo de navegación esperado. Estos flujos representan acciones comunes tanto para el cliente como para el propietario, facilitando el análisis de usabilidad y asegurando la coherencia del diseño:

- **Wireflow – Cliente (Web):** Desde el inicio de sesión o registro, el cliente puede ver sus reservas, acceder al menú, seleccionar una sede, consultar la disponibilidad de mesas, reservar y visualizar la ubicación en mapa.

wireflow1_web

- **Wireflow – Propietario (Web):** Luego del acceso, el propietario puede gestionar mesas (añadir o eliminar), visualizar reservas por sede, mesa o persona, y mantener el control completo sobre la operatividad del local.

wireflow2_web

- **Wireflow – Cliente (Mobile):** Similar al flujo web, adaptado a navegación móvil con vistas más compactas y controles táctiles.

wireflow1_mobile

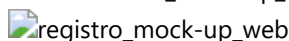
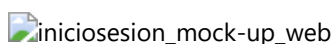
- **Wireflow – Propietario (Mobile):** Pensado para una gestión rápida desde dispositivos móviles, permite acceder a reservas, editar disponibilidad y revisar información clave en tiempo real.

wireflow2_mobile

5.4.3. Applications Mock-ups

Versión Web

La pantalla de inicio de sesión y registro son similares para ambos segmentos, allí el cliente o propietario ingresa sus credenciales para acceder al sistema o por el otro lado, puede llenar un formulario de creación de cuenta para nuevos usuarios, con campos de correo y contraseña.



Cliente

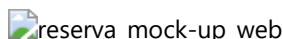
- **Inicio:** Vista principal del cliente donde puede ver sus reservas activas, acceder al menú y ver las sedes disponibles.



- **Menú:** Sección donde el cliente puede explorar las categorías de productos disponibles en la cafetería.



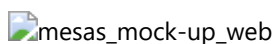
- **Vista de reservas:** Panel donde el cliente puede revisar la lista de sedes e información básica de cada una.



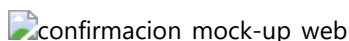
- **Detalle de la sede:** Información de la sede seleccionada, con dirección, horarios, contacto y acceso a las mesas disponibles con filtros de capacidad y zona. Además, muestra las mesas de la sede seleccionada, agrupadas por zonas, con su estado y horarios disponibles.



- **Reserva de mesa (pop-up):** Ventana emergente para seleccionar la hora de la reserva de la mesa elegida.



- **Confirmación de reserva:** Mensaje que confirma que la reserva fue realizada exitosamente.



- **Mapa de la sede (pop-up):** Muestra un mapa con la ubicación exacta de la sede seleccionada para facilitar la llegada del cliente.

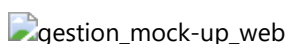


Propietario

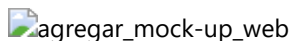
- **Inicio:** Muestra las sedes a cargo del propietario y el resumen de reservas por sede.



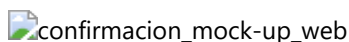
- **Gestión de mesas:** Panel para ver todas las mesas de una sede, con opción de editar o eliminar cada una.



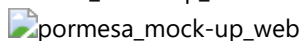
- **Agregar mesa (pop-up):** Ventana para añadir nuevas mesas especificando zona, capacidad e identificador.



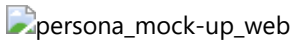
- **Confirmación de agregado:** Mensaje que informa que la mesa se añadió correctamente.



- **Vista de reservas por mesa:** Permite al propietario revisar las reservas filtradas por cada mesa, con su disponibilidad por horario.

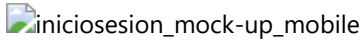


- **Vista de reservas por persona:** Muestra una lista de personas que han reservado, con los detalles y hora.



Versión Móvil

La pantalla de inicio de sesión y registro son similares para ambos segmentos, allí el cliente o propietario ingresa sus credenciales para acceder al sistema o por el otro lado, puede llenar un formulario de creación de cuenta para nuevos usuarios, con campos de correo y contraseña.



Cliente

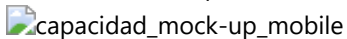
- **Inicio:** Muestra la reserva activa (si existe), y un acceso rápido a las sedes disponibles.



- **Vista de sedes:** Despliega una lista de sedes con su descripción, dirección y número de mesas disponibles.



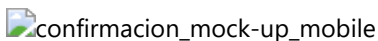
- **Detalle de sede:** Muestra información general de la sede seleccionada (dirección, horario y contacto), incluye una sección donde se visualizan las mesas disponibles, agrupadas por zonas y con sus capacidades, las cuales son una lista ampliada de mesas con scroll para ver más opciones, incluyendo filtros de zona.



- **Reserva de mesa (pop-up):** Selector de horario para la mesa elegida, mostrando disponibilidad por hora.



- **Confirmación de reserva:** Mensaje que informa que la reserva fue realizada con éxito.



- **Mapa de la sede (pop-up):** Ventana emergente que muestra un mapa con la ubicación de la sede.



- **Menu:** Visualización de productos con nombre, tipo y precio (ej. cafés, postres, etc).



Propietario

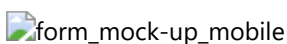
- **Inicio:** Pantalla principal con vista de sedes administradas, cada una con datos clave y acceso a reservas.



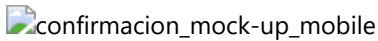
- **Gestión de mesas:** Lista de todas las mesas con capacidad, zona y botón de eliminación.



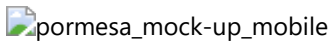
- **Agregar nueva mesa (pop-up):** Formulario para añadir una mesa, solicitando zona, capacidad e identificador.



- **Confirmación de agregado:** Mensaje de éxito que confirma que la mesa fue registrada.



- **Detalle de sede:** El propietario puede visualizar cómo el cliente ve las opciones disponibles de reserva.



- **Detalle de reserva por persona:** Vista filtrada para ver las reservas agrupadas por nombre de cliente, fecha y mesa.

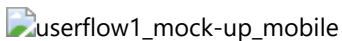
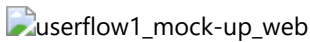


5.4.4. Applications User Flow Diagrams

Los diagramas de flujo de usuario permiten entender cómo los usuarios interactúan con la aplicación desde una perspectiva lógica, no visual. Estos diagramas incluyen decisiones condicionales, caminos alternativos y respuestas del sistema según la acción realizada por el usuario.

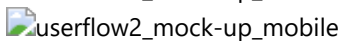
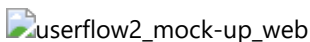
- **User Flow – Cliente:**

1. Inicia sesión o se registra
2. Visualiza sus reservas o accede al menú
3. Selecciona una sede
4. Consulta mesas disponibles
5. Selecciona mesa y hora
6. Reserva
7. Confirmación
8. (Opcional) Consulta ubicación en mapa



- **User Flow – Propietario:**

1. Inicia sesión o se registra
2. Visualiza su(s) sede(s)
3. Gestiona mesas (añadir o eliminar)
4. Consulta reservas por persona o por mesa
5. Toma decisiones administrativas



5.5. Applications Prototyping

Web application prototyping

- Clientes

Se diseñaron prototipos que permiten a los clientes explorar restaurantes, realizar reservas de mesas, y visualizar menús digitales desde la plataforma web, con una interfaz intuitiva enfocada en la experiencia de usuario.



Video explicación:

<https://drive.google.com/file/d/1FppOgE1BkclITFJZ26mgpkJoDIqalVaq/view?usp=sharing>

- Administradores

Se diseñó una interfaz administrativa que facilita la gestión de reservas, control de disponibilidad de mesas y configuración de menús, permitiendo a cada sede de una cafetería tener un control completo desde la web.

 web_prototype_administrador

Video explicación:

https://drive.google.com/file/d/1kcd_FJk4l8dhdRntgm0UZbYrUqKcuO6/view?usp=drive_link

Mobile application prototyping

- Clientes

Se desarrollaron prototipos móviles que permiten a los usuarios buscar restaurantes cercanos, reservar mesas fácilmente, ver recomendaciones personalizadas y acceder a menús interactivos, priorizando la usabilidad en dispositivos móviles.

 mobile_prototype_cliente

Video explicación:

https://drive.google.com/file/d/1H52_e9udVbAlttnXDAsrhZpsWwABluJw/view?usp=drive_link

Capítulo VI: Product Implementation, Validation & Deployment

6.1. Software Configuration Management

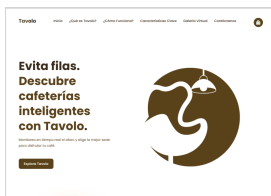
6.1.1. Software Development Environment Configuration

Project Management:

- **WhatsApp:** Se utilizó WhatsApp como una herramienta de comunicación para mantener el contacto con los miembros del equipo, mandar recordatorios, realizar encuestas y colaborar en cierta medida.



- **Landing Page:** Para dar vida a la primera impresión de nuestra landing page, decidimos usar HTML, CSS y JavaScript que son ampliamente conocidas y utilizadas en la creación de páginas web. Estas tecnologías permitieron crear una página atractiva, funcional y adaptada a las necesidades del proyecto.



Requirements Management:

- **Trello:** Herramienta para la gestión ágil de tareas, empleada con enfoque Kanban. Se organizaron listas por etapas, lo que permitió un control visual y eficiente del progreso.



- **Lucidchart:** Utilizamos Lucidchart como herramienta para realizar diagramas y documentación.



Product UX/UI Design:

- **Figma:** Empleamos Figma para el diseño de la interfaz de usuario (UI) y la experiencia de usuario (UX) de la aplicación Tavolo. Figma permitió la creación de prototipos, wireframes y diseños visuales que guiarán el desarrollo del producto final.



- **UXPressia:** Utilizamos UXPressia como herramienta especializada en la creación de mapas de experiencia de usuario, perfiles de clientes y otros elementos relacionados con el diseño de UX. UXPressia nos permitió visualizar y comprender la experiencia del usuario en profundidad, ayudando a mejorar la usabilidad y el diseño de la aplicación.

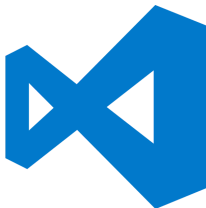


- **Canva:** Incorporamos Canva en la etapa de diseño para la creación de recursos gráficos adicionales, como imágenes, gráficos y presentaciones, que mejoró la estética y la presentación del producto.



Software Development:

- **IDE (Entorno de Desarrollo Integrado):** Utilizamos Visual Studio Code como nuestro IDE principal debido a su capacidad para facilitar el desarrollo web moderno y eficiente. Visual Studio Code ofrece soporte robusto para los lenguajes y tecnologías clave que utilizamos, como HTML, CSS, JavaScript y frameworks como Angular o Vue.js. Además, su interfaz intuitiva y extensibilidad mediante plugins nos permitió trabajar de manera más productiva, asegurando una codificación de calidad y una depuración eficaz.



- **HTML5:** HyperText Markup Language, o por sus siglas HTML, es un lenguaje de etiquetado para páginas web. Será empleado en el desarrollo del proyecto para la presentación del contenido en la aplicación.

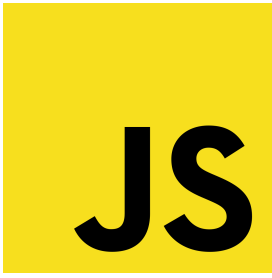


- **CSS:** Cascading Style Sheets es un lenguaje que maneja el diseño y presentación de las páginas web, el cual va de la mano con HTML.



- **JavaScript:** Es un lenguaje de programación interpretado, dialecto del estándar ECMAScript. Se define como orientado a objetos, basado en prototipos, imperativo, débilmente tipado y dinámico. Se utilizará para elaborar

la interfaz de usuario dentro de la aplicación.



Software Documentation:

- **GitHub:** Empleamos GitHub como plataforma principal para la documentación del código y la colaboración en el desarrollo. Utilizamos el README.md para documentar el proceso de desarrollo.



6.1.2. Source Code Management

Para garantizar la eficiencia y evitar conflictos en el desarrollo de soluciones, los proyectos de Tavolo se gestionaron en una organización de GitHub. Dentro de esta organización, se encuentran los repositorios correspondientes a cada proyecto. Aquí están los enlaces a los repositorios:

Landing page: <https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-landing-page>

Report: <https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-report>

Web: <https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-web>

Backend: <https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-backend>

En cuanto al manejo del Gitflow, fue de la siguiente forma:

En el desarrollo de Tavolo, cada cambio que se realizó en los archivos se marcó con un mensaje con el formato "Conventional Commits". Esta práctica facilitó la identificación de los cambios realizados en cada commit, permitió un seguimiento más eficiente del proyecto. Además, este modelo incluye la rama develop y main, que contuvo las versiones finales y estables del proyecto. Para mantener una organización clara del proyecto, se creó una rama específica para cada integrante del equipo. Esto permitió un seguimiento más detallado y una mejor organización del código.

6.1.3. Source Code Style Guide & Conventions

Nuestro equipo adoptó las siguientes convenciones y guías de estilo para garantizar un código fuente coherente, legible y mantenible en los diferentes lenguajes y tecnologías utilizados en nuestra solución

HTML:

Para el lenguaje HTML, nos planteamos utilizar las convenciones descritas en la guía "HTML Style Guide and Coding Conventions":

- Usar nombres de elementos en minúsculas
- Cerrar todos los elementos HTML
- Usar nombres de atributos en minúsculas
- Usar atributos en imágenes
- Evitar líneas de código largas
- Usar sintaxis simple para los enlaces para las hojas de estilo y para cargar script externos

CSS:

Para el lenguaje CSS, utilizaremos las siguientes prácticas para alcanzar un código coherente, sostenible y ordenado:

- Utilizar minúsculas y guiones para los nombres de propiedades
- Utilizar un espacio después de los dos puntos y un punto y coma para separar pares propiedad-valor.
- Agrupar reglas CSS relacionadas y separarlas con una línea en blanco.
- Utilizar nombres de clases que sean descriptivos y reflejen el propósito del elemento.
- Separar los nombres de las clases y ID con un guión

JavaScript:

Para el lenguaje JavaScript, seguimos las siguientes prácticas para escribir un código legible, eficiente y mantenible:

- Las declaraciones de variables se realizaron al principio de su ámbito.
- Se prefirió el uso de `const` y `let` sobre `var`.
- Se incluyó comentarios descriptivos para explicar la funcionalidad de componentes, servicios y secciones complejas.
- Se aplicaron principios de programación reactiva y patrones de diseño adecuados para la construcción de la aplicación web.

Vue.js:

En el desarrollo frontend con Vue.js, seguimos las recomendaciones oficiales del Vue Style Guide:

- Organización del componente `.vue` en orden: `template`, `script`, `style`.
- Nombres de archivos coinciden con el nombre del componente.
- Propiedades documentadas, tipadas y ordenadas.
- Separación clara de lógica en métodos y componentes hijos.
- Estilos `scoped` para evitar conflictos globales.

Java:

En la capa backend desarrollada con Java, seguimos las convenciones propuestas por la Guía oficial de estilo de Java de Oracle:

- Uso de CamelCase para nombres de clases y métodos.
- Un archivo `.java` por clase pública.
- Bloques `if`, `for`, `while` siempre con llaves `{}` incluso para una sola línea.
- Separación lógica del código en paquetes por funcionalidad: `controller`, `service`, `repository`, `model`.
- Aplicación de principios SOLID para una arquitectura mantenible y escalable.

6.1.4. Software Deployment Configuration

El despliegue del sistema Tavolo se realizará utilizando Microsoft Azure, una plataforma en la nube confiable y escalable que permite alojar tanto la aplicación web como la base de datos de forma segura y eficiente. A continuación, se detallan los pasos principales del proceso de despliegue:

1. Crear los recursos en Azure como grupo de recursos, App Service (backend), Static Web App (landing page), y Azure SQL Database.
2. Seleccionar entorno de ejecución en App Service.
3. Publicar el backend desde GitHub o carga manual al App Service.
4. Subir la landing page a Azure Static Web Apps.
5. Configurar la cadena de conexión al servidor SQL.
6. Ejecutar scripts de creación de tablas y relaciones.
7. Activar HTTPS, definir reglas de firewall e IPs permitidas.
8. Configurar roles y permisos en la base de datos.
9. Usar Azure Monitor para registrar métricas y errores.
10. Configurar escalado automático del App Service según demanda.

6.2. Landing Page, Services & Applications Implementation.

En esta sección se documenta el proceso completo de implementación, validación y despliegue de las diversas soluciones tecnológicas que conforman Tavolo. **Partiendo del Product Backlog establecido**, cada sprint representa una iteración enfocada en entregar incrementos funcionales del producto, demostrando así el avance progresivo en el desarrollo de la solución.

La implementación de Tavolo comprende múltiples componentes interconectados:

- **Landing Page:** La puerta de entrada informativa para potenciales usuarios, diseñada para comunicar claramente la propuesta de valor.
- **Web Services (Backend API):** El núcleo funcional que gestiona la lógica de negocio, procesamiento de datos y comunicación con los sensores IoT.
- **Web Applications:** Interfaces para comensales y administradores que permiten interactuar con el sistema desde navegadores web.

El desarrollo incremental se estructuró en sprints, cada uno con objetivos específicos y entregables definidos. Para cada sprint se documentan las siguientes etapas:

1. Sprint Planning: Planificación detallada de las historias de usuario seleccionadas.
2. Backlog y asignación de tareas: Distribución de responsabilidades entre los miembros del equipo.
3. Development Evidence: Registro de los avances y contribuciones al código.
4. Testing Suite: Validación funcional y técnica de los componentes implementados.
5. Execution Evidence: Demostración del producto en funcionamiento.
6. Services Documentation: Documentación técnica de las APIs y servicios.
7. Deployment: Publicación de los componentes en sus respectivos entornos.
8. Collaboration Insights: Análisis de la dinámica del equipo durante el sprint.

A lo largo de este capítulo, se presenta una visión transparente del **proceso iterativo que llevó a la materialización de Tavolo** como una solución funcional.

6.2.1. Sprint 1

6.2.1.1. Sprint Planning 1.

Sprint #1	Sprint 1
Sprint Planning Background	
Date	9/5/2025
Time	11:00 am

Sprint #1	Sprint 1
Location	REunion virtual vía Discord
Prepared By	Jimena Cama
Attendees (to planning meeting)	- Baldeon Fabian, Aldo Alberto - Cama Salvatierra, Jimena Tamara - Castillo Castillo, Jair Alexander - La Torre Valle, Franz Jair - Quezada Portalatino, Barbara Susana
Sprint Goal & User Stories	
Sprint 1 Goal	En este sprint la meta a cumplir fue desarrollar el backend de la plataforma, la aplicación web funcional y la landing page.
Sprint 1 Velocity	17
Sum of Story Points	81

6.2.1.2. Aspect Leaders and Collaborators

Para lograr una implementación eficiente del Sprint 1, se estableció una estructura clara de liderazgo y colaboración entre los miembros del equipo. Se identificaron cinco aspectos principales que abarcaron el alcance completo del sprint: Frontend Web Cliente, Frontend Web Administrador, Backend API, Testing & Documentación, y Landing Page. Esta organización permitió asegurar que cada área tuviera un responsable directo y colaboradores que apoyaron en tareas específicas.

La matriz de liderazgo y colaboración (LACX) detalla cómo se distribuyeron las responsabilidades para cada aspecto del sprint:

Team Member (Last Name, First Name)	GitHub Username	Frontend Web Cliente Leader (L) / Collaborator (C)	Frontend Web Administrador Leader (L) / Collaborator (C)	Backend API Leader (L) / Collaborator (C)	Testing & Documentación Leader (L) / Collaborator (C)	Landing Page Leader (L) / Collaborator (C)
Baldeon Fabian, Aldo Alberto	CodAress	L	C	C		
Cama Salvatierra, Jimena Tamara	aksonie		L		C	C
Castillo Castillo, Jair Alexander	U202211390		C	C	L	

Team Member (Last Name, First Name)	GitHub Username	Frontend Web Cliente Leader (L) / Collaborator (C)	Frontend Web Administrador Leader (L) / Collaborator (C)	Backend API Leader (L) / Collaborator (C)	Testing & Documentación Leader (L) / Collaborator (C)	Landing Page Leader (L) / Collaborator (C)
La Torre Valle, Franz Jair	FranzJairLTV	C			C	L
Quezada Portalatino, Barbara Susana	BarbaraQP15	C		L		C

Responsabilidades por aspecto:

1. Frontend Web Cliente:

- **Líder:** Baldeon Fabian, Aldo Alberto
- **Enfoque:** Implementación de interfaces para visualización de disponibilidad de mesas en tiempo real, registro e inicio de sesión de comensales, y proceso de reserva.
- **Tareas principales:** Desarrollo de componentes de visualización de mesas, formularios de autenticación y pantallas principales.

2. Frontend Web Administrador:

- **Líder:** Cama Salvatierra, Jimena Tamara
- **Enfoque:** Desarrollo del panel administrativo con funcionalidades para gestión de sedes, visualización de reservas y configuración de mesas.
- **Tareas principales:** Implementación de pantallas de inicio del administrador, visualización de sedes asignadas y gestión de reservas.

3. Backend API:

- **Líder:** Quezada Portalatino, Barbara Susana
- **Enfoque:** Desarrollo de la API RESTful con servicios para autenticación, gestión de mesas, reservas y operaciones administrativas.
- **Tareas principales:** Implementación de endpoints, integración con base de datos y lógica de negocio.

4. Testing & Documentación:

- **Líder:** Castillo Castillo, Jair Alexander
- **Enfoque:** Diseño y ejecución de pruebas, documentación de servicios y evidencia de funcionamiento.
- **Tareas principales:** Creación de casos de prueba, documentación Swagger para la API y recopilación de evidencias.

5. Landing Page:

- **Líder:** La Torre Valle, Franz Jair
- **Enfoque:** Diseño e implementación de la landing page promocional de Tavolo.
- **Tareas principales:** Desarrollo de secciones informativas, galería virtual y formulario de contacto.

Esta organización permitió una clara asignación de responsabilidades, facilitó la comunicación entre subequipos y aseguró que todas las áreas del sprint contaran con el enfoque adecuado. Los líderes de cada aspecto coordinaron con

sus colaboradores para definir tareas específicas y garantizar el cumplimiento de los criterios de aceptación de cada historia de usuario asignada.

6.2.1.3. Sprint Backlog 1.

Sprint #		Sprint 1					
User Story		Work-Item/Task					
Id	Title	Id	Title	Description	Estimation (Hours)	Assigned To	Status
US001	Ver sección "¿Qué es Tavolo?"	TA1	Crear componente informativo	Diseñar y codificar el componente de la sección "¿Qué es Tavolo?"	1	Quezada Portalatino, Barbara Susana	Done
		TA2	Integrar contenido e imágenes	Subir textos e imágenes proporcionados al frontend	1	Quezada Portalatino, Barbara Susana	Done
US002	Ver sección "Características clave"	TA1	Crear layout de características	Maquetar y estilizar la sección de características	1	Quezada Portalatino, Barbara Susana	Done
		TA2	Implementar íconos e interactividad	Asociar íconos representativos a cada característica con animaciones	1	Quezada Portalatino, Barbara Susana	Done
US003	Ver la galería virtual	TA1	Crear carrusel de imágenes	Implementar galería interactiva con carrusel	1	Cama Salvatierra, Jimena Tamara	Done
		TA2	Integrar imágenes reales de sedes	Subir imágenes y validar visualización responsive	1	Cama Salvatierra, Jimena Tamara	Done
US004	Usar el formulario de contacto	TA1	Diseñar formulario	Maquetar campos necesarios y validaciones	1	Quezada Portalatino, Barbara Susana	Done
		TA2	Configurar envío de datos	Configurar envío POST con mensajes de confirmación	2	Quezada Portalatino, Barbara Susana	Done

Sprint #	Sprint 1						
US005	Ver menú digital de una sede	TA1	Crear vista para menú por sede	Diseñar vista filtrada por sede seleccionada	2	La Torre Valle, Franz	Done
		TA2	Mostrar platos con categorías	Agregar cards por categoría con imagen, nombre y precio	2	La Torre Valle, Franz	Done
US006	Visualizar reserva activa	TA1	Crear vista de reservas activas	Mostrar reserva activa del usuario al iniciar sesión	2	La Torre Valle, Franz	Done
		TA2	Agregar detalles de la reserva	Visualizar sede, horario, cantidad de personas y estado	2	La Torre Valle, Franz	Done
US007	Ver disponibilidad de mesas en tiempo real	TA1	Integrar disponibilidad con backend	Mostrar mesas libres en tiempo real según sede y horario	4	Baldeon Fabian, Aldo	Done
		TA2	Mostrar visualmente mesas ocupadas/libres	Agregar colores o íconos para representar disponibilidad	1	Baldeon Fabian, Aldo	Done
US008	Reservar una mesa	TA1	Crear formulario de reserva	Formulario con sede, fecha, hora y cantidad de personas	1	Quezada Portalatino, Barbara	Done
		TA2	Enviar reserva al backend	Conexión con API para guardar reserva y mostrar confirmación	1/2	Quezada Portalatino, Barbara	Done
US010	Registro de cuenta de comensal	TA1	Maquetar formulario de registro	Campos: nombre, correo, contraseña	1	Baldeon Fabian, Aldo	Done
		TA2	Validar datos e integrarlo con backend	Validar datos y enviar a API para crear cuenta	1	Baldeon Fabian, Aldo	Done
US011	Inicio de sesión de cuenta de comensal	TA1	Diseñar pantalla de login	Formulario de ingreso con validación	2	Baldeon Fabian, Aldo	Done

Sprint
**Sprint 1**

		TA2	Manejo de sesión y redirección	Guardar token en localStorage y redirigir según rol	2	Baldeon Fabian, Aldo Alberto	Done
US012	Agregar nuevas mesas	TA1	Diseñar formulario de creación	Formulario para admins para añadir mesas	2	Castillo Castillo, Jair Alexander	Done
		TA2	Enviar datos a backend	Validar y enviar datos de la nueva mesa al servidor	2	Castillo Castillo, Jair Alexander	Done
US013	Visualizar reservas realizadas	TA1	Listar reservas del comensal	Mostrar historial con fechas y estados	2	Cama Salvatierra, Jimena Tamara	Done
		TA2	Agregar opciones de filtro por fecha/estado	Filtros por mes o estado (activa, cancelada, completada)	1	Cama Salvatierra, Jimena Tamara	Done
US014	Ver la sede asignada	TA1	Mostrar sede asignada en perfil	Mostrar sede que administra el usuario (admin)	1/2	Cama Salvatierra, Jimena Tamara	Done
		TA2	Obtener sede desde el backend	Lógica para obtener sede del usuario y mostrarla	1/2	Cama Salvatierra, Jimena Tamara	Done
US015	Eliminar mesas	TA1	Crear botón de eliminación	Agregar ícono de papelera para eliminar mesa	1	Castillo Castillo, Jair Alexander	Done
		TA2	Confirmar y eliminar en backend	Confirmación con modal y eliminación en API	1	Castillo Castillo, Jair Alexander	Done
US016	Inicio de sesión de administrador	TA1	Crear vista de login para admins	Vista distinta a la de comensales, con validación	2	Baldeon Fabian, Aldo Alberto	Done

Sprint # **Sprint 1**

US017	Registro de cuenta de administrador	TA2	Asignar token y redirigir a dashboard	Lógica de inicio de sesión y navegación hacia panel	1	Baldeon Fabian, Aldo Alberto	Done
		TA1	Crear formulario con rol "admin"	Campos de registro con rol predefinido como admin	1	Baldeon Fabian, Aldo Alberto	Done
		TA2	Validar correo corporativo	Validación de correo institucional para asegurar rol	2	Baldeon Fabian, Aldo Alberto	Done
US018	Acceder a todos los módulos del sistema	TA1	Mostrar menú de navegación	Menú para navegación entre módulos: reservas, mesas, reportes	2	La Torre Valle, Franz Jair	Done
		TA2	Proteger rutas con roles	Validar acceso solo para administradores	2	La Torre Valle, Franz Jair	Done

6.2.1.4. Development Evidence for Sprint Review.

En este primer sprint, el equipo ha logrado importantes avances en la implementación de los componentes esenciales de Tavolo. Se ha desarrollado con éxito la **landing page, el frontend web para comensales y administradores, así como el backend API** que soporta las funcionalidades principales del sistema.

Entre los logros destacables se encuentran:

- Implementación completa de la landing page con todas las secciones informativas y el formulario de contacto
- Desarrollo del sistema de autenticación para comensales y administradores
- Implementación del módulo de visualización de mesas en tiempo real
- Desarrollo del sistema de reservas y visualización de reservas activas
- Implementación del panel de administración para gestión de mesas y visualización de reservas

A continuación, se presenta una tabla detallada de los commits más relevantes realizados en los diferentes repositorios durante el Sprint 1:

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on (Date)
https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-landing-page	main	a72e3b4	feat: add landing page	Creación de estructura HTML base y estilos CSS para la landing page con las secciones principales	10/05/2025

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on (Date)
https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-web	feature/auth	e23a9c1	feat: sistema de autenticación	Implementación del login y registro para usuarios comensales y administradores con manejo de tokens y sesiones	10/05/2025
https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-web	feature/client-dashboard	f84b6d2	feat: dashboard de comensal	Creación del panel principal para comensales con visualización de disponibilidad de mesas y menú	12/05/2025
https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-web	feature/booking-system	g59c7e3	feat: sistema de reservas	Implementación del formulario de reserva y visualización de reservas activas para comensales	13/05/2025
https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-web	feature/admin-panel	h31d8f4	feat: panel de administrador	Desarrollo del dashboard para administradores con funcionalidades de gestión de mesas y visualización de sede asignada	14/05/2025
https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-web	feature/table-management	i76e2a9	feat: gestión de mesas	Implementación de funcionalidades para agregar y eliminar mesas desde el panel de administrador	15/05/2025
https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-backend	feature/api-base	j42f1e5	feat: estructura base de la API	Configuración inicial del proyecto Spring Boot con dependencias y estructura de paquetes	10/05/2025
https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-backend	feature/auth-endpoints	k93g7h2	feat: endpoints de autenticación	Implementación de servicios REST para registro e inicio de sesión con roles diferenciados	11/05/2025

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on (Date)
https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-backend	feature/table-endpoints	l28j5k6	feat: endpoints para gestión de mesas	Desarrollo de servicios REST para crear, eliminar y consultar estado de mesas	13/05/2025
https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-backend	feature/booking-endpoints	m67n4p9	feat: endpoints para reservas	Implementación de servicios REST para realizar reservas y consultar reservas activas	14/05/2025
https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-backend	feature/sede-endpoints	o15r3s8	feat: endpoints para sedes	Desarrollo de servicios REST para consultar información de sedes y asignación a administradores	15/05/2025

6.2.1.5. Testing Suite Evidence for Sprint Review.

En esta sección se presenta el conjunto de Unit Tests, Integration Tests y Acceptance Tests automatizados para los Web Services relacionados con los User Stories especificados en el Sprint.

Relación de Commits de Testing

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on (Date)
tavolo-backend	feature/unit-tests	abc123	Add unit tests for reservation service	Implemented tests for create, update and delete methods	2023-10-15
tavolo-backend	feature/integration-tests	def456	Add integration tests for reservation API	Testing the complete flow of reservation endpoints	2023-10-16
tavolo-backend	feature/bdd-tests	ghi789	Add BDD tests for reservation feature	Added Cucumber tests with Gherkin scenarios for reservation management	2023-10-18

Unit Tests

TableCommandServiceTest

Esta clase contiene pruebas unitarias para validar el comportamiento del servicio **TableCommandService**, responsable de gestionar las operaciones relacionadas con la creación de mesas en el sistema.

```

21 public class TableCommandServiceTest {
22     @Mock 2 usages
23     private com.tavolo.platform.booking.infrastructure.persistence.jpa.repositories.TableRepository tableRepository;
24
25     @Mock 2 usages
26     private ExternalHeadquarterService externalHeadquarterService;
27
28     @InjectMocks 3 usages
29     private TableCommandServiceImpl tableCommandService;
30
31     @BeforeEach
32     public void setUp() {
33         MockitoAnnotations.openMocks(this);
34
35         // Inyección manual de los mocks
36         ReflectionTestUtils.setField(tableCommandService, name: "tableRepository", tableRepository);
37         ReflectionTestUtils.setField(tableCommandService, name: "externalHeadquarterService", externalHeadquarterService);
38     }
39
40     @Test
41     public void testHandleCreateTableCommand() {
42         // Crear un comando de ejemplo con los parámetros correctos
43         Integer tableNumber = 1;
44         Integer capacity = 4;
45         Long headquarterId = 1L;
46
47         // Configuración del mock para que retorne true cuando se verifica si existe el headquarter
48         when(externalHeadquarterService.existsHeadquarter(headquarterId)).thenReturn(true);
49
50         CreateTableCommand command = new CreateTableCommand(tableNumber, capacity, headquarterId);
51         Table mockTable = new Table(new TableDetails(tableNumber, capacity), new HeadquarterId(headquarterId));
52         when(tableRepository.save(any(Table.class))).thenReturn(mockTable);
53
54         Optional<Table> result = tableCommandService.handle(command);
55
56         assertTrue(result.isPresent()); // Verificar que se crea la mesa
57     }
58 }

```

Este test unitario valida:

- La creación correcta de comandos para crear mesas
- El comportamiento del servicio cuando verifica la existencia de una sede
- El correcto guardado de la mesa en el repositorio
- La devolución del resultado esperado

Integration Tests

TableCommandServiceIntegrationTest

Esta clase de prueba integra los distintos componentes del sistema para validar el flujo completo de creación de mesas:

```

@SpringBootTest
@Transactional
public class TableCommandServiceIntegrationTest {
    @Autowired
    private TableCommandServiceImpl tableCommandService;

    @Test
    public void testHandleCreateTableCommandIntegration() {
        // Crear un comando de ejemplo con los parámetros correctos
        Integer tableNumber = 1; // Número de mesa como Integer
        Integer capacity = 4; // Capacidad como Integer
        Long headquarterId = 1L; // ID de la sede

        CreateTableCommand command = new CreateTableCommand(tableNumber, capacity, headquarterId);
        Optional<Table> result = tableCommandService.handle(command);
        assertTrue(result.isPresent()); // Verificar que se crea la mesa
    }
}

```

Esta prueba de integración verifica:

- El funcionamiento correcto del servicio en un entorno transaccional
- La comunicación correcta entre los distintos componentes del sistema
- La persistencia adecuada de los datos

Autenticacion (AuthFeature.feature)

Feature: Autenticación de usuarios

Scenario: Registro exitoso de un nuevo usuario

Given el usuario proporciona datos válidos para registro

When se envía la solicitud de registro

Then el sistema crea el usuario y devuelve un estado 201

Scenario: Fallo en el registro por datos inválidos

Given el usuario proporciona datos inválidos

When se envía la solicitud de registro

Then el sistema devuelve un error de validación

6.2.1.6. Execution Evidence for Sprint Review.

En este primer sprint, el equipo Tavolo ha logrado implementar con éxito los componentes fundamentales del sistema, creando una base sólida para el desarrollo futuro del proyecto. Se han completado todas las historias de usuario planificadas de la parte web, alcanzando las principales funcionalidades tanto para comensales como para administradores.

- **Landing page completa** con información del producto, galería y formulario de contacto
- **Sistema de autenticación** diferenciado para comensales y administradores
- **Visualización de disponibilidad de mesas en tiempo real**
- **Sistema de reservas** con confirmación y visualización de reservas activas
- **Panel administrativo** para gestión de mesas y visualización de la sede asignada

Landing Page



Evita filas. Descubre cafeterías inteligentes con Tavolo.

Monitorea en tiempo real el aforo y elige la mejor sede para disfrutar tu café.

Explora Tavolo



¿Qué es Tavolo?

Tavolo es una aplicación innovadora que te permite monitorear en tiempo real el aforo de cafeterías y hacer reservas desde tu dispositivo móvil. Con Tavolo, puedes disfrutar de tu café sin preocuparte por las largas filas.

¿Cómo Funciona?

Con sensores de aforo y una aplicación intuitiva, puedes monitorear en tiempo real la disponibilidad de las cafeterías y hacer reservas desde tu dispositivo móvil.



Sensores detectan aforo

Los sensores de Tavolo detectan el aforo en tiempo real, permitiéndote conocer la cantidad de personas en la cafetería.



Muestra disponibilidad en tiempo real

La aplicación Tavolo muestra la disponibilidad en tiempo real de las cafeterías, permitiéndote elegir la mejor opción.



El usuario reserva y ve el menú en línea

Una vez que eliges la cafetería, puedes reservar tu mesa y ver el menú en línea, facilitando tu experiencia.

Características Clave

Tavolo ofrece una serie de características clave que mejoran tu experiencia en cafeterías. Desde la reserva de mesas hasta la consulta de disponibilidad en tiempo real, Tavolo está diseñado para hacer tu vida más fácil.



Menú digital y vista del local

Explora el menú digital y disfruta de una vista panorámica del local para elegir tu lugar ideal.



Consulta de disponibilidad en tiempo real

Consulta la disponibilidad en tiempo real de las cafeterías y evita las largas filas.



Sugerencia de otras sedes

Si la sede que elegiste está llena, Tavolo te sugiere otras sedes cercanas con disponibilidad.

¿Tienes una cafetería?

Tavolo es la solución perfecta para optimizar la experiencia de tus clientes. Con nuestra aplicación, podrás ofrecer un servicio eficiente y cómodo, mejorando la satisfacción del cliente y aumentando la lealtad a tu marca.

- ✓ Datos en tiempo real
- ✓ Gestión de espacios eficiente.
- ✓ Canal directo con clientes.



Contáctanos

Si tienes alguna pregunta o necesitas más información sobre Tavolo, no dudes en contactarnos. Estamos aquí para ayudarte.

☐ Acepto los términos y condiciones

Enviar Mensaje

Tavolo

Tavolo es una aplicación innovadora que te permite monitorear en tiempo real el aforo de cafeterías y hacer reservas desde tu dispositivo móvil. Con Tavolo, puedes disfrutar de tu café sin preocuparte por las largas filas.

Secciones

Inicio
¿Qué es Tavolo?
Características Clave
Cómo Funciona
Contáctanos

Redes Sociales

Facebook
Instagram
Twitter
Linkedin

Sistema para Comensales



¡Bienvenido!

Registrarse

Email

correo@ejemplo.com

Contraseña

contraseña



Confirmar contraseña

confirmar contraseña



Regístrate

¿Ya tienes un cuenta? [Iniciar sesión](#)

Conoce nuestras sedes y reserva!



Urqu Coffee Shop

Dirección: Jirón Áncash 251, Centro de Lima

Horario:

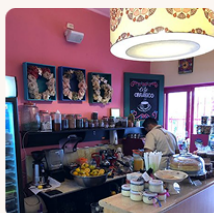
Lunes -Viernes: 9:00 am - 8 pm

Sábado: 10:00 am - 6 pm

Domingo: 10:00 am - 5 pm

Ver mesas

Ver en Mapa



Homemade Hecho en Casa

Dirección: Calle Henry Revett 259, Miraflores

Horario:

Lunes -Viernes: 9:00 am - 8 pm

Sábado: 10:00 am - 6 pm

Domingo: 10:00 am - 5 pm

Ver mesas

Ver en Mapa

Mis reservas

Revisa tus reservas y no llegues tarde

Urqe Coffe Shop

Mesa: #6
Zona: Terraza
Hora: 6:00 p.m

Ver local en Mapa

Homemade Hecho en Casa

Mesa: #2
Zona: Terraza
Hora: 10:00 p.m

Ver local en Mapa

Bisetti

Mesa: #2
Zona: Terraza
Hora: 10:00 p.m

Ver local en Mapa

Revisa el menú

Lo que tenemos para ofrecerte

Ver todo

Tostadas andinas con crema de queso de cabra

\$ 20.00

Una reinterpretación gourmet de las tostadas matutinas, esta delicia fusiona ingredientes tradicionales peruanos con un toque contemporáneo. El plato presenta dos rebanadas de pan artesanal de quinua y kiwicha, ligeramente tostadas en mantequilla de hierbas. Sobre ellas, una capa generosa de crema batida de queso de cabra infundada con huacatay

Panel de Administración



¡Bienvenido de vuelta!

Iniciar Sesión

Email

correo@ejemplo.com

Contraseña

contraseña

Iniciar Sesión

¿No tienes una cuenta todavía? [Crear cuenta](#)



Homemade Hecho en Casa

Dirección: Calle Henry Revett 259, Miraflores

Horario:

Lunes -Viernes: 9:00 am - 8 pm

Sábado: 10:00 am - 6 pm

Domingo: 10:00 am - 5 pm

Número: +01 6782345



Ver reservas

Revisa las reservas

Ver todo

Mesa #1

Capacidad: 5 personas

Zona: Terraza

Hora:

8:00

9:00

10:00

11:00

12:00

13:00

14:00

15:00

16:00

17:00

18:00

19:00

20:00

Mesa #2

Capacidad: 4 personas

Zona: Sala principal

Hora:

8:00

9:00

10:00

11:00

12:00

13:00

14:00

15:00

16:00

17:00

18:00

19:00

20:00

Gestionar mesas

+ Agregar mesa

Mesa #2

ID: 2

Capacidad: 3 personas

Zona: Sala principal

Estado: AVAILABLE

Eliminar

Mesa #1

ID: 1

Capacidad: 2 personas

Zona: Sala principal

Estado: AVAILABLE

Eliminar

Mesa #3

ID: 3

Capacidad: 2 personas

Zona: Sala principal

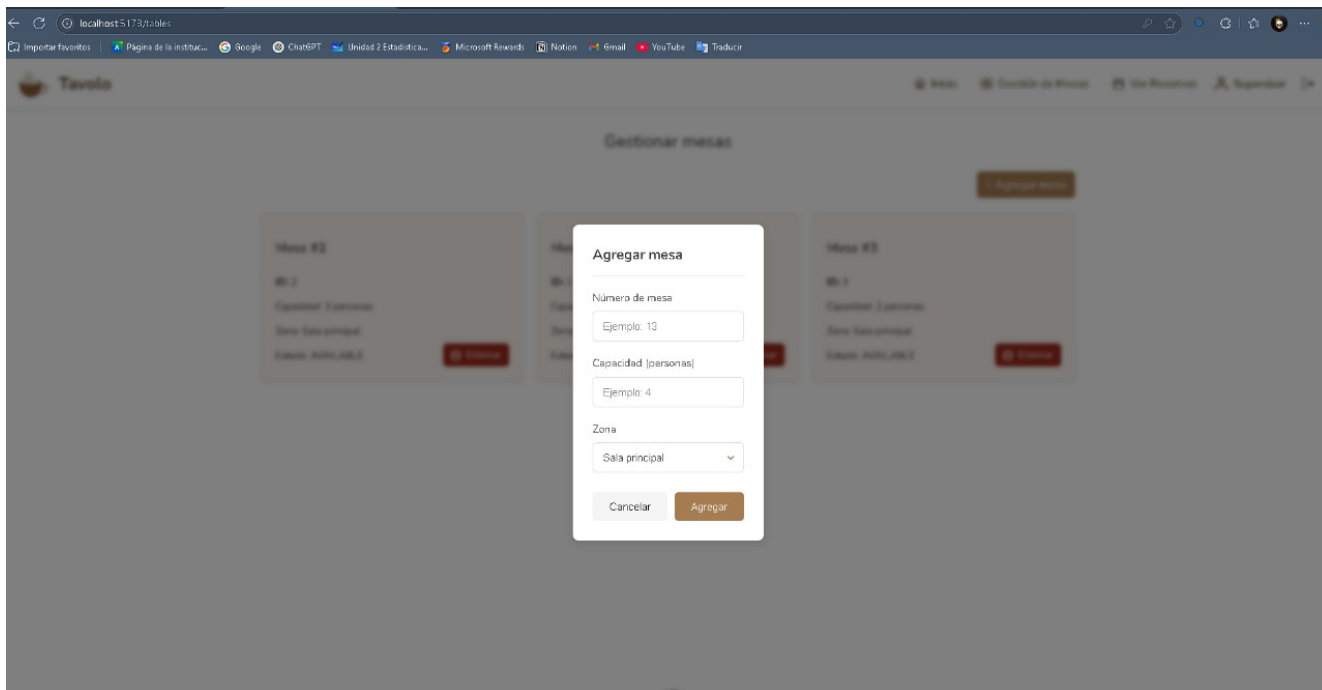
Estado: AVAILABLE

Eliminar

PROFESSEUR : M.DA ROS

◆ 167 / 226 ◆

BTS SIO BORDEAUX - LYCÉE GUSTAVE EIFFEL



Tavolo

[Inicio](#)

[Gestión mesas](#)

[Ver reservas](#)

[Cerrar sesión](#)

Ver reservas

Ver por mesa ▾

Mesa #2

Capacidad: 4 personas

Zona: Sala principal

Hora:

8:00 9:00 10:00 11:00 12:00 13:00 14:00 15:00 16:00 17:00 18:00 19:00 20:00

Mesa #2

Capacidad: 4 personas

Zona: Sala principal

Hora:

8:00 9:00 10:00 11:00 12:00 13:00 14:00 15:00 16:00 17:00 18:00 19:00 20:00

Mesa #2

Capacidad: 4 personas

Zona: Sala principal

Hora:

8:00 9:00 10:00 11:00 12:00 13:00 14:00 15:00 16:00 17:00 18:00 19:00 20:00

Video Explicativo

El video muestra el flujo completo desde la landing page hasta las funcionalidades de reserva para comensales y gestión para administradores, demostrando la integración exitosa de todos los componentes desarrollados durante este sprint.

Video Demostración Sprint 1 - Tavolo

El equipo ha completado el 100% de las historias de usuario planificadas para este sprint, cumpliendo con los criterios de aceptación establecidos. La aplicación es funcional en su estado actual y proporciona las funcionalidades básicas

necesarias para la gestión de mesas y reservas. En los **próximos sprints** se agregarán características adicionales **como notificaciones, reportes estadísticos** y la integración completa con los sensores IoT físicos.

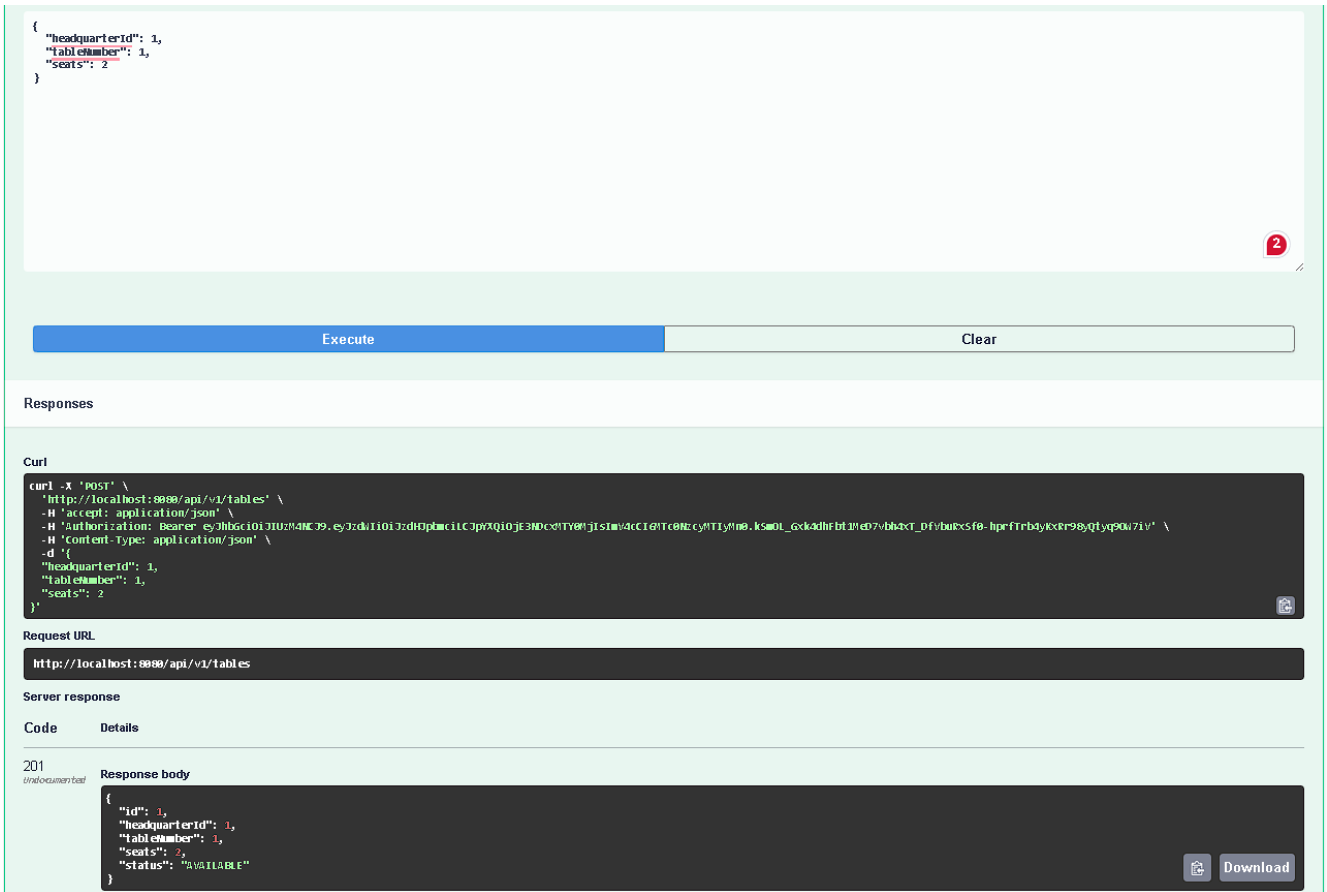
6.2.1.7. Services Documentation Evidence for Sprint Review.

Durante el Sprint 1, el equipo ha logrado documentar completamente los endpoints de la API de Tavolo utilizando Swagger. Esta documentación proporciona una referencia clara y accesible para desarrolladores frontend, facilitando la integración con los servicios backend.

Categoría	Endpoint	Método HTTP	Descripción
Autenticación	<code>/api/v1/authentication/sign-up</code>	POST	Registrar nuevo usuario
Autenticación	<code>/api/v1/authentication/sign-in</code>	POST	Iniciar sesión de usuario
Reservas	<code>/api/v1/bookings</code>	GET	Listar todas las reservas
Reservas	<code>/api/v1/bookings</code>	POST	Crear nueva reserva
Reservas	<code>/api/v1/bookings/{id}</code>	GET	Obtener detalles de reserva específica
Reservas	<code>/api/v1/bookings/client/{clientId}</code>	GET	Obtener reservas de un cliente específico
Sedes	<code>/api/v1/headquarters</code>	GET	Listar todas las sedes
Sedes	<code>/api/v1/headquarters</code>	POST	Crear nueva sede
Sedes	<code>/api/v1/headquarters/{headquarterId}</code>	GET	Obtener detalles de sede específica
Roles	<code>/api/v1/roles</code>	GET	Listar todos los roles
Usuarios	<code>/api/v1/users</code>	GET	Listar todos los usuarios
Usuarios	<code>/api/v1/users/{userId}</code>	GET	Obtener detalles de usuario específico
Mesas	<code>/api/v1/tables</code>	GET	Listar todas las mesas
Mesas	<code>/api/v1/tables</code>	POST	Crear nueva mesa
Mesas	<code>/api/v1/tables/{tableId}</code>	GET	Obtener detalles de mesa específica
Mesas	<code>/api/v1/tables/{tableId}/schedule</code>	GET	Obtener horario de una mesa específica

Evidencia de pruebas realizadas

Para verificar la funcionalidad de la API, se realizaron pruebas exhaustivas utilizando la interfaz Swagger. A continuación se muestran algunos ejemplos de estas pruebas:



Resultado exitoso de la creación de una mesa

Repositorio y commits relacionados

Repositorio: <https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-backend>

Commit ID	Descripción	Autor	Fecha
c739042	feat(booking): implementar endpoint para consultar reservas por ID	Baldeon Fabian, Aldo Alberto	07/05/2025
l28j5k6	feat(iam): implement user authentication and registration system	Castillo Castillo, Jair Alexander	11/05/2025
m67n4p9	feat: feat(booking,branching): implement reservation system with branch management	Baldeon Fabian, Aldo Alberto	11/05/2025
o15r3s8	feature(iam): add bounded context iam	Castillo Castillo, Jair Alexander	12/05/2025
p45t7u9	feature(shared): add add bounded context shared	Castillo Castillo, Jair Alexander	12/05/2025

6.2.1.8. Software Deployment Evidence for Sprint Review.

Durante este primer sprint, el equipo Tavolo ha implementado una estrategia de despliegue completa para asegurar que todos los componentes del sistema estén disponibles en entornos de producción. Se ha desplegado exitosamente la landing page, el backend API y la aplicación web frontend, utilizando servicios en la nube para garantizar disponibilidad, escalabilidad y seguridad.

Landing Page Deployment

La landing page de Tavolo fue desplegada utilizando GitHub Pages, aprovechando la integración directa con nuestro repositorio para un flujo de trabajo simplificado y automático.

URL: <https://upc-pre-202510-1asi0572-2952-tavolo.github.io/tavolo-landing-page/>

Evidencia del despliegue

Landing Page 1

Landing Page 2

Landing Page 3

Landing Page 4

Backend API

La API de Tavolo fue desplegada **por ahora de manera local**, se ha probado todas las funcionalidades de la plataforma de manera exitosa.

Evidencia del Swagger

Swagger UI

Swagger UI

Swagger UI

Frontend Web Application

La aplicación web de Tavolo se preparó para ser desplegada en Firebase Hosting, que ofrece una plataforma eficiente para aplicaciones React con rápidos tiempos de carga y distribución global. Sin embargo, **por ahora** se maneja de **manera local**

Evidencia de las pantallas

Pantalla de Comensal

Mis reservas

Revisa tus reservas y no llegues tarde

Urqe Coffe Shop

Mesa: #6
Zona: Terraza
Hora: 6:00 p.m

Ver local en Mapa

Homemade Hecho en Casa

Mesa: #2
Zona: Terraza
Hora: 10:00 p.m

Ver local en Mapa

Bisetti

Mesa: #2
Zona: Terraza
Hora: 10:00 p.m

Ver local en Mapa

Revisa el menú

Lo que tenemos para ofrecerte

Ver todo

Tostadas andinas con crema de queso de cabra

\$ 20.00

Una reinterpretación gourmet de las tostadas matutinas, esta delicia fusiona ingredientes tradicionales peruanos con un toque contemporáneo. El plato presenta dos rebanadas de pan artesanal de quinua y kiwicha, ligeramente tostadas en mantequilla de hierbas. Sobre ellas, una capa generosa de crema batida de queso de cabra infundada con huacatay

Homemade Hecho en Casa

Café con opciones para vegetarianos y veganos, ideal para desayunos y brunchs.

Entradas

Platos principales

Postres

Bebidas



Tostadas andinas con crema de queso de cabra

S/ 20.00

Una reinterpretación gourmet de las tostadas matutinas, esta delicia fusiona ingredientes tradicionales peruanos con un toque contemporáneo. El plato presenta dos rebanadas de pan artesanal de quinua y kiwicha, ligeramente tostadas en mantequilla de hierbas. Sobre ellas, una capa generosa de crema batida de queso de cabra infundada con huacatay



Wrap de quinua y camote

S/ 35.00

Tortilla integral rellena con camote asado en especias andinas, quinua roja cocida al dente, palta en cubos, tomates cherry y brotes de alfalfa. Acompañado con una salsa de yogurt vegetal con hierbabuena.



Tostada verde tropical

S/ 18.00

Lunes -Viernes: 9:00 am - 8 pm
Sábado: 10:00 am - 6 pm
Domingo: 10:00 am - 5 pm

[Ver en Mapa](#)



Lunes -Viernes: 9:00 am - 8 pm
Sábado: 10:00 am - 6 pm
Domingo: 10:00 am - 5 pm

[Ver en Mapa](#)

Café con opciones para vegetarianos y veganos, ideal para desayunos y brunchs.

Reservar

Reservar

Reservar

Reservar

Reservar

Homemade Hecho en Casa

Café con opciones para vegetarianos y veganos, ideal para desayunos y brunchs.

Filtrar mesas

Capacidad

Select

5 personas

4 personas

3 personas

2 personas

1 persona

Zona

Select

Terraza

Ventana

Sala principal

Se reservó correctamente la mesa

Mesa #1

Capacidad: 5 personas

Zona: Terraza

Reservar

Mesa #2

Capacidad: 4 personas

Zona: Ventana

Reservar

Mesa #3

Capacidad: 3 personas

Zona: Sala principal

Reservar

Mesa #4

Capacidad: 2 personas

Zona: Terraza

Reservar

Mesa #5

Capacidad: 1 persona

Zona: Terraza

Reservar

Mesa #6

Capacidad: 5 personas

Zona: Ventana

Reservar

Mesa #7

Capacidad: 5 personas

Zona: Sala principal

Reservar

Mesa #8

Capacidad: 4 personas

Zona: Terraza

Reservar

Mesa #9

Capacidad: 3 personas

Zona: Terraza

Reservar

Mesa #10

Capacidad: 1 persona

Zona: Sala principal

Reservar

Mesa #11

Capacidad: 3 personas

Zona: Terraza

Reservar

Mesa #12

Capacidad: 2 personas

Zona: Sala principal

Reservar

Conoce nuestras sedes y reserva!

Sede #1



Cerrar

Horario:

Lunes -Viernes: 9:00 am - 8 pm
Sábado: 10:00 am - 6 pm
Domingo: 10:00 am - 5 pm

Ver mesas

Ver en Mapa

Ver mesas

Ver en Mapa

Pantalla de Administrador



Homemade Hecho en Casa

Dirección: Calle Henry Revett 259, Miraflores

Horario:

Lunes -Viernes: 9:00 am - 8 pm

Sábado: 10:00 am - 6 pm

Domingo: 10:00 am - 5 pm

Número: +01 6782345



Ver reservas

Revisa las reservas

Ver todo

Mesa #1

Capacidad: 5 personas

Zona: Terraza

Hora:

8:00

9:00

10:00

11:00

12:00

13:00

14:00

15:00

16:00

17:00

18:00

19:00

20:00

Mesa #2

Capacidad: 4 personas

Zona: Sala principal

Hora:

8:00

9:00

10:00

11:00

12:00

13:00

14:00

15:00

16:00

17:00

18:00

19:00

20:00

Gestionar mesas

+ Agregar mesa

Mesa #2

ID: 2

Capacidad: 3 personas

Zona: Sala principal

Estado: AVAILABLE

Eliminar

Mesa #1

ID: 1

Capacidad: 2 personas

Zona: Sala principal

Estado: AVAILABLE

Eliminar

Mesa #3

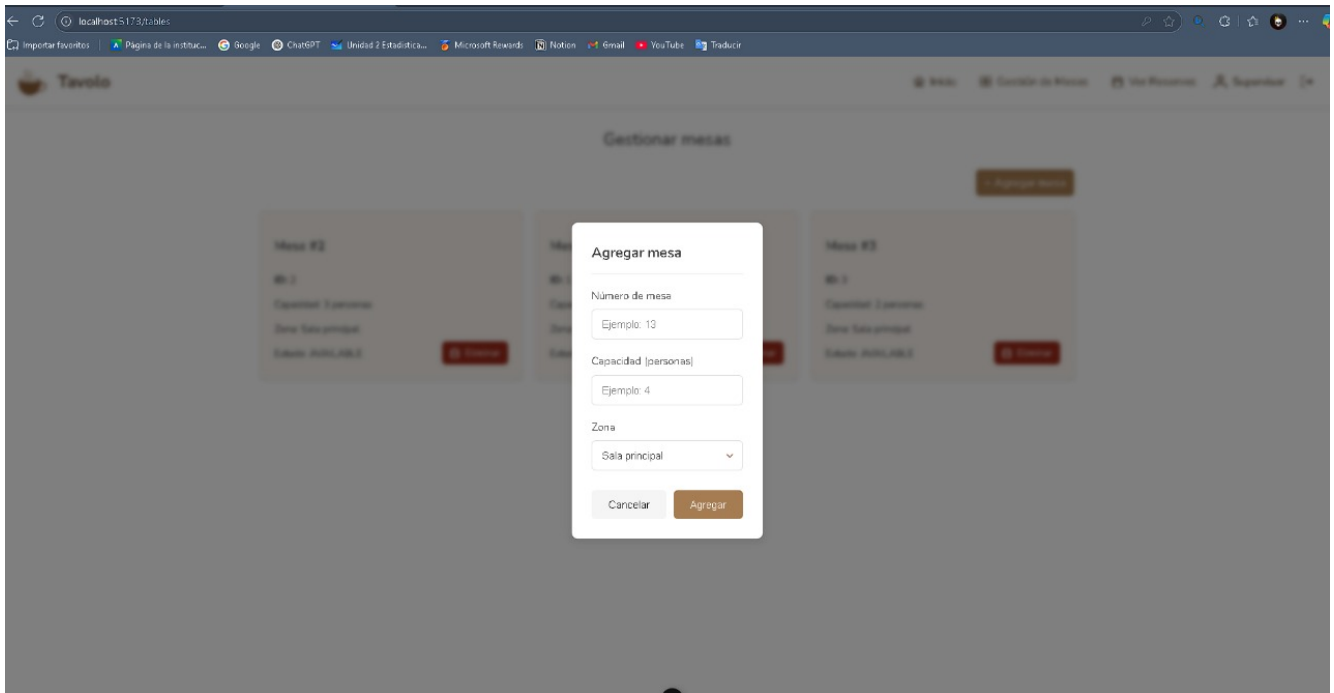
ID: 3

Capacidad: 2 personas

Zona: Sala principal

Estado: AVAILABLE

Eliminar



Tavolo

InicioGestión mesasVer reservasCerrar sesión

Gestionar mesas

+ Agregar mesa

Mesa #1

Capacidad: 5 personas

Zona: Terraza

Eliminar

Mesa #2

Capacidad: 4 personas

Zona: Ventana

Eliminar

Mesa #3

Capacidad: 4 personas

Zona: Ventana

Eliminar

Mesa #4

Capacidad: 2 personas

Zona: Terraza

Eliminar

Mesa #5

Capacidad: 1 persona

Zona: Terraza

Eliminar

Mesa #6

Capacidad: 4 personas

Zona: Ventana

Eliminar

Mesa #7

Capacidad: 5 personas

Zona: Sala principal

Eliminar

Mesa #8

Capacidad: 4 personas

Zona: Terraza

Eliminar

Mesa #9

Capacidad: 3 personas

Zona: Terraza

Eliminar

Mesa #10

Capacidad: 1 persona

Zona: Sala principal

Eliminar

Mesa #11

Capacidad: 3 personas

Zona: Terraza

Eliminar

Mesa #12

Capacidad: 2 personas

Zona: Sala principal

Eliminar

Se agregó correctamente la mesa

Ver reservas

Seleccionar

Por mesa

Por persona

Mesa #2

Capacidad: 4 personas

Zona: Sala principal

Hora:

8:00

9:00

10:00

11:00

12:00

13:00

14:00

15:00

16:00

17:00

18:00

19:00

20:00

Mesa #2

Capacidad: 4 personas

Zona: Sala principal

Hora:

8:00

9:00

10:00

11:00

12:00

13:00

14:00

15:00

16:00

17:00

18:00

19:00

20:00

Mesa #2

Capacidad: 4 personas

Zona: Sala principal

Hora:

8:00

9:00

10:00

11:00

12:00

13:00

14:00

15:00

16:00

17:00

18:00

19:00

20:00

Ver reservas

Ver por persona

Carla Zapata

Mesa #2

Capacidad: 4 personas

Zona: Sala principal

Hora: 18:00

Maribel Torres

Mesa #2

Capacidad: 4 personas

Zona: Sala principal

Hora: 18:00

Anthony Gutierrez

6.2.1.9. Team Collaboration Insights during Sprint.

Durante el Sprint 1, el equipo Tavolo demostró una colaboración efectiva y coordinada, permitiendo el cumplimiento exitoso de todas las historias de usuario planificadas. A continuación, se detallan los aspectos clave de la dinámica colaborativa del equipo:

Distribución del trabajo y contribuciones

El análisis de las contribuciones muestra una distribución equilibrada de responsabilidades:

Miembro del equipo	Áreas principales	Contribuciones clave	Pull requests	Issues closed
Baldeon Fabian, Aldo	Backend API, Frontend (Cliente), Testing	Sistema de autenticación, visualización de disponibilidad de mesas, endpoints de reservas	8	7
Cama Salvatierra, Jimena	Frontend (Administrador), Frontend (Cliente)	Panel administrativo, visualización de sede asignada, visualización de reservas	6	6
Castillo Castillo, Jair	Backend API, Testing	Endpoints de mesas y autenticación, pruebas unitarias, documentación API	7	6
La Torre Valle, Franz	Frontend (Cliente), Frontend (Administrador)	Menú digital, reserva activa, módulos de navegación, protección de rutas	6	5
Quezada Portalatino, Barbara	Frontend (Cliente), Landing Page	Sistema de reservas, landing page completa, formulario de contacto	7	6

Colaboración en componentes principales

Backend API

- **Líderes:** Aldo Baldeon y Jair Castillo
- **Logros:** Implementación completa de endpoints para autenticación, gestión de mesas, reservas y sedes.
- **Dinámica de trabajo:** Aldo se enfocó en los servicios de autenticación y reservas, mientras que Jair desarrolló los endpoints para mesas y configuró la documentación Swagger. La colaboración entre ambos permitió una integración fluida de los diferentes servicios.

Frontend Web (Cliente y Administrador)

- **Líderes:** Jimena Cama (Administrador) y Franz La Torre (Cliente)
- **Colaboradores:** Aldo Baldeon, Barbara Quezada, Jair Castillo
- **Logros:** Desarrollo de todas las interfaces de usuario para comensales y administradores con excelente usabilidad.
- **Dinámica de trabajo:** Jimena coordinó la implementación del panel administrativo mientras Franz dirigió el desarrollo de las interfaces para comensales. Barbara contribuyó significativamente con el sistema de reservas, mientras que Aldo aportó en la autenticación y visualización de mesas. Jair apoyó en la integración de componentes críticos.

Landing Page

- **Líder:** Barbara Quezada
- **Logros:** Diseño e implementación completa de la landing page con todas las secciones requeridas.
- **Dinámica de trabajo:** Barbara lideró el diseño y desarrollo de la landing page, incorporando información clara sobre el producto y asegurando una experiencia visual atractiva para los visitantes.

Testing

- **Líderes:** Aldo Baldeon y Jair Castillo

- **Logros:** Verificación de la funcionalidad de todos los componentes implementados.
- **Dinámica de trabajo:** Crearon y ejecutaron casos de prueba para validar el correcto funcionamiento de la aplicación, asegurando que todos los criterios de aceptación fueran cumplidos.

Herramientas de colaboración utilizadas

- **GitHub:** Control de versiones y seguimiento de issues
- **Discord:** Comunicación diaria y coordinación entre miembros
- **Figma:** Diseño colaborativo de interfaces
- **Google Meet:** Reuniones de planificación y retrospectiva

Metodología de trabajo

El equipo adoptó un enfoque ágil con las siguientes prácticas:

1. **Daily Standup:** Reuniones semanales de 2 horas para compartir avances y obstáculos.
2. **Revisión de código:** Cada PR requería al menos una revisión de otro miembro del equipo.
3. **Documentación continua:** Actualización constante de la documentación de API y componentes.

Desafíos enfrentados y soluciones

- **Desafío:** Integración entre frontend y backend en desarrollo paralelo.
Solución: Definición temprana de contratos de API con Swagger y uso de datos mockeados para pruebas.
- **Desafío:** Coordinación del trabajo entre múltiples colaboradores en los componentes frontend.
Solución: División clara de responsabilidades y uso de ramas de características específicas.

Lecciones aprendidas

1. La documentación temprana de la API facilitó significativamente el trabajo paralelo.
2. La asignación de líderes por componente con colaboradores definidos mejoró la calidad del código.
3. Las sesiones de pair programming aceleraron la resolución de problemas complejos.
4. El uso de prototipos y mockups antes de la implementación redujo la necesidad de cambios posteriores.

Métricas de colaboración

- **Tiempo promedio de resolución de issues:** 1.5 días
- **Tiempo promedio de revisión de PRs:** 4 horas
- **Porcentaje de código con revisión por pares:** 100%

Este Sprint 1 ha establecido una base sólida para la colaboración continua del equipo Tavolo. La distribución equilibrada del trabajo, el liderazgo compartido y el enfoque en la calidad han permitido entregar todas las funcionalidades planificadas cumpliendo con los criterios de aceptación establecidos.

6.2.2. Sprint 2

6.2.2.1.Sprint Planning 2.

Sprint #2	Sprint 2
Sprint Planning	
Background	
Date	10/6/2025
Time	5:00 pm

Sprint #2	Sprint 2
Location	Reunión virtual vía Discord
Prepared By	Jimena Cama
Attendees (to planning meeting)	- Baldeon Fabian, Aldo Alberto - Cama Salvatierra, Jimena Tamara - Castillo Castillo, Jair Alexander - La Torre Valle, Franz Jair - Quezada Portalatino, Barbara Susana
Sprint Goal & User Stories	
Sprint 2 Goal	En este sprint la meta a cumplir fue desarrollar el backend de la plataforma, la aplicación web funcional y la landing page.
Sprint 2 Velocity	15
Sum of Story Points	

6.2.2.2.Aspect Leaders and Collaborators.

Para lograr una implementación eficiente del Sprint 2, se estableció una estructura clara de liderazgo y colaboración entre los miembros del equipo. Se identificaron cinco aspectos principales que abarcaron el alcance completo del sprint: Frontend Web Superadministrador, Frontend Movil Cliente, Embedded application y el Edge. Esta organización permitió asegurar que cada área tuviera un responsable directo y colaboradores que apoyaron en tareas específicas.

La matriz de liderazgo y colaboración (LACX) detalla cómo se distribuyeron las responsabilidades para cada aspecto del sprint:

Team Member	GitHub Username	Frontend Web Superadministrador Leader (L) / Collaborator (C)	Frontend Móvil Cliente Leader (L) / Collaborator (C)	Embedded Application Leader (L) / Collaborator (C)	Edge Leader (L) / Collaborator (C)
Baldeon Fabian, Aldo Alberto	CodAress		C	L	L
Cama Salvatierra, Jimena Tamara	aksonie	L	C	C	C
Castillo Castillo, Jair Alexander	U202211390		C	C	C
La Torre Valle, Franz Jair	FranzJairLTV		C	C	C
Quezada Portalatino, Barbara Susana	BarbaraQP15		L	C	C

Responsabilidades por aspecto:

- Frontend Web Superadministrador:

Líder: Cama Salvatierra, Jimena Tamara

Enfoque: Desarrollo del dashboard para la gestión y administración del sistema desde el perfil superadministrador.

Tareas principales: Implementación de interfaces para la supervisión general, manejo de usuarios y configuraciones avanzadas.

- Frontend Móvil Cliente:

Líder: Baldeon Fabian, Aldo Alberto

Enfoque: Implementación de la app móvil para clientes, incluyendo filtros de mesas, reservas, formularios de reserva y menús.

Tareas principales: Desarrollo de pantallas para búsqueda de mesas, detalle de sedes, formulario de reservas y navegación en la app.

Colaboradores: Cama Salvatierra, Jimena Tamara; Castillo Castillo, Jair Alexander; La Torre Valle, Franz Jair; Quezada Portalatino, Barbara Susana.

- Embedded Application:

Líder: Baldeon Fabian, Aldo Alberto

Enfoque: Desarrollo del sistema embebido que interactúa con el entorno físico y la aplicación móvil/web.

Tareas principales: Implementación de lógica para dispositivos integrados, comunicación con backend y manejo de eventos en tiempo real.

Colaboradores: Cama Salvatierra, Jimena Tamara; Castillo Castillo, Jair Alexander; La Torre Valle, Franz Jair; Quezada Portalatino, Barbara Susana.

- Edge:

Líder: Baldeon Fabian, Aldo Alberto

Enfoque: Soporte y tareas específicas relacionadas con el procesamiento en borde (edge computing) para optimización y rapidez.

Tareas principales: Implementación de funcionalidades de procesamiento local y sincronización con la nube.

Colaboradores: Cama Salvatierra, Jimena Tamara; Castillo Castillo, Jair Alexander; La Torre Valle, Franz Jair; Quezada Portalatino, Barbara Susana.

6.2.2.3.Sprint Backlog 2.

Sprint #				Sprint 2			
User Story				Work-Item / Task			
Id	Title	Id	Title	Description	Estimation (Hours)	Assigned to	Status (To do / In process / To review / Done)

US001	Ver sección "¿Qué es Tavolo?"	TA1	Crear componente móvil informativo	Mostrar texto informativo desde app Flutter	2	Quezada Portalatino, Barbara	Done
		TA2	Cargar contenido desde API	Conectar con backend para cargar descripción	2	Quezada Portalatino, Barbara	Done
US002	Ver sección "Características clave"	TA1	Mostrar características en app	Lista con íconos y descripciones desde backend	3	Quezada Portalatino, Barbara	Done
		TA2	Agregar navegación a cada módulo	Botones que redirigen a pantallas relevantes	2	Quezada Portalatino, Barbara	Done
US003	Ver la galería virtual	TA1	Implementar galería en Flutter	Carrusel de imágenes de sedes desde API	3	Cama Salvatierra, Jimena	Done
		TA2	Optimizar carga desde backend	Endpoint optimizado para envío de imágenes	2	Baldeon Fabian, Aldo	Done
US005	Ver menú digital de una sede	TA1	Diseñar pantalla mobile del menú	Mostrar platos por categoría en Flutter	2	La Torre Valle, Franz	Done
		TA2	Optimizar endpoint de menú	Agregar filtros por sede y categoría	2	Baldeon Fabian, Aldo	Done
US006	Visualizar reserva activa	TA1	Mostrar reserva al iniciar en app	Leer reserva desde backend y mostrar detalles	3	La Torre Valle, Franz	Done
		TA2	Conexión en tiempo real (polling)	Verificar estado activo periódicamente	2	Baldeon Fabian, Aldo	Done
US007	Ver disponibilidad de mesas en tiempo real	TA1	Integración con microcontrolador	Recepción en backend de estados vía MQTT	4	Castillo Castillo, Jair	Done
		TA2	Visualización embebida en app Flutter	Mostrar íconos de disponibilidad en pantalla	3	Castillo Castillo, Jair	Done
US008	Reservar una mesa	TA1	Diseñar formulario en Flutter	Interfaz para elegir sede, fecha y personas	2	Quezada Portalatino, Barbara	Done
		TA2	Conectar con backend para crear reserva	POST a API y confirmación visual	2	La Torre Valle, Franz	Done

US009	Visualizar más sedes en un mapa	TA1	Crear vista de sedes en app móvil	Mostrar lista de sedes disponibles con nombre y dirección	3	Quezada Portalatino, Barbara	Done
		TA2	Implementar mapa interactivo	Al presionar "Ver en mapa", abrir modal con ubicación vía API de mapas	3	Quezada Portalatino, Barbara	Done
		TA3	Conectar con backend para obtener sedes	Endpoint que devuelva lista actualizada de sedes activas	2	Baldeon Fabian, Aldo	Done
US010	Registro de cuenta de comensal	TA1	Vista de registro en app móvil	Campos y validaciones en Flutter	2	Baldeon Fabian, Aldo	Done
		TA2	Validar datos desde backend	Evitar duplicidad y errores comunes	2	Castillo Castillo, Jair	Done
US011	Inicio de sesión de cuenta de comensal	TA1	Implementar login móvil	Lógica y guardado de token local	3	Castillo Castillo, Jair	Done
		TA2	Validar credenciales en backend	Envío seguro de usuario y password	3	Baldeon Fabian, Aldo	Done
US013	Visualizar reservas realizadas	TA1	Lista de historial de reservas	Mostrar reservas pasadas en app	3	Cama Salvatierra, Jimena	Done
		TA2	Agregar filtros por estado y fecha	Dropdowns y chips para filtrar	2	Cama Salvatierra, Jimena	Done
US014	Ver la sede asignada	TA1	Mostrar sede en perfil móvil	Vista en Flutter para administradores	2	Cama Salvatierra, Jimena	Done
		TA2	Obtener sede desde backend	Consulta protegida según token	2	Cama Salvatierra, Jimena	Done
US018	Acceder a todos los módulos del sistema	TA1	Menú de navegación en app móvil	Drawer o barra inferior para navegar entre módulos	2	La Torre Valle, Franz	Done
		TA2	Control de accesos desde backend	Middleware según roles y permisos	3	La Torre Valle, Franz	Done

6.2.2.4. Development Evidence for Sprint Review.

En este segundo sprint, el equipo ha logrado importantes avances en la implementación de los componentes esenciales de Tavolo. Se ha desarrollado con éxito la **landing page, el frontend web para comensales, el mobile app y administradores, así como el backend API** que soporta las funcionalidades principales del sistema.

Entre los logros destacables se encuentran:

- Implementación completa de la landing page con todas las secciones informativas y el formulario de contacto
- Desarrollo del sistema de autenticación para comensales y administradores
- Implementación del módulo de visualización de mesas en tiempo real
- Desarrollo del sistema de reservas y visualización de reservas activas
- Implementación del panel de administración para gestión de mesas y visualización de reservas

A continuación, se presenta una tabla detallada de los commits más relevantes realizados en los diferentes repositorios durante el Sprint 2:

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on (Date)
https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-landing-page	main	a72e3b4	feat: add landing page	Creación de estructura HTML base y estilos CSS para la landing page con las secciones principales	10/05/2025
https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-web	feature/auth	e23a9c1	feat: sistema de autenticación	Implementación del login y registro para usuarios comensales y administradores con manejo de tokens y sesiones	10/05/2025
https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-web	feature/client-dashboard	f84b6d2	feat: dashboard de comensal	Creación del panel principal para comensales con visualización de disponibilidad de mesas y menú	12/05/2025
https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-web	feature/booking-system	g59c7e3	feat: sistema de reservas	Implementación del formulario de reserva y visualización de reservas activas para comensales	13/05/2025
https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-web	feature/admin-panel	h31d8f4	feat: panel de administrador	Desarrollo del dashboard para administradores con funcionalidades de gestión de mesas y visualización de sede asignada	14/05/2025

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on (Date)
https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-web	feature/table-management	i76e2a9	feat: gestión de mesas	Implementación de funcionalidades para agregar y eliminar mesas desde el panel de administrador	15/05/2025
https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-backend	feature/api-base	j42f1e5	feat: estructura base de la API	Configuración inicial del proyecto Spring Boot con dependencias y estructura de paquetes	10/05/2025
https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-backend	feature/auth-endpoints	k93g7h2	feat: endpoints de autenticación	Implementación de servicios REST para registro e inicio de sesión con roles diferenciados	11/05/2025
https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-backend	feature/table-endpoints	l28j5k6	feat: endpoints para gestión de mesas	Desarrollo de servicios REST para crear, eliminar y consultar estado de mesas	13/05/2025
https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-backend	feature/booking-endpoints	m67n4p9	feat: endpoints para reservas	Implementación de servicios REST para realizar reservas y consultar reservas activas	14/05/2025
https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-backend	feature/sede-endpoints	o15r3s8	feat: endpoints para sedes	Desarrollo de servicios REST para consultar información de sedes y asignación a administradores	15/05/2025

6.2.2.5. Testing Suite Evidence for Sprint Review.

En esta sección se presenta el conjunto de Unit Tests, Integration Tests y Acceptance Tests automatizados para los Web Services relacionados con los User Stories especificados en el Sprint.

Relación de Commits de Testing

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on (Date)
tavolo-backend	feature/unit-tests	abc123	Add unit tests for reservation service	Implemented tests for create, update and delete methods	2023-10-15

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on (Date)
tavolo-backend	feature/integration-tests	def456	Add integration tests for reservation API	Testing the complete flow of reservation endpoints	2023-10-16
tavolo-backend	feature/bdd-tests	ghi789	Add BDD tests for reservation feature	Added Cucumber tests with Gherkin scenarios for reservation management	2023-10-18

Unit Tests

TableCommandServiceTest

Esta clase contiene pruebas unitarias para validar el comportamiento del servicio `TableCommandService`, responsable de gestionar las operaciones relacionadas con la creación de mesas en el sistema.

```

21 public class TableCommandServiceTest {
22     @Mock 2 usages
23     private com.tavolo.platform.booking.infrastructure.persistence.jpa.repositories.TableRepository tableRepository;
24
25     @Mock 2 usages
26     private ExternalHeadquarterService externalHeadquarterService;
27
28     @InjectMocks 3 usages
29     private TableCommandServiceImpl tableCommandService;
30
31     @BeforeEach
32     public void setUp() {
33         MockitoAnnotations.openMocks(this);
34
35         // Inyección manual de los mocks
36         ReflectionTestUtils.setField(tableCommandService, "tableRepository", tableRepository);
37         ReflectionTestUtils.setField(tableCommandService, "externalHeadquarterService", externalHeadquarterService);
38     }
39
40     @Test
41     public void testHandleCreateTableCommand() {
42         // Crear un comando de ejemplo con los parámetros correctos
43         Integer tableNumber = 1;
44         Integer capacity = 4;
45         Long headquarterId = 1L;
46
47         // Configuración del mock para que retorne true cuando se verifica si existe el headquarter
48         when(externalHeadquarterService.existsHeadquarter(headquarterId)).thenReturn(true);
49
50         CreateTableCommand command = new CreateTableCommand(tableNumber, capacity, headquarterId);
51         Table mockTable = new Table(new TableDetails(tableNumber, capacity), new HeadquarterId(headquarterId));
52         when(tableRepository.save(any(Table.class))).thenReturn(mockTable);
53
54         Optional<Table> result = tableCommandService.handle(command);
55
56         assertTrue(result.isPresent()); // Verificar que se crea la mesa
57     }
58 }

```

Este test unitario valida:

- La creación correcta de comandos para crear mesas

- El comportamiento del servicio cuando verifica la existencia de una sede
- El correcto guardado de la mesa en el repositorio
- La devolución del resultado esperado

Integration Tests

TableCommandServiceIntegrationTest

Esta clase de prueba integra los distintos componentes del sistema para validar el flujo completo de creación de mesas:

```
@SpringBootTest
@Transactional
public class TableCommandServiceIntegrationTest {
    @Autowired
    private TableCommandServiceImpl tableCommandService;

    @Test
    public void testHandleCreateTableCommandIntegration() {
        // Crear un comando de ejemplo con los parámetros correctos
        Integer tableNumber = 1; // Número de mesa como Integer
        Integer capacity = 4; // Capacidad como Integer
        Long headquarterId = 1L; // ID de la sede

        CreateTableCommand command = new CreateTableCommand(tableNumber, capacity, headquarterId);
        Optional<Table> result = tableCommandService.handle(command);
        assertTrue(result.isPresent()); // Verificar que se crea la mesa
    }
}
```

Esta prueba de integración verifica:

- El funcionamiento correcto del servicio en un entorno transaccional
- La comunicación correcta entre los distintos componentes del sistema
- La persistencia adecuada de los datos

Autenticacion (AuthFeature.feature)

Feature: Autenticación de usuarios

Scenario: Registro exitoso de un nuevo usuario

Given el usuario proporciona datos válidos para registro

When se envía la solicitud de registro

Then el sistema crea el usuario y devuelve un estado 201

Scenario: Fallo en el registro por datos inválidos

Given el usuario proporciona datos inválidos

When se envía la solicitud de registro

Then el sistema devuelve un error de validación

6.2.2.6. Execution Evidence for Sprint Review.

En este segundo sprint, el equipo Tavolo ha logrado implementar con éxito los componentes fundamentales del sistema, creando una base sólida para el desarrollo futuro del proyecto. Se han completado todas las historias de

usuario planificadas de la parte web, alcanzando las principales funcionalidades tanto para comensales como para administradores.

- **Landing page completa** con información del producto, galería y formulario de contacto
- **Sistema de autenticación** diferenciado para comensales y administradores
- **Visualización de disponibilidad de mesas en tiempo real**
- **Sistema de reservas** con confirmación y visualización de reservas activas
- **Panel administrativo** para gestión de mesas y visualización de la sede asignada

Landing Page

Tavolo[Inicio](#)[¿Qué es Tavolo?](#)[¿Cómo Funciona?](#)[Características Clave](#)[Galería Virtual](#)[Contáctanos](#)

Evita filas. Descubre cafeterías inteligentes con Tavolo.

Monitorea en tiempo real el aforo y elige la mejor sede para disfrutar tu café.

[Explora Tavolo](#)





¿Qué es Tavolo?

Tavolo es una aplicación innovadora que te permite monitorear en tiempo real el aforo de cafeterías y hacer reservas desde tu dispositivo móvil. Con Tavolo, puedes disfrutar de tu café sin preocuparte por las largas filas.

¿Cómo Funciona?

Con sensores de aforo y una aplicación intuitiva, puedes monitorear en tiempo real la disponibilidad de las cafeterías y hacer reservas desde tu dispositivo móvil.



Sensores detectan aforo

Los sensores de Tavolo detectan el aforo en tiempo real, permitiéndote conocer la cantidad de personas en la cafetería.



Muestra disponibilidad en tiempo real

La aplicación Tavolo muestra la disponibilidad en tiempo real de las cafeterías, permitiéndote elegir la mejor opción.



El usuario reserva y ve el menú en línea

Una vez que eliges la cafetería, puedes reservar tu mesa y ver el menú en línea, facilitando tu experiencia.

Características Clave

Tavolo ofrece una serie de características clave que mejoran tu experiencia en cafeterías. Desde la reserva de mesas hasta la consulta de disponibilidad en tiempo real, Tavolo está diseñado para hacer tu vida más fácil.



Menú digital y vista del local

Explora el menú digital y disfruta de una vista panorámica del local para elegir tu lugar ideal.



Consulta de disponibilidad en tiempo real

Consulta la disponibilidad en tiempo real de las cafeterías y evita las largas filas.



Sugerencia de otras sedes

Si la sede que elegiste está llena, Tavolo te sugiere otras sedes cercanas con disponibilidad.

¿Tienes una cafetería?

Tavolo es la solución perfecta para optimizar la experiencia de tus clientes. Con nuestra aplicación, podrás ofrecer un servicio eficiente y cómodo, mejorando la satisfacción del cliente y aumentando la lealtad a tu marca.

- ✓ Datos en tiempo real
- ✓ Gestión de espacios eficiente.
- ✓ Canal directo con clientes.



Contáctanos

Si tienes alguna pregunta o necesitas más información sobre Tavolo, no dudes en contactarnos. Estamos aquí para ayudarte.

☐ Acepto los términos y condiciones

Enviar Mensaje

Tavolo

Tavolo es una aplicación innovadora que te permite monitorear en tiempo real el aforo de cafeterías y hacer reservas desde tu dispositivo móvil. Con Tavolo, puedes disfrutar de tu café sin preocuparte por las largas filas.

Secciones

Inicio
¿Qué es Tavolo?
Características Clave
Cómo Funciona
Contáctanos

Redes Sociales

Facebook
Instagram
Twitter
LinkedIn

Sistema para Comensales



¡Bienvenido!

Registrarse

Email

correo@ejemplo.com

Contraseña

contraseña



Confirmar contraseña

confirmar contraseña



Regístrate

¿Ya tienes un cuenta? [Iniciar sesión](#)

Conoce nuestras sedes y reserva!



Urqu Coffee Shop

Dirección: Jirón Áncash 251, Centro de Lima

Horario:

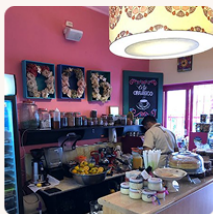
Lunes -Viernes: 9:00 am - 8 pm

Sábado: 10:00 am - 6 pm

Domingo: 10:00 am - 5 pm

Ver mesas

Ver en Mapa



Homemade Hecho en Casa

Dirección: Calle Henry Revett 259, Miraflores

Horario:

Lunes -Viernes: 9:00 am - 8 pm

Sábado: 10:00 am - 6 pm

Domingo: 10:00 am - 5 pm

Ver mesas

Ver en Mapa

Mis reservas

Revisa tus reservas y no llegues tarde

Urqe Coffe Shop

Mesa: #6
Zona: Terraza
Hora: 6:00 p.m

Ver local en Mapa

Homemade Hecho en Casa

Mesa: #2
Zona: Terraza
Hora: 10:00 p.m

Ver local en Mapa

Bisetti

Mesa: #2
Zona: Terraza
Hora: 10:00 p.m

Ver local en Mapa

Revisa el menú

Lo que tenemos para ofrecerte

Ver todo

Tostadas andinas con crema de queso de cabra

\$ 20.00

Una reinterpretación gourmet de las tostadas matutinas, esta delicia fusiona ingredientes tradicionales peruanos con un toque contemporáneo. El plato presenta dos rebanadas de pan artesanal de quinua y kiwicha, ligeramente tostadas en mantequilla de hierbas. Sobre ellas, una capa generosa de crema batida de queso de cabra infundada con huacatay

Panel de Administración



¡Bienvenido de vuelta!

Iniciar Sesión

Email

correo@ejemplo.com

Contraseña

contraseña

Iniciar Sesión

¿No tienes una cuenta todavía? [Crear cuenta](#)



Homemade Hecho en Casa

Dirección: Calle Henry Revett 259, Miraflores

Horario:

Lunes -Viernes: 9:00 am - 8 pm

Sábado: 10:00 am - 6 pm

Domingo: 10:00 am - 5 pm

Número: +01 6782345



Ver reservas

Revisa las reservas

Ver todo

Mesa #1

Capacidad: 5 personas

Zona: Terraza

Hora:

8:00

9:00

10:00

11:00

12:00

13:00

14:00

15:00

16:00

17:00

18:00

19:00

20:00

Mesa #2

Capacidad: 4 personas

Zona: Sala principal

Hora:

8:00

9:00

10:00

11:00

12:00

13:00

14:00

15:00

16:00

17:00

18:00

19:00

20:00

Gestionar mesas

+ Agregar mesa

Mesa #2

ID: 2

Capacidad: 3 personas

Zona: Sala principal

Estado: AVAILABLE

Eliminar

Mesa #1

ID: 1

Capacidad: 2 personas

Zona: Sala principal

Estado: AVAILABLE

Eliminar

Mesa #3

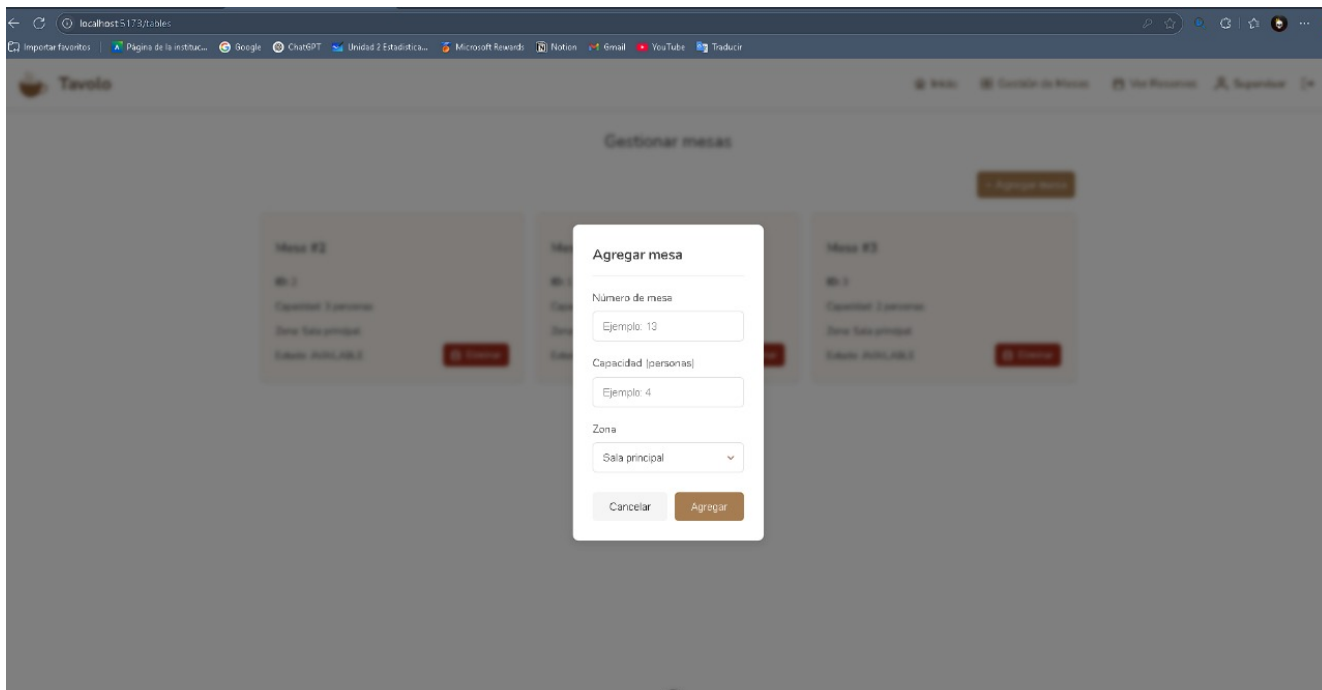
ID: 3

Capacidad: 2 personas

Zona: Sala principal

Estado: AVAILABLE

Eliminar



Tavolo

[Inicio](#)

[Gestión mesas](#)

[Ver reservas](#)

[Cerrar sesión](#)

Ver reservas

Ver por mesa ▾

Mesa #2

Capacidad: 4 personas

Zona: Sala principal

Hora:

8:00 9:00 10:00 11:00 12:00 13:00 14:00 15:00 16:00 17:00 18:00 19:00 20:00

Mesa #2

Capacidad: 4 personas

Zona: Sala principal

Hora:

8:00 9:00 10:00 11:00 12:00 13:00 14:00 15:00 16:00 17:00 18:00 19:00 20:00

Mesa #2

Capacidad: 4 personas

Zona: Sala principal

Hora:

8:00 9:00 10:00 11:00 12:00 13:00 14:00 15:00 16:00 17:00 18:00 19:00 20:00

Video Explicativo

El video muestra el flujo completo desde la landing page hasta las funcionalidades de reserva para comensales y gestión para administradores, demostrando la integración exitosa de todos los componentes desarrollados durante este sprint.

Video Demostración Sprint 2 - Tavolo

El equipo ha completado el 100% de las historias de usuario planificadas para este sprint, cumpliendo con los criterios de aceptación establecidos. La aplicación es funcional en su estado actual y proporciona las funcionalidades básicas

necesarias para la gestión de mesas y reservas. En los **próximos sprints** se agregarán características adicionales **como notificaciones, reportes estadísticos** y la integración completa con los sensores IoT físicos.

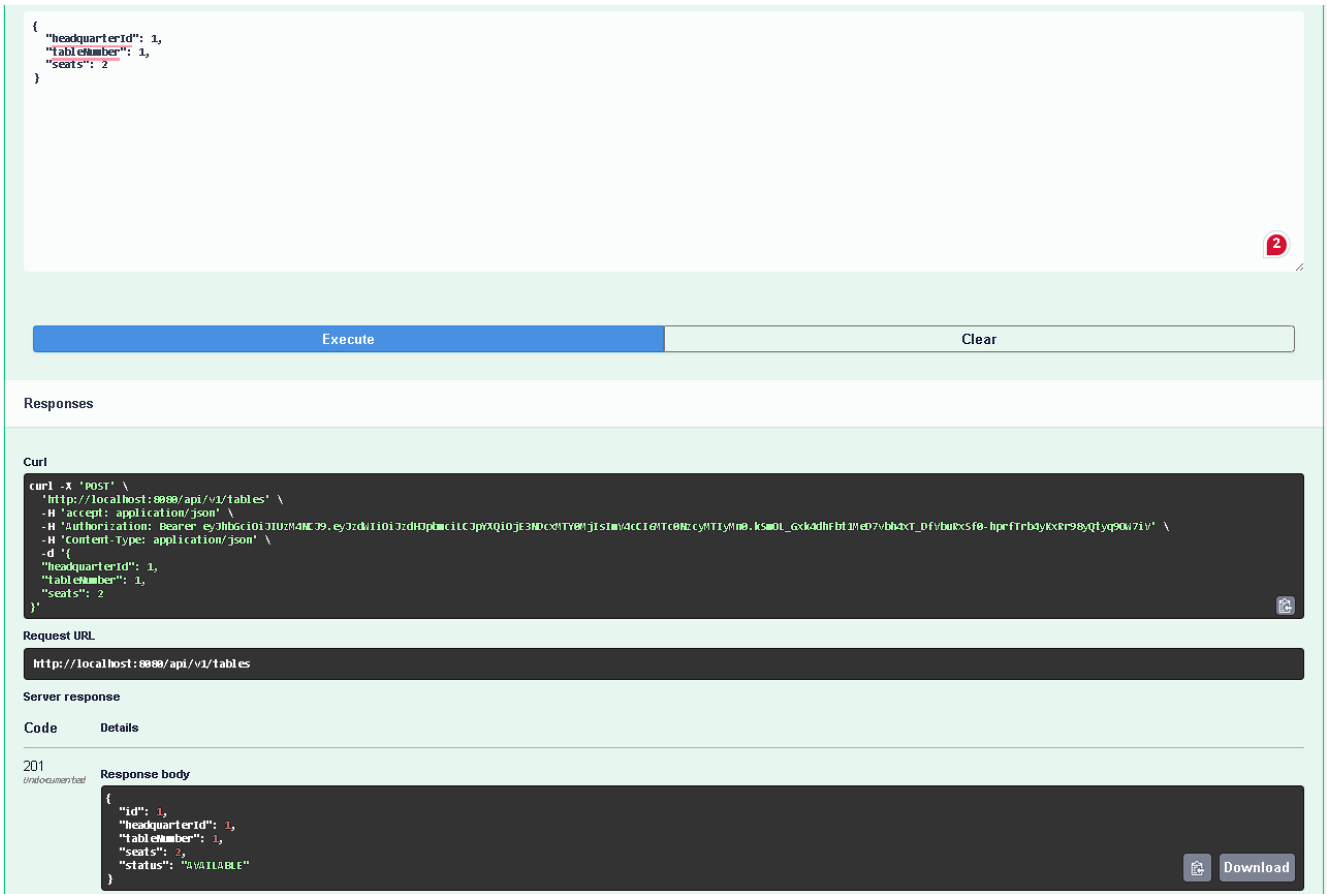
6.2.2.7. Services Documentation Evidence for Sprint Review.

Durante el Sprint 2, el equipo ha logrado documentar completamente los endpoints de la API de Tavolo utilizando Swagger. Esta documentación proporciona una referencia clara y accesible para desarrolladores frontend, facilitando la integración con los servicios backend.

Categoría	Endpoint	Método HTTP	Descripción
Autenticación	/api/v1/authentication/sign-up	POST	Registrar nuevo usuario
Autenticación	/api/v1/authentication/sign-in	POST	Iniciar sesión de usuario
Reservas	/api/v1/bookings	GET	Listar todas las reservas
Reservas	/api/v1/bookings	POST	Crear nueva reserva
Reservas	/api/v1/bookings/{id}	GET	Obtener detalles de reserva específica
Reservas	/api/v1/bookings/client/{clientId}	GET	Obtener reservas de un cliente específico
Sedes	/api/v1/headquarters	GET	Listar todas las sedes
Sedes	/api/v1/headquarters	POST	Crear nueva sede
Sedes	/api/v1/headquarters/{headquarterId}	GET	Obtener detalles de sede específica
Roles	/api/v1/roles	GET	Listar todos los roles
Usuarios	/api/v1/users	GET	Listar todos los usuarios
Usuarios	/api/v1/users/{userId}	GET	Obtener detalles de usuario específico
Mesas	/api/v1/tables	GET	Listar todas las mesas
Mesas	/api/v1/tables	POST	Crear nueva mesa
Mesas	/api/v1/tables/{tableId}	GET	Obtener detalles de mesa específica
Mesas	/api/v1/tables/{tableId}/schedule	GET	Obtener horario de una mesa específica

Evidencia de pruebas realizadas

Para verificar la funcionalidad de la API, se realizaron pruebas exhaustivas utilizando la interfaz Swagger. A continuación se muestran algunos ejemplos de estas pruebas:



Resultado exitoso de la creación de una mesa

Repositorio y commits relacionados

Repositorio: <https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-backend>

Commit ID	Descripción	Autor	Fecha
c739042	feat(booking): implementar endpoint para consultar reservas por ID	Baldeon Fabian, Aldo Alberto	07/05/2025
l28j5k6	feat(iam): implement user authentication and registration system	Castillo Castillo, Jair Alexander	11/05/2025
m67n4p9	feat: feat(booking,branching): implement reservation system with branch management	Baldeon Fabian, Aldo Alberto	11/05/2025
o15r3s8	feature(iam): add bounded context iam	Castillo Castillo, Jair Alexander	12/05/2025
p45t7u9	feature(shared): add add bounded context shared	Castillo Castillo, Jair Alexander	12/05/2025

6.2.2.8. Software Deployment Evidence for Sprint Review.

Durante este segundo sprint, el equipo Tavolo ha implementado una estrategia de despliegue completa para asegurar que todos los componentes del sistema estén disponibles en entornos de producción. Se ha desplegado exitosamente la landing page, el backend API y la aplicación web frontend, utilizando servicios en la nube para garantizar disponibilidad, escalabilidad y seguridad.

Landing Page Deployment

La landing page de Tavolo fue desplegada utilizando GitHub Pages, aprovechando la integración directa con nuestro repositorio para un flujo de trabajo simplificado y automático.

URL: <https://upc-pre-202510-1asi0572-2952-tavolo.github.io/tavolo-landing-page/>

Evidencia del despliegue

Landing Page 1

Landing Page 2

Landing Page 3

Landing Page 4

Backend API

La API de Tavolo fue desplegada en render, se ha probado todas las funcionalidades de la plataforma de manera exitosa.

URL: <https://tavolo-backend.onrender.com/swagger-ui/index.html>

Evidencia del Swagger

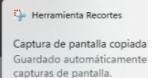
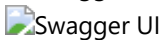
Apache 2.0

<https://tavolo-backend.onrender.com> - Generated server url

Authorize

 $\wedge$ 

^

 $\wedge$ 

BTS SIO BORDEAUX - LYCÉE GUSTAVE EIFFEL

La aplicación web de Tavolo se preparó para ser desplegada en Render, que ofrece una plataforma eficiente para aplicaciones Vuejs con rápidos tiempos de carga y distribución global.

URL: <https://tavolo-web-develop.onrender.com>

Evidencia de las pantallas

Pantalla de Comensal

Mis reservas

Revisa tus reservas y no llegues tarde

Urqe Coffe Shop

Mesa: #6
Zona: Terraza
Hora: 6:00 p.m

Ver local en Mapa

Homemade Hecho en Casa

Mesa: #2
Zona: Terraza
Hora: 10:00 p.m

Ver local en Mapa

Bisetti

Mesa: #2
Zona: Terraza
Hora: 10:00 p.m

Ver local en Mapa

Revisa el menú

Lo que tenemos para ofrecerte

Ver todo

Tostadas andinas con crema de queso de cabra

\$ 20.00

Una reinterpretación gourmet de las tostadas matutinas, esta delicia fusiona ingredientes tradicionales peruanos con un toque contemporáneo. El plato presenta dos rebanadas de pan artesanal de quinua y kiwicha, ligeramente tostadas en mantequilla de hierbas. Sobre ellas, una capa generosa de crema batida de queso de cabra infundada con huacatay

Homemade Hecho en Casa

Café con opciones para vegetarianos y veganos, ideal para desayunos y brunchs.

Entradas

Platos principales

Postres

Bebidas

Tostadas andinas con crema de queso de cabra

S/ 20.00

Una reinterpretación gourmet de las tostadas matutinas, esta delicia fusiona ingredientes tradicionales peruanos con un toque contemporáneo. El plato presenta dos rebanadas de pan artesanal de quinua y kiwicha, ligeramente tostadas en mantequilla de hierbas. Sobre ellas, una capa generosa de crema batida de queso de cabra infundada con huacatay

Wrap de quinua y camote

S/ 35.00

Tortilla integral rellena con camote asado en especias andinas, quinua roja cocida al dente, palta en cubos, tomates cherry y brotes de alfalfa. Acompañado con una salsa de yogurt vegetal con hierbabuena.

Tostada verde tropical

S/ 18.00

Homemade Hecho en Casa

Café con opciones para vegetarianos y veganos, ideal para desayunos y brunchs.

Filtrar mesas

Capacidad

Select

5 personas

4 personas

3 personas

2 personas

1 persona

Zona

Select

Terraza

Ventana

Sala principal

Se reservó correctamente la mesa

Mesa #1

Capacidad: 5 personas

Zona: Terraza

Reservar

Mesa #2

Capacidad: 4 personas

Zona: Ventana

Reservar

Mesa #3

Capacidad: 3 personas

Zona: Sala principal

Reservar

Mesa #4

Capacidad: 2 personas

Zona: Terraza

Reservar

Mesa #5

Capacidad: 1 persona

Zona: Terraza

Reservar

Mesa #6

Capacidad: 5 personas

Zona: Ventana

Reservar

Mesa #7

Capacidad: 5 personas

Zona: Sala principal

Reservar

Mesa #8

Capacidad: 4 personas

Zona: Terraza

Reservar

Mesa #9

Capacidad: 3 personas

Zona: Terraza

Reservar

Mesa #10

Capacidad: 1 persona

Zona: Sala principal

Reservar

Mesa #11

Capacidad: 3 personas

Zona: Terraza

Reservar

Mesa #12

Capacidad: 2 personas

Zona: Sala principal

Reservar

Conoce nuestras sedes y reserva!

Sede #1



Cerrar

Horario:

Lunes -Viernes: 9:00 am - 8 pm
Sábado: 10:00 am - 6 pm
Domingo: 10:00 am - 5 pm

Ver mesas

Ver en Mapa

Ver mesas

Ver en Mapa

Pantalla de Administrador



Homemade Hecho en Casa

Dirección: Calle Henry Revett 259, Miraflores

Horario:

Lunes -Viernes: 9:00 am - 8 pm

Sábado: 10:00 am - 6 pm

Domingo: 10:00 am - 5 pm

Número: +01 6782345



Ver reservas

Revisa las reservas

Ver todo

Mesa #1

Capacidad: 5 personas

Zona: Terraza

Hora:

8:00

9:00

10:00

11:00

12:00

13:00

14:00

15:00

16:00

17:00

18:00

19:00

20:00

Mesa #2

Capacidad: 4 personas

Zona: Sala principal

Hora:

8:00

9:00

10:00

11:00

12:00

13:00

14:00

15:00

16:00

17:00

18:00

19:00

20:00

Gestionar mesas

+ Agregar mesa

Mesa #2

ID: 2

Capacidad: 3 personas

Zona: Sala principal

Estado: AVAILABLE

Eliminar

Mesa #1

ID: 1

Capacidad: 2 personas

Zona: Sala principal

Estado: AVAILABLE

Eliminar

Mesa #3

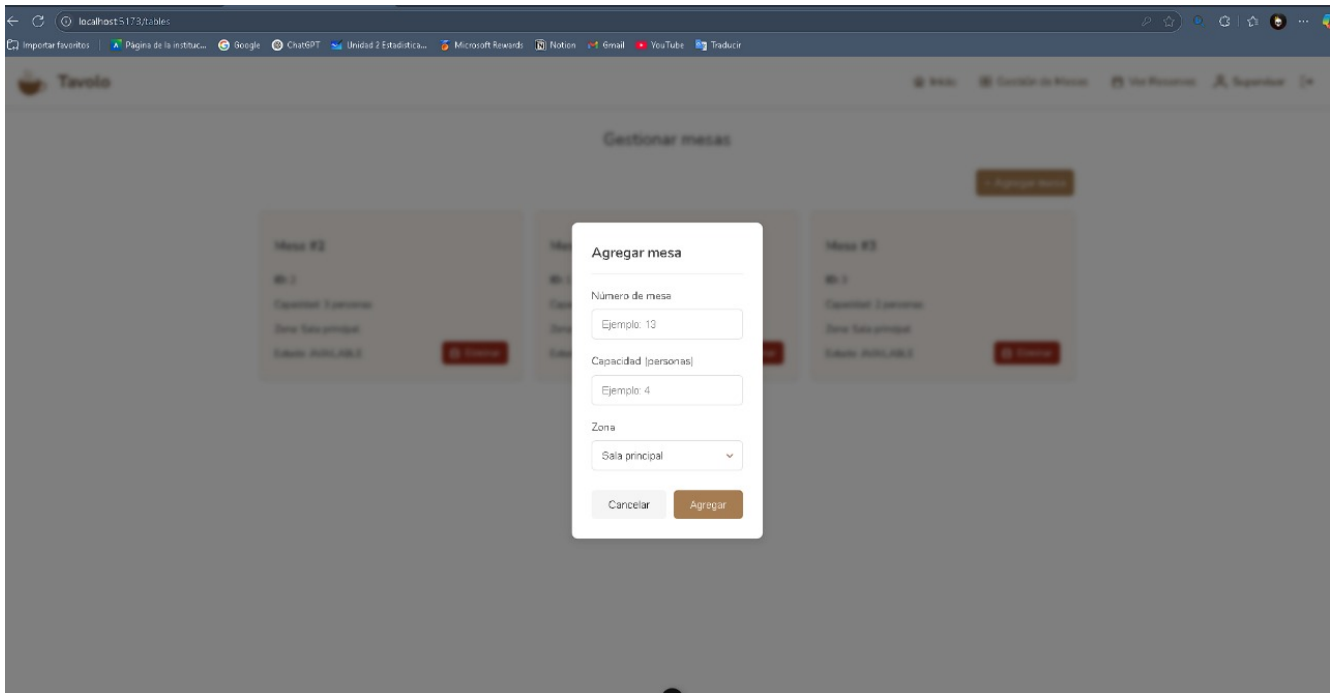
ID: 3

Capacidad: 2 personas

Zona: Sala principal

Estado: AVAILABLE

Eliminar



Tavolo

InicioGestión mesasVer reservasCerrar sesión

Gestionar mesas

+ Agregar mesa

Mesa #1

Capacidad: 5 personas

Zona: Terraza

Eliminar

Mesa #2

Capacidad: 4 personas

Zona: Ventana

Eliminar

Mesa #3

Capacidad: 4 personas

Zona: Ventana

Eliminar

Mesa #4

Capacidad: 2 personas

Zona: Terraza

Eliminar

Mesa #5

Capacidad: 1 persona

Zona: Terraza

Eliminar

Mesa #6

Capacidad: 4 personas

Zona: Ventana

Eliminar

Mesa #7

Capacidad: 5 personas

Zona: Sala principal

Eliminar

Mesa #8

Capacidad: 4 personas

Zona: Terraza

Eliminar

Mesa #9

Capacidad: 3 personas

Zona: Terraza

Eliminar

Mesa #10

Capacidad: 1 persona

Zona: Sala principal

Eliminar

Mesa #11

Capacidad: 3 personas

Zona: Terraza

Eliminar

Mesa #12

Capacidad: 2 personas

Zona: Sala principal

Eliminar

Se agregó correctamente la mesa

Ver reservas

Seleccionar

Por mesa

Por persona

Mesa #2

Capacidad: 4 personas

Zona: Sala principal

Hora:

8:00

9:00

10:00

11:00

12:00

13:00

14:00

15:00

16:00

17:00

18:00

19:00

20:00

Mesa #2

Capacidad: 4 personas

Zona: Sala principal

Hora:

8:00

9:00

10:00

11:00

12:00

13:00

14:00

15:00

16:00

17:00

18:00

19:00

20:00

Mesa #2

Capacidad: 4 personas

Zona: Sala principal

Hora:

8:00

9:00

10:00

11:00

12:00

13:00

14:00

15:00

16:00

17:00

18:00

19:00

20:00

Ver reservas

Ver por persona

Carla Zapata

Mesa #2

Capacidad: 4 personas

Zona: Sala principal

Hora: 18:00

Maribel Torres

Mesa #2

Capacidad: 4 personas

Zona: Sala principal

Hora: 18:00

Anthony Gutierrez

Pantalla Superadministrador

Panel Administrativo

Tu centro de gestión de creación de sedes y supervisores

Estadísticas del Sistema

0 Sedes registradas

0 Supervisores activos

0 Productos en menú

Acciones Rápidas

Gestionar Sedes

Gestionar Menú

Gestionar Supervisores

Cafetería El Aroma

~ Sabores que inspiran momentos ~

Entradas
Platos Principales
Postres
Bebidas

+ Agregar plato

No hay productos disponibles en esta categoría.

¡Vuelve pronto para descubrir nuevas delicias!

Tavolo

Gestión de Sedes

Sedes
Supervisores

Sedes Disponibles

+ Nueva Sede

Supervisores

Miraflores

+ Asignar Supervisor

Selecciona una sede para asignar un supervisor

Crear Nueva Sede

Información Básica

Nombre de la Sede *

Ingrese el nombre de la sede

Teléfono Fijo Teléfono Móvil

Ej: 4567890 Ej: 987654321

Ubicación

Calle * Número *

Ingrese solo números Ingrese solo letras

Ciudad * Código Postal *

Ingrese la ciudad Ej: 1234

País *

Ingrese el país

Coordenadas

Latitude: No seleccionada Longitud: No seleccionada



Sedes Disponibles

+ Nueva Sede

🔍 Buscar sedes...

Miraflores Activa

Registro de Supervisores

Crear una nueva cuenta de supervisor

Nombre de Usuario

Ingrese nombre de usuario

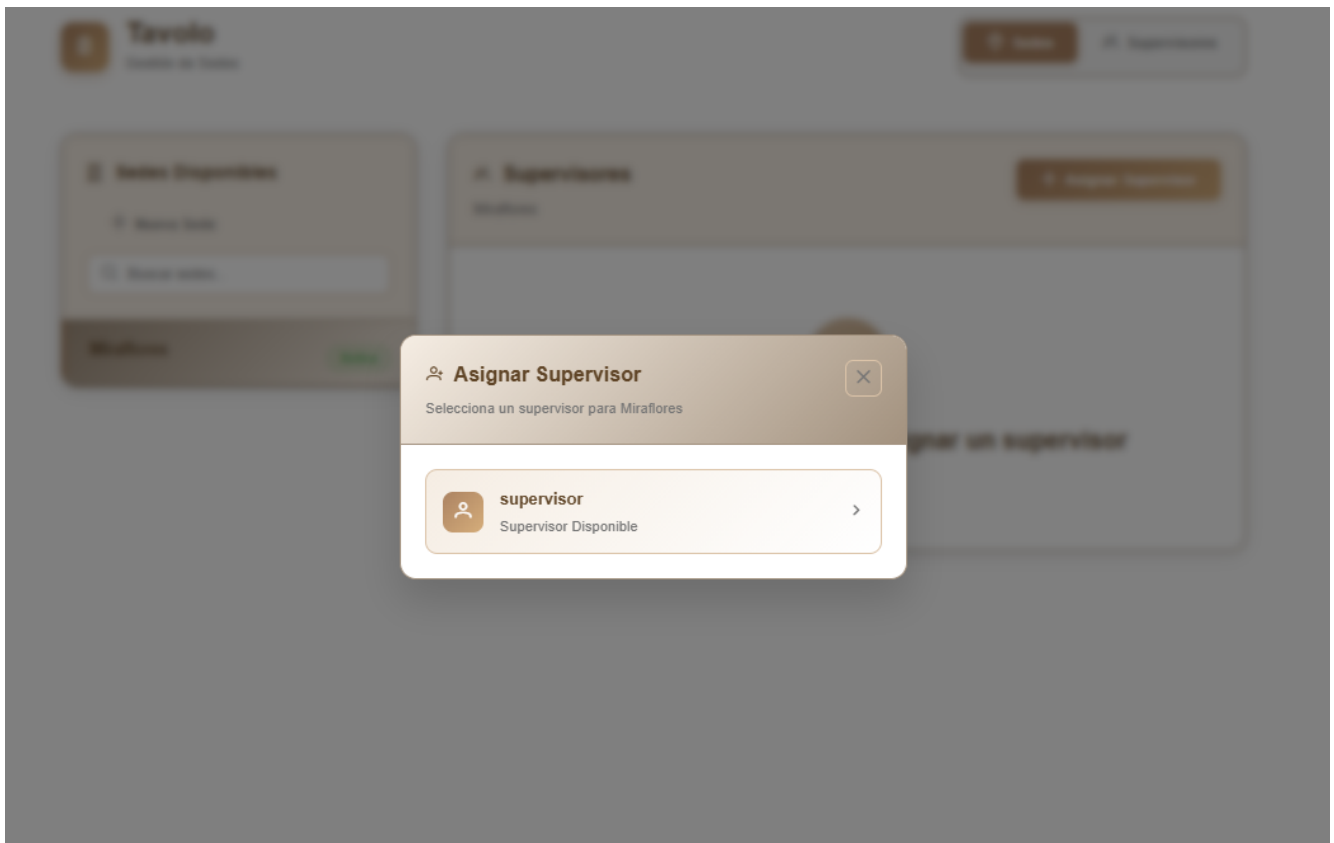
Contraseña

Confirmar Contraseña

Registrar Supervisor

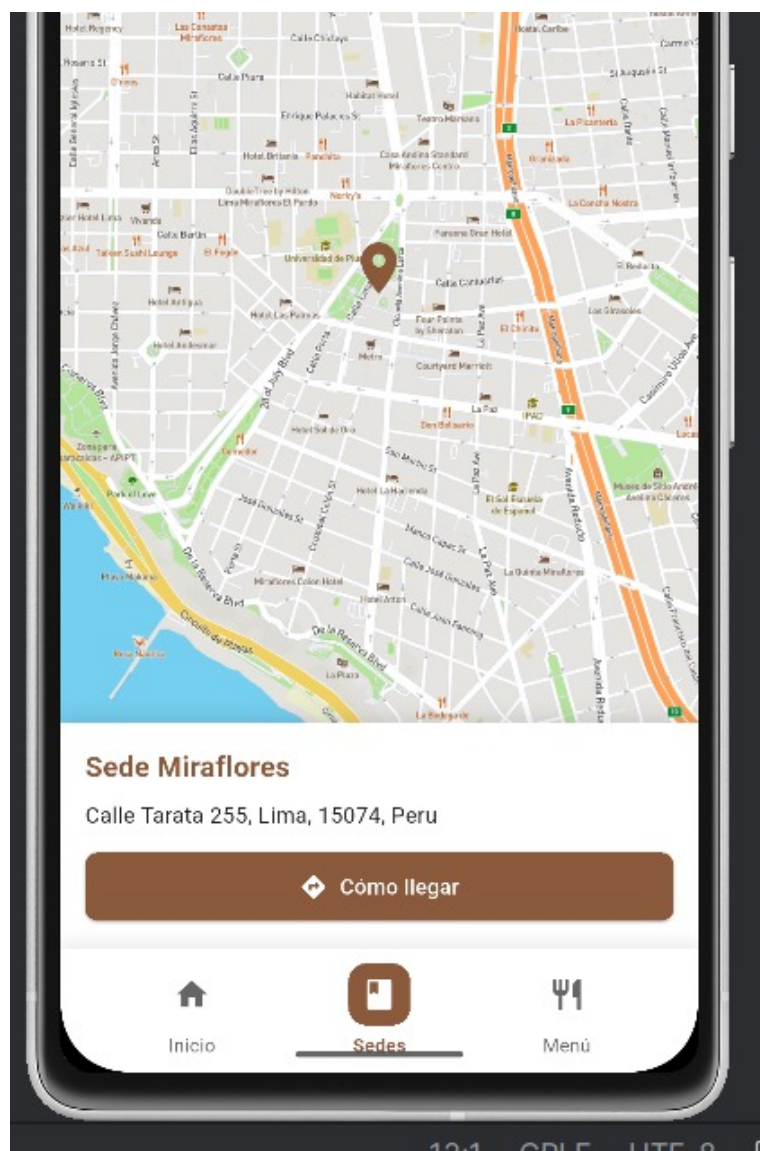
Supervisores Registrados

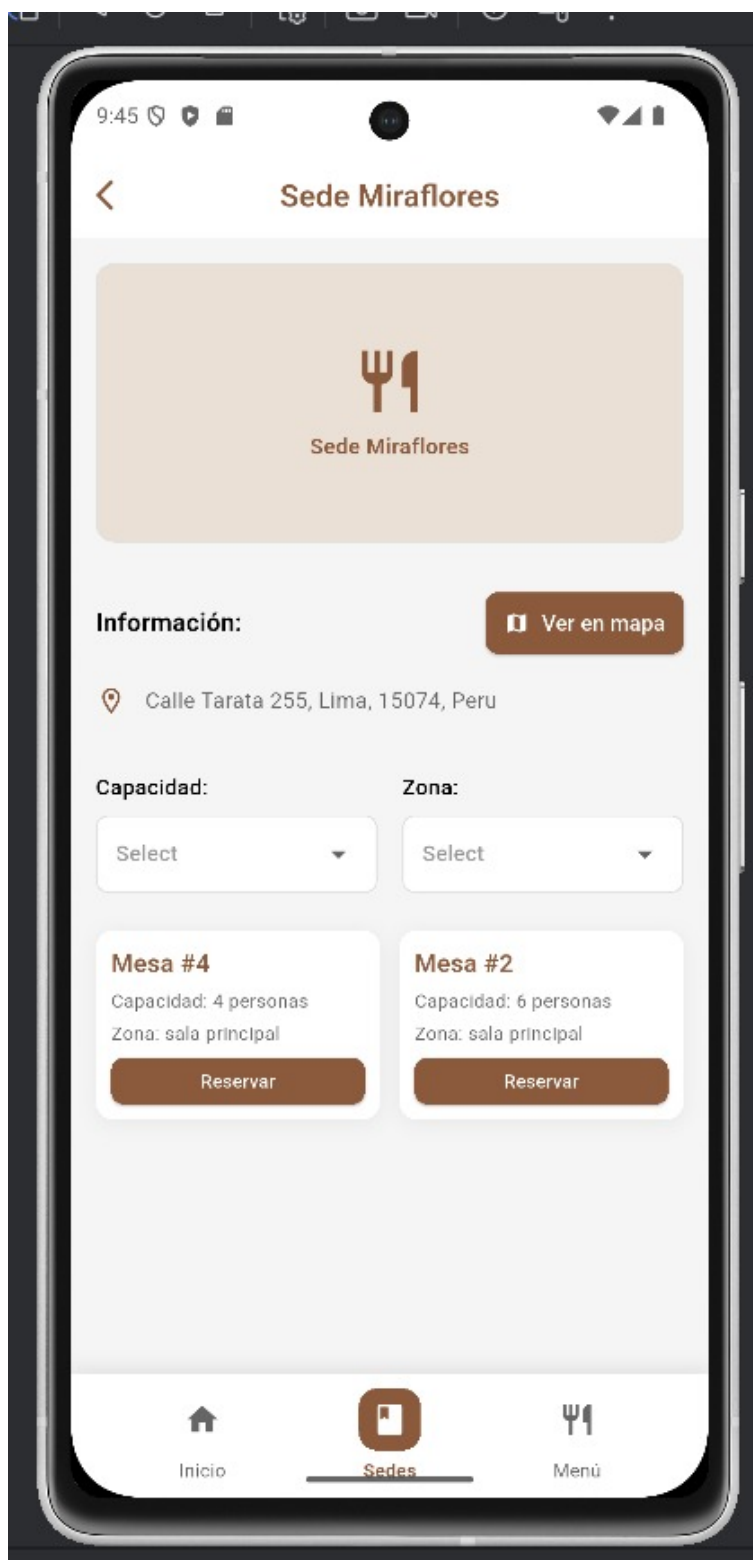
 supervisor

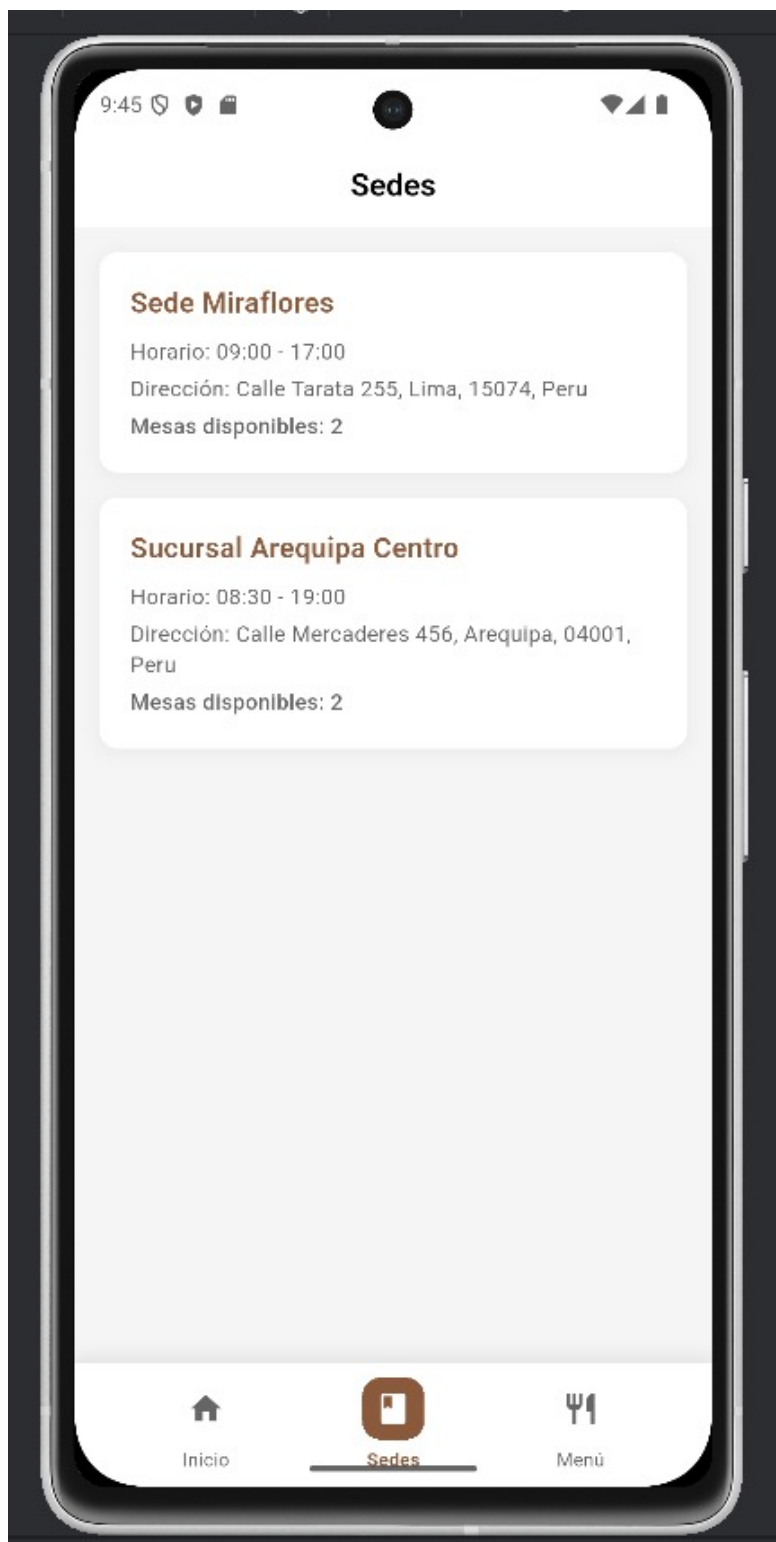


Mobile Application

Pantalla de Comensal



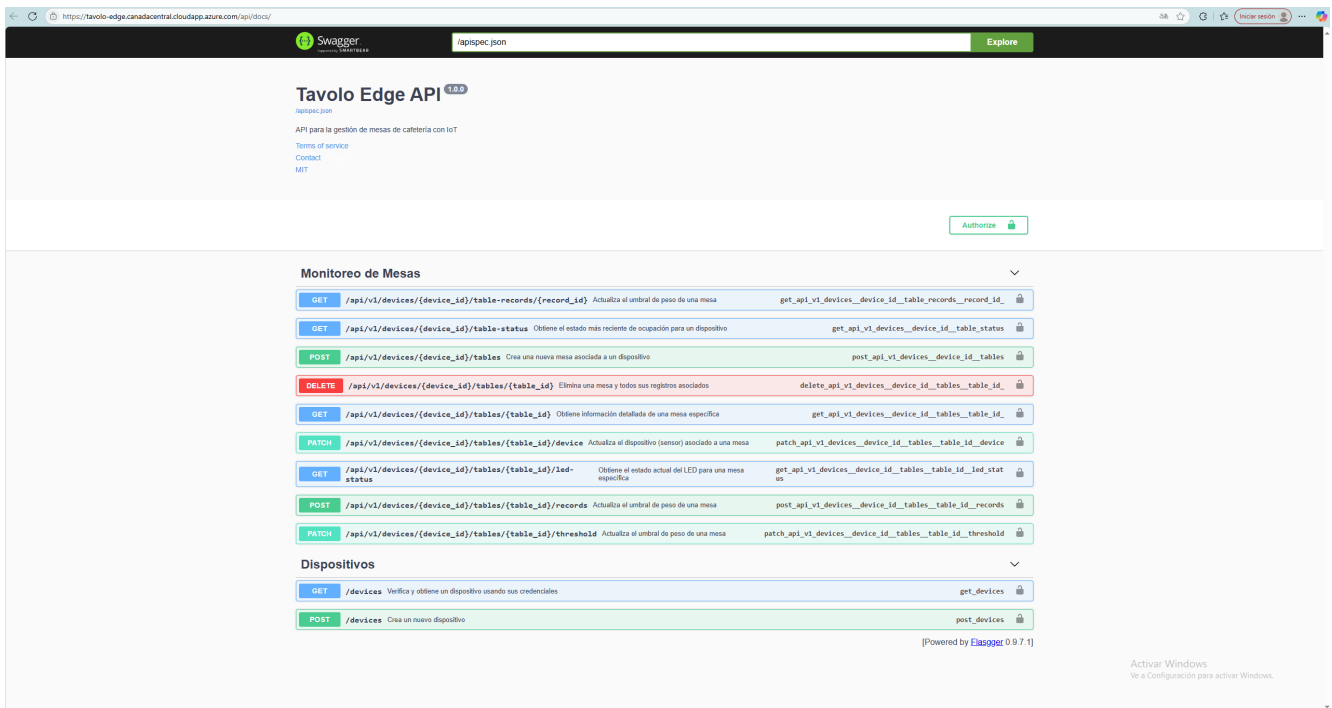




Edge API

El API Edge desplegada en Azure sirve como punto de entrada a las integraciones con nuestro sistema embedded, gestionando y enroutando solicitudes externas de forma segura y eficiente

URL: <https://tavolo-edge.canadacentral.cloudapp.azure.com/api/docs/>



6.2.2.9. Team Collaboration Insights during Sprint.

Durante el Sprint 2, el equipo Tavolo demostró una colaboración efectiva y coordinada, permitiendo el cumplimiento exitoso de todas las historias de usuario planificadas. A continuación, se detallan los aspectos clave de la dinámica colaborativa del equipo:

Distribución del trabajo y contribuciones

El análisis de las contribuciones muestra una distribución equilibrada de responsabilidades:

Miembro del equipo	Áreas principales	Contribuciones clave	Pull requests	Issues closed
Baldeon Fabian, Aldo	Backend API, Frontend (Cliente), Testing	Sistema de autenticación, visualización de disponibilidad de mesas, endpoints de reservas	8	7
Cama Salvatierra, Jimena	Frontend (Administrador), Frontend (Cliente)	Panel administrativo, visualización de sede asignada, visualización de reservas	6	6
Castillo Castillo, Jair	Backend API, Testing	Endpoints de mesas y autenticación, pruebas unitarias, documentación API	7	6
La Torre Valle, Franz	Frontend (Cliente), Frontend (Administrador)	Menú digital, reserva activa, módulos de navegación, protección de rutas	6	5
Quezada Portalatino, Barbara	Frontend (Cliente), Landing Page	Sistema de reservas, landing page completa, formulario de contacto	7	6

Colaboración en componentes principales

Backend API

- **Líderes:** Aldo Baldeon y Jair Castillo
- **Logros:** Implementación completa de endpoints para autenticación, gestión de mesas, reservas y sedes.
- **Dinámica de trabajo:** Aldo se enfocó en los servicios de autenticación y reservas, mientras que Jair desarrolló los endpoints para mesas y configuró la documentación Swagger. La colaboración entre ambos permitió una integración fluida de los diferentes servicios.

Frontend Web (Cliente y Administrador)

- **Líderes:** Jimena Cama (Administrador) y Franz La Torre (Cliente)
- **Colaboradores:** Aldo Baldeon, Barbara Quezada, Jair Castillo
- **Logros:** Desarrollo de todas las interfaces de usuario para comensales y administradores con excelente usabilidad.
- **Dinámica de trabajo:** Jimena coordinó la implementación del panel administrativo mientras Franz dirigió el desarrollo de las interfaces para comensales. Barbara contribuyó significativamente con el sistema de reservas, mientras que Aldo aportó en la autenticación y visualización de mesas. Jair apoyó en la integración de componentes críticos.

Landing Page

- **Líder:** Barbara Quezada
- **Logros:** Diseño e implementación completa de la landing page con todas las secciones requeridas.
- **Dinámica de trabajo:** Barbara lideró el diseño y desarrollo de la landing page, incorporando información clara sobre el producto y asegurando una experiencia visual atractiva para los visitantes.

Testing

- **Líderes:** Aldo Baldeon y Jair Castillo
- **Logros:** Verificación de la funcionalidad de todos los componentes implementados.
- **Dinámica de trabajo:** Crearon y ejecutaron casos de prueba para validar el correcto funcionamiento de la aplicación, asegurando que todos los criterios de aceptación fueran cumplidos.

Herramientas de colaboración utilizadas

- **GitHub:** Control de versiones y seguimiento de issues
- **Discord:** Comunicación diaria y coordinación entre miembros
- **Figma:** Diseño colaborativo de interfaces
- **Google Meet:** Reuniones de planificación y retrospectiva

Metodología de trabajo

El equipo adoptó un enfoque ágil con las siguientes prácticas:

1. **Daily Standup:** Reuniones semanales de 2 horas para compartir avances y obstáculos.
2. **Revisión de código:** Cada PR requería al menos una revisión de otro miembro del equipo.
3. **Documentación continua:** Actualización constante de la documentación de API y componentes.

Desafíos enfrentados y soluciones

- **Desafío:** Integración entre frontend y backend en desarrollo paralelo.
Solución: Definición temprana de contratos de API con Swagger y uso de datos mockeados para pruebas.
- **Desafío:** Coordinación del trabajo entre múltiples colaboradores en los componentes frontend.
Solución: División clara de responsabilidades y uso de ramas de características específicas.

Lecciones aprendidas

1. La documentación temprana de la API facilitó significativamente el trabajo paralelo.
2. La asignación de líderes por componente con colaboradores definidos mejoró la calidad del código.
3. Las sesiones de pair programming aceleraron la resolución de problemas complejos.
4. El uso de prototipos y mockups antes de la implementación redujo la necesidad de cambios posteriores.

Métricas de colaboración

- **Tiempo promedio de resolución de issues:** 1.5 días
- **Tiempo promedio de revisión de PRs:** 4 horas
- **Porcentaje de código con revisión por pares:** 100%

Este Sprint 2 ha establecido una base sólida para la colaboración continua del equipo Tavolo. La distribución equilibrada del trabajo, el liderazgo compartido y el enfoque en la calidad han permitido entregar todas las funcionalidades planificadas cumpliendo con los criterios de aceptación establecidos.

6.2.3. Sprint 3

6.2.3.1.Sprint Planning 3.

Sprint #3	Sprint 3
Sprint Planning Background	
Date	15/6/2025
Time	6:00 pm
Location	Reunión virtual vía Discord
Prepared By	Aldo Baldeon
Attendees (to planning meeting)	- Baldeon Fabian, Aldo Alberto - Cama Salvatierra, Jimena Tamara - Castillo Castillo, Jair Alexander - La Torre Valle, Franz Jair - Quezada Portalatino, Barbara Susana
Sprint Goal & User Stories	
Sprint 3 Goal	En este sprint la meta fue implementar el dispositivo IoT físico basado en ESP32 para detección de ocupación de mesas en tiempo real, integrándolo completamente con el sistema backend y las aplicaciones frontend.
Sprint 3 Velocity	18
Sum of Story Points	18

6.2.3.2.Aspect Leaders and Collaborators.

Para el Sprint 3, el equipo se organizó en torno a la implementación del dispositivo IoT físico y su integración completa con el ecosistema Tavolo. Se estableció una estructura colaborativa donde todos los miembros participaron activamente en el montaje del dispositivo físico, mientras que se asignaron líderes específicos para cada aspecto técnico del proyecto.

La matriz de liderazgo y colaboración (LACX) para este sprint se enfocó en la implementación del hardware y la integración IoT:

Team Member	GitHub Username	IoT Device Hardware Leader (L) / Collaborator (C)	Embedded Software Leader (L) / Collaborator (C)	API Integration Leader (L) / Collaborator (C)	Hardware Assembly Leader (L) / Collaborator (C)
Baldeon Fabian, Aldo Alberto	CodAress	L	L	L	C
Cama Salvatierra, Jimena Tamara	aksonie	C	C	C	C
Castillo Castillo, Jair Alexander	U202211390	C	C	L	C
La Torre Valle, Franz Jair	FranzJairLTV	C	C	C	C
Quezada Portalatino, Barbara Susana	BarbaraQP15	C	C	C	C

Responsabilidades por aspecto:

- **IoT Device Hardware:**

- Líder: Baldeon Fabian, Aldo Alberto
- Enfoque: Diseño e implementación del dispositivo físico basado en ESP32 con LEDs indicadores y botones de control
- Tareas principales: Montaje del circuito, configuración de componentes, pruebas de hardware

- **Embedded Software:**

- Líder: Baldeon Fabian, Aldo Alberto
- Enfoque: Desarrollo del firmware para ESP32 que gestiona sensores, LEDs y comunicación WiFi
- Tareas principales: Programación en C++, manejo de interrupciones, comunicación HTTP con el backend

- **API Integration:**

- Líder: Castillo Castillo, Jair Alexander
- Enfoque: Modificación y extensión de la API backend para recibir y procesar datos del dispositivo IoT
- Tareas principales: Nuevos endpoints, autenticación de dispositivos, procesamiento de datos en tiempo real

- **Hardware Assembly:**

- Participación colaborativa de todo el equipo
- Enfoque: Montaje físico del dispositivo siguiendo el modelo de Wokwi
- Tareas principales: Soldadura, conexiones, pruebas de circuito, calibración

6.2.3.3.Sprint Backlog 3.

Sprint	User Story	Work-Item / Task	Id	Title	Description	Estimation (Hours)	Assigned To	Status
Sprint 3	Como administrador quiero recibir datos de ocupación en tiempo real desde sensores IoT para monitorear el estado de las mesas automáticamente	Task	T01	Implementar circuito físico ESP32	Montar el dispositivo físico con ESP32, LEDs y botones siguiendo el modelo de Wokwi	12	Todo el equipo	Done
		Task	T02	Desarrollar firmware ESP32	Programar el microcontrolador para detectar ocupación y comunicarse con el backend	16	Aldo Baldeon	Done
		Task	T03	Integrar API IoT con backend	Modificar backend para recibir y procesar datos del dispositivo IoT	14	Jair Castillo	Done
		Task	T04	Pruebas de integración IoT	Validar comunicación completa entre dispositivo, backend y frontend	10	Todo el equipo	Done
Sprint 3	Como comensal quiero ver el estado actualizado de las mesas en tiempo real para tomar decisiones informadas sobre mi reserva	Task	T05	Actualizar frontend para datos en tiempo real	Modificar interfaces para mostrar datos actualizados desde los sensores IoT	8	Franz La Torre	Done
		Task	T06	Implementar notificaciones push	Sistema de notificaciones cuando cambia el estado de las mesas	6	Barbara Quezada	Done
		Task	T07	Optimizar app móvil para IoT	Actualizar la aplicación móvil para recibir datos en tiempo real	8	Jimena Cama	Done

6.2.3.4.Development Evidence for Sprint Review.

En este tercer sprint, el equipo Tavolo logró un hito fundamental: la implementación exitosa del dispositivo IoT físico que convierte nuestro sistema en una solución integral de gestión de cafeterías con detección automática de ocupación. Este sprint representó la materialización de la visión IoT del proyecto, donde todos los componentes digitales desarrollados en sprints anteriores se integran con hardware real.

Logros destacables del Sprint 3:

- **Implementación del dispositivo IoT físico** basado en ESP32 con LEDs indicadores tricolor y botones de control
- **Integración completa con el ecosistema backend** mediante nuevos endpoints especializados para IoT
- **Actualización de las aplicaciones frontend** para mostrar datos de sensores en tiempo real
- **Pruebas exhaustivas de la comunicación** entre dispositivo físico y plataforma digital

Dispositivo IoT Físico Implementado

El dispositivo se basó en el modelo de simulación Wokwi desarrollado previamente y incluye:

Componentes del circuito:

- ESP32 DevKit V1 como microcontrolador principal
- 3 LEDs indicadores (rojo, verde, azul) para estados de mesa
- 2 botones pulsadores para interacción manual
- 3 resistencias de 200Ω para protección de LEDs
- Conexiones WiFi para comunicación con el backend

Funcionalidades implementadas:

- Detección automática de ocupación de mesa
- Indicadores LED tricolor para estados: libre (verde), ocupada (rojo), reservada (azul)
- Botones para simulación manual de eventos de ocupación
- Comunicación bidireccional con el sistema backend
- Actualización en tiempo real del estado en todas las interfaces

6.2.3.5. Testing Suite Evidence for Sprint Review.

Durante el Sprint 3, se implementó una suite de pruebas especializada para validar la funcionalidad del dispositivo IoT y su integración con el sistema completo. Las pruebas se realizaron físicamente con el dispositivo, abarcando desde el nivel de hardware hasta la integración end-to-end, ejecutándose directamente en las ramas de desarrollo correspondientes.

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on (Date)
tavolo-embedded	main	abc123	Add hardware validation tests	Implementación de pruebas para validación física del circuito y componentes	2025-06-20
tavolo-embedded	feature/http-client	ghi789	Add end-to-end IoT communication tests	Pruebas completas de comunicación dispositivo físico-backend-frontend	2025-06-22

6.2.3.6. Execution Evidence for Sprint Review.

En este tercer sprint, el equipo Tavolo ha logrado la implementación exitosa del componente IoT físico, completando así la visión integral del sistema de gestión de cafeterías. El dispositivo embebido desarrollado permite la detección

automática de ocupación de mesas en tiempo real, integrándose perfectamente con las aplicaciones web y móvil desarrolladas en sprints anteriores.

Funcionalidades implementadas en el Sprint 3:

- **Dispositivo IoT físico funcional** con ESP32, LEDs tricolor y botones de control
- **Detección automática de ocupación** mediante sensores y botones de simulación
- **Comunicación en tiempo real** entre dispositivo y sistema backend
- **Actualización automática** del estado de mesas en todas las interfaces
- **Dashboard mejorado** con datos en tiempo real de sensores IoT

 Dispositivo ESP32 Montado

Dispositivo ESP32 completamente montado con LEDs indicadores y botones de control

 Circuito en Funcionamiento

LEDs indicadores mostrando diferentes estados: libre (verde), ocupada (rojo), reservada (azul)

6.2.3.7.Services Documentation Evidence for Sprint Review.

Todos los endpoints existentes de los sprints anteriores se mantuvieron completamente funcionales, incluyendo autenticación, gestión de mesas, reservas, sedes, roles y usuarios.

Categoría	Endpoint	Método HTTP	Nuevas Funcionalidades
Mesas	/api/v1/tables/{tableId}	GET	Incluye datos de sensor IoT asociado
Reservas	/api/v1/bookings	GET	Filtra por estado real de ocupación IoT
Sedes	/api/v1/headquarters/{headquarterId}/tables	GET	Muestra estado IoT de todas las mesas

La documentación completa de la API IoT está disponible en:

URL Swagger: <https://tavolo-backend.onrender.com/swagger-ui/index.html#/IoT>

6.2.3.8.Software Deployment Evidence for Sprint Review.

Durante el Sprint 3, se completó el despliegue integral del ecosistema Tavolo, incluyendo la configuración de la infraestructura para soportar los dispositivos IoT y la comunicación en tiempo real. Este sprint representó la culminación de la arquitectura distribuida con componentes físicos y digitales funcionando en conjunto.

Arquitectura de Despliegue Completa

El sistema Tavolo ahora opera con la siguiente infraestructura desplegada:

1. Landing Page (GitHub Pages)

URL: <https://upc-pre-202510-1asi0572-2952-tavolo.github.io/tavolo-landing-page/>

- Estática, optimizada para SEO
- Formulario de contacto funcional
- Responsive design

2. Backend API (Render Cloud)

URL: <https://tavolo-backend.onrender.com>

- API REST completa con endpoints IoT
- Base de datos PostgreSQL en la nube
- Documentación Swagger disponible
- WebSocket para comunicación en tiempo real

3. Frontend Web Application (Render)

URL: <https://tavolo-web-develop.onrender.com>

- Aplicación Vue.js con PWA capabilities
- Integración completa con IoT
- Dashboard en tiempo real
- Notificaciones push

4. Edge API (Azure Cloud)

URL: <https://tavolo-edge.canadacentral.cloudapp.azure.com>

- Microservicio para procesamiento edge
- Balanceador de carga para dispositivos IoT
- Cache distribuido para datos en tiempo real

5. Mobile Application

- Disponible a través de u celular Android

6.2.3.9.Team Collaboration Insights during Sprint.

Durante el Sprint 3, el equipo Tavolo experimentó su mayor desafío colaborativo al integrar componentes físicos con el ecosistema digital desarrollado. Este sprint requirió una coordinación excepcional entre todos los miembros, ya que el montaje del dispositivo IoT físico demandó la participación activa de todo el equipo, mientras que las tareas de desarrollo se distribuían según las especialidades de cada miembro.

Distribución del Trabajo y Contribuciones

Miembro del equipo	Áreas principales	Contribuciones clave	Pull requests	Issues closed	Participación hardware
Baldeon Fabian, Aldo	IoT Hardware Leader, Embedded Software	Diseño del circuito, firmware ESP32, integración con backend	12	10	Líder en montaje
Cama Salvatierra, Jimena	Hardware Assembly	Notificaciones push móviles, optimización para datos IoT	8	7	Colaboradora activa
Castillo Castillo, Jair	Backend IoT API, Hardware Assembly	Endpoints IoT, WebSocket real-time, documentación API	10	9	Colaboradora activa
La Torre Valle, Franz	Hardware Assembly	Dashboard tiempo real, visualización datos sensores	7	6	Colaboradora activa
Quezada Portalatino, Barbara	Hardware Assembly	Sistema notificaciones web, UX para datos IoT	6	5	Colaboradora activa

Herramientas de Colaboración Especializadas

- **GitHub**: Control de versiones con ramas específicas para IoT
- **Wokwi Simulator**: Simulación previa del circuito antes del montaje físico
- **Discord**: Canal específico #iot-hardware para comunicación técnica
- **Google Drive**: Compartir esquemas, fotos del proceso y documentación técnica

6.3. Validation Interviews.

6.3.1. Diseño de Entrevistas.

Para las entrevistas de validación del Sprint 3, se diseñaron preguntas específicas que evalúan tanto las funcionalidades existentes como las nuevas implementaciones del sistema IoT y las mejoras realizadas en este sprint.

Segmento clientes

Navegación y Usabilidad General:

- ¿Cómo fue tu experiencia haciendo una reserva? ¿Pudiste entender qué mesas y horarios estaban disponibles?
- ¿Qué tan útil te pareció poder revisar o cancelar tu reserva desde la app?

Funcionalidades IoT y Tiempo Real:

- ¿Notaste que el estado de las mesas se actualiza automáticamente? ¿Qué te pareció esta funcionalidad?
- ¿Qué opinas sobre poder ver el estado real de ocupación de las mesas antes de llegar a la cafetería?
- ¿Te generó más confianza saber que la información de disponibilidad viene directamente de sensores en las mesas?

Aplicación Móvil Mejorada:

- ¿Pudiste acceder fácilmente a todas las funciones desde tu teléfono?

Segmento administradores

Gestión de Mesas y Operaciones:

- ¿Qué tan clara y práctica te resultó la función para agregar, editar o eliminar mesas?

Sistema IoT y Monitoreo en Tiempo Real:

- ¿Qué opinas sobre recibir datos automáticos del estado de las mesas a través de sensores?
- ¿Te resultó útil poder ver el estado real de ocupación sin tener que verificar físicamente cada mesa?

Dashboard y Reportes:

- ¿Te gustaría tener acceso a más estadísticas o reportes sobre el uso de las mesas?

Eficiencia Operativa:

- ¿Recomendarías este sistema a otros administradores de cafeterías? ¿Por qué?

6.3.2. Registro de Entrevistas.

Propietarios de cafeterías

Entrevista #1

Entrevistado:

Duración:

Link:

Entrevista #2

Entrevistado:

Duración:

Link:

Clientes de cafeterías

Entrevista #1

Entrevistado:

Duración:

Link:

Entrevista #2

Entrevistado:

Duración:

Link:

6.3.3. Evaluaciones según heurísticas

6.4. Video About-the-Product

En este video demostrativo presentamos Tavolo, nuestra plataforma integral para gestión de cafeterías. El contenido muestra:



Enlace:

[Video About The Product de Tavolo](#)

El video demuestra cómo estas tres interfaces interactúan en tiempo real para optimizar la operación de las cafeterías, desde que un cliente realiza una reserva hasta que se refleja en los reportes de gestión de reservas de un supervisor.

Conclusiones Generales del Proyecto

- El proyecto TAVOLO se enfoca en optimizar la experiencia en cafeterías peruanas mediante una solución tecnológica IoT que gestiona la ocupación de mesas y reservas. Para los comensales, ofrece visualización del menú, disponibilidad de mesas en tiempo real y reservas; para los administradores, permite la gestión de mesas y reservas de la sede asignada. El objetivo es mejorar la experiencia del cliente y la eficiencia operativa del negocio.
- La investigación y análisis de requerimientos identificaron la problemática de la incertidumbre sobre la disponibilidad de mesas en cafeterías, especialmente en horas pico, lo que afecta la experiencia del cliente. Se definieron segmentos objetivo claros (comensales y administradores) y se analizaron competidores como Eat App y OpenTable, destacando la ventaja de TAVOLO en la detección automática de ocupación en tiempo real mediante sensores IoT.
- El equipo demostró un liderazgo compartido y colaborativo, asumiendo responsabilidades específicas en cada capítulo y utilizando herramientas como GitHub y metodologías como Gitflow para una gestión eficiente del proyecto. Se establecieron metas claras, se planificaron tareas según las fortalezas individuales y se mantuvo una comunicación constante, lo que permitió cumplir los objetivos y fortalecer la cohesión del equipo.
- El desarrollo de la app móvil consideró principios de usabilidad y experiencia de usuario (UX), permitiendo una navegación intuitiva, rápida y accesible. Esto garantiza que los comensales puedan interactuar con el sistema sin necesidad de asistencia técnica, lo que refuerza su autonomía y satisfacción.
- TAVOLO fue diseñado con una arquitectura modular que permite escalar a múltiples sedes y escenarios de uso. Su backend en Java (Spring Boot), combinado con una app Flutter, una app web y dispositivos embebidos, proporciona una base tecnológica sólida adaptable a restaurantes, coworkings y otros negocios con gestión de espacios.

Bibliografía

1. **Las MIPYME en cifras 2022:** Publicado por la Oficina General de Evaluación de Impacto y Estudios Económicos (OGEEIE) de Produce. ([Ver publicación](#))
2. **Memoria Institucional del Ministerio de la Producción 2022:** Documento oficial que resume las acciones y logros del ministerio en ese año. ([Ver Memoria](#))
3. **Informe de ComexPerú (basado en datos oficiales):** Las micro y pequeñas empresas en el Perú Resultados en 2022. ([Ver Informe Comex](#))
4. **Google Developers:** Material Design Guidelines. Guía oficial para la creación de interfaces centradas en la experiencia del usuario. ([Ver guía](#))
5. **IoT For All:** Using Microcontrollers and Sensors in Smart Environments. Artículo técnico sobre aplicaciones de sensores IoT en espacios físicos. ([Ver artículo](#))

Anexos

- **Video de Exposición TB1:** <https://drive.google.com/drive/u/0/folders/1YN0UBGC28osxHvRTsBrLSjwX2CAVLH1t>
- **Repositorio del Proyecto en GitHub:** <https://github.com/orgs/upc-pre-202510-1asi0572-2952-tavolo/repositories>
- **Organización del Proyecto en GitHub:** <https://github.com/upc-pre-202510-1asi0572-2952-tavolo>
- **Informe del Proyecto:** <https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-report>
- **Backend del Proyecto:** <https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-backend>
- **Landing Page del Proyecto:** <https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-landing-page>

- **Frontend Web del Proyecto:** <https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-web>
- **Frontend Movil del Proyecto:** <https://github.com/upc-pre-202510-1asi0572-2952-tavolo/tavolo-mobile>
- **Registro de Entrevista #1 (Propietarios/Cliente - Diego Morales/Elias Yauri Paucar):**
<https://drive.google.com/file/d/1CVDQx8Ue15xk7yt8oukkN1S87bAc7TeB/view?usp=sharing>
- **Registro de Entrevista #2 (Propietarios - Jenna Vasquez):**
https://drive.google.com/file/d/1WTxiDxw4vaabZhFvibL8mHLINcrJR0_9/view?usp=sharing
- **Registro de Entrevista #2 (Clientes - Aleshka Fernández):** <https://youtu.be/QGSisj7iF-k>
- **Video prototyping web cliente:** <https://drive.google.com/file/d/1FppOgE1BkclITFJZ26mgpkJoDlqalVaq/view?usp=sharing>
- **Video prototyping web administrador:** https://drive.google.com/file/d/1kcd_FJk4l8dhdRntgm0UZbYrUqKcuO6/view?usp=drive_link
- **Video prototyping mobile cliente:** https://drive.google.com/file/d/1H52_e9udVbAltnnXDAsrhZpsWwABluJw/view?usp=drive_link
- **Diagrama de Organization System:** https://drive.google.com/file/d/1OKUWtDfdjzkCRD_nNvbWuFwYXwBbUSHL/view?usp=sharing
- **Diagrama de C4** <https://drive.google.com/file/d/1C4Bcv6pZgL29nQ34p1o3DiLyqJqWyyxY/view?usp=sharing>
- **Registro de Entrevista de Validación #1 (Propietarios - Franchesko Montesinos):**
<https://drive.google.com/file/d/1BakcVds5atZrFJ7rZIf7gbyW3izZaHkM/view>
- **Registro de Entrevista de Validación #1 (Clientes - Jenna Portalatino):**
<https://drive.google.com/file/d/1BRU-wJS04vY4o1B6tNWp8De5KIXhHKA/v/view>
- **Registro de Entrevista de Validación #2 (Propietarios - Diego Salinas):**
<https://drive.google.com/file/d/1Q9JL3E64J4BqAubMJNZ6rc8pWE40EuIU/view>